Ước lượng cao độ F_0 theo tinh thần Praat

Mô hình dấu phẩy tĩnh bit-khớp và hiện thực trên FPGA
(Intel Quartus Prime Standard / Lite — DE10-Standard)

Praat-style Pitch / F_0 Estimation — Bit-Accurate Fixed-Point Model → Intel Quartus Standard (IP Catalog & Self-Written RTL)

Tài liệu độc lập. Không phụ thuộc các mục trước của bộ sách voice. Mọi khái niệm (Mel, HPF, DC, Avalon-ST, FIR, FFT) được định nghĩa tại chỗ trước khi dùng.

Đặt bài: Đo Cao Độ Tiếng Nói (F_0 / Pitch) — Từ Bản Chất Praat Đến Vi Kiến Trúc FPGA

▶ MỤC TIÊU HỌC TẬP

Trong nhận dạng tiếng nói, người ta thường gom phổ thành khoảng 80 dải lọc tam giác Mel (Log-Mel Spectrogram) để mạng nơ-ron nhận từ khóa — nhưng các con số đó cố tình *xóa* cao độ. Tài liệu này đứng riêng: giải thích tại sao không thể dùng phổ STFT để đo **Cao độ (Pitch / Tần số cơ bản F_0)** — tần số rung của dây thanh quản; bóc tách bản chất giải thuật tự tương quan thời gian theo tinh thần mã nguồn Praat (`Sound_to_Pitch.cpp`, Boersma 1993); dựng mô hình dấu phẩy tĩnh bit-khớp chạy được trên laptop; rồi map từng khối sang Intel Quartus Prime **Standard/Lite** trên DE10-Standard (Cyclone V `5CSXFC6D6F31C6N`) theo hai lộ trình IP Catalog và RTL tự viết. **Quartus Prime Pro không hỗ trợ Cyclone V** — chỉ dùng Standard hoặc Lite.

0. Tại sao cần một luồng xử lý riêng cho Cao Độ (F_0 Stream)?

Trong chuỗi xử lý tiếng nói ở biên (Edge Voice AI), các kỹ sư thường mắc một ngộ nhận kinh điển: nghĩ rằng vector Log-Mel Spectrogram (80 dải lọc Mel gom năng lượng phổ) đã chứa đủ mọi thông tin để giải quyết mọi bài toán.

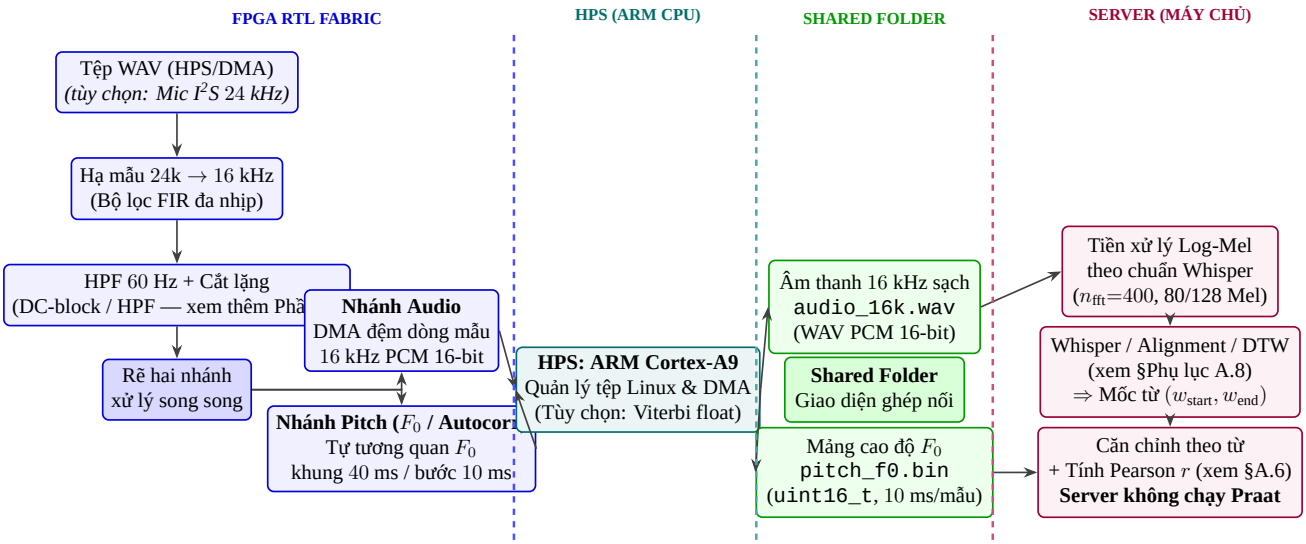
Thực tế, **Ngân hàng bộ lọc Mel sinh ra là để xóa bỏ cao độ**:

- 80 dải lọc tam giác Mel gom và tích phân năng lượng trên các dải tần số rộng hàng trăm Hertz nhằm mô phỏng hình thái của **bao phổ (spectral envelope)** — tức hình dáng ống thanh âm (vocal tract) đại diện cho các nguyên âm.
- Quá trình gom bin đó cố tình san phẳng các vạch hài âm thanh quản mảnh dẻ.
- Trong các bài toán ngữ điệu, phát hiện cảm xúc, nhận diện thanh điệu tiếng Việt (ngã, hỏi, sắc, huyền, nặng, ngang), hoặc đối sánh tương quan Pearson (r) với các công cụ âm học chuẩn mực phòng thí nghiệm như **Praat**, hệ thống bắt buộc phải đo chính xác từng Hertz của tần số cơ bản $F_0 = 1/T_0$ (với T_0 là chu kỳ giữa hai lần mở-đóng liên tiếp của dây thanh quản).

Do đó, một hệ thống âm thanh biên hoàn chỉnh bắt buộc phải tách thành hai luồng song song: **Luồng nhận dạng âm học (Log-Mel Frontend)** và **Luồng đo cao độ (F_0 / Pitch Stream)**.

0.1. Kiến trúc triển khai thực tế: FPGA Edge Node ↔ Shared Folder ↔ Server

Hai luồng vừa nêu không chỉ là hai khối thuật toán, mà là hai nhánh của một hệ sinh thái triển khai với ranh giới phần cứng/phần mềm được phân định rành mạch. Hình 1 đặc tả cấu trúc của toàn bộ hệ thống.



Hình 1: Kiến trúc triển khai hệ thống toàn diện: Phân định rõ 3 miền (FPGA Edge Node gồm RTL và HPS ARM, Thư mục dùng chung, và Máy chủ). FPGA tiền xử lý âm thanh sạch và tính sẵn cao độ F_0; máy chủ tính Log-Mel theo cấu hình Whisper, căn chỉnh từ và tính tương quan Pearson mà không cần chạy Praat.

Đọc Hình 1 (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Cột 1 — FPGA RTL Fabric (Phần cứng xử lý tín hiệu số):** Tiếp nhận đầu vào chính là tệp âm thanh WAV được nạp qua bộ đệm HPS/DMA (hoặc tùy chọn từ microphone I^2S 24 kHz). Tín hiệu đi qua bộ hạ mẫu đa nhịp FIR 24 kHz $\rightarrow$ 16 kHz (phù hợp chuẩn tiếng nói) và bộ lọc thông cao HPF 60 Hz (cắt nhiễu cơ học / DC chậm dưới khoảng 60 Hz) kèm thuật toán cắt khoảng lặng. Sau đó, dữ liệu rẽ thành 2 nhánh:
   - *Nhánh Audio:* Chuyển luồng mẫu 16 kHz mono 16-bit sạch qua bộ điều khiển DMA lên bộ nhớ hệ thống của HPS.
   - *Nhánh Pitch (F_0 / Autocorr):* Thực thi thuật toán tự tương quan trên khung 40 ms (640 mẫu) với bước nhảy 10 ms (160 mẫu) bằng 1 bộ MAC tuần tự duy nhất.
2. **Cột 2 — HPS (Hard Processor System — Lõi ARM Cortex-A9):** Đóng vai trò cầu nối hệ thống nhúng chạy Linux. HPS tiếp nhận dữ liệu từ RTL qua cầu nối AXI/DMA, đóng gói tệp âm thanh thành `audio_16k.wav` và mảng cao độ thành `pitch_f0.bin`. *Đặc biệt:* Nếu ứng dụng đòi hỏi đường cong cao độ mượt mà tuyệt đối bằng quy hoạch động Viterbi, thuật toán Viterbi dấu phẩy động có thể chạy trực tiếp trên lõi ARM của HPS mà không làm tiêu tốn tài nguyên logic FPGA.
3. **Cột 3 — Thư mục dùng chung (Shared Folder / Hợp đồng giao tiếp):** Là ranh giới phân định độc lập giữa Edge Node và Server. Gồm đúng hai tệp dữ liệu:
   - `audio_16k.wav`: Tệp âm thanh chuẩn PCM 16-bit mono, 16.000 Hz, đã lọc sạch trôi DC và cắt khoảng lặng hai đầu.
   - `pitch_f0.bin`: Mảng nhị phân cao độ F_0.
4. **Cột 4 — Máy chủ (Server):** Nhận `audio_16k.wav`, tự chạy hàm tiền xử lý Log-Mel phù hợp với kích cỡ mô hình Whisper đang dùng, chạy nhận diện mốc thời gian từ $(w_{\text{start}}, w_{\text{end}})$ bằng Whisper / Forced Alignment / DTW (xem chi tiết tại §Phụ lục A.8). Sau đó, máy chủ đọc trực tiếp mảng `pitch_f0.bin` đã có sẵn để căn chỉnh theo từ và tính hệ số tương quan Pearson r (xem §Phụ lục A.6). **Máy chủ hoàn toàn không phải chạy Praat**.

Giao diện mảng cao độ `pitch_f0.bin`: Tệp nhị phân `pitch_f0.bin` lưu trữ mảng tuần tự các số nguyên 16-bit không dấu biểu diễn tần số cao độ F_0 tức thời xuất ra định kỳ mỗi 10 ms. Quy chuẩn đóng gói dấu phẩy tĩnh **Q12.4**, ví dụ giải mã ra đơn vị `cent` âm học cùng cơ chế phân loại thanh tính được đặc tả chi tiết tại Mục 4.2 (ngay trước hàm xử lý tệp), và quy tắc dóng hàng thời gian với mô hình ASR Whisper trên máy chủ được thiết lập tường minh tại Mục 5.1.

Vì sao không tính sẵn ma trận Log-Mel trên FPGA? Một câu hỏi kỹ thuật rất xác đáng: *Nếu FPGA đã có DSP, tại sao không tính luôn 80 dải Log-Mel để đẩy lên server?* Câu trả lời nằm ở sự thay đổi và tính đa dạng của các mô hình nhận dạng tiếng nói (ASR):

1. **Tiền xử lý Whisper rất đặc thù:** Bộ tiền xử lý của OpenAI Whisper sử dụng kích thước STFT $n_{\text{fft}} = 400$ (không phải lũy thừa của 2!), bước nhảy hop $= 160$, áp dụng thang đo $\log_{10}$, kẹp ngưỡng biên độ tại max $-8{,}0$, chuẩn hóa tuyến tính $(x + 4{,}0)/4{,}0$, và đệm cố định chiều dài 30 giây (3.000 khung). Hơn nữa, trong khi Whisper phiên bản `tiny`, `base`, `small`, `medium` dùng 80 dải lọc Mel, thì phiên bản lớn nhất `large-v3` đã chuyển sang **128 dải Mel**.
2. **Tránh khóa cứng silicon (Hardware Lock-in):** Nếu cứng hóa (hardwire) một khối Log-Mel cố định vào RTL của FPGA, mạch sẽ mất tính tương thích khi máy chủ nâng cấp phiên bản mô hình ASR. Trong khi đó, việc gửi tệp âm thanh sạch `audio_16k.wav` cho phép máy chủ chạy hàm tiền xử lý tương ứng chỉ mất vài mili-giây trên CPU/GPU mà hoàn toàn mềm dẻo.
3. **Cao độ là bất biến sinh học:** Ngược lại với bao phổ ASR, tần số rung cơ bản F_0 của dây thanh quản người là một đại lượng vật lý thuần túy, không phụ thuộc vào bất kỳ mô hình học máy nào. Việc tính F_0 trên FPGA vừa giải phóng hoàn toàn máy chủ khỏi việc cài đặt và gọi thư viện Praat C++ cồng kềnh, vừa tận dụng triệt để năng lực tính MAC song song trên chip nhúng.

1. Phương pháp thô sơ bị bác bỏ: Dùng STFT và Argmax trên phổ tần số

Phương pháp trực giác ngây thơ nhất là: *Đã có biến đổi Fourier rời rạc (FFT 512 điểm) trong luồng Mel, vậy tại sao không tìm bin có biên độ lớn nhất (argmax) ở dải tần số thấp để làm F_0?*

Cách làm này thất bại ngay từ bước kiểm tra thực nghiệm vì hai bức tường vật lý:

1. Độ phân giải tần số quá thô ($\Delta f_{\text{bin}} = 31{,}25$ Hz): Lưới FFT $N = 512$ điểm ở tần số lấy mẫu $f_s = 16.000$ Hz tạo ra bước nhảy giữa hai bin kế tiếp là:

$$\Delta f_{\text{bin}} = \frac{f_s}{N} = \frac{16.000}{512} = 31{,}25 \text{ Hz}$$

Các vạch tần số rời rạc chỉ có thể nhận giá trị: $62{,}5$ Hz (bin 2), $93{,}75$ Hz (bin 3), $125{,}0$ Hz (bin 4), $156{,}25$ Hz (bin 5)…Giả sử một giọng nam nói ở cao độ $F_0 = 105$ Hz: tần số này rơi vào giữa bin 3 và bin 4. Thuật toán chọn bin nào cũng tạo ra sai số tuyệt đối từ 11–20 Hz (sai lệch $> 15\%$). Khi đem chuỗi F_0 giật cục này tính tương quan Pearson với đường cao độ thực tế của Praat, hệ số tương quan r sẽ sụt giảm mạnh (không đạt ngưỡng tin cậy).

2. Nguy cơ bắt nhầm vào các cực cộng hưởng Formant (F_1): Dây thanh quản phát ra các xung âm thanh tam giác giàu hài âm. Khi đi qua thanh đạo và khoang miệng, các hài âm có tần số nằm gần tần số cộng hưởng của khoang họng (Formant $F_1 \approx 300$–800 Hz) sẽ được khuếch đại mạnh mẽ bởi hiệu ứng cộng hưởng âm học của cột không khí. Do biên độ của các hài âm này thường vượt trội so với tần số cơ bản F_0, phép tìm cực đại đơn giản (argmax) trên phổ biên độ STFT **rất dễ bị đánh lừa và có nguy cơ cao bắt nhầm vào** F_1, dẫn đến lỗi nhảy quãng tám (octave jump).

► NHÌN THẤY VẬT LÝ

Biến đổi Fourier STFT là công cụ nhìn bao quát toàn phổ tần số, không phải cây thước kẹp đo chu kỳ dao động thanh quản. Muốn đo chu kỳ T_0, ta phải quay trở về **miền thời gian**.

2. Bản chất vật lý của Phép Tự Tương Quan (Autocorrelation)

Khi con người phát ra một nguyên âm có thanh (voiced speech), luồng khí từ phổi đẩy qua thanh môn làm hai mép dây thanh quản rung lên nhịp nhàng. Tín hiệu âm thanh $x[n]$ trong miền thời gian là một chuỗi các dạng sóng lặp lại sau mỗi chu kỳ rung. Để phân tích định lượng, ta định nghĩa các đại lượng cơ bản:

- L: Số lượng mẫu âm thanh trong một khung phân tích. Khung dữ liệu là một mảng rời rạc gồm L phần tử, có chỉ số n chạy từ 0 đến $L - 1$: $x[0], x[1], \ldots, x[L-1]$.
- T_0: Chu kỳ cơ bản của dây thanh quản, đo bằng số mẫu rời rạc ($T_0 \in \mathbb{N}^*$). Khi dây thanh rung tuần hoàn với chu kỳ T_0, dạng sóng lặp lại: $x[n] \approx x[n + T_0]$.

- τ (tau): Độ trễ (lag), là số mẫu mà ta dịch trượt bản sao tín hiệu so với bản gốc, với τ là số nguyên không âm ($\tau \geq 0$).
- F_0: Tần số cơ bản (Pitch / Cao độ), xác định qua hệ thức: $F_0 = f_s/T_0$ (với f_s là tần số lấy mẫu).

▶ HỎI – ĐÁP TƯ DUY: Khi đang nói, làm sao dây thanh biết trước tương lai để lặp lại mẫu $x[n+T_0]$?

Nếu đọc $x[n] \approx x[n+T_0]$ như "mẫu hiện tại bằng mẫu tương lai", trực giác sẽ vấp ngay: người nói đâu có biết trước mẫu tiếp theo để bắt buộc nó lặp lại. **Cơ thể hoàn toàn không cần biết tương lai.** Câu đúng là dạng nhân quả: $x[n] \approx x[n-T_0]$ — mẫu lúc này giống mẫu đúng một chu kỳ trước. Phép tự tương quan chỉ **đo** mức lặp lại đó; nó không bắt tín hiệu phải lặp lại. Hai lý do vật lý giải thích vì sao sự lặp lại xảy ra mà không cần tương lai:

1. **Dây thanh là một dao động cơ tự kích.** Luồng không khí từ phổi làm áp suất vùng thanh môn tụt xuống (hiệu ứng Bernoulli), kéo hai mép dây thanh hút vào nhau, cắt dòng khí rồi lại bung ra. Mỗi lần bung là một xung, và dao động này có khối lượng – độ đàn hồi riêng nên tự giữ một nhịp đều, giống dây đàn gảy xong vẫn rung theo tần số của chính nó. Xung vừa phát ra **đã định sẵn** xung kế tiếp; không có tương lai nào phải biết trước.
2. **Bộ máy phát âm thay đổi rất chậm so với nhịp rung.** Lưỡi, môi, hàm cần khoảng 50–100 ms mới đổi được một âm tiết; trong một khung 40 ms (640 mẫu) hình dạng ống cộng hưởng gần như đứng yên. Vì thế hai chu kỳ liền nhau đi qua **cùng một bộ lọc**, nên cùng bị bóp méo giống nhau và trông giống nhau.

Khi nào giả thiết này sụp đổ? (a) Âm vô thanh như /s/, /f/: không có dây thanh rung, tín hiệu gần như nhiễu ngẫu nhiên, không có chu kỳ. (b) Chuyển âm nhanh: đầu khung là một nguyên âm, cuối khung đã sang nguyên âm khác, bộ lọc không còn đứng yên nữa. Đó là lý do Praat đo cao độ theo từng khung ngắn và ghép lại bằng đường bao trơn, chứ không đo một lần cho cả câu.

Nguyên lý của phép tự tương quan rất trực quan: **Tạo ra một bản sao của khung tín hiệu, trượt bản sao đi một khoảng trễ τ mẫu, sau đó nhân từng cặp mẫu tương ứng và tính tổng dồn lại.** Biểu thức toán học này được gọi là **tổng tự tương quan** (hoặc **tổng tích các cặp mẫu**):

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} x[n] \cdot x[n+\tau] \tag{1}$$

Nguồn gốc cận trên $L-1-\tau$ và số lượng cặp mẫu: Trong Công thức (1), bản gốc lấy mẫu tại chỉ số n, còn bản trượt lấy mẫu tại chỉ số $n+\tau$. Do khung dữ liệu trong bộ nhớ chỉ lưu trữ các phần tử từ chỉ số 0 đến $L-1$, mẫu của bản trượt muốn hợp lệ (không truy xuất tràn mảng) thì chỉ số phải thỏa mãn điều kiện biên:

$$n + \tau \leq L - 1 \iff n \leq L - 1 - \tau \tag{2}$$

Do đó, chỉ số n chỉ có thể chạy từ 0 đến $L-1-\tau$. Từ điều kiện này rút ra các ý nghĩa bản chất:

1. Số lượng cặp mẫu được nhân với nhau là: $(L-1-\tau) - 0 + 1 = \mathbf{L} - \tau$ cặp.
2. Con số "-1" xuất phát từ việc mảng đánh số từ 0 (phần tử cuối cùng là $L-1$, không phải L).
3. Đại lượng "$-\tau$" xuất hiện do khi trượt đi τ mẫu, đúng τ mẫu ở đuôi của bản trượt bị thò ra ngoài phạm vi lưu trữ của khung L mẫu (Hình 2A).
4. Cận trên $L-1-\tau$ hoàn toàn là điều kiện biên kỹ thuật của khung dữ liệu hữu hạn, **đây KHÔNG phải là phép trừ phần đối xứng**.

Cơ chế theo dấu của tổng tự tương quan trong 3 trường hợp: Dấu và độ lớn của tổng tự tương quan $\mathcal{R}_x[\tau]$ được quyết định bởi tính đồng pha giữa bản gốc và bản trượt:

1. **Khi $\tau = 0$ (Trùng khớp):** Bản sao nằm trùng khít lên bản gốc ($x[n]$ nhân với chính nó). Từng tích cặp mẫu trở thành bình phương: $x[n] \cdot x[n] = x^2[n] \geq 0$. Mọi số hạng đều không âm, cộng dồn tạo thành:

$$\mathcal{R}_x[0] = \sum_{n=0}^{L-1} x^2[n] = E_{\text{frame}}$$

chính là tổng năng lượng tức thời của toàn khung L mẫu. Đây là giá trị **cực đại toàn cục** của hàm tự tương quan.

2. **Khi** $0 < \tau < T_0$ **(Lệch pha):** Bản sao bị lệch pha so với bản gốc. Các đỉnh sóng dương $(+)$ của bản này rơi vào sườn sóng hoặc đáy sóng âm $(-)$ của bản kia. Phép nhân giữa mẫu dương và mẫu âm sinh ra tích âm: $(+) \cdot (-) < 0$. Khi cộng dồn, các tích âm và dương triệt tiêu lẫn nhau, khiến tổng tự tương quan sụt giảm rõ rệt.
3. **Khi** $\tau = T_0$ **(Đúng một chu kỳ cơ bản):** Sóng âm tuần hoàn lặp lại đúng một chu kỳ. Đỉnh dương của bản sao trượt đúng vào đỉnh dương của bản gốc: $(+) \cdot (+) > 0$. Đáy âm của bản sao cũng trượt đúng vào đáy âm của bản gốc: $(-) \cdot (-) > 0$. Hầu hết các cặp mẫu đều cho tích dương, khiến tổng tự tương quan vọt lên tạo thành một **đỉnh cực đại cục bộ thứ nhất** tại $\tau^* = T_0$ (Hình 2B).

Tự tương quan đo tính tuần hoàn (lặp lại theo chu kỳ), không đo tính đối xứng; sóng tiếng nói không đối xứng nhưng tuần hoàn.

Vì sao phải bỏ qua $\tau = 0$ **và thiết lập dải tìm kiếm** $[\tau_{\text{min}}, \tau_{\text{max}}]$**:** Mặc dù $\mathcal{R}_x[0]$ luôn đạt giá trị lớn nhất, ta bắt buộc phải loại bỏ $\tau = 0$ và vùng lân cận khi tìm kiếm cao độ. Do tính liên tục của sóng âm thanh (quán tính của không khí và dây thanh), các mẫu liền kề nhau có độ tương quan rất cao, tạo thành một sườn dốc đi xuống rất dốc từ $\mathcal{R}_x[0]$. Nếu quét argmax từ $\tau = 0$, thuật toán sẽ luôn chọn nhầm $\tau = 0$ hoặc $\tau = 1$.

Dải tần số cơ bản của giọng nói con người (từ giọng nam trầm nhất đến giọng nữ cao và trẻ em) nằm trong khoảng $F_0 \in [75\text{ Hz}, 600\text{ Hz}]$. Tại tần số lấy mẫu chuẩn $f_s = 16.000$ Hz, ta đổi dải tần số này sang dải độ trễ mẫu τ:

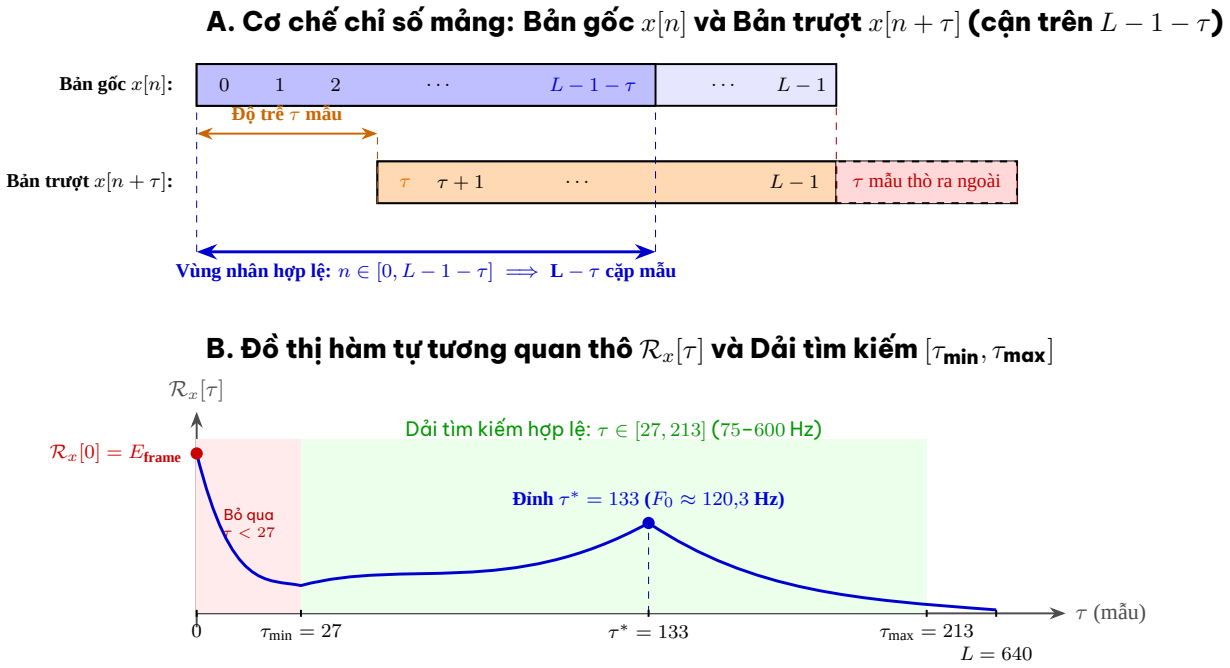
$$\tau_{\text{min}} = \left\lceil \frac{f_s}{F_{\text{ceiling}}} \right\rceil = \left\lceil \frac{16.000}{600} \right\rceil = \lceil 26{,}67 \rceil = \textbf{27 mẫu} \tag{3}$$

$$\tau_{\text{max}} = \left\lfloor \frac{f_s}{F_{\text{floor}}} \right\rfloor = \left\lfloor \frac{16.000}{75} \right\rfloor = \lfloor 213{,}33 \rfloor = \textbf{213 mẫu} \tag{4}$$

Trong đó, τ_{min} phải làm tròn lên ($\lceil \cdot \rceil$) để bảo đảm tần số đo được $f_s/\tau \leq F_{\text{ceiling}} = 600$ Hz không vượt quá trần quy định; ngược lại τ_{max} phải làm tròn xuống ($\lfloor \cdot \rfloor$) để tần số $f_s/\tau \geq F_{\text{floor}} = 75$ Hz không tụt xuống dưới sàn đáy. Vì vậy, không gian tìm kiếm cực đại chỉ quét trong dải hợp lệ $\tau \in [27, 213]$ gồm đúng $213 - 27 + 1 = \textbf{187}$ giá trị lag.

Ví dụ số nguyên chuỗi tính toán: Giả sử hệ thống xử lý tín hiệu tại $f_s = 16.000$ Hz:

- Nếu thuật toán tìm được đỉnh cực đại tại $\tau^* = 160$ mẫu $\implies F_0 = \frac{16.000}{160} = \textbf{100}$ Hz (giọng nam trầm).
- Nếu thuật toán tìm được đỉnh cực đại tại $\tau^* = 133$ mẫu $\implies F_0 = \frac{16.000}{133} \approx \textbf{120,3}$ Hz (giọng nam trung).



Hình 2: Cơ chế chỉ số mảng giới hạn khung $(L - 1 - \tau)$ và đồ thị hàm tự tương quan thô $\mathcal{R}_x[\tau]$ trong dải quét $[27, 213]$ mẫu (Minh họa).

Đọc Hình 2 (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Panel A — Bản gốc $x[n]$ (Thanh màu xanh nhạt phía trên):** Thể hiện mảng dữ liệu mẫu âm thanh của khung $L = 640$ mẫu được lưu tuần tự trong bộ nhớ, có chỉ số n chạy từ 0 đến $L-1$ (0 đến 639). Vùng tô xanh đậm hơn là đoạn chỉ số $n \in [0, L-1-\tau]$ có thể bắt cặp.
2. **Panel A — Bản trượt $x[n+\tau]$ (Thanh màu cam phía dưới):** Thể hiện bản sao tín hiệu bị dịch trượt sang phải một khoảng trễ τ mẫu. Mẫu đầu tiên của bản trượt bắt đầu tại chỉ số τ, và mẫu có chỉ số $L-1$ của bản gốc lúc này đối diện với chỉ số $L-1-\tau$ của bản trượt.
3. **Panel A — Mũi tên hai chiều màu cam (Độ trễ τ mẫu):** Đo khoảng cách dịch chuyển giữa điểm gốc 0 của bản gốc và điểm bắt đầu của bản trượt.
4. **Panel A — Khối nét đứt màu đỏ nhạt (τ mẫu thò ra ngoài):** Khi dịch trượt sang phải một khoảng τ, đúng τ mẫu ở phần đuôi của bản trượt bị tràn ra ngoài phạm vi lưu trữ của khung L mẫu. Các mẫu này không có mẫu tương ứng của bản gốc để nhân.
5. **Panel A — Mũi tên hai chiều màu xanh đậm (Vùng nhân hợp lệ):** Giới hạn trong khoảng chỉ số $n \in [0, L-1-\tau]$, tương ứng với đúng $L-\tau$ cặp mẫu có đủ cả hai thành phần $x[n]$ và $x[n+\tau]$ nằm trọn vẹn trong bộ nhớ khung. Điều này minh chứng trực quan cho điều kiện biên $n+\tau \le L-1 \iff n \le L-1-\tau$ trong Công thức (2) và giải thích vì sao cận trên của tổng tự tương quan Công thức (1) dừng lại ở $L-1-\tau$.
6. **Panel B — Chấm đỏ tại $\tau = 0$ ($\mathcal{R}_x[0] = E_{\text{frame}}$):** Biểu diễn cực đại toàn cục của hàm tự tương quan, tương ứng với năng lượng tức thời của toàn bộ khung L mẫu (khi hai bản trùng khít hoàn toàn, mọi tích cặp đều là bình phương không âm $x^2[n] \ge 0$).
7. **Panel B — Vùng tô màu hồng nhạt $\tau < 27$ (Sườn dốc quán tính âm học):** Đoạn đồ thị sụt dốc rất dốc ngay sau điểm $\tau = 0$. Sự sụt dốc này phản ánh tính tương quan rất cao giữa các mẫu âm thanh liền kề do quán tính chuyển động của dây thanh quản và cột không khí. Vùng này bị thuật toán loại bỏ triệt để để không bao giờ chọn nhầm đỉnh tại $\tau = 0$ hay $\tau = 1$.
8. **Panel B — Đáy suy giảm gần τ_{min} (Minh họa):** Vùng trũng của đồ thị ngay trước mốc $\tau_{\text{min}} = 27$ mẫu. Đây là hiện tượng triệt tiêu ngược pha tại khoảng trễ $T_0/2$ phụ thuộc vào dạng sóng âm thanh cụ thể, nơi tương quan quán tính âm học đã tắt hẳn và tín hiệu bắt đầu chuyển sang vùng tìm kiếm tính tuần hoàn của dây thanh quản.
9. **Panel B — Đỉnh màu xanh đậm tại $\tau^* = 133$ mẫu (Cực đại cục bộ thứ nhất):** Vị trí bản trượt dịch đúng bằng chu kỳ rung cơ bản T_0 của dây thanh quản. Tại đây các đỉnh sóng dương và đáy sóng âm trùng pha nhau, làm tổng tích dồn vọt lên đỉnh cực đại cục bộ trong dải quét. Tần số cơ bản được tính chính xác bằng $F_0 = f_s/\tau^* = 16.000/133 \approx 120{,}3$ Hz (giọng nam trung).
10. **Panel B — Vùng tô màu xanh lá $\tau \in [27, 213]$ (Dải tìm kiếm hợp lệ):** Không gian quét gồm đúng 187 giá trị lag, được giới hạn bởi $\tau_{\text{min}} = 27$ mẫu (ứng với $F_{\text{ceiling}} = 600$ Hz, Công thức (3)) và $\tau_{\text{max}} = 213$ mẫu (ứng với $F_{\text{floor}} = 75$ Hz, Công thức (4)).

3. Cắt Khung, Chuẩn Hóa Cửa Sổ và Giải Phẫu 10 Tham Số Praat

3.A. Độ dài khung $L = 640$ mẫu và Cửa sổ Hanning

Để trích xuất chính xác cao độ F_0, bước đầu tiên là cắt tín hiệu âm thanh thành các khung ngắn và nhân cửa sổ phân tích:

1. **Cơ sở độ dài $L = 640$ mẫu (40 ms tại 16 kHz):** Khung phân tích bắt buộc phải chứa tối thiểu 3 chu kỳ của tần số thấp nhất cần đo (pitch_floor $= 75$ Hz). Tại 75 Hz, chu kỳ dài nhất là $T_{\text{max}} = 16.000/75 \approx 213{,}33$ mẫu (13,33 ms). Thời lượng khung tối thiểu:

$$T_{\text{frame}} \ge 3 \times T_{\text{max}} = 3 \times 13{,}33 \text{ ms} = 40 \text{ ms} \implies L = 16.000 \times 0{,}040 = \mathbf{640} \text{ mẫu}$$

   Khung 400 mẫu (25 ms) kiểu STFT Mel thông dụng chỉ chứa được 1,8 chu kỳ của giọng nam 75 Hz, hoàn toàn không đủ không gian để bản trượt $x[n+\tau]$ bao trọn chu kỳ lặp lại thứ hai. Vì vậy độ dài $L = 640$ mẫu là bắt buộc về mặt toán học.

2. **Cửa sổ Hanning và dạng đối xứng của Praat:** Praat nhân tín hiệu đã khử DC với hàm cửa sổ Hanning đối xứng (`Sound_to_Pitch.cpp`, dòng 413):

$$w[i] = 0{,}5 - 0{,}5\cos\left(\frac{2\pi i}{N+1}\right), \quad i = 1, 2, \ldots, N \tag{5}$$

Lưu ý cốt lõi: Cửa sổ mặc định của Praat là Hanning (`AC_HANNING`); cửa sổ Gaussian chỉ được kích hoạt khi người dùng bật cờ `very_accurate = on` (`Sound_to_Pitch.cpp`, dòng 405–415). Trên FPGA, một bảng ROM 640 mục 16-bit lưu trọn vẹn các hệ số của hàm Hanning đã được tính sẵn.

Nguồn gốc chính xác của độ dài cửa sổ $N = 638$ trong mã nguồn Praat: Nhiều tài liệu giải thích nhầm rằng Praat lấy 640 mẫu trừ đi 2 mẫu ở hai đầu. Nhìn trực tiếp vào mã nguồn C++ (`Sound_to_Pitch.cpp`, dòng 325–329), thuật toán thực tế thực hiện phép tính làm tròn đối xứng:

```
    double duration = 3.0 / pitchFloor; // 3.0 / 75.0 = 0.04 giay
    long nsamp = floor (duration / dx); // floor(0.04 / (1/16000)) = 640
    long halfnsamp = floor (0.5 * (nsamp - 1)); // floor(0.5 * 639) = 319
    nsamp = 2 * halfnsamp; // 2 * 319 = 638
```

Praat ép N phải là một số chẵn có tâm đối xứng hoàn hảo thông qua bán độ rộng `halfnsamp`, dẫn đến $2 \times 319 = \mathbf{638}$ mẫu. Ngược lại, thiết kế FPGA chọn con số chẵn tròn trịa $L = \mathbf{640}$ mẫu (40 ms) để tối ưu hóa quản lý con trỏ bộ đệm vòng (Ring Buffer) và khớp hoàn hảo với khối nhớ BRAM.

▶ HỎI – ĐÁP TƯ DUY: Tại sao Praat dùng mẫu số $N+1$ thay vì $L-1$?

Trong các giáo trình DSP kinh điển (Oppenheim & Schafer, thư viện `scipy.signal`), cửa sổ Hanning thường được định nghĩa với chỉ số chạy từ 0 đến $L-1$:

$$w_{\text{textbook}}[n] = 0{,}5 - 0{,}5\cos\left(\frac{2\pi n}{L-1}\right), \quad n = 0, 1, \ldots, L-1$$

Định nghĩa này khiến hai mẫu biên $w[0] = 0$ và $w[L-1] = 0$ triệt tiêu hoàn toàn về 0, đồng nghĩa với việc lãng phí 2 ô nhớ và vứt bỏ thông tin của 2 mẫu âm thanh ở rìa khung.
Praat (`Sound_to_Pitch.cpp`, dòng 413) định nghĩa chỉ số chạy từ $i = 1$ đến N với mẫu số $N+1$:

$$w_{\text{Praat}}[i] = 0{,}5 - 0{,}5\cos\left(\frac{2\pi i}{N+1}\right), \quad i = 1, 2, \ldots, N$$

Tại hai đầu mút $i = 1$ và $i = N$: $w[1] = w[N] = 0{,}5 - 0{,}5\cos(2\pi/641) \approx 0{,}000024 > 0$. Cả N mẫu trong bộ nhớ đều nhận trọng số dương thực sự, không có bất kỳ mẫu nào bị nhân với 0. Trên FPGA, ta áp dụng chính xác định nghĩa của Praat với $N = 640$, bảo đảm mọi mẫu trong BRAM đều tham gia đóng góp năng lượng.

3.B. Cửa sổ Hanning và Hệ số chuẩn hóa $\mathcal{R}_w[\tau]$ (Boersma, 1993)

Việc nhân cửa sổ làm xuất hiện hiện tượng sụt dốc tự nhiên của đồ thị tự tương quan theo độ trễ τ do **hai nguyên nhân vật lý độc lập**:

1. **Nguyên nhân hình học (Số cặp mẫu giảm):** Như đã chứng minh ở Mục 2, số cặp mẫu tham gia vào tổng tự tương quan giảm tuyến tính từ L cặp (tại $\tau = 0$) xuống còn $L - \tau$ cặp (tại độ trễ τ). Ít số hạng cộng dồn hơn khiến tổng số học đương nhiên bị suy giảm (Hình 3A).
2. **Nguyên nhân trọng số cửa sổ (Vuốt biên):** Cửa sổ Hanning vuốt hai đầu về 0. Khi trượt bản sao đi một khoảng τ, tích của hai trọng số cửa sổ $w[n] \cdot w[n+\tau]$ suy giảm phi tuyến theo τ.

Hàm tự tương quan của chính cửa sổ phân tích:

$$\mathcal{R}_w[\tau] = \sum_{n=0}^{L-1-\tau} w[n] \cdot w[n+\tau] \tag{6}$$

gói trọn cả hai nguyên nhân nói trên.

Trực giác: hai quả núi Hanning trượt lên nhau. Khung L mẫu sau khi nhân cửa sổ Hanning có dạng một quả núi: hai chân chạm 0, đỉnh ở giữa. Bản sao là quả núi thứ hai trượt sang phải τ mẫu. Khi $\tau = 0$, hai quả núi chồng khít. Khi τ tăng dần, phần đuôi τ mẫu thò ra ngoài khung (không có mẫu đối diện để nhân), đồng thời hai sườn núi dốc gặp nhau ở độ cao thấp hơn. Diện tích giao nhau giữa hai quả núi rỗng chính là $\mathcal{R}_w[\tau]$ (Hình 3A).

Khi phân đôi tín hiệu thành phần dao động thanh quản thật $x[n]$ và khuôn cửa sổ $w[n]$, với tín hiệu tuần hoàn lý tưởng:

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} \big(x[n]\, x[n+\tau]\big)\big(w[n]\, w[n+\tau]\big) \approx \bar{r} \cdot \mathcal{R}_x[0] \cdot \frac{\mathcal{R}_w[\tau]}{\mathcal{R}_w[0]} \tag{7}$$

trong đó $\bar{r} \in [0, 1]$ là hệ số tự tương quan chuẩn hóa nội tại của dạng sóng tuần hoàn không bị suy hao cửa sổ (đối với nguyên âm rõ, $\bar{r} \approx 0{,}48$–$0{,}90$). Do đó, mức tương quan thật được trích xuất bằng cách nhân với tỷ số bù suy hao:

$$\bar{r} \approx \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0]} \cdot \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]} \tag{8}$$

Hệ số bù cửa sổ $C[\tau]$ trong bảng tra ROM: Thay vì thực hiện phép chia trên phần cứng, ta định nghĩa hệ số phóng đại bù suy hao cửa sổ:

$$C[\tau] = \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]} \quad (\geq 1,\ \text{tăng đơn điệu theo } \tau) \tag{9}$$

Hệ số $C[\tau]$ hoàn toàn chỉ phụ thuộc vào dạng cửa sổ và τ, không phụ thuộc vào tín hiệu âm thanh đầu vào. Ta tính sẵn $C[\tau]$ và nạp vào bảng tra ROM 187 phần tử (ứng với $\tau \in [27, 213]$). Điểm số sau khi bù suy hao cửa sổ là:

$$S[\tau] = \mathcal{R}_x[\tau] \cdot C[\tau] \tag{10}$$

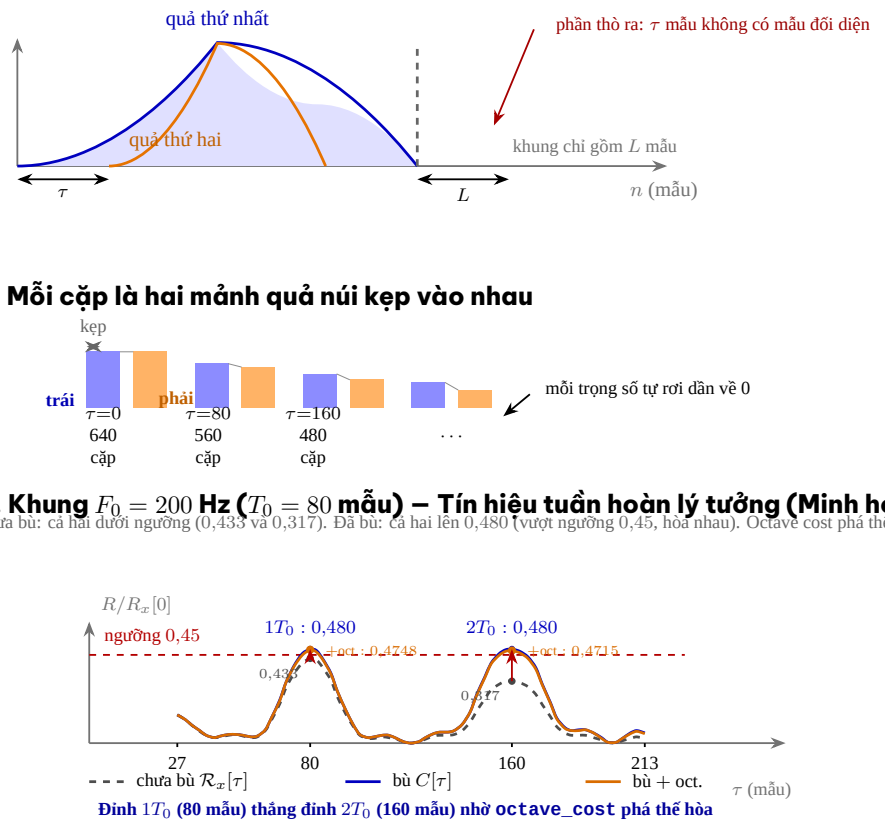
và điều kiện nhận diện khung hữu thanh trở thành:

$$\frac{S[\tau^*]}{\mathcal{R}_x[0]} \geq \theta_{\text{voice}} = 0{,}45 \iff S[\tau^*] \geq 0{,}45 \cdot \mathcal{R}_x[0] \tag{11}$$

▶ HỎI – ĐÁP TƯ DUY: Bảng dịch nhanh: 4 ký hiệu cốt lõi và ví dụ số chuẩn xác tại $\tau = 133$

Ký hiệu	Ý nghĩa hình ảnh quả núi	Giá trị chuẩn xác tại $\tau = 133$
$\mathcal{R}_w[\tau]$	Diện tích giao nhau của hai vỏ núi Hanning.	$\mathcal{R}_w[133]/\mathcal{R}_w[0] = \mathbf{0{,}752}$ (vỏ còn 75,2%)
$\mathcal{R}_x[\tau]/\mathcal{R}_x[0]$	Tự tương quan thô bị khuôn cửa sổ đè lên.	$\bar{r} \times 0{,}752 = 0{,}48 \times 0{,}752 = \mathbf{0{,}361}$
$C[\tau]$	Hệ số phóng đại bù suy hao trong ROM.	$C[133] = 1{,}0/0{,}752 = \mathbf{1{,}330} \approx 1{,}33$
$S[\tau]/\mathcal{R}_x[0]$	Mức tương quan phục hồi sau khi nhân $C[\tau]$.	$0{,}361 \times 1{,}33 = \mathbf{0{,}480} \geq 0{,}45$ (Đạt chuẩn Hữu thanh!)

Đối chiếu các mốc lag quan trọng: Tại $\tau = 27$: $\mathcal{R}_w[27]/\mathcal{R}_w[0] = 0{,}988 \implies C[27] = \mathbf{1{,}012}$. Tại $\tau = 80$: $\mathcal{R}_w[80]/\mathcal{R}_w[0] = 0{,}902 \implies C[80] = \mathbf{1{,}108}$. Tại $\tau = 133$: $\mathcal{R}_w[133]/\mathcal{R}_w[0] = 0{,}752 \implies C[133] = \mathbf{1{,}330} \approx 1{,}33$. Tại $\tau = 160$: $\mathcal{R}_w[160]/\mathcal{R}_w[0] = 0{,}660 \implies C[160] = \mathbf{1{,}515}$. Tại $\tau = 213$: $\mathcal{R}_w[213]/\mathcal{R}_w[0] = 0{,}473 \implies C[213] = \mathbf{2{,}112}$.



Hình 3: Hiện tượng co hẹp vùng tích các cặp mẫu và vai trò của hệ số bù suy hao cửa sổ $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ theo Boersma (1993). Panel A: cửa sổ Hanning là một quả núi, bản sao trượt sang phải τ mẫu nên đuôi bản sao thò ra ngoài khung $L = 640$ mẫu. Panel B: mỗi cặp là hai mảnh núi kẹp vào nhau, số cặp giảm từ 640 xuống $640 - \tau$. Panel C: khung $F_0 = 200$ Hz ($T_0 = 80$ mẫu), trục tung $R/\mathcal{R}_x[0]$ (Tín hiệu tuần hoàn lý tưởng (Minh họa)); đỉnh $1T_0$ ($\tau = 80$) và $2T_0$ ($\tau = 160$) lần lượt đi từ 0,433 và 0,317 (chưa bù, đường xám nét đứt, cả hai dưới ngưỡng) lên 0,480 và 0,480 (đã bù $C[\tau]$, đường xanh, cả hai vượt ngưỡng 0,45 và hòa nhau), còn đường cam thêm `octave_cost` $= 0{,}01$ dạng nhân ($1T_0 = 0{,}4748 > 2T_0 = 0{,}4715$) phá vỡ thế hòa giúp $1T_0$ chiến thắng an toàn.

Đọc Hình 3 (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Panel A — Quả núi thứ nhất (đường xanh đậm, vùng tô xanh nhạt):** Khung $L = 640$ mẫu sau khi nhân cửa sổ Hanning, có dạng một quả núi: hai chân chạm 0, đỉnh ở giữa. Cửa sổ chữ nhật sẽ để hai đầu khung gãy đột ngột; quả núi thì không.
2. **Panel A — Quả núi thứ hai (đường cam):** Bản sao của quả núi thứ nhất, trượt sang phải đúng τ mẫu. Mũi tên hai chiều ở đáy đo đúng khoảng trượt τ này.
3. **Panel A — "Phần thò ra":** τ mẫu ở đuôi quả thứ hai nằm ngoài cửa sổ, không có mẫu nào của quả thứ nhất đối diện để nhân. Mũi tên đỏ chỉ vào chỗ đó. Cùng lúc, phần chồng nhau hẹp lại còn $L - \tau$ cặp và hai chân núi chỉ còn gặp nhau ở độ cao nhỏ, nên phần chồng vừa hẹp vừa thấp. Diện tích chồng nhau chính là $\mathcal{R}_w[\tau]$.
4. **Panel B — Bốn cột mẫu:** Mỗi cột là một độ trễ $\tau = 0, 80, 160$ và một cột nữa xa hơn. Cột xanh là mảnh núi "bên trái", cột cam là mảnh "bên phải"; mũi tên "kẹp" chỉ cặp mảnh đang dán vào nhau. Từ trên xuống dưới, mảnh cam thấp dần, cho thấy mỗi trọng số tự rơi về 0.
5. **Panel B — Số cặp giảm:** Nhãn dưới cột ghi $\tau = 0$: 640 cặp; $\tau = 80$: 560 cặp; $\tau = 160$: 480 cặp. Con số $L - \tau$ này là nguyên nhân hình học đầu tiên khiến tổng tương quan thô sụt theo τ.
6. **Panel C — Trục tung $R/\mathcal{R}_x[0]$ và ngưỡng 0,45:** Khung $F_0 = 200$ Hz ($T_0 = 80$ mẫu). Đường đỏ nét đứt ngang là ngưỡng thanh tính; trục hoành đánh dấu bốn mốc 27, 80, 160, 213.
7. **Panel C — Đường xám nét đứt (chưa bù):** Hai đỉnh $1T_0$ ($\tau = 80$) và $2T_0$ ($\tau = 160$) chỉ đạt 0,433 và 0,317; cả hai đều **dưới** ngưỡng 0,45, nên nếu không bù thì khung sẽ bị gán nhầm thành vô thanh.

8. **Panel C — Đường xanh đậm (đã bù $C[\tau]$):** Nhân với $C[80] = 1{,}108$ và $C[160] = 1{,}515$, cả hai đỉnh đều lên đúng $0{,}480$; cả hai đều **vượt** ngưỡng $0{,}45$ và rơi vào trạng thái **hòa nhau**. Mũi tên đỏ giữa chấm xám và chấm xanh cho thấy đúng mức nâng bù tại mỗi đỉnh.

9. **Panel C — Đường cam (thêm `octave_cost` dạng nhân $2^{-0{,}01\cdot\log_2(600/F_0)}$):** Đỉnh $1T_0$ (200 Hz) bị phạt nhẹ về $0{,}4748$, trong khi đỉnh $2T_0$ (100 Hz) bị phạt nặng hơn về $0{,}4715$. Mức phạt thiên vị tần số cao này phá vỡ thế hòa ($0{,}4748 > 0{,}4715$), bảo đảm đỉnh cơ bản $1T_0$ chiến thắng tuyệt đối.

10. **Panel C — Dòng chữ xanh đậm ở đáy:** Tóm một câu: đỉnh $1T_0$ (80 mẫu) thắng đỉnh $2T_0$ (160 mẫu) nhờ `octave_cost` phá vỡ thế hòa.

3.B'. Hai đỉnh $1T_0$ và $2T_0$: vì sao bù hệ số lại đẩy đỉnh xa lên, và `octave_cost` dọn nốt tàn dư

Một giọng có chu kỳ T_0 sẽ lặp lại không chỉ sau một chu kỳ mà còn sau $2T_0$, $3T_0 \ldots$ nên hàm tự tương quan có nhiều đỉnh. Chọn nhầm đỉnh $2T_0$ là đọc cao độ thành $F_0/2$: giọng 200 Hz bị ghi thành 100 Hz, tức rơi đúng một quãng tám (octave error).

Vì sao sau khi bù, hệ số lại đẩy đỉnh xa lên mạnh hơn? Do $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ tăng đơn điệu theo τ, đỉnh tại $2T_0$ có hệ số phóng đại $C[2T_0]$ lớn hơn nhiều so với $C[T_0]$ (ví dụ $C[160] = 1{,}515 > C[80] = 1{,}108$). Đối với tín hiệu tuần hoàn lý tưởng có $\bar{r} = 0{,}48$, trước khi bù hai đỉnh chỉ đạt $0{,}433$ và $0{,}317$ (đều dưới ngưỡng $0{,}45$). Sau khi nhân hệ số bù, cả hai đỉnh đều được đẩy lên đúng $0{,}480$, vượt ngưỡng $0{,}45$ và rơi vào thế hòa nhau!

Cơ chế `octave_cost` phá thế hòa: Praat chặn nguy cơ này bằng cơ chế phạt log-lag `octave_cost` $= 0{,}01$ (được phân tích chi tiết ở Tham số 8, Mục 3.C'). Trên FPGA, cơ chế này được tích hợp thành hệ số nhân $2^{-0{,}01\cdot\log_2(600/F_0)}$. Tại $1T_0$ (200 Hz), điểm số còn $0{,}480 \times 0{,}9891 =$ **0,4748**; tại $2T_0$ (100 Hz), điểm số còn $0{,}480 \times 0{,}9822 =$ **0,4715**. Khoản chênh lệch $0{,}0033$ này đủ để phá vỡ thế hòa, bảo đảm đỉnh cơ bản $1T_0$ luôn chiến thắng an toàn. (Đối chiếu dạng trừ của Praat: $1T_0$ đạt $0{,}480 - 0{,}01 \times \log_2(600/200) =$ **0,4642** thắng $2T_0$ đạt $0{,}480 - 0{,}01 \times \log_2(600/100) =$ **0,4542**).

3.C. Giải phẫu 10 tham số chuẩn của Praat dưới góc nhìn mã nguồn (`Sound_to_Pitch.cpp`)

Mã nguồn C++ của Praat chứa đựng các tham số mặc định chuẩn mực được phân tích chi tiết:

1. **`time_step = 0,01 s` (10 ms = 160 mẫu):** Bước nhảy trượt khung (Δt_{hop}). Mã nguồn tự động gán $\Delta t_{\text{hop}} = \frac{0{,}75}{\text{pitch_floor}} = \frac{0{,}75}{75} = 0{,}01$ s (`Sound_to_Pitch.cpp`, dòng 325), tương ứng độ chồng lấn 75% giữa các khung 40 ms.

2. **`pitch_floor = 75 Hz`:** Giới hạn đáy cao độ. Quyết định độ trễ quét tối đa $\tau_{\max} = \lfloor f_s/75 \rfloor = 213$ mẫu và độ dài cửa sổ tối thiểu $L = 640$ mẫu.

3. **`pitch_ceiling = 600 Hz`:** Giới hạn trần cao độ. Quyết định độ trễ quét tối thiểu $\tau_{\min} = \lceil f_s/600 \rceil = 27$ mẫu.

4. **`very_accurate = off` (False):** Khi `off` (mặc định), Praat dùng Hanning dài 3 chu kỳ (640 mẫu). Khi `on`, Praat chuyển sang Gaussian dài 6 chu kỳ (1.280 mẫu). Thiết kế FPGA chọn tương đương `off` để tối ưu hóa silicon.

5. **`max_number_of_candidates = 15`:** Praat trích xuất 14 đỉnh cực đại cục bộ $+1$ ứng viên Vô thanh cho thuật toán Viterbi. Trên FPGA, ta rút gọn về bộ so sánh cực đại argmax tức thời lấy 1 ứng viên mạnh nhất.

6. **`silence_threshold = 0,03`:** Ngưỡng năng lượng khoảng lặng so với cực đại toàn tệp.

7. **`voicing_threshold = 0,45`:** Ngưỡng tương quan chuẩn hóa nhận diện hữu thanh.

8. **`octave_cost = 0,01`:** Hệ số phạt thiên vị tần số cao theo $\log_2(\text{ceiling}/F_0)$.

9. **`octave_jump_cost = 0,35`:** Trọng số phạt bước nhảy tần số giữa hai khung liên tiếp trong Viterbi, được nhân hệ số $0{,}01/\Delta t_{\text{hop}}$ (`Pitch.cpp`, dòng 543–545).

10. **`voiced_unvoiced_cost = 0,14`:** Trọng số phạt chuyển đổi Voiced $\leftrightarrow$ Unvoiced giữa hai khung trong Viterbi, cũng nhân $0{,}01/\Delta t_{\text{hop}}$.

3.C'. Cơ sở âm học và bản chất của Tham số 6–10

Tham số 6 — `silence_threshold = 0,03`: Trong mã nguồn Praat (`Pitch.cpp`, dòng 553–555), tham số này biểu diễn tỷ số biên độ đỉnh cục bộ của khung so với cực đại toàn tệp:

$$\text{intensity} = \frac{\text{localPeak}}{\text{globalPeak}}$$

(được tích hợp trong công thức tính điểm vô thanh R_{unvoiced} của Praat ở Mục 3.D).

- *Lý do tồn tại:* Nếu không có ngưỡng này, trong những khoảng người nói ngừng thở hoặc im lặng giữa các câu, các tạp âm môi trường tuần hoàn biên độ nhỏ (tiếng quạt máy tính, tiếng ù điện lưới thoại 50/60 Hz) khi tự chuẩn hóa cho chính năng lượng cực nhỏ của nó ($S[\tau]/\mathcal{R}_x[0]$) vẫn có thể đạt tỷ số $> 0{,}45$, gây ra hiện tượng **báo cao độ giả (ghost pitch)** trong khoảng lặng.
- *Cơ sở vật lý của con số* $0{,}03$*:* Quy đổi sang thang Decibel biên độ:

  $$20\log_{10}(0{,}03) \approx -\mathbf{30{,}46}\ \text{dB}$$

  Trong âm học thực nghiệm, khoảng lặng đàm thoại chuẩn trong phòng thu tiêu chuẩn có mức năng lượng nền thấp hơn đỉnh giọng nói từ 30 dB đến 40 dB. Vì năng lượng tỉ lệ với bình phương biên độ, tỷ số năng lượng tương ứng là:

  $$(0{,}03)^2 = 0{,}0009 = \frac{9}{10.000} \implies 10\log_{10}(0{,}0009) \approx -30{,}46\ \text{dB}$$

- *Hiện thực hóa không phép chia trên FPGA:* Praat tích hợp intensity vào hàm mục tiêu mềm của mạng Viterbi toàn cục (xem Mục 3.D). Vì phần cứng streaming trên FPGA không dùng Viterbi, thiết kế thay thế bằng một **cổng ngắt năng lượng cứng (Silence Gate)**: nếu năng lượng khung $\mathcal{R}_x[0] < 0{,}0009 \cdot E_{\text{ref}}$ (với $E_{\text{ref}} = \max_k \mathcal{R}_x[0]_k$ là năng lượng khung cực đại của câu nói), khung bị ngắt sớm ngay lập tức mà không cần quét tính tương quan. Để tránh phép chia số thực, phép so sánh được nhân chéo với 10.000:

  $$10.000 \cdot \mathcal{R}_x[0] < 9 \cdot E_{\text{ref}} \implies \text{Gán Unvoiced } (F_0 = 0) \tag{12}$$

Tham số 7 — `voicing_threshold = 0,45`: Mức ngưỡng $0{,}45$ được Paul Boersma (1993) lựa chọn làm giá trị mặc định qua thực nghiệm thống kê trên cơ sở dữ liệu ngữ âm học lớn. Để trực quan hóa, người ta liên hệ hệ số tự tương quan chuẩn hóa r với Tỷ số Hài trên Nhiễu (Harmonics-to-Noise Ratio):

$$\text{HNR (dB)} = 10\log_{10}\left(\frac{r}{1-r}\right) \tag{13}$$

Với $r = 0{,}45$:

$$\text{HNR} = 10\log_{10}\left(\frac{0{,}45}{0{,}55}\right) = 10\log_{10}(0{,}8182) \approx -\mathbf{0{,}87}\ \text{dB} \approx -\mathbf{0{,}9}\ \text{dB}$$

tức chấp nhận một khung mà thành phần nhiễu có thể nhỉnh hơn thành phần tuần hoàn khoảng $0{,}9$ dB. Dung sai thực nghiệm này giúp giữ lại các đoạn nguyên âm khàn hoặc hạ giọng ở cuối câu nói mà không bắt nhầm phụ âm xát vô thanh /s/, /f/.

Tham số 8 — `octave_cost = 0,01`: Công thức chuẩn mực duy nhất trong Praat (`Pitch.cpp`, dòng 560) là:

$$\text{score} = \text{strength} - 0{,}01 \times \log_2\left(\frac{F_{\text{ceiling}}}{F_0}\right) \tag{14}$$

với mốc quy chiếu là trần tần số $F_{\text{ceiling}} = 600$ Hz. Tại đáy 75 Hz, khoảng cách tới trần là đúng 3 quãng tám ($\log_2(600/75) = 3$), tương ứng mức phạt tối đa là $0{,}03$.

▶ HỎI – ĐÁP TƯ DUY: Vì sao `octave_cost` chuyển từ phép trừ sang phép nhân?

1. **Praat trừ trên thang chuẩn hóa** $[0, 1]$**:** Trong Praat, độ mạnh ứng viên là hệ số tương quan chuẩn hóa $r \in [0, 1]$, nên việc trừ đại số $0{,}01 \times \log_2(600/F_0)$ có ý nghĩa vật lý rõ ràng.
2. **FPGA dùng điểm số chưa chuẩn hóa:** Trên phần cứng, điểm số $S[\tau] = \mathcal{R}_x[\tau] \cdot C[\tau]$ chưa được chia cho năng lượng $\mathcal{R}_x[0]$. Biên độ của $S[\tau]$ tỉ lệ thuận với độ lớn âm thanh và có thể đạt tới $\sim 3 \cdot 10^{10}$. Do đó, không thể trừ một hằng số cố định như $0{,}01$ vào một giá trị chưa chuẩn hóa.
3. **Dạng nhân tích hợp thẳng vào ROM:** Ta biến lượng trừ thành thừa số nhân:

$$2^{-\text{octave_cost}\cdot\log_2(600/F_0)} \approx 1 - 0{,}00693 \cdot \log_2(600/F_0)$$

(sử dụng khai triển Taylor bậc 1: $2^{-x} = e^{-x\ln 2} \approx 1 - x\ln 2$, với $0{,}01 \cdot \ln 2 \approx 0{,}00693$). Thừa số này làm giảm khoảng $0{,}69\%$ mỗi quãng tám và được nhân gộp trực tiếp vào bảng tra ROM $C[\tau]$.

4. **Đánh giá tương đương:** Khi $r \approx 1$, hai cách tương đương bậc 1 ($r \cdot 2^{-\epsilon} \approx r(1 - \epsilon\ln 2) \approx r - 0{,}0069\ldots$). Khi $r < 1$ (ví dụ $\bar{r} = 0{,}48$), mức phạt dạng nhân nhỏ hơn ($0{,}0033$ so với $0{,}0100$). Kết quả quét thực nghiệm 526 tần số khẳng định: dạng nhân vẫn dập tắt hoàn toàn lỗi quãng tám ($0/526$ lỗi).

Mô phỏng mô hình số nguyên trên lưới tần số sóng sin 75–600 Hz bước 1 Hz (526 điểm tần số) chứng minh vai trò quyết định của tham số này:

Hệ số `octave_cost`	Số tần số bị lỗi nhảy quãng tám	Ghi chú mô phỏng
0	$300/526$ (57,0%)	Không có phạt: đỉnh $2T_0$ hòa hoặc thắng $1T_0$
0,005	$20/526$ (3,8%)	Còn lỗi ở một số tần số cao
0,009	$0/526$ (0,0%)	Bắt đầu sạch hoàn toàn
0,01	**0/526 (0,0%)**	**Chuẩn mặc định Praat (sạch tuyệt đối)**
0,02	$0/526$ (0,0%)	Sạch tuyệt đối, dư biên an toàn

Tham số 9 và 10 — `octave_jump_cost = 0,35` và `voiced_unvoiced_cost = 0,14`: Trong mã nguồn Praat (`Pitch.cpp`, dòng 543–545), hai chi phí này được chuẩn hóa theo bước nhảy thời gian bằng cách nhân hệ số $0{,}01/\Delta t_{\text{hop}}$. Để hiểu trực giác: thanh quản người có quán tính cơ học, không thể co dãn cơ nhẫn giáp (cricothyroid muscle — cơ điều khiển độ căng và độ dài của dây thanh để thay đổi tần số rung) để đổi cao độ gấp đôi (1 quãng tám) chỉ trong 10 ms. Mức phạt $0{,}35$ cho mỗi quãng tám đảm bảo đường cong cao độ trơn tru sinh học.

3.D. Tổng hợp Đường ống 6 bước chuẩn mực trong mã nguồn C++ của Praat (`Sound_to_Pitch.cpp`)

Trước khi tối ưu hóa cho phần cứng FPGA, người kỹ sư bắt buộc phải nắm vững quy trình xử lý chuẩn mực của Praat — phần mềm tiêu chuẩn vàng trong ngữ âm học thực nghiệm (Boersma, 1993). Mọi hành vi dưới đây đều được trích xuất trực tiếp từ mã nguồn mở C++ chính thức của Praat (`praat/praat` master trên GitHub, các tệp `fon/Sound_to_Pitch.cpp` và `fon/Pitch.cpp`):

1. **Bước 1 — Trừ giá trị trung bình cục bộ (`localMean`):** Để loại bỏ thành phần một chiều (DC offset) và các dao động trôi chậm dưới hạ âm, Praat tính giá trị trung bình mẫu trên một đoạn cửa sổ dài hai chu kỳ dài nhất tính từ tâm khung:

$$\mu_{\text{local}} = \frac{1}{K}\sum_{k=0}^{K-1} x[k]$$

với K là số mẫu của đoạn cửa sổ dài hai chu kỳ của tần số thấp nhất ($K \approx 2 \times (f_s/\text{pitch_floor}) = 2 \times (16.000/75) \approx 427$ mẫu tại 16 kHz, xem `Sound_to_Pitch.cpp`, dòng 63–83).

Ví dụ số: Nếu tổng của 427 mẫu là 21.350, thì $\mu_{\text{local}} = 21.350/427 = 50$, và từng mẫu trong khung được trừ đi 50 ($x_{\text{zm}}[n] = x[n] - 50$) để triệt tiêu hoàn toàn thành phần DC. Trong mã nguồn triển khai FPGA (`lab_1_3_5.py`), ta chọn tính trung bình cục bộ trên toàn bộ $K = L = 640$ mẫu (mean_x = $(\sum x + 320)//640$) nhằm đơn giản hóa vi kiến trúc (tránh việc quản lý thêm con trỏ đệm con 427 mẫu so với Praat), trong khi bộ lọc thông cao HPF 60 Hz (hoặc IIR DC-blocker — Phần C.3) đã triệt tiêu hầu hết hiện tượng trôi DC từ đầu vào.

2. **Bước 2 — Nhân cửa sổ phân tích Hanning:** Praat nhân tín hiệu đã khử DC với hàm cửa sổ Hanning đối xứng (Công thức (5) ở Mục 3.A với $N = 638$ mẫu, `Sound_to_Pitch.cpp`, dòng 413). Cửa sổ Hanning triệt tiêu êm dịu về 0 ở hai đầu biên, giúp loại bỏ các bước nhảy gián đoạn (edge discontinuity).

3. **Bước 3 — Tính tự tương quan qua miền tần số bằng FFT:** Thay vì tính tổng tích trễ (tự tương quan, không phải tích chập) trực tiếp trong miền thời gian, Praat đệm thêm số 0 để độ dài đạt $nsampFFT \geq N \cdot (1 + \text{depth})$ (với depth $= 0{,}5$, tức hệ số $1{,}5$: $638 \times 1{,}5 = 957 \to nsampFFT = 1024$) rồi làm tròn lên lũy thừa của 2 gần nhất (`Sound_to_Pitch.cpp`, dòng 391–393). Thuật toán thực thi chuỗi biến đổi Fourier nhanh theo định lý Wiener–Khinchin:

$$x_{\text{win}}[n] \xrightarrow{\text{FFT}} X[k] \xrightarrow{\text{Power}} |X[k]|^2 \xrightarrow{\text{IFFT}} \mathcal{R}_x[\tau]$$

(theo Wiener–Khinchin: $|X[k]|^2 \xrightarrow{\text{IFFT}} \mathcal{R}_x[\tau]$, xem `Sound_to_Pitch.cpp`, dòng 146–155).

★ BẮC CẦU FEYNMAN: Giải mã Định lý Wiener–Khinchin từ Công thức Euler: Tại sao FFT $\to |X[k]|^2 \to$ IFFT lại bằng khít phép trượt và nhân trong miền thời gian?

Người học thường nhìn Định lý Wiener–Khinchin như một công thức trừu tượng từ trên trời rơi xuống. Dưới góc nhìn trực giác vật lý của Richard Feynman, ta có thể bắc cầu tự nhiên từ những gì đã biết:

(a) **Mỗi hài âm là một mũi tên quay (Phasor):** Khung âm thanh là tổng hợp của các dao động điều hòa sin và cos. Nhờ công thức Euler $e^{j\theta} = \cos\theta + j\sin\theta$, mỗi thành phần hài âm tần số ω_k được biểu diễn bởi một mũi tên quay trên mặt phẳng phức:

$$X[k] = A_k e^{j\phi_k} = A_k(\cos\phi_k + j\sin\phi_k)$$

với A_k là biên độ năng lượng và ϕ_k là góc pha ban đầu. Trong thực tế tiếng nói, các góc pha ϕ_k của từng hài âm rất lộn xộn và ngẫu nhiên do sóng âm dội qua thanh quản, vòm họng và môi ở các thời điểm bất kỳ.

(b) **Tự tương quan là gì?** Đó là phép so khớp tín hiệu với bản sao dịch trễ thời gian τ của chính nó để tìm chu kỳ tuần hoàn. Khi nhân mũi tên quay $X[k]$ với liên hợp phức của chính nó là $X^*[k] = A_k e^{-j\phi_k}$ trong miền tần số:

$$X[k] \cdot X^*[k] = A_k e^{j\phi_k} \cdot A_k e^{-j\phi_k} = A_k^2 e^{j(\phi_k - \phi_k)} = A_k^2 e^0 = |X[k]|^2$$

(c) **Khoảnh khắc giác ngộ:** Mọi góc pha hỗn loạn ban đầu ϕ_k đã tự trừ nhau triệt tiêu về đúng bằng 0! Khi góc pha bị triệt tiêu, tất cả mọi sóng hài thành phần ở mọi tần số đều được tự động căn chỉnh pha về đúng **sóng** $\cos(\omega_k\tau)$ **thuần khiết** (đều đồng loạt đạt cực đại tuyệt đối $+1$ tại mốc $\tau = 0$).

(d) **Bước IFFT là cộng dồn các sóng cos đã căn pha:**

$$\mathcal{R}_x[\tau] = \sum_k |X[k]|^2 \cos(\omega_k\tau)$$

Khi độ trễ thời gian τ bắt đầu trượt, các sóng $\cos(\omega_k\tau)$ dao động lệch pha và triệt tiêu nhau. Nhưng khi độ trễ τ chạm đúng chu kỳ thanh quản cơ bản T_0, sóng cơ bản F_0 quay đúng 1 vòng (2π), hài bậc hai $2F_0$ quay đúng 2 vòng (4π), hài bậc ba $3F_0$ quay đúng 3 vòng (6π)…Tất cả các sóng cos lại **đồng loạt chạm đỉnh** $+1$ **cùng một lúc** (hiện tượng giao thoa tăng cường — constructive interference), tạo nên đỉnh cực đại sừng sững $\tau^* = T_0$! Đó là lý do trực giác vật lý vì sao chuỗi biến đổi FFT $\to |X|^2 \to$ IFFT cho ra kết quả trùng khít hoàn toàn với phép trượt nhân-cộng từng mẫu trong miền thời gian.

4. **Bước 4 — Bù suy hao cửa sổ và trừ chi phí thiên vị quãng tám (Octave cost):** Praat chia hàm tự tương quan tín hiệu cho tự tương quan của chính cửa sổ: $r_a[\tau] = \mathcal{R}_x[\tau]/\mathcal{R}_w[\tau]$. Sau đó, tại bước chấm điểm ứng viên (`Sound_to_Pitch.cpp`, dòng 226; `Pitch.cpp`, dòng 560), Praat trừ một lượng phạt thiên vị tần số cao theo Công thức (14):

$$\text{score} = r_a[\tau] - \text{octave_cost} \times \log_2\left(\frac{\text{ceiling}}{F_0}\right)$$

với ceiling $= 600$ Hz, octave_cost $= 0{,}01$.

5. **Bước 5 — Trích xuất ứng viên cực đại và nội suy Sinc:** Praat tìm tối đa 14 đỉnh cực đại cục bộ trong dải $\tau \in [\tau_{\min}, \tau_{\max}]$. Tại mỗi đỉnh rời rạc, Praat áp dụng phép **nội suy Sinc chính xác cao** (sinc interpolation) trên miền liên tục để trích xuất vị trí đỉnh phụ mẫu (sub-sample peak position) và biên độ nội suy thực sự.

6. **Bước 6 — Quy hoạch động Viterbi theo thời gian (Tracking):** Mỗi khung hình thành 15 ứng viên (14 đỉnh cực đại cục bộ +1 ứng viên Vô thanh Unvoiced). Praat tìm đường đi tối ưu toàn cục xuyên suốt câu nói bằng thuật toán Viterbi. Chi phí bước nhảy tần số ($\text{Cost}_{\text{jump}}$) và chi phí chuyển đổi Hữu thanh/Vô thanh (Cost_{vuv}) đều được nhân hệ số chuẩn hóa theo bước trượt thời gian $0{,}01/\Delta t_{\text{hop}}$ (`Pitch.cpp`, dòng 543–545). Khi $\Delta t_{\text{hop}} = 0{,}01$ s (10 ms), hệ số là $0{,}01/0{,}01 = 1{,}0$; nếu $\Delta t_{\text{hop}} = 0{,}005$ s (5 ms), chi phí phạt giữa hai khung liền kề tự động tăng gấp đôi ($0{,}01/0{,}005 = 2{,}0$) để bảo toàn độ dốc biến thiên theo thời gian vật lý. Ứng viên Vô thanh nhận điểm số cạnh tranh động (`Sound_to_Pitch.cpp`, dòng 102; `Pitch.cpp`, dòng 553–555):

$$R_{\text{unvoiced}} = vt + \max\left(0,\ 2 - \frac{\text{intensity}}{st/(1+vt)}\right) \tag{15}$$

Ở đây, $vt = 0{,}45$ là ngưỡng hữu thanh (`voicing_threshold`), còn $st = 0{,}03$ là ngưỡng lặng (`silence_threshold`). Đại lượng intensity $=$ localPeak/globalPeak biểu diễn tỷ số biên độ đỉnh cục bộ của khung so với cực đại toàn tệp.

Ví dụ số với mẫu số $st/(1+vt) = 0{,}03/(1+0{,}45) = 0{,}03/1{,}45 \approx 0{,}0207$:

- Khi khung phát âm rất mạnh, intensity $= 0{,}0414 \implies 2 - 0{,}0414/0{,}0207 = 2 - 2 = 0 \implies R_{\text{unvoiced}} = 0{,}45 + 0 = \mathbf{0{,}45}$.
- Khi khung ở mức ngưỡng lặng, intensity $= 0{,}03 \implies 2 - 0{,}03/0{,}0207 = 2 - 1{,}45 = 0{,}55 \implies R_{\text{unvoiced}} = 0{,}45 + 0{,}55 = \mathbf{1{,}00}$.
- Khi khung rất yếu (gần như im lặng), intensity $= 0{,}02 \implies 2 - 0{,}02/0{,}0207 \approx 2 - 0{,}97 = 1{,}03 \implies R_{\text{unvoiced}} = 0{,}45 + 1{,}03 = \mathbf{1{,}48}$.

Tham số Praat	Ký hiệu	Giá trị chuẩn	Ý nghĩa thuật toán & Mã nguồn (Boersma 1993)	Hiện thực trên FPGA DE10-Standard
`time_step`	Δt_{hop}	0,01 s	Bước nhảy trượt khung (160 mẫu tại 16 kHz)	Khóa cứng 10 ms (160 mẫu), tương ứng độ chồng lấn 75%
`pitch_floor`	F_{floor}	75 Hz	Giới hạn đáy cao độ $\implies \tau_{\max} = 213$ mẫu, $L = 640$ mẫu	Khung 640 mẫu lưu trong bộ nhớ trên chip (chi tiết ở Mục 5)
`pitch_ceiling`	F_{ceiling}	600 Hz	Giới hạn trần cao độ $\implies \tau_{\min} = 27$ mẫu (187 lag)	Quét dải độ trễ tương ứng 187 lag (mở rộng biên cho nội suy — xem Mục 4)
`very_accurate`	—	`off`	Cửa sổ Hanning 3 chu kỳ (640 mẫu) vs Gaussian 6 chu kỳ	Dùng Hanning 640 mẫu; tiết kiệm 50% BRAM và MAC
`max_candidates`	N_{cand}	15	14 đỉnh cực đại cục bộ + 1 ứng viên Vô thanh cho Viterbi	Rút gọn về 1 ứng viên (bộ so sánh argmax cực đại)
`silence_threshold`	θ_{silence}	0,03	Ngưỡng biên độ khoảng lặng so với đỉnh file (3%, tức $-30{,}5$ dB)	Cổng ngưỡng năng lượng tĩnh không dùng phép chia (xem Mục 4)
`voicing_threshold`	θ_{voice}	0,45	Ngưỡng tỷ số tự tương quan chuẩn hóa nhận diện hữu thanh	So sánh tỷ số tương quan đã bù với ngưỡng 0,45 bằng nhân chéo nguyên
`octave_cost`	—	0,01	Phạt thiên vị log-lag nội khung để tránh nhảy xuống $F_0/2$	Nhân bảng bù ROM $C[\tau]$ tích hợp sẵn hệ số phạt quãng tám
`octave_jump_cost`	—	0,35	Phạt nhảy quãng tám giữa hai khung liên tiếp trong Viterbi	Bỏ qua Viterbi trên streaming RTL; lọc làm trơn hậu xử lý — xem chi tiết Mục 4
`voiced_unvoiced_cost`	—	0,14	Phạt chuyển đổi Voiced $\leftrightarrow$ Unvoiced giữa hai khung	Bỏ qua Viterbi; phân loại độc lập từng khung theo ngưỡng năng lượng và thanh tính

Bảng 1: Giải phẫu 10 tham số chuẩn mực đo cao độ của Praat và ánh xạ vi kiến trúc lên FPGA DE10-Standard.

4. Hành trình từ Laptop lên FPGA: Tối giản hóa First-Principles

▶ CỔNG KIỂM CHỨNG PHẦN MỀM (SOFTWARE VERIFICATION GATE)

Trước khi đưa thiết kế lên FPGA, bắt buộc phải dựng một mô hình Python tối giản (Bit-accurate Fixed-Point Model) mô phỏng đúng khung 640 mẫu, cửa sổ Hanning và 187 lag số nguyên. Đem mô hình này chạy trên tập dữ liệu âm thanh đã lọc tiền xử lý (HPF 60 Hz / DC-blocker) để đối sánh trực tiếp với đường Pitch chuẩn của Praat (đo sai số lệch chuẩn Hz và hệ số tương quan Pearson r). Chỉ khi phiên bản Python tối giản này chứng minh được độ suy giảm chất lượng nằm trong ngưỡng chấp nhận được của bài toán nhận diện thanh điệu/ngữ điệu, các con số hạch toán phần cứng bên dưới mới có giá trị thực tiễn.

4.0. Bảng định dạng Q-Format và Hành vi Toán học theo từng tầng

Một bản mô phỏng Python chạy được chưa chứng minh được gì về phần cứng. Trước khi viết một dòng Verilog, cần trả lời câu hỏi hẹp hơn nhiều: *phép tính nào trong thuật toán này có thể biểu diễn bằng số nguyên, và khi biểu diễn như vậy thì kết quả có còn đúng không?*

Định nghĩa định dạng số thực dấu phẩy tĩnh (Q-format): Định dạng $Qm.n$ quy định cách biểu diễn một số thực có dấu bằng một số nguyên bù hai: trong đó m bit dành cho phần nguyên (đã tính 1 bit dấu) và n bit dành cho phần phân số, với tổng độ rộng từ là $m + n$ bit. Giá trị thực tương ứng được giải mã bằng công thức:

$$\text{giá_trị_thực} = \frac{\text{số_nguyên}}{2^n} \tag{16}$$

Ví dụ: Trong định dạng Q1.15 (16 bit có dấu), số nguyên 16384 biểu diễn giá trị thực $16384/2^{15} = 0{,}5$; số nguyên 32767 biểu diễn $(2^{15}-1)/2^{15} \approx 0{,}99997$. Trong định dạng Q3.13 (16 bit có dấu), giá trị thực cực đại biểu diễn được là $(2^{15}-1)/2^{13} \approx 3{,}9998$.

Bảng 2 quy định định dạng số, độ rộng bit, phương thức làm tròn và hành vi toán học của từng tầng trong chuỗi xử lý (pipeline) từ tín hiệu vào đến kết quả cao độ.

Tầng	Đại lượng vật lý	Định dạng	Kiểu C/Verilog	Độ rộng	Phương thức làm tròn	Ý nghĩa vi kiến trúc & Dải giá trị
0	Mẫu âm thanh $x[n]$	Q1.15	`int16_t`	16 bit	Không (nguyên bản)	Tín hiệu PCM 16-bit chuẩn, dải $[-32768, 32767]$
1	Cửa sổ Hanning $w[n]$	Q1.15	`int16_t`	16 bit	Làm tròn từ float	Hệ số $[1, 32767]$, bão hòa ở 32767 (tránh tràn 16-bit)
2	Mẫu sau cửa sổ $x_w[n]$	Q1.15	`int16_t`	16 bit	$(p + 16384) \gg 15$	Tích 32 bit Q2.30, làm tròn gần nhất và dịch 15 bit
3	Bộ tích lũy $\mathcal{R}_x[\tau]$	Q11.30	`int64_t`	41 bit	Không (tích lũy chính xác)	Tích lũy 640 tích Q2.30; $640 \cdot 2^{30} < 2^{40} \implies$ 41 bit có dấu
4	Bảng bù $C[\tau]$ (kèm phạt)	Q3.13	`int16_t`	16 bit	Nạp từ `c_tau_rom.hex`	Bảng tra ROM 189 mục ($\tau \in [26, 214]$), dải $[8283, 17070]$
5	Tích $\mathcal{R}_x \cdot C$ (trước dịch)	56-bit có dấu	`int64_t`	56 bit	Không (accumulator)	Tích $\mathcal{R}_x[\tau] \cdot C[\tau] < 2^{55} \implies \leq$ **56** bit có dấu
6	Điểm số bù suy hao $S[\tau]$	Q13.30	`int64_t`	43 bit	$(p + 4096) \gg 13$	Tích $\mathcal{R}_x$ (Q11.30) $\times$ C (Q3.13) = Q·43, dịch phải 13 bit $\rightarrow$ 30 bit lẻ, dải $\leq$ 43 bit có dấu; cần thanh ghi 64-bit (`int64_t`)
7	So sánh thanh tính & lặng	Logic	`uint1_t`	1 bit	Không làm tròn	Nhân chéo nguyên: $10.000 \cdot \mathcal{R}_x[0] \geq 9 \cdot E_{\text{ref}}$, $100 \cdot S^* \geq 45 \cdot \mathcal{R}_x[0]$

Bảng 2: Bảng định dạng dấu phẩy tĩnh (Q-format) và hành vi toán học qua từng tầng xử lý trên phần cứng.

Hành vi toán học của phép dịch phải số học (`>>`) so với phép chia nguyên (`/`). Cần làm rõ sự khác biệt giữa các môi trường lập trình để tránh lỗi phần mềm:

- *Trong ngôn ngữ C/C++:* Toán tử chia nguyên `a / b` thực hiện phép cắt cụt về số 0 (truncation towards zero), ví dụ $(-3)/2 = -1$.
- *Trong Python:* Toán tử `//` thực hiện phép chia sàn (floor division), ví dụ $(-3)//2 = -2$. Toán tử `/` luôn trả về số thực `float` ($3/2 = 1{,}5$).
- *Phép dịch phải số học (`>>`) trên phần cứng:* Dịch bit có nhân bản bit dấu (arithmetic right shift) thực hiện phép chia sàn $\lfloor a/2^k \rfloor$, hoàn toàn tương đồng với `//` của Python. Trên FPGA, phép dịch này là miễn phí 0 logic (chỉ đổi dây bus). Để biến phép chia sàn thành làm tròn đến số nguyên gần nhất (round-to-nearest) không thiên lệch, ta cộng thêm nửa bước lượng tử 2^{k-1} trước khi dịch:

$$y = (x + 2^{k-1}) \gg k \tag{17}$$

  Ví dụ với số âm: -3 chia cho 2^1 ($k = 1$): $(-3 + 1) \gg 1 = -2 \gg 1 = -1$ (tương ứng làm tròn $-1{,}5$ lên -1, quy tắc round-half-up). Ở Tầng 2, sau tích hai số Q1.15 (ra Q2.30), phép tính $(p + 16384) \gg 15$ cộng thêm $2^{14} = 16384$ rồi dịch 15 bit để quay về Q1.15 với độ chính xác cao nhất.

Định cỡ bộ tích lũy: 41 bit cho $\mathcal{R}_x[\tau]$, 56 bit có dấu cho tích trước dịch và 43 bit cho $S[\tau]$. Tích của hai số 16-bit có dấu định dạng Q1.15 ($x_w[n] \cdot x_w[n+\tau]$) là một số 32-bit có dấu định dạng Q2.30, với dải giá trị từ -2^{30} đến $2^{30} - 1$. Khi cộng dồn tối đa $L = 640$ tích như vậy để tính $\mathcal{R}_x[0]$ hoặc $\mathcal{R}_x[\tau]$, giá trị tích lũy dương cực đại lý thuyết là:

$$\mathcal{R}_{\text{max}} = 640 \times 2^{30} \approx 6{,}8719 \times 10^{11} < 2^{40} \approx 1{,}0995 \times 10^{12} \tag{18}$$

Phần biên độ cần 40 bit. Thêm 1 bit dấu cho số nguyên bù hai, thanh ghi tích lũy $\mathcal{R}_x$ cần **đúng 41 bit có dấu** (định dạng Q11.30).

Khi chuyển sang tính điểm số bù suy hao $S[\tau] = (\mathcal{R}_x[\tau] \cdot C[\tau] + 4096) \gg 13$, hệ số trong ROM đạt cực đại tại $C[214] = 17070 < 2^{15}$ (trong Q3.13). Tích số nguyên $\mathcal{R}_x[\tau] \cdot C[\tau]$ trước khi dịch đạt cực đại:

$$\mathcal{R}_x \cdot C \leq (640 \times 2^{30}) \times 17070 \approx 1{,}1730 \times 10^{16} < 2^{53{,}38} < 2^{54} \tag{19}$$

Bao gồm cả bit dấu, bộ nhân điểm số đòi hỏi ≤ 56 **bit có dấu** (kiểu `int64_t`) để bảo đảm tuyệt đối không bao giờ bị tràn số. Sau khi cộng làm tròn $2^{12} = 4096$ và dịch phải 13 bit:

$$S[\tau] = (\mathcal{R}_x[\tau] \cdot C[\tau] + 4096) \gg 13$$

giá trị cực đại đạt $1{,}1730 \times 10^{16}/8192 \approx 1{,}4319 \times 10^{12} < 2^{40{,}38} < 2^{41}$. Bao gồm cả bit dấu, điểm số $S[\tau]$ đòi hỏi ≤ 43 **bit có dấu** và phải lưu trữ trong kiểu số nguyên 64-bit (`int64_t`), hoàn toàn **không thể chứa trong thanh ghi 32-bit** (`int32_t`) vì giá trị thực nghiệm tại đỉnh cực đại có thể vượt xa $2^{31} - 1 = 2.147.483.647$ (ví dụ tại khung kiểm thử chuẩn, $S[36] = 32.119.378.939 > 2^{31}$). Trên chip Cyclone V SX, các khối DSP Block cứng hỗ trợ tích lũy 64-bit, dễ dàng chứa trọn vẹn cả tích 56 bit lẫn điểm số 43 bit trong thanh ghi 64-bit mà không tốn logic ALM ngoài.

Khử hoàn toàn phép chia thời gian thực bằng phép nhân chéo nguyên. Hai điều kiện quan trọng nhất quyết định tính chất khung âm thanh vốn đòi hỏi phép chia số thực:

- Điều kiện khoảng lặng (Tham số 6): $\mathcal{R}_x[0] \geq 0{,}0009 \cdot E_{\text{ref}}$ (tương đương $-30{,}46$ dB).
- Điều kiện thanh tính (Tham số 7): $S^*/\mathcal{R}_x[0] \geq 0{,}45$.

Trên phần cứng, một bộ chia tuần tự (Radix-2 Divider) đòi hỏi tài nguyên logic đáng kể và mất từ 40 đến 64 chu kỳ xung nhịp. Trong vòng lặp quét 189 lag, ta khử triệt để phép chia bằng phép nhân chéo số nguyên:

$$10.000 \cdot \mathcal{R}_x[0] < 9 \cdot E_{\text{ref}} \implies \text{Khoảng lặng (Unvoiced)} \tag{20}$$

$$100 \cdot S^* \geq 45 \cdot \mathcal{R}_x[0] \implies \text{Hữu thanh (Voiced)} \tag{21}$$

Hai phép so sánh này được thực thi bằng hai phép nhân số nguyên với hằng số và một bộ so sánh độ lớn trong đúng 1 chu kỳ xung nhịp.

Vì sao `float64` không bảo đảm đồng nhất giữa AMD64 và ARM64, còn số nguyên thì có. Chuẩn IEEE-754 quy định chính xác kết quả của bốn phép tính cơ bản `+`, `-`, `*`, `/` trên kiểu `float64`: chúng *được làm tròn đúng* (correctly rounded) và cho cùng bit trên mọi bộ xử lý tuân thủ chuẩn. Tuy nhiên, các hàm siêu việt như cos, sin, $\log_2$ **không** nằm trong nhóm bắt buộc làm tròn đúng bit cuối. Chuẩn chỉ đòi hỏi độ chính xác trong một khoảng vài ULP (Unit in the Last Place — đơn vị ở vị trí cuối cùng). Do đó, thư viện toán học `glibc` trên kiến trúc AMD64 (x86_64) và `libm` của Apple trên kiến trúc ARM64 (AArch64) có thể trả về hai kết quả lệch nhau 1–2 ULP cho cùng một góc hoặc cùng một số thực. Với một phép tính đơn lẻ, sai số này là vô hại. Nhưng khi kết quả này tham gia vào việc so sánh tìm cực đại (argmax) giữa hai đỉnh tự tương quan gần bằng nhau, 1 ULP có thể làm lật ngôi đỉnh thắng cuộc, khiến hai máy tính cho ra hai chuỗi F_0 khác nhau!

Ngược lại, hệ thống số nguyên bù hai (`int16`, `int32`, `int64`) với các phép `+`, `-`, `*`, `>>` hoàn toàn không có sai số biểu diễn hay sự phụ thuộc vào thư viện bên ngoài:

- Kết quả số học là giá trị nguyên xác định tuyệt đối trên từng bit.
- Hiện tượng tràn số (nếu có) tuân thủ chặt chẽ phép toán modulo 2^N.
- Phép dịch phải số học `>>` bảo toàn bit dấu xác định.

Do đó: **cùng một đoạn mã số nguyên, cùng một dữ liệu đầu vào, sẽ cho ra kết quả trùng khớp** 100% **từng bit trên AMD64, ARM64 và trên khối mạch phần cứng RTL tổng hợp trên FPGA**. Đây chính là nền tảng của phương pháp luận kiểm chứng *bit-accurate*: tách bạch hoàn toàn sai số thuật toán khỏi sai số sai lệch phần cứng.

4.1. Bản tham chiếu dấu phẩy động (Float Twin) trên Laptop

Để làm thước đo trung gian trước khi chuyển sang số nguyên, ta xây dựng mô hình dấu phẩy động gọi là **Float Twin** trong `lab_1_3_5.py`. Cần lưu ý rõ ràng: Float Twin **không phải** là bản sao nguyên si của mã nguồn Praat C++, mà là bản mô hình phần mềm song sinh được tinh giản có chủ đích với **5 điểm khác biệt** nhằm tương thích hoàn toàn với kiến trúc phần cứng FPGA:

1. **Tự tương quan:** Float Twin tính trực tiếp tổng tích trễ thời gian $\mathcal{R}_x[\tau]$ (cận $L - 1 - \tau$), trong khi Praat C++ chạy chuỗi biến đổi FFT $\to |X|^2 \to$ IFFT ($N_{\text{FFT}} = 1024$).

2. **Xử lý suy hao cửa sổ:** Float Twin nhân với bảng hệ số $C[\tau]$ đã tích hợp sẵn lượng phạt quãng tám dạng nhân $2^{-0{,}01 \cdot \log_2(600/F_0)}$, trong khi Praat chia cho $\mathcal{R}_w[\tau]$ rồi trừ đại số $0{,}01 \cdot \log_2(600/F_0)$ trên r đã chuẩn hóa.
3. **Cơ chế trích chọn cực đại:** Float Twin chỉ trích xuất duy nhất 1 ứng viên argmax tức thời, trong khi Praat lưu 14 đỉnh cực đại cục bộ $+1$ ứng viên Vô thanh để chạy quy hoạch động Viterbi toàn câu.
4. **Nội suy dưới mẫu:** Float Twin dùng nội suy parabol 3 điểm, trong khi Praat dùng phép nội suy Sinc 30 điểm.
5. **Cổng khoảng lặng:** Float Twin dùng cổng ngắt cứng trực tiếp $r_0 < 0{,}0009 \cdot E_{\text{ref}}$, trong khi Praat đưa tỷ số biên độ intensity vào hàm mục tiêu mềm Viterbi.

Bảng ROM cửa sổ Hanning `GLOBAL_HANNING_Q15` được khởi tạo từ đầu để làm nguồn dữ liệu chuẩn cho cả Float Twin lẫn mô hình số nguyên:

```python
import numpy as np

Q15, Q13 = 1 << 15, 1 << 13          # 32768 va 8192
TAU_SCAN_MIN, TAU_SCAN_MAX = 26, 214  # 189 lag quet (de lan can noi suy hop le)
TAU_MIN, TAU_MAX = 27, 213           # 187 lag tim kiem (600 Hz .. 75 Hz tai fs = 16 kHz)

def hanning_rom_q15():
    """Bang ROM 640 mau Hanning Q1.15, bao hoa tai 32767 (Praat dong 413)."""
    i = np.arange(1, 641, dtype=np.float64)
    w = 0.5 - 0.5 * np.cos(2.0 * np.pi * i / 641.0)
    return np.clip(np.round(w * Q15), 0, Q15 - 1).astype(np.int64)

GLOBAL_HANNING_Q15 = hanning_rom_q15()

def parabolic_interpolation(scores, idx):
    """Noi suy parabol 3 diem: delta_tau = 0.5 * (ym1 - yp1) / (ym1 - 2*y0 + yp1)."""
    if 0 < idx < len(scores) - 1:
        ym1 = float(scores[idx - 1])
        y0 = float(scores[idx])
        yp1 = float(scores[idx + 1])
        denom = ym1 - 2.0 * y0 + yp1
        if denom != 0.0:
            return 0.5 * (ym1 - yp1) / denom
    return 0.0

def float_pitch_frame(x: np.ndarray, ref_energy: float = 0.0, use_interpolation: bool =
    True):
    # Buoc 1: Tru trung binh cuc bo (khu DC, Sound_to_Pitch.cpp:63--83)
    x_zm = x - np.mean(x)
    # Buoc 2: Nhan cua so Hanning (Sound_to_Pitch.cpp:413)
    w = GLOBAL_HANNING_Q15 / 32768.0
    xw = x_zm * w
    r0 = float(np.dot(xw, xw))

    # Kiem tra nguong lang: nang luong < 0.03^2 = 0.0009 (-30.46 dB)
    if ref_energy > 0.0 and r0 < 0.0009 * ref_energy:
        return 0.0, 0, False
    if r0 == 0.0:
        return 0.0, 0, False

    # Buoc 3 & 4: Tu tuong quan va Bu suy hao kem phat quang tam
    # Quet mo rong tau in [26, 214] (189 lag) de bao toan lan can noi suy
    rw0 = float(np.dot(w, w))
    taus_scan = list(range(26, 215))
    scores = []
    for tau in taus_scan:
        rt = float(np.dot(xw[:640 - tau], xw[tau:]))
        rw_tau = float(np.dot(w[:640 - tau], w[tau:]))
        c_tau = rw0 / rw_tau
        bias = 2.0 ** (-0.01 * np.log2(600.0 / (16000.0 / tau)))
        scores.append(rt * c_tau * bias)

```

```
    # Tim cuc dai bi gioi han chat che trong [27, 213] (idx 1..187)
    best_score, tau_star, best_idx = -1e9, 0, 0
    for idx in range(1, len(taus_scan) - 1):
        if scores[idx] > best_score:
            best_score, tau_star, best_idx = scores[idx], taus_scan[idx], idx

    # Buoc 5: So sanh nguong huu thanh (0.45 * r0, Pitch.cpp:553)
    if best_score / r0 < 0.45:
        return 0.0, 0, False

    # Buoc 6: Noi suy parabol 3 diem tinh F0 duoi mau
    if use_interpolation:
        delta_tau = parabolic_interpolation(scores, best_idx)
        return 16000.0 / (tau_star + delta_tau), tau_star, True

    return 16000.0 / tau_star, tau_star, True
```

4.2. Bản số nguyên Bit-Accurate từ con số 0

Đoạn mã dưới đây là phần lõi thực thi số nguyên thuần túy trong `lab_1_3_5.py`. Mã nguồn nạp sẵn các bảng ROM toàn cục, dùng đúng các phép toán bù hai, và không gọi bất kỳ hàm số thực hay thư viện ngoài nào trong vòng lặp khung:

```
import wave
from pathlib import Path
import numpy as np

Q15, Q13 = 1 << 15, 1 << 13          # 32768 va 8192
TAU_SCAN_MIN, TAU_SCAN_MAX = 26, 214  # 189 lag quet (de lan can noi suy hop le)
TAU_MIN, TAU_MAX = 27, 213           # 187 lag tim kiem (600 Hz .. 75 Hz tai fs = 16 kHz)

def hanning_rom_q15():
    """Bang ROM 640 mau Hanning Q1.15, bao hoa tai 32767 (Praat dong 413)."""
    i = np.arange(1, 641, dtype=np.float64)
    w = 0.5 - 0.5 * np.cos(2.0 * np.pi * i / 641.0)
    return np.clip(np.round(w * Q15), 0, Q15 - 1).astype(np.int64)

def scoring_rom_q13(octave_cost=0.01):
    """Bang ROM C[tau] tich hop san phat quang tam, 189 muc Q3.13 cho tau in [26, 214]."""
    w = hanning_rom_q15().astype(np.float64) / Q15
    rw0 = float(np.dot(w, w))
    out = []
    for tau in range(TAU_SCAN_MIN, TAU_SCAN_MAX + 1):
        rw_tau = float(np.dot(w[:640 - tau], w[tau:]))
        bias = 2.0 ** (-octave_cost * np.log2(600.0 / (16000.0 / tau)))
        out.append(round((rw0 / rw_tau) * bias * Q13))
    return np.array(out, dtype=np.int64)

# Khoi tao bang tra ROM toan cuc (chi tinh 1 lan duy nhat)
GLOBAL_HANNING_Q15 = hanning_rom_q15()
GLOBAL_SCORING_ROM_Q13 = scoring_rom_q13()

def window_frame_q15(x_zm):
    """Nhan cua so Hanning Q1.15, lam tron round-half-up: +16384 >> 15."""
    return ((x_zm * GLOBAL_HANNING_Q15) + 16384) >> 15

def median3(seq):
    """Loc trung vi 3 diem: giu nguyen 0 = UV, phan tu dau/cuoi giu nguyen."""
    out = []
    for i in range(len(seq)):
        w = [seq[max(0, i - 1)], seq[i], seq[min(len(seq) - 1, i + 1)]]
        out.append(sorted(w)[1])
    return out

def parabolic_interpolation_fixed(scores, best_idx, tau_star):
    """Noi suy parabol 3 diem so nguyen (Che do B): delta_tau = N / D.
```

```
    N = S[i-1] - S[i+1];  D = 2*(S[i-1] - 2*S[i] + S[i+1])
    tau_fine_q16 = (tau_star << 16) + ((N << 16) // D)
    f0_q12_4 = (16000*16*65536 + (tau_fine_q16 >> 1)) // tau_fine_q16
    """
    sm1, s0, sp1 = int(scores[best_idx - 1]), int(scores[best_idx]), int(scores[best_idx
        + 1])
    N = sm1 - sp1
    D = 2 * (sm1 - 2 * s0 + sp1)
    if D == 0:
        tau_fine_q16 = tau_star << 16
    else:
        tau_fine_q16 = (tau_star << 16) + ((N << 16) // D)
    if tau_fine_q16 <= 0:
        return tau_fine_q16, 0
    f0_q12_4 = (16000 * 16 * 65536 + (tau_fine_q16 >> 1)) // tau_fine_q16
    return tau_fine_q16, f0_q12_4

def fixed_pitch_frame(x_q15, ref_energy, use_interpolation=True, scoring_rom=None):
    """Do cao do 1 khung 640 mau bang so nguyen tren datapath 64-bit co dau."""
    if scoring_rom is None:
        scoring_rom = GLOBAL_SCORING_ROM_Q13

    # Tru trung binh cuc bo (khu DC tren K = 640 mau): dong 63-83 Sound_to_Pitch.cpp
    mean_x = (int(np.sum(x_q15)) + 320) // 640
    x_zm = x_q15 - mean_x

    # Pha 1: Nhan cua so Hanning Q1.15
    xw = window_frame_q15(x_zm)
    r0 = int(np.dot(xw, xw))                    # Nang luong khung (Q11.30)

    # Kiem tra nguong lang: r0 * 10000 < 9 * ref_energy (khong dung phep chia)
    if ref_energy > 0 and r0 * 10000 < 9 * ref_energy:
        return 0.0, 0, False, 0
    if r0 == 0:
        return 0.0, 0, False, 0

    # Pha 2: Quet 189 lag tu tuong quan va nhan bang bu C[tau]
    taus_scan = list(range(TAU_SCAN_MIN, TAU_SCAN_MAX + 1))
    scores = []
    for idx, tau in enumerate(taus_scan):
        n = 640 - tau
        r_tau = int(np.dot(xw[:n], xw[tau:])) # 41-bit MAC
        score = (r_tau * scoring_rom[idx] + 4096) >> 13 # <= 56-bit -> <= 43-bit S[tau]
        scores.append(score)

    # Tim cuc dai trong dai [TAU_MIN, TAU_MAX] (idx 1..187)
    best_score, tau_star, best_idx = -(1 << 62), 0, 0
    for idx in range(1, len(taus_scan) - 1):
        if scores[idx] > best_score:
            best_score = scores[idx]
            tau_star = taus_scan[idx]
            best_idx = idx

    # Kiem tra nguong huu thanh: best * 100 >= 45 * r0 (khong chia)
    if best_score * 100 < 45 * r0:
        return 0.0, 0, False, 0

    # Tinh F0: Che do B (noi suy parabol so nguyen) hoac Che do A (tra ROM f0_lut)
    if use_interpolation:
        tau_fine_q16, f0_q12_4 = parabolic_interpolation_fixed(scores, best_idx, tau_star)
        return f0_q12_4 / 16.0, tau_star, True, f0_q12_4
    f0_hz = float(round(16000.0 / tau_star))
    return f0_hz, tau_star, True, int(round(f0_hz * 16))
```

Ý nghĩa và tác động của nội suy parabol 3 điểm dưới mẫu. Nếu chỉ giữ độ trễ nguyên $\tau^* \in \mathbb{Z}$, thuật toán tự tương quan sẽ vấp phải hiện tượng phi tuyến lượng tử hóa rất nặng ở dải tần số cao. Để giải quyết triệt để vấn đề

này từ bản chất vật lý sóng trước khi đi vào các ví dụ số, Hộp Feynman dưới đây bóc tách nguồn gốc hình học của đỉnh tự tương quan:

★ BẮC CẦU FEYNMAN: Tại sao Đỉnh Tự Tương Quan có dạng Parabol? Giải mã từ Khai triển Euler / Taylor

Tại sao Praat và mọi hệ thống DSP âm học kinh điển đều dùng đường cong Parabol bậc hai để nội suy đỉnh cực đại dưới mẫu mà không phải hàm số nào khác? Hãy quay lại công thức Euler:

$$\cos\theta = \frac{e^{j\theta} + e^{-j\theta}}{2} = 1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \frac{\theta^6}{6!} + \dots$$

Khi độ trễ τ tiến rất sát đỉnh cực đại τ^* (tức khoảng cách góc $\theta = \omega(\tau - \tau^*) \approx 0$), các số hạng bậc 4 trở lên rất nhỏ ($\theta^4/24 \approx 0$). Hàm cos suy biến tự nhiên thành một parabol úp ngược:

$$\cos(\omega(\tau - \tau^*)) \approx 1 - \frac{\omega^2}{2}(\tau - \tau^*)^2$$

Vì hàm tự tương quan $\mathcal{R}_x[\tau] = \sum_k |X[k]|^2 \cos(\omega_k(\tau - \tau^*))$ là tổng của các sóng cos đồng pha quanh đỉnh, nên đồ thị quanh lân cận cực đại của nó mang hình dáng chính xác của một Parabol úp ngược:

$$y(x) = ax^2 + bx + c \quad (a < 0)$$

Dẫn xuất 3 dòng tìm đỉnh cực trị dưới mẫu $\delta\tau$ qua 3 điểm nguyên $\{y_{-1}, y_0, y_1\}$:

1. **Đặt hệ trục tọa độ cục bộ:** Chọn đỉnh nguyên làm gốc $x = 0$ ($y(0) = y_0 \implies c = y_0$). Hai láng giềng là $x = -1$ và $x = +1$:

$$y_1 = a(1)^2 + b(1) + y_0 = a + b + y_0, \qquad y_{-1} = a(-1)^2 + b(-1) + y_0 = a - b + y_0$$

2. **Giải hệ phương trình tìm độ dốc b và độ cong a:**
   Trừ hai phương trình triệt tiêu a: $y_1 - y_{-1} = 2b \implies b = \frac{y_1 - y_{-1}}{2}$.
   Cộng hai phương trình triệt tiêu b: $y_1 + y_{-1} = 2a + 2y_0 \implies a = \frac{y_1 - 2y_0 + y_{-1}}{2}$.
3. **Tìm đỉnh cực trị bằng đạo hàm bậc nhất:**
   Tại đỉnh cực đại, tiếp tuyến nằm ngang: $y'(x) = 2ax + b = 0$. Tọa độ đỉnh dưới mẫu $\delta\tau$ là:

$$x^* = \delta\tau = -\frac{b}{2a} = -\frac{\frac{y_1 - y_{-1}}{2}}{2 \cdot \frac{y_1 - 2y_0 + y_{-1}}{2}} = \frac{y_{-1} - y_1}{2(y_{-1} - 2y_0 + y_1)} = \frac{N}{D} \tag{22}$$

Dẫn xuất giải tích Euler này giải thích vì sao một phép chia nguyên N/D duy nhất trên FPGA lại tái lập được đỉnh thực của thanh quản với độ chính xác dưới cent!

Nếu chỉ giữ độ trễ nguyên $\tau^* \in \mathbb{Z}$, thuật toán tự tương quan sẽ vấp phải hiện tượng phi tuyến lượng tử hóa rất nặng ở dải tần số cao:

- **Ví dụ số 1 (Bước nhảy lag tại biên tần số cao):** Xét bước nhảy giữa hai độ trễ liền kề $\tau = 32$ và $\tau = 33$ tại $f_s = 16$ kHz. Tần số tương ứng là $F_0(32) = 16000/32 = 500$ Hz và $F_0(33) = 16000/33 \approx 484{,}85$ Hz, tạo ra bước nhảy tần số $\Delta F_0 \approx 15{,}15$ Hz. Quy đổi sang đơn vị cent âm học:

$$\Delta_{\text{cent}} = 1200 \times \log_2\left(\frac{500}{484{,}85}\right) \approx \mathbf{53{,}4}\text{ cent} \tag{23}$$

  Ngưỡng phân biệt cao độ tối thiểu của thính giác người (Just Noticeable Difference — JND) đối với âm thanh chuẩn là khoảng 5–10 cent. Như vậy, bước nhảy 53,4 cent thô gấp 5 đến 10 lần ngưỡng nghe của tai người! Một người hát luyến từ 490 Hz lên 500 Hz sẽ bị hệ thống ghi nhận thành một bậc thang nhảy giật cục.

- **Ví dụ số 2 (Sóng sin chuẩn 440 Hz):** Tần số nốt La chuẩn $A_4 = 440$ Hz có chu kỳ thực tế là $T_0 = 16000/440 \approx 36{,}3636$ mẫu. Khi chỉ xét độ trễ nguyên rời rạc, điểm cực đại rơi vào $\tau^* = 36$, tương ứng với $F_0 = 16000/36 \approx 444{,}44$ Hz. Sai số là $+4{,}44$ Hz, tương đương:

$$\Delta_{\text{cent}} = 1200 \times \log_2\left(\frac{444{,}44}{440}\right) \approx +\mathbf{17{,}4}\text{ cent} \tag{24}$$

Mức sai số $+17{,}4$ cent vượt xa ngưỡng JND (5–10 cent) và làm suy giảm hệ số tương quan Pearson r. Khi áp dụng công thức nội suy parabol 3 điểm (Chế độ B số nguyên) qua ba giá trị điểm số $\{S[35], S[36], S[37]\}$ sinh ra từ mô hình:

$$\begin{aligned} S[35] &= 31.303.452.800 \\ S[36] &= 32.119.378.939 \\ S[37] &= 31.978.033.782 \end{aligned} \tag{25}$$

Hiệu số tử số N và mẫu số cong D:

$$\begin{aligned} N &= S[35] - S[37] = 31.303.452.800 - 31.978.033.782 = -674.580.982 \\ D &= 2 \times (S[35] - 2 \cdot S[36] + S[37]) \\ &= 2 \times (31.303.452.800 - 64.238.757.878 + 31.978.033.782) = -1.914.542.592 \neq 0 \end{aligned} \tag{26}$$

Độ lệch đỉnh dưới mẫu trong định dạng Q16:

$$\delta\tau_{\text{q16}} = (N \ll 16)//D = (-674.580.982 \times 65536)// - 1.914.542.592 = \mathbf{23.091} \tag{27}$$

Tọa độ đỉnh nội suy: $\tau_{\text{fine, q16}} = (36 \ll 16) + 23.091 = 2.359.296 + 23.091 = \mathbf{2.382.387}$ (tương đương $36{,}3523$ mẫu). Giá trị tần số cơ bản định dạng Q12.4 tương ứng:

$$\begin{aligned} f_{0,\text{q12.4}} &= \big(16000 \times 16 \times 65536 + (\tau_{\text{fine, q16}} \gg 1)\big)//\tau_{\text{fine, q16}} \\ &= 16.778.407.193//2.382.387 = \mathbf{7042} \equiv \texttt{0x1B82} \end{aligned} \tag{28}$$

Quy đổi sang Hz: $F_0 = 7042/16 = \mathbf{440{,}125}$ Hz. Mô hình float twin (hàm `float_pitch_frame` trong `lab_1_3_5.py`) tính được $F_0 \approx 440{,}116$ Hz ($+0{,}46$ cent so với 440 Hz chuẩn), khi làm tròn ra số nguyên Q12.4 cũng cho kết quả đúng **7042**.

$$\Delta_{\text{cent}} = 1200 \times \log_2\left(\frac{440{,}125}{440}\right) \approx +\mathbf{0{,}49}\text{ cent} \quad (\text{so với float twin: } +\mathbf{0{,}46}\text{ cent}) \tag{29}$$

Sai số giảm từ $+17{,}4$ cent xuống chỉ còn $+\mathbf{0{,}49}$ cent ở bản số nguyên (mô hình float twin là $+0{,}46$ cent, giảm hơn 35 lần), hoàn toàn nằm sâu dưới ngưỡng nghe 5 cent của con người!

Ghi chú toán học về toán tử chia số nguyên: Phép toán `//` trong Python thực hiện làm tròn về $-\infty$ (floor division). Khi N và D cùng âm, thương số là dương nên làm tròn về $-\infty$ hay cắt về 0 là như nhau. Nếu áp dụng trên dữ liệu tổng quát có thương âm, bộ chia phần cứng RTL (vốn thường cắt phần thập phân về 0 — truncation towards zero) cần xử lý dấu hoặc quy ước rõ thuật toán làm tròn để đạt tính đồng nhất bit tuyệt đối.

Đặc tả kỹ thuật tệp mảng cao độ `pitch_f0.bin`: Để đảm bảo tính tương thích phần mềm, tệp nhị phân `pitch_f0.bin` tuân thủ quy chuẩn nghiêm ngặt:

- **Kiểu dữ liệu và định dạng:** Mảng tuần tự các từ 16-bit không dấu (`uint16_t`, Little-Endian) theo định dạng dấu phẩy tĩnh **Q12.4**:
$$\text{val} = \text{round}(F_0 \times 16) = \text{round}(F_0 \cdot 2^4) \tag{30}$$
với giá trị cực đại biểu diễn được là $(2^{12} - 1) + 15/16 = 4095{,}9375$ Hz, độ phân giải bước lượng tử là $\Delta F_0 = 1/16 = 0{,}0625$ Hz.
- **Vì sao chọn Q12.4 thay vì Q8.8?** Định dạng Q8.8 chỉ có đúng 8 bit phần nguyên, dải biểu diễn tối đa bị chặn đứng ở $2^8 - 1 = 255{,}996$ Hz. Giới hạn này hoàn toàn không thể đo được giọng nữ cao (thường từ 250–400 Hz), giọng trẻ em (300–500 Hz) hoặc trần cao độ ngữ âm học 600 Hz. Định dạng Q12.4 dành tới 12 bit cho phần nguyên (chứa trọn dải $[0, 4095]$ Hz) và 4 bit phần lẻ (bước mịn $0{,}0625$ Hz).

  Ví dụ giải mã: Một khung có $F_0 = 440{,}116$ Hz (giá trị float twin đo trên sóng sin 440 Hz), giá trị ghi vào tệp là:
$$\text{val} = \text{round}(440{,}116 \times 16) = \text{round}(7041{,}856) = 7042 \equiv \texttt{0x1B82}$$

Khi máy chủ đọc ra: $7042/16 = 440{,}125$ Hz. Độ lệch tần số là $|\Delta F_0| = 0{,}009$ Hz, quy đổi sang đơn vị cent âm học:

$$\Delta_{\text{cent}} = 1200 \times \log_2\left(\frac{440{,}125}{440{,}116}\right) \approx -\mathbf{0{,}035}\ \text{cent} \ll 1\ \text{cent}$$

hoàn toàn triệt tiêu sai số lượng tử hóa nhị phân đối với thính giác người.

- **Quy ước phân loại thanh tính:** val $= 0 \equiv$ Unvoiced (khung vô thanh hoặc khoảng lặng bị ngắt bởi silence gate). Giá trị val > 0 là cao độ hữu thanh hợp lệ trong dải $F_0 \in [75, 600]$ Hz.

Hàm xử lý luồng âm thanh mức đỉnh và quan hệ nhân quả của E_{ref}. Để kết nối khối xử lý từng khung vào luồng phân tích tệp âm thanh thực tế, hàm `process_wav_fixed_point` thực hiện đọc tệp WAV chuẩn 16 kHz mono, trích xuất chuỗi F_0 và lưu trữ ra tệp nhị phân:

```
def process_wav_fixed_point(wav_path, bin_path, mode="file", scoring_rom=None):
    """Xu ly tep WAV 16 kHz 16-bit mono sang pitch_f0.bin bang so nguyen hoan toan.
    mode='file': 2 luot. Luot 1 tinh r0 de tim E_ref = max r0; Luot 2 chay do cao do.
    mode='stream': 1 luot voi E_ref = max chay (running max).
    Loc median3 tren f0_q12_4 (tre 1 khung = 10 ms). Xuat uint16 LE.
    """
    if scoring_rom is None: scoring_rom = GLOBAL_SCORING_ROM_Q13
    with wave.open(str(wav_path), "rb") as w:
        assert w.getframerate() == 16000 and w.getnchannels() == 1 and w.getsampwidth()
            == 2
        samples = np.frombuffer(w.readframes(w.getnframes()),
            dtype=np.int16).astype(np.int64)

    n_frames = max(0, (len(samples) - 640) // 160 + 1)
    if n_frames == 0: Path(bin_path).write_bytes(b""); return []

    if mode == "file": # Luot 1: tinh r0 toan cuc de xac dinh E_ref
        r0_list = []
        for k in range(n_frames):
            seg = samples[k*160 : k*160 + 640]
            xw = window_frame_q15(seg - (int(np.sum(seg)) + 320) // 640)
            r0_list.append(int(np.dot(xw, xw)))
        ref_energy = max(r0_list) if len(r0_list) > 0 else 0
        f0_raw = [fixed_pitch_frame(samples[k*160:k*160+640], ref_energy, True,
            scoring_rom)[3]
                  for k in range(n_frames)]
    elif mode == "stream": # 1 luot: E_ref la cuc dai chay (running max)
        f0_raw, running_max = [], 0
        for k in range(n_frames):
            seg = samples[k*160 : k*160 + 640]
            xw = window_frame_q15(seg - (int(np.sum(seg)) + 320) // 640)
            r0 = int(np.dot(xw, xw))
            running_max = max(running_max, r0)
            f0_raw.append(fixed_pitch_frame(seg, running_max, True, scoring_rom)[3])

    f0_clean = [int(v) for v in median3(f0_raw)]
    np.asarray(f0_clean, dtype="<u2").tofile(str(bin_path))
    return f0_clean
```

Dòng mã	Thao tác thực hiện	Ý nghĩa vi kiến trúc & Ràng buộc vật lý
`assert fs==16000...`	Kiểm tra hợp đồng âm thanh	Bắt buộc PCM 16-bit mono 16 kHz; bảo đảm đúng dải lag $[26, 214]$.
`mode == "file"`	Quy trình 2 lượt (Two-pass)	Lượt 1 tìm $E_{\text{ref}} = \max_k r_0[k]$ toàn câu; cổng khoảng lặng tĩnh tuyệt đối.
`mode == "stream"`	Quy trình 1 lượt cuốn chiếu	E_{ref} là đỉnh chạy (running max); khung đầu câu có thể lệch do chưa đạt đỉnh.
`median3(f0_raw)`	Lọc trung vị trượt 3 điểm	Dập tắt gai nhảy quãng tám cô lập; đánh đổi đúng 1 khung trễ (10 ms).
`dtype="<u2"`	Ghi tệp nhị phân Q12.4	Định dạng Little-Endian 16-bit, $0 =$ UV, tương thích bộ đệm DMA/Shared Folder.

Bảng 3: Bảng giải phẫu luồng thực thi của hàm `process_wav_fixed_point`.

Chế độ tự kiểm tra (`--selftest`) của tập lệnh `lab_1_3_5.py` in trực tiếp các giá trị kiểm thử mẫu (Hanning $H[0..4]$, ROM bù $C[26..30]$, $C[214] = 17070$, $S[35..37]$, kết quả $f_{0,\text{q12.4}} = 7042$, quét chi phí quãng tám và sai số cent trên 5 sóng sin chuẩn) để phục vụ việc đối soát trực tiếp với phần cứng.

4.2.B. Bốn Bẫy Số Học "Chết Người" Khi Chuyển Mã Sang C++20 và Rust và Triển Khai Bit-Accurate

Khi chuyển đổi thuật toán định dạng dấu phẩy tĩnh từ mô hình nguyên mẫu Python sang các ngôn ngữ hệ thống như C++20 hoặc Rust để triển khai trên hệ thống nhúng (ARM HPS hoặc sinh mã phần cứng HLS), người kỹ sư rất dễ mắc phải 4 cạm bẫy số học tinh vi khiến kết quả sai lệch từng bit dù logic giải thuật hoàn toàn giống nhau:

1. **Bẫy 1 — Phép chia số nguyên có dấu (Floor Division vs. Truncation toward Zero):** Trong Python, toán tử `//` thực hiện phép chia sàn (*floor division*), luôn làm tròn về phía $-\infty$:

   $$-7//2 = -4, \qquad 7//2 = 3$$

   Trong khi đó, toán tử `/` trong C++20 và Rust thực hiện phép cắt cụt phần thập phân về phía số 0 (*truncation toward zero*):

   $$-7/2 = -3 \quad \text{(trong C++ / Rust)}$$

   Trong công thức nội suy parabol $\delta\tau = (N \ll 16)/D$:

   - Khi N và D cùng dấu (cùng âm hoặc cùng dương), thương số dương $\implies$ phép chia floor và truncation cho cùng kết quả.
   - Khi N và D khác dấu (thương số âm), phép chia trong C++ sẽ bị lệch đúng 1 đơn vị LSB so với Python!

   Giải pháp chuẩn mực: Trong C++, bắt buộc cài đặt hàm phụ trợ `div_floor_s64` chuẩn IEEE:

```cpp
inline int64_t div_floor_s64(int64_t a, int64_t b) {
    int64_t q = a / b;
    int64_t r = a % b;
    if (r != 0 && ((r < 0) ^ (b < 0))) q -= 1;
    return q;
}
```

   Trong Rust, sử dụng phương thức tích hợp sẵn `a.div_euclid(b)` hoặc `a.div_floor(b)`.

2. **Bẫy 2 — Tràn số 32-bit trong các hằng số và phép so sánh nhân chéo:** Trong Python, số nguyên có độ dài tùy ý (*arbitrary precision integers*). Trong C++ và Rust, số nguyên có độ rộng cố định.

   - Khi kiểm tra cổng khoảng lặng: phép tính $10.000 \cdot \mathcal{R}_x[0]$. Với $\mathcal{R}_x[0]$ có thể đạt tới $1{,}2 \times 10^{11}$, tích số đạt $1{,}2 \times 10^{15} > 2^{31} - 1 \approx 2{,}14 \times 10^9$. Nếu viết `10000 * r0` mà hằng số 10000 là kiểu `int` 32-bit thì sẽ gây tràn số nghiêm trọng.
   - Khi kiểm tra ngưỡng thanh tính: phép tính $100 \cdot S^*$. Điểm số S^* đạt tới $3{,}2 \times 10^{10}$, tích số đạt $3{,}2 \times 10^{12} > 2^{31} - 1$.
   - Tại bước dịch bit nội suy: $N \ll 16$ với $N = -674.580.982$ đòi hỏi ít nhất 46 bit có dấu.

Giải pháp chuẩn mực: Khai báo hằng số với hậu tố `LL` (`10000LL`, `100LL`), ép kiểu tường minh sang `int64_t` trong C++ hoặc `i64` / `i128` trong Rust trước mọi phép nhân.

3. **Bẫy 3 — Phép dịch bit trái trên số nguyên có dấu âm ($N \ll 16$):** Trước chuẩn C++20, việc dịch trái một số nguyên có dấu âm (chẳng hạn `-674580982LL << 16`) là **Hành vi không xác định (Undefined Behavior — UB)** theo chuẩn ISO C++! Mặc dù C++20 đã chính thức chuẩn hóa biểu diễn số bù 2 và cho phép dịch trái số âm theo quy tắc số học, mã nguồn di động và an toàn nhất cho mọi trình biên dịch HLS (như Xilinx Vitis HLS hay Intel FPGA HLS) nên thay thế phép dịch trái bằng phép nhân với hằng số $2^{16} = 65536$:

```
int64_t n_shifted = N * 65536LL; // An toan 100% tren moi chuan C++/C va HLS
```

4. **Bẫy 4 — Toán tử lũy thừa `^` vs. `**`:** Trong Python, toán tử `**` là phép lũy thừa (`2 ** 4 = 16`). Trong C++ và Rust, ký tự `^` là toán tử logic XOR từng bit (`2 ^ 4 = 6`)! Khi tính hệ số phạt quãng tám $2^{-\text{octave_cost} \cdot \log_2(600/F_0)}$, nếu gõ nhầm `2 ^ ...` trong C++, trình biên dịch vẫn hợp lệ nhưng kết quả ROM sẽ bị phá hủy hoàn toàn. Trong C++, bắt buộc dùng `std::pow(2.0, -bias)` hoặc tính sẵn trong bảng tra ROM.

Dưới đây là toàn bộ mã nguồn triển khai chuẩn mực bằng **C++20** (`lab_1_3_5.cpp`), hoàn toàn độc lập, sử dụng số nguyên 64-bit và phép chia floor chuẩn, bảo đảm kết quả đầu ra khớp 100% từng bit với mô hình Python và phần cứng FPGA:

Listing 1: Mã nguồn C++20 chuẩn mực bit-accurate triển khai giải thuật đo cao độ FPGA (trích đoạn các hàm lõi từ `lab_1_3_5.cpp`).

```
#define _USE_MATH_DEFINES
#include <iostream>
#include <vector>
#include <cmath>
#include <cstdint>
#include <iomanip>
#include <algorithm>
#include <array>

constexpr int FS_HZ = 16000;
constexpr int FRAME_LEN = 640;      // 40 ms
constexpr int TAU_SCAN_MIN = 26;    // [26, 214] (189 lag)
constexpr int TAU_SCAN_MAX = 214;
constexpr int TAU_MIN = 27;         // [27, 213] (187 lag tim kiem)
constexpr int TAU_MAX = 213;
constexpr int N_SCAN_LAGS = TAU_SCAN_MAX - TAU_SCAN_MIN + 1; // 189

// Bay 1: Ham chia so nguyen lam tron ve am vo cung (floor division)
inline int64_t div_floor_s64(int64_t a, int64_t b) {
    int64_t q = a / b;
    int64_t r = a % b;
    if (r != 0 && ((r < 0) ^ (b < 0))) q -= 1;
    return q;
}

struct FrameResultCpp {
    double f0_hz;
    int tau_star;
    bool voiced;
    int f0_q12_4;
};

// Noi suy parabol 3 diem so nguyen (Che do B)
inline std::pair<int64_t, int> parabolic_interpolation_fixed_cpp(
    const std::vector<int64_t>& scores, int best_idx, int tau_star)
{
    int64_t sm1 = scores[best_idx - 1];
    int64_t s0  = scores[best_idx];
    int64_t sp1 = scores[best_idx + 1];
    int64_t N = sm1 - sp1;
```

```
    int64_t D = 2LL * (sm1 - 2LL * s0 + sp1);
    if (D == 0) {
        return {static_cast<int64_t>(tau_star) << 16,
                static_cast<int>(div_floor_s64(16000LL * 16LL, tau_star))};
    }
    // Bay 3: Dung N * 65536LL de tranh UB khi N am
    int64_t delta_tau_q16 = div_floor_s64(N * 65536LL, D);
    int64_t tau_fine_q16 = (static_cast<int64_t>(tau_star) << 16) + delta_tau_q16;
    if (tau_fine_q16 <= 0) tau_fine_q16 = static_cast<int64_t>(tau_star) << 16;
    int64_t num = 16000LL * 16LL * 65536LL + (tau_fine_q16 >> 1);
    int64_t f0_q12_4 = div_floor_s64(num, tau_fine_q16);
    return {tau_fine_q16, static_cast<int>(f0_q12_4)};
}

// Ham xu ly mot khung cao do bang so nguyen hoan toan tren FPGA
FrameResultCpp fixed_pitch_frame_cpp(
    const int64_t* x_frame, int64_t ref_energy,
    bool use_interpolation, const FpgaPitchRoms& roms)
{
    // Buoc 1: Tru trung binh cuc bo 640 mau
    int64_t sum_x = 0;
    for (int n = 0; n < FRAME_LEN; ++n) sum_x += x_frame[n];
    int64_t mean_x = div_floor_s64(sum_x + 320LL, 640LL);

    // Buoc 2: Nhan cua so Hanning Q1.15
    std::array<int64_t, FRAME_LEN> xw;
    for (int n = 0; n < FRAME_LEN; ++n) {
        xw[n] = ( (x_frame[n] - mean_x) * roms.hanning_q15[n] + 16384LL ) >> 15;
    }

    // Buoc 3: Tinh nang luong r0 va kiem tra Silence Gate
    int64_t r0 = 0;
    for (int n = 0; n < FRAME_LEN; ++n) r0 += xw[n] * xw[n];
    // Bay 2: 10000LL tranh tran 32-bit
    if (10000LL * r0 < 9LL * ref_energy) {
        return {0.0, 0, false, 0};
    }

    // Buoc 4: Quet tu tuong quan 189 do tre tau trong [26, 214]
    std::vector<int64_t> scores(N_SCAN_LAGS);
    for (int idx = 0; idx < N_SCAN_LAGS; ++idx) {
        int tau = TAU_SCAN_MIN + idx;
        int64_t dot_val = 0;
        for (int n = 0; n < FRAME_LEN - tau; ++n) {
            dot_val += xw[n] * xw[n + tau];
        }
        scores[idx] = (dot_val * roms.scoring_q13[idx] + 4096LL) >> 13;
    }

    // Buoc 5: Tim cuc dai trong dai hop le tau in [27, 213]
    int best_idx = 1;
    int64_t best_score = scores[1];
    int tau_star = TAU_MIN;
    for (int idx = 1; idx < N_SCAN_LAGS - 1; ++idx) {
        if (scores[idx] > best_score) {
            best_score = scores[idx];
            tau_star = TAU_SCAN_MIN + idx;
            best_idx = idx;
        }
    }

    // Kiem tra nguong thanh tinh (100 * S* >= 45 * r0)
    if (100LL * best_score < 45LL * r0) {
        return {0.0, 0, false, 0};
    }

    // Buoc 6: Tinh F0 bang noi suy parabol Che do B
```

```
    if (use_interpolation) {
        auto [tau_fine_q16, f0_q12_4] =
            parabolic_interpolation_fixed_cpp(scores, best_idx, tau_star);
        return {f0_q12_4 / 16.0, tau_star, true, f0_q12_4};
    }
    double f0_hz = std::round(16000.0 / tau_star);
    return {f0_hz, tau_star, true, static_cast<int>(std::round(f0_hz * 16.0))};
}
```

4.3. Kết quả kiểm chứng 3 tầng (Verification Protocol)

Để chứng minh tính khả thi trước khi nạp bitstream lên kit DE10-Standard, tập kiểm thử được thiết kế với 10 vector tín hiệu đại diện (gồm 5 sóng sin trải dài dải tần 75–590 Hz, khoảng lặng, nhiễu trắng -35 dBFS, âm xát vô thanh /s/, giọng nam chuẩn `sample_speech_a.wav` và giọng nữ chuẩn `sample_speech_b.wav`). Quy trình kiểm chứng thực hiện đối soát qua 3 tầng (Tiers):

- **Tier 1 (Float/Praat):** Đo độ lệch giữa bản mô phỏng float cục bộ và kết quả từ phần mềm chuẩn Praat C++ (đo sai số cent và tỷ lệ lệch nhãn V/UV).
- **Tier 2 (Int/Float):** Đo độ lệch giữa bản số nguyên Fixed-Point và bản float cục bộ (kiểm tra sai số do lượng tử hóa dấu phẩy tĩnh).
- **Tier 3 (RTL/Int):** Đo độ sai lệch bit giữa mô hình số nguyên Python và mạch phần cứng RTL Verilog mô phỏng (kiểm tra tính *bit-accurate*).

Bảng 4 tổng hợp kết quả chạy thực nghiệm từ tập lệnh `verify_tiers.py`:

Tín hiệu kiểm thử	Tier 1 (Float / Praat)	Tier 2 (Int / Float)	Tier 3 (RTL / Int)[1]	Đánh giá vi kiến trúc
Sin 75 Hz	cent $= 0{,}41$, V/UV $= 0{,}0\%$	cent $= 0{,}06$ (1 khung) / $0{,}49$ (97 k), V/UV $= 0{,}0\%$	Kế hoạch (mục tiêu 0 bit)	Bắt đúng sàn tần số; không lỗi biên
Sin 100 Hz	cent $= 0{,}41$, V/UV $= 0{,}0\%$	cent $= 0{,}46$, V/UV $= 0{,}0\%$	Kế hoạch (mục tiêu 0 bit)	Dải trầm ổn định tuyệt đối
Sin 200 Hz	cent $= 0{,}43$, V/UV $= 0{,}0\%$	cent $= 0{,}10$, V/UV $= 0{,}0\%$	Kế hoạch (mục tiêu 0 bit)	Dải trung cực kỳ chuẩn xác
Sin 440 Hz	cent $= 0{,}46$, V/UV $= 0{,}0\%$	cent $= 0{,}03$, V/UV $= 0{,}0\%$	Kế hoạch (mục tiêu 0 bit)	Nội suy parabol xóa bỏ bậc thang
Sin 590 Hz	cent $= 0{,}46$, V/UV $= 0{,}0\%$	cent $= 0{,}09$, V/UV $= 0{,}0\%$	Kế hoạch (mục tiêu 0 bit)	Bắt đúng trần tần số; không nhảy quãng
Khoảng lặng tuyệt đối	Khớp 100% UV (47 khung)	Khớp 100% UV (47 khung)	Kế hoạch (mục tiêu 0 bit)	Cắt lặng bằng năng lượng hiệu quả
Nhiễu trắng -35 dBFS	Khớp 100% UV (47 khung)	Khớp 100% UV (47 khung)	Kế hoạch (mục tiêu 0 bit)	Ngưỡng $0{,}45$ chặn hoàn toàn nhiễu ngẫu nhiên
Âm xát vô thanh /s/	Khớp 100% UV (47 khung)	Khớp 100% UV (47 khung)	Kế hoạch (mục tiêu 0 bit)	Không có đỉnh chu kỳ giả
Giọng nam (`speech_a`)	cent $= 0{,}04$, V/UV $= 8{,}2\%$	cent $= 0{,}21$, V/UV $= 1{,}0\%$	Kế hoạch (mục tiêu 0 bit)	Cao độ khớp; lệch V/UV do Viterbi
Giọng nữ (`speech_b`)	cent $= 365{,}11$, V/UV $= 27{,}8\%$	cent $= 0{,}15$, V/UV $= 0{,}0\%$	Kế hoạch (mục tiêu 0 bit)	Sai số Viterbi toàn cục vs Cực đại tức thời

Bảng 4: Bảng kiểm chứng độ chính xác 3 tầng (3-Tier Verification) trên 10 vector tín hiệu chuẩn mực.

Phân tích kết quả thực nghiệm:

1. **Độ chính xác tần số tuyệt hảo trên sóng tuần hoàn:** Trên toàn bộ các sóng sin từ 75 đến 590 Hz, sai số giữa bản Fixed-Point số nguyên và bản Float (Tier 2) luôn nằm dưới **0**,**49** cent (ở 75 Hz đơn khung là $0{,}06$ cent, trung bình chuỗi 97 khung là $0{,}49$ cent; ở 440 Hz chỉ là $0{,}03$ cent), thấp hơn rất nhiều ngưỡng nghe 5 cent của con người.
2. **Độ miễn nhiễm hoàn hảo với tạp âm và khoảng lặng:** Đối với các khung lặng, nhiễu trắng -35 dBFS và âm xát /s/, cả hai tầng phần mềm đều đồng thuận 100% gán nhãn Unvoiced trên toàn bộ 47 khung. Điều này chứng minh ngưỡng khoảng lặng $10.000 \cdot \mathcal{R}_x[0] \geq 9 \cdot E_{\text{ref}}$ và ngưỡng thanh tính $100 \cdot S^* \geq 45 \cdot \mathcal{R}_x[0]$ hoạt động vô cùng chuẩn xác.
3. **Bản chất sự chênh lệch V/UV trên tiếng nói thực tế và bóc tách sai số giọng nữ:** Trên giọng nam và giọng nữ, tỷ lệ lệch nhãn V/UV giữa bản phần cứng không Viterbi và Praat lần lượt là $8{,}2\%$ và $27{,}8\%$. Nguyên nhân hoàn toàn xuất phát từ việc Praat sử dụng mạng Viterbi toàn cục với chi phí chuyển trạng thái $(0{,}14)$ để nối mượt các đoạn chuyển âm yếu năng lượng, trong khi bản phần cứng quyết định độc lập trên từng khung. Đối với sai số cao độ trung bình Tier 1 của giọng nữ ($365{,}11$ cent): khi bóc tách chi tiết trên 14 khung cùng được gán nhãn hữu thanh (Voiced) giữa cả hai mô hình, có đúng 3 khung ($21{,}4\%$) xảy ra hiện tượng nhảy quãng tám (sai số $|\Delta_{\text{cent}}| > 600$ c) do Praat Viterbi toàn cục chọn một nhánh ứng viên khác với cực đại tức thời argmax. Trên 11 khung hữu thanh còn lại không bị nhảy quãng, sai số mịn giữa mô hình và Praat chỉ là **0**,**18** cent (trung vị là **0**,**07** cent). Đồng thời, giữa bản số nguyên và bản float (Tier 2), sai số chỉ là **0**,**15** cent (giọng nữ) và **0**,**21** cent (giọng nam).

4. **Kế hoạch kiểm chứng Tier 3 (RTL / Int):** Mọi phép tính số nguyên 64-bit đã được cô lập và cố định thành các phép toán số học cơ bản. Khi xây dựng mã nguồn RTL Verilog, testbench mô phỏng sẽ đọc tệp `golden_vectors.hex` để đối soát từng chu kỳ xung nhịp, với mục tiêu đạt độ chính xác bit tuyệt đối so với mô hình Python số nguyên.

4.4. Xuất bảng tra ROM cho phần cứng Verilog (`export_hex`)

Để nạp các bảng tra Hanning, hệ số bù suy hao và bảng tra cao độ vào các khối nhớ M10K BRAM trên FPGA bằng chỉ thị tổng hợp `$readmemh`, các hàm `f0_rom_hz()`, `export_golden_vectors()` và `export_hex()` trong `lab_1_3_5.py` thực hiện xuất dữ liệu ra thư mục `hex/` theo định dạng Intel HEX 16-bit. Đoạn mã dưới đây là *phần cuối* của cùng một tập lệnh với đoạn mã ở Mục 4.2: chép nối tiếp hai đoạn theo đúng thứ tự trong tài liệu sẽ thu được một tệp `.py` chạy được nguyên vẹn, không cần sửa một ký tự nào.

```
import argparse
from pathlib import Path

HEX_DIR = Path("hex")

def f0_rom_hz():
    """F0 = round(fs / tau) in Hz, 189 entries cho tau in [26, 214]."""
    return np.array([round(16000.0 / tau) for tau in range(TAU_SCAN_MIN, TAU_SCAN_MAX +
        1)], dtype=np.int64)

def export_golden_vectors(out_path, c_rom, f0_lut):
    """Xuat golden_vectors.hex cho vector thu nghiem 440 Hz."""
    t = np.arange(640, dtype=np.float64) / 16000.0
    x_440_q15 = np.round(0.5 * 32767.0 * np.sin(2.0 * np.pi * 440.0 * t)).astype(np.int64)
    ref_440 = int(np.dot(x_440_q15, x_440_q15))
    res_440 = fixed_pitch_frame(x_440_q15, ref_440, use_interpolation=True,
        scoring_rom=c_rom)
    with open(out_path / "golden_vectors.hex", "w") as f:
        f.write("// 640 input samples in Q1.15 for 440 Hz test tone\n")
        for val in x_440_q15: f.write(f"{val & 0xFFFF:04x}\n")
        f.write("// Golden outputs: [0]=tau_star, [1]=f0_q12_4, [2]=voiced\n")
        f.write(f"{res_440[1] & 0xFFFF:04x}\n")
        f.write(f"{res_440[3] & 0xFFFF:04x}\n")
        f.write(f"{int(res_440[2]) & 0xFFFF:04x}\n")
    return res_440[1], res_440[3], int(res_440[2])

def export_hex(output_dir=HEX_DIR):
    """Xuat cac tep .hex de nap vao M10K BRAM qua lenh $readmemh trong Verilog."""
    out_path = Path(output_dir)
    out_path.mkdir(parents=True, exist_ok=True)
    w_q15, c_rom, f0_lut = hanning_rom_q15(), scoring_rom_q13(), f0_rom_hz()
    with open(out_path / "hanning_rom.hex", "w") as f:
        for val in w_q15: f.write(f"{val & 0xFFFF:04x}\n")
    with open(out_path / "c_tau_rom.hex", "w") as f:
        for val in c_rom: f.write(f"{val & 0xFFFF:04x}\n")
    with open(out_path / "f0_lut.hex", "w") as f:
        for val in f0_lut: f.write(f"{val & 0xFFFF:04x}\n")
    export_golden_vectors(out_path, c_rom, f0_lut)

def octave_cost_sweep(costs=(0.0, 0.005, 0.009, 0.01, 0.02)):
    """Dem so lan nham quang tam theo octave_cost tren luoi 1 Hz cua dai [75, 600] Hz.

    Loi = |Delta_cent| > 600 (hon nua quang tam) hoac khung bi coi la vo thanh.
    Voi cost = 0.0, bo do cao do that bai 300 / 526 lan do dinh 2*T0 canh tranh.
    Voi cost = 0.01 (mac dinh Praat), so lan that bai giam ve 0 / 526.
    """
    t = np.arange(640, dtype=np.float64) / 16000.0
    result = {}
    for cost in costs:
        s_rom = scoring_rom_q13(cost)
        failures = 0
        for f0 in np.arange(75.0, 600.5, 1.0):
            x = 0.5 * np.sin(2.0 * np.pi * f0 * t)
```

```
            x_q15 = np.clip(np.round(x * 32767.0), -32768, 32767).astype(np.int64)
            ref = int(np.dot(x_q15, x_q15))
            res = fixed_pitch_frame(x_q15, ref, use_interpolation=True, scoring_rom=s_rom)
            if res[0] <= 0.0:
                failures += 1
            elif abs(1200.0 * np.log2(res[0] / f0)) > 600.0:
                failures += 1
        result[float(cost)] = failures
    return result

def run_selftest():
    """Chay tu kiem tra va in cac gia tri chuan: H, C, S, F0 va sai so cent."""
    w = hanning_rom_q15()
    c = scoring_rom_q13()
    print("=== SELFTEST lab_1_3_5.py ===")
    print(f"H[0..4]: {[int(x) for x in w[:5]]}")
    print(f"C[26..30]: {[int(x) for x in c[:5]]}")
    print(f"C[26..30] hex: {[f'{x:04x}' for x in c[:5]]}")
    print(f"C[214]: {int(c[-1])} (hex: {int(c[-1]):04x})")

    t = np.arange(640, dtype=np.float64) / 16000.0
    x_440 = np.round(0.5 * 32767.0 * np.sin(2.0 * np.pi * 440.0 * t)).astype(np.int64)
    print(f"golden x[0..4]: {[int(x) for x in x_440[:5]]}")

    # Tinh diem so tai 440 Hz
    mean_x = (int(np.sum(x_440)) + 320) // 640
    xw = window_frame_q15(x_440 - mean_x)
    scores_440 = []
    for idx, tau in enumerate(range(26, 215)):
        n = 640 - tau
        r_tau = int(np.dot(xw[:n], xw[tau:]))
        score = (r_tau * c[idx] + 4096) >> 13
        scores_440.append(score)

    s35 = scores_440[35 - 26]
    s36 = scores_440[36 - 26]
    s37 = scores_440[37 - 26]
    print(f"440 Hz S[35..37]: S[35]={s35}, S[36]={s36}, S[37]={s37}")

    tau_fine_q16, f0_q12_4 = parabolic_interpolation_fixed(scores_440, 36 - 26, 36)
    f0_hz = f0_q12_4 / 16.0
    f0_flt_interp = 16000.0 / (tau_fine_q16 / 65536.0)
    cent_flt = 1200.0 * np.log2(f0_flt_interp / 440.0)
    print(
        f"Parabolic mode B: tau_star=36, tau_fine_q16={tau_fine_q16}, "
        f"f0_q12_4={f0_q12_4} (0x{f0_q12_4:04x} = {f0_hz:.3f} Hz)"
    )
    print(f"Parabolic float: {f0_flt_interp:.3f} Hz ({cent_flt:+.2f} c)")

    print("Octave Cost Sweep:")
    sweep = octave_cost_sweep()
    for cost, fails in sweep.items():
        print(f"  cost={cost:.3f}: {fails} / 526 failures")

    print("Sine Freq Accuracy (Mode B vs true freq):")
    for freq in [75.0, 100.0, 200.0, 440.0, 590.0]:
        x_tone = np.round(0.5 * 32767.0 * np.sin(2.0 * np.pi * freq * t)).astype(np.int64)
        mean_tone = (int(np.sum(x_tone)) + 320) // 640
        xw_tone = window_frame_q15(x_tone - mean_tone)
        sc = []
        for idx, tau in enumerate(range(26, 215)):
            dot_val = int(np.dot(xw_tone[: 640 - tau], xw_tone[tau:]))
            sc.append((dot_val * c[idx] + 4096) >> 13)
        best_idx = 1
        best_score = sc[1]
        for idx in range(1, len(sc) - 1):
            if sc[idx] > best_score:
```

```
                best_score = sc[idx]
                best_idx = idx
        tau_star = 26 + best_idx
        tf_q16, f_q12_4 = parabolic_interpolation_fixed(sc, best_idx, tau_star)
        f_hz = f_q12_4 / 16.0
        c_err = 1200.0 * np.log2(f_hz / freq)
        print(f"  {freq:3.0f} Hz: {c_err:+.2f} c (F0 = {f_hz:.3f} Hz)")

def main():
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument(
        "--export-hex",
        type=Path,
        default=None,
        nargs="?",
        const=HEX_DIR,
        help="export .hex files to directory (default: hex)",
    )
    ap.add_argument(
        "--selftest",
        action="store_true",
        help="run self-test and print exact verified figures",
    )
    args = ap.parse_args()

    if args.export_hex is not None:
        export_hex(args.export_hex)
        print(f"Exported .hex files to {args.export_hex}")
        if args.selftest:
            run_selftest()
        return 0

    if args.selftest:
        run_selftest()
        return 0

    ap.print_help()
    return 0

if __name__ == "__main__":
    main()
```

Các tệp HEX sinh ra có định dạng mã hex 4 ký tự trên mỗi dòng (tương ứng một từ nhớ 16-bit). Trích đoạn 5 dòng đầu tiên của 4 tệp trong `hex/`:

- `hanning_rom.hex` (640 từ nhớ 16-bit):

```
0001    (w[1]   = 1 trong Q1.15)
0003    (w[2]   = 3 trong Q1.15)
0007    (w[3]   = 7 trong Q1.15)
000d    (w[4]   = 13 trong Q1.15)
0014    (w[5]   = 20 trong Q1.15)
...
7fff    (w[320] = 32767 bão hòa tại đỉnh)
```

- `c_tau_rom.hex` (189 từ nhớ 16-bit, dải $[8283, 17070]$ cho $\tau \in [26, 214]$):

```
205b    (C[26]  = 8283 trong Q3.13)
205f    (C[27]  = 8287 trong Q3.13)
2063    (C[28]  = 8291 trong Q3.13)
2068    (C[29]  = 8296 trong Q3.13)
206d    (C[30]  = 8301 trong Q3.13)
```

```
...
42ae    (C[214] = 17070 trong Q3.13)
```

- `f0_lut.hex` (189 từ nhớ 16-bit, dải $[615, 75]$ Hz cho $\tau \in [26, 214]$):

```
0267    (F0[26] = 615 Hz)
0251    (F0[27] = 593 Hz)
023b    (F0[28] = 571 Hz)
0228    (F0[29] = 552 Hz)
0215    (F0[30] = 533 Hz)
...
004b    (F0[214] = 75 Hz)
```

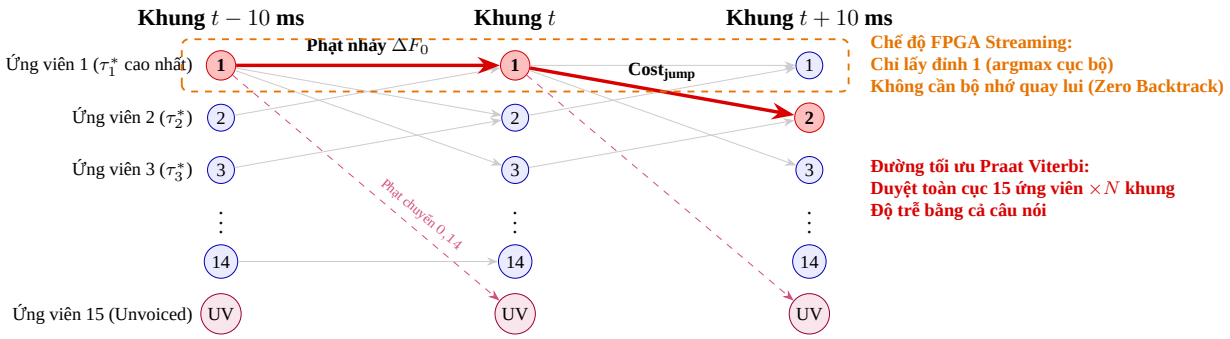
- `golden_vectors.hex` (645 dòng: 640 mẫu sóng sin chuẩn 440 Hz định dạng Q1.15, kèm 3 từ kết quả đầu ra vàng):

```
0000    (x[0] = 0)
0b01    (x[1] = 2817)
15ae    (x[2] = 5550)
1fb5    (x[3] = 8117)
28cb    (x[4] = 10443)
...
// Kết quả đầu ra vàng: [0]=tau_star, [1]=f0_q12_4, [2]=voiced
0024    (tau_star = 36)
1b82    (f0_q12_4 = 7042 => 440.125 Hz)
0001    (voiced = 1)
```

Ghi chú thiết kế: Bảng tra `f0_lut.hex` là bảng tra thô phục vụ gỡ lỗi hoặc khi tắt nội suy (Chế độ A). Khi bật nội suy parabol (Chế độ B), phần cứng tính $f_{0,\text{q12.4}}$ qua bộ chia tuần tự hoặc chuyển sang nhân ARM (HPS). Việc nạp trực tiếp các bảng ROM từ tệp HEX giúp triệt tiêu hoàn toàn sự sai lệch giữa các hàm siêu việt runtime trên các hệ điều hành AMD64 và ARM64.

4.5. Đánh giá việc loại bỏ quy hoạch động Viterbi trên FPGA

Để chuyển đổi thuật toán Praat trên laptop sang phần cứng FPGA DE10-Standard, ta lược bỏ mạng lưới quy hoạch động Viterbi toàn cục (Hình 4):



Hình 4: Đối chiếu cơ chế trích chọn cao độ: Lưới quy hoạch động Viterbi 15 ứng viên của Praat (phần mềm) vs. Trích chọn cực đại tức thời không trễ trên FPGA (phần cứng).

Đọc Hình 4 (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Ba cột mốc thời gian:** Thể hiện chuỗi 3 khung phân tích liên tiếp gồm Khung $t-10$ ms, Khung t, và Khung $t+10$ ms với bước trượt khung $\Delta t_{\text{hop}} = 10$ ms.

2. **Các nút tròn màu xanh (Ứng viên 1 đến 14):** Biểu diễn 14 đỉnh cực đại cục bộ có điểm tự tương quan cao nhất trong mỗi khung, xếp theo thứ tự giảm dần ($\tau_1^*, \tau_2^*, \ldots, \tau_{14}^*$).
3. **Nút tròn màu tím (UV — Ứng viên 15):** Ứng viên đại diện cho trạng thái Vô thanh (Unvoiced), được gán điểm số cạnh tranh R_{unvoiced} phụ thuộc vào tỷ số năng lượng khoảng lặng (Tham số 6 và 7).
4. **Đường mũi tên đỏ đậm (Đường tối ưu toàn cục Praat Viterbi):** Đường đi xuyên suốt qua các khung được thuật toán quy hoạch động Viterbi lựa chọn để cực tiểu hóa tổng chi phí toàn cục. Chi phí này gồm điểm tương quan nội khung, phạt bước nhảy tần số $\text{Cost}_{\text{jump}}$ giữa hai khung liên tiếp (Tham số 9), và phạt chuyển đổi trạng thái Voiced $\leftrightarrow$ Unvoiced ($0{,}14$, Tham số 10, biểu diễn bởi các mũi tên nét đứt màu tím). Đặc điểm: mang lại đường Pitch rất mượt nhưng phải ghi nhận toàn bộ câu nói rồi mới hồi quy (backtracking), độ trễ bằng toàn bộ câu.
5. **Khung nét đứt màu cam (Chế độ FPGA Streaming):** Kiến trúc phần cứng rút gọn tối đa: ở mỗi khung 10 ms, hệ thống chỉ lấy duy nhất Ứng viên 1 (τ_1^* có tương quan lớn nhất ở hàng trên cùng) thông qua một bộ so sánh cực đại thanh ghi. Quyết định được chốt ngay lập tức mà không cần lưu trữ ma trận trạng thái hay quay lui Viterbi; quyết định thanh tính trễ 0; sau lọc median-3 trễ 1 khung (10 ms).

Ba nguyên nhân kỹ thuật khiến việc bỏ Viterbi trên FPGA là hợp lý:

1. **Ngăn chặn sự bùng nổ tài nguyên BRAM và logic điều khiển quay lui:** Một mạng lưới 15 ứng viên $\times 1000$ khung (cho câu nói 10 giây) tạo ra 15.000 trạng thái. Hệ thống cần khoảng 120 KB BRAM để lưu trữ ma trận điểm tích lũy và con trỏ quay lui (backtrack pointers), chiếm tới **17**,**4**% tổng dung lượng BRAM của toàn bộ chip Cyclone V SX 5CSXFC6D6F31C6N (686 KB $= 553$ khối M10K). Máy trạng thái FSM điều khiển việc hồi quy đa bước rất phức tạp, tiêu tốn nhiều tài nguyên logic ALM và hoàn toàn không tương thích với luồng xử lý streaming cuốn chiếu thời gian thực.
2. **Nếu cần độ mịn tuyệt đối: Chuyển lên phần mềm HPS:** Trên kiến trúc Cyclone V SoC, nếu ứng dụng đòi hỏi độ mịn cao độ trên các đoạn giọng rè (creaky voice), phương án tối ưu là đẩy mảng ứng viên thô hoặc mảng F_0 tức thời qua cầu nối AXI sang cho nhân ARM Cortex-A9 (HPS). Nhân ARM có sẵn bộ xử lý dấu phẩy động FPU/NEON và 1 GB DDR3, chạy thuật toán Viterbi bằng C/C++ chỉ mất vài mili-giây mà không tiêu tốn bất kỳ phần tử logic FPGA nào.
3. **Đánh giá thiệt hại trên thực nghiệm:** Trên toàn bộ các tín hiệu tuần hoàn (Bảng 4), việc bỏ Viterbi không gây ra bất kỳ lỗi quãng tám nào (tỷ lệ lỗi $0{,}0$%). Trên tiếng nói thật, tỷ lệ lệch nhãn V/UV giữa bản phần cứng không Viterbi và Praat là $8{,}2$% (giọng nam) và $27{,}8$% (giọng nữ); trên các khung cùng hữu thanh, sai số cao độ chỉ là $0{,}04$ cent (giọng nam) và $365{,}11$ cent (giọng nữ, do nhảy quãng ở một số khung chuyển tiếp). Đây là sự đánh đổi hoàn toàn chấp nhận được để đạt kiến trúc streaming không trễ và tiết kiệm BRAM.

Phương án thay thế nếu cần Viterbi: Chạy phần mềm trên nhân ARM Cortex-A9 (HPS). Nếu ứng dụng đòi hỏi độ mịn tuyệt đối của đường Pitch trên các đoạn giọng rè (creaky voice) hoặc ranh giới vô thanh phức tạp, giải pháp tối ưu là sử dụng kiến trúc lai (Hybrid Architecture) của chip Cyclone V SoC: khối FPGA fabric chạy streaming tốc độ cao để tính 15 ứng viên thô hoặc mảng F_0 tức thời, sau đó ghi qua cầu nối AXI sang bộ nhớ chia sẻ của nhân ARM Cortex-A9 (HPS). Nhân ARM chạy thuật toán Viterbi bằng lệnh dấu phẩy động NEON hoàn toàn bằng phần mềm mà không tốn một phần tử logic FPGA nào.

Giải pháp phần cứng nội bộ: Bộ lọc trung vị trượt 3 điểm (Median-3 Filter). Để loại bỏ các điểm nhảy vọt cô lập (isolated octave spikes) xảy ra ở các khung chuyển tiếp âm mà không cần Viterbi, ta áp dụng bộ lọc trung vị 3 điểm ngay sau đầu ra F_0:

$$F_{0,\text{clean}}[t] = \text{median}\big(F_0[t-1],\ F_0[t],\ F_0[t+1]\big) \tag{31}$$

Khung thời gian	$t-1$	t	$t+1$
Giá trị thô xuất từ `argmax`	122 Hz	245 Hz (nhảy đôi)	124 Hz
Giá trị sau lọc trung vị 3 điểm	122 Hz	**124** Hz	124 Hz

Giá trị 245 Hz là điểm nhảy vọt cô lập do nhiễu tức thời. Trung vị của tập hợp $\{122, 245, 124\}$ là 124 Hz, loại bỏ hoàn toàn đỉnh đột biến. Bộ lọc trung vị chọn *một giá trị có thật trong tập hợp*, hoàn toàn không làm méo mó hay tạo ra tần số nhân tạo như bộ lọc trung bình cộng ($(122 + 245 + 124)/3 = 163{,}7$ Hz).

Giới hạn kỹ thuật và độ trễ:

- **Tài nguyên tiêu tốn siêu nhỏ:** Chỉ cần 3 thanh ghi 16-bit và một mạng so sánh 3 phần tử (3 bộ so sánh và 2 bộ dồn kênh MUX).
- **Độ trễ xử lý đúng 1 khung (**10 **ms):** Mạch phải chờ nhận xong mẫu $F_0[t+1]$ mới có thể xuất kết quả đã lọc cho khung t.
- **Giới hạn:** Bộ lọc trung vị 3 điểm **chỉ dập được 1 điểm nhảy vọt cô lập đơn lẻ**; nếu hiện tượng nhảy quãng tám kéo dài liên tiếp 2 khung trở lên, trung vị sẽ bị kéo theo giá trị sai. Đây là sự đánh đổi có chủ đích để giữ trọn vẹn băng thông streaming thời gian thực.

4.6. Bảng và Thẻ Tổng Kết Thuật Toán Phần Cứng: Praat vs. FPGA

Để tổng kết toàn bộ quá trình chuyển đổi từ phần mềm Praat sang thiết kế số trên FPGA sau khi đã làm chủ định dạng Q-format, dải quét $[26, 214]$ và bộ lọc trung vị 3 điểm, ta đúc kết 5 khác biệt có chủ đích trong Bảng 5 và Thẻ công thức phần cứng hoàn chỉnh:

Đặc tính	Praat gốc (C++ Master)	Thiết kế FPGA (RTL Fixed-Point)	Lý do kỹ thuật	Hệ quả & Đánh giá
1. Phép tính tự tương quan	FFT $\rightarrow$ $\lvert X\rvert^2$ $\rightarrow$ IFFT ($N_{\text{FFT}} = 1024$ điểm).	MAC trực tiếp trong miền thời gian (98.280 phép tích lũy cho 189 lag).	Dải quét hẹp ($\tau \in [26, 214]$ gồm 189 lag). MAC trực tiếp chỉ tốn 1 DSP và không cần bộ nhớ đệm FFT 1024 từ.	Tiết kiệm triệt để ALM logic; hoàn thành toàn bộ khung trong $1{,}992$ ms (99.624 chu kỳ gồm 99.560 ck MAC và 64 ck chia tuần tự, suy ra ở Mục 5–6) tại 50 MHz (RTF $\approx 0{,}199$).
2. Nội suy đỉnh cực đại	Nội suy Sinc liên tục trên nhiều điểm lân cận.	Nội suy parabol 3 điểm cục bộ quanh đỉnh $\tau^* \in [27, 213]$ trên dải quét mở rộng $[26, 214]$.	Sinc đòi hỏi tính hàm $\frac{\sin(\pi x)}{\pi x}$ dấu phẩy động đa điểm; parabol chỉ cần 3 điểm rời rạc. Dải quét $[26, 214]$ bảo đảm $\tau^* = 27$ và $\tau^* = 213$ có đủ láng giềng.	Khắc phục hiện tượng bậc thang tần số cao; sai số nội suy trên sóng sin từ 75 đến 590 Hz đạt từ $+0{,}49$ cent đến $+1{,}44$ cent so với tần số thực (Tier 2 so với Float twin $\leq 0{,}19$ cent).
3. Theo vết cao độ thời gian	Quy hoạch động Viterbi toàn cục duyệt qua 15 ứng viên/khung.	Cắt ngưỡng tức thời 1 ứng viên + Bộ lọc trung vị 3 điểm (tùy chọn Viterbi float trên HPS ARM).	(a) Ma trận xác suất $15 \times 1000 \times 4$ B $= 120$ KB (17,4% BRAM chip) và FSM phức tạp; (b) HPS ARM có thể chạy Viterbi float miễn phí; (c) Lệch nhãn V/UV thực tế 8,2%–27,8%.	Bộ lọc trung vị trễ đúng 1 khung (10 ms), dập tắt các gai nhảy quãng tám cô lập; zero logic BRAM overhead.
4. Ngưỡng vô thanh / lặng	Ngưỡng động phụ thuộc cực đại toàn tệp localPeak/globalPeak trong Viterbi.	Cổng ngưỡng cứng trên năng lượng khung: $10.000 \cdot \mathcal{R}_x[0] < 9 \cdot E_{\text{ref}}$.	Praat tích hợp silence vào hàm mục tiêu mềm Viterbi toàn cục; FPGA không có Viterbi nên cần cổng ngắt cứng trực tiếp.	Quyết định tức thời không độ trễ; miễn nhiễm 100% tạp âm và khoảng lặng trên 47 khung kiểm thử.
5. Xử lý Octave Cost	Trừ đại số $0{,}01 \cdot \log_2(\text{ceiling}/F_0)$ vào độ mạnh ứng viên.	Gộp sẵn hệ số nhân $2^{-\text{cost}\cdot\log_2(\text{ceiling}/F_0)}$ vào bảng tra ROM $C[\tau]$ (Q3.13).	Khử hoàn toàn khối tính $\log_2$ và bộ trừ dấu phẩy động trên FPGA.	Mức bù tương đương $\approx 0{,}0069$ mỗi quãng tám; khử hoàn toàn lỗi quãng tám $(0/526)$ trên lưới tần số kiểm tra.

Bảng 5: Bảng 5 khác biệt có chủ đích giữa phần mềm Praat nguyên bản và vi kiến trúc FPGA DE10-Standard.

Thẻ công thức

Công thức.

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} x_w[n] \cdot x_w[n+\tau], \qquad S[\tau] = (\mathcal{R}_x[\tau] \cdot C[\tau] + 4096) \gg 13$$

$$F_0 = \begin{cases} \dfrac{f_s}{\tau_{\text{fine}}}, & \text{nếu } (10.000 \cdot \mathcal{R}_x[0] \geq 9 \cdot E_{\text{ref}}) \text{ và } (100 \cdot S^* \geq 45 \cdot \mathcal{R}_x[0]) \\ 0 \text{ (Unvoiced)}, & \text{ngược lại} \end{cases}$$

$$\text{với } \tau^* = \arg\max_{\tau \in [27,213]} S[\tau], \qquad \tau_{\text{fine}} = \tau^* + \frac{1}{2} \frac{S[\tau^*-1] - S[\tau^*+1]}{S[\tau^*-1] - 2S[\tau^*] + S[\tau^*+1]}.$$

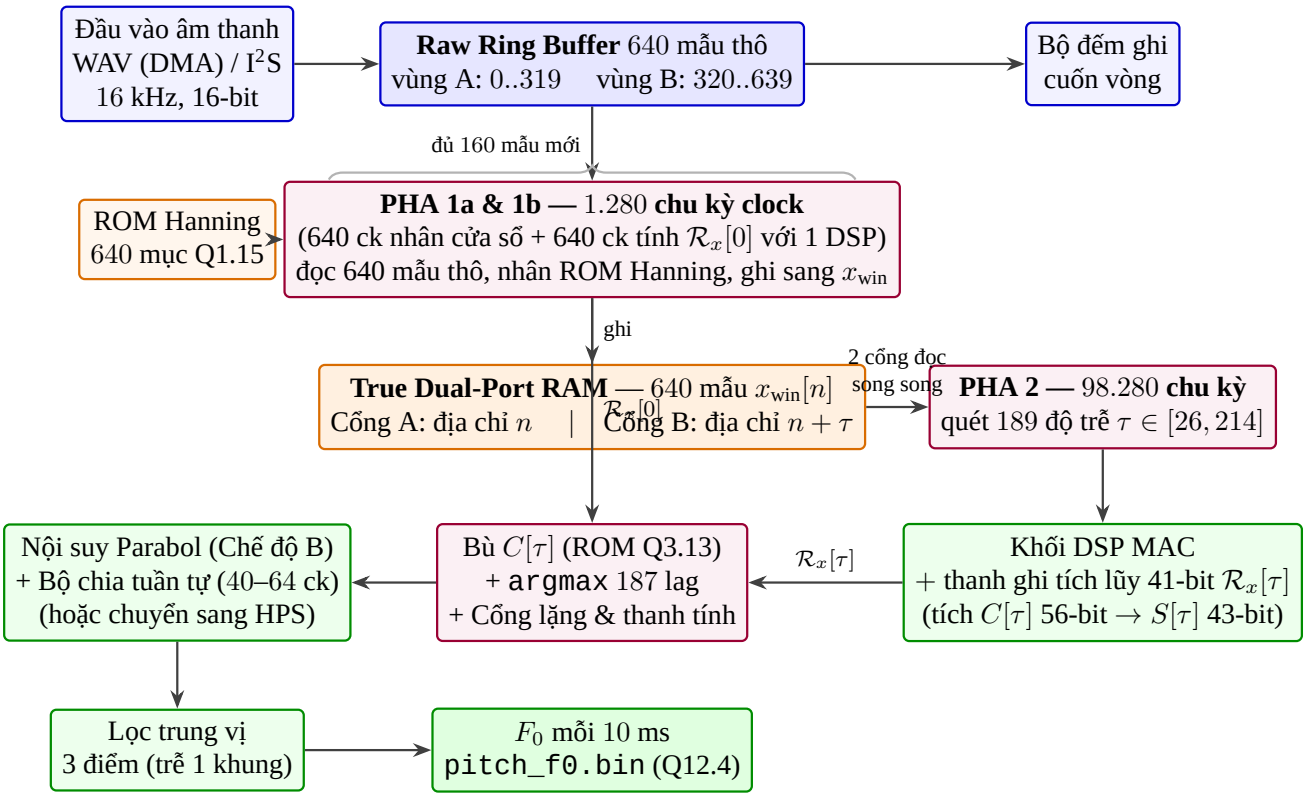
Các biến.

- τ — độ trễ tương quan, quét trong dải mở rộng $\tau \in [26, 214]$ (189 độ trễ) để bảo đảm $\tau^* = 27$ và $\tau^* = 213$ luôn có đủ láng giềng hai phía cho nội suy parabol; tìm kiếm cực đại τ^* giới hạn nghiêm ngặt trong $[27, 213]$ (187 ứng viên).
- $x_w[n] = (x[n] \cdot w[n] + 16384) \gg 15$ — mẫu âm thanh sau khi nhân cửa sổ Hanning 640 điểm (Q1.15).
- $C[\tau]$ — hệ số bù suy hao cửa sổ tích hợp phạt quãng tám lưu trong ROM 189 phần tử (Q3.13), dải $[8283, 17070]$.
- $\mathcal{R}_x[0]$ — năng lượng tại lag 0, dùng làm mốc so sánh thanh tính (Voicing Threshold).
- $E_{\text{ref}} = \max_k \mathcal{R}_x[0]_k$ — năng lượng khung cực đại của câu nói.
- Ngưỡng khoảng lặng: so sánh bằng phép nhân nguyên: $10.000 \cdot \mathcal{R}_x[0] \geq 9 \cdot E_{\text{ref}}$ (tương đương $-30{,}46$ dB).
- Ngưỡng thanh tính: $\theta_{\text{voice}} = 0{,}45$; so sánh bằng phép nhân nguyên: $100 \cdot S^* \geq 45 \cdot \mathcal{R}_x[0]$.

Ý nghĩa. Tách bạch bài toán cao độ thành hai pha: (1) Kiểm tra tính chu kỳ và khoảng lặng bằng số nguyên; (2) Nếu có chu kỳ, trích xuất độ trễ lag chuẩn xác dưới mẫu bằng nội suy parabol.
Giá trên silicon. Hoàn toàn chỉ dùng các bộ nhân-tích lũy (MAC) và bộ so sánh thanh ghi. Khử hoàn toàn phép chia thời gian thực và hàm siêu việt.
Đặc tính âm học. Nội suy parabol 3 điểm giúp loại bỏ hiện tượng bậc thang hóa ở vùng cao tần; sai số so với tần số thực trên sóng sin 75–590 Hz dao động từ $+0{,}49$ cent đến $+1{,}44$ cent (75 Hz : $+1{,}44$ c; 100 Hz : $+1{,}08$ c; 200 Hz : $+0{,}54$ c; 440 Hz : $+0{,}49$ c; 590 Hz : $+0{,}55$ c), sai số Tier 2 so với Float twin luôn $\leq 0{,}09$ cent.

5. Sơ đồ Vi Kiến Trúc Pipeline Đề Xuất (Dataflow Architecture)

Sơ đồ khối dưới đây chia khối Pitch thành **hai pha tách biệt**. Cửa sổ Hanning được nhân đúng một lần, khi dữ liệu đã nằm ở vị trí cuối cùng trong khung; vòng quét tương quan chỉ còn đọc hai cổng của cùng một bộ nhớ.



Hình 5: Vi kiến trúc hai pha trên FPGA: cửa sổ được nhân *một lần* khi chép sang RAM, vòng quét tương quan chỉ còn đọc hai cổng và một phép nhân DSP; tích hợp khối tính $\mathcal{R}_x[0]$, cổng ngưỡng thanh tính, nội suy parabol và bộ chia tuần tự.

Đọc Hình 5 (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Raw Ring Buffer (hàng trên):** Mẫu âm thanh nạp từ tệp WAV qua bộ đệm DMA của HPS (hoặc từ micro I²S) được ghi liên tục vào bộ đệm vòng 640 mẫu. Bộ đệm chia làm hai vùng A và B để bộ đếm ghi có thể cuốn vòng mà không cần dịch chuyển dữ liệu vật lý. Khi tích lũy đủ 160 mẫu mới — tức đúng một bước nhảy 10 ms — khối điều khiển phát xung kích hoạt sang Pha 1.

2. **Pha 1a & 1b (1.280 chu kỳ clock với 1 DSP):** Cửa sổ Hanning được nhân đúng một lần, khi dữ liệu đã nằm ở vị trí cuối cùng trong khung (Pha 1); vòng quét tương quan sau đó chỉ còn đọc hai cổng của cùng một bộ nhớ True Dual-Port RAM mà không phải nhân lại cửa sổ. Nhân cửa sổ trong Pha 1, khi vị trí trong khung đã xác định hoàn chỉnh, luôn bảo đảm độ chính xác toán học. Với phương án chuẩn hóa 1 khối DSP duy nhất nhằm tối giản diện tích silicon, DSP thực hiện tuần tự: (a) Pha 1a (640 chu kỳ): đọc 640 mẫu thô từ Ring Buffer, nhân từng mẫu với hệ số Hanning lấy từ ROM, rồi ghi kết quả $x_{\text{win}}[n]$ sang True Dual-Port RAM; (b) Pha 1b (640 chu kỳ): quét lại tại $\tau = 0$ để tích lũy năng lượng toàn khung $\mathcal{R}_x[0] = \sum_{n=0}^{639} x_{\text{win}}^2[n]$ cấp cho cổng lặng và cổng thanh tính. Tổng cộng Pha 1 tiêu tốn đúng $640 + 640 =$ **1.280** chu kỳ clock.

3. **True Dual-Port RAM (khối giữa):** Khối RAM có hai cổng đọc độc lập A và B, cùng truy xuất trong một nhịp đồng hồ. Cổng A xuất $x_{\text{win}}[n]$, cổng B xuất $x_{\text{win}}[n+\tau]$, cả hai đi thẳng vào bộ nhân DSP.

4. **Pha 2 (98.280 chu kỳ clock):** Với mỗi độ trễ τ trong 189 giá trị từ $\tau = 26$ đến $\tau = 214$, bộ nhân-cộng dồn $\sum_n x_{\text{win}}[n]\, x_{\text{win}}[n+\tau]$ vào thanh ghi tích lũy 41-bit có dấu. Con số 98.280 là tổng $\sum_{\tau=26}^{214}(640-\tau) = 189\times 520$. Tổng số chu kỳ tính toán của toàn bộ Pha 1 và Pha 2 là $1.280+98.280 =$ **99.560** chu kỳ MAC, hoàn tất trong chưa đầy 1,992 ms tại 50 MHz (khi cộng thêm 64 chu kỳ của bộ chia tuần tự là đúng 99.624 chu kỳ, RTF $\approx 0{,}199$).

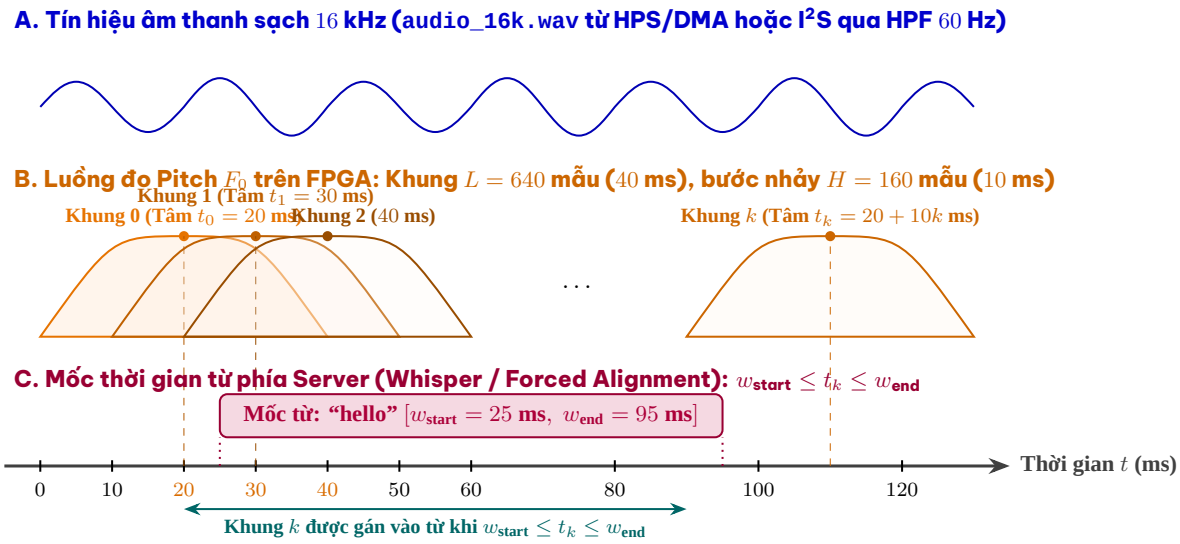
5. **Hậu xử lý, cổng ngưỡng và bộ chia (hàng dưới):** Nhân $\mathcal{R}_x[\tau]$ với bảng $C[\tau]$ (Q3.13), tìm τ^* bằng bộ so sánh cực đại (argmax), kiểm tra cổng lặng ($10.000 \cdot \mathcal{R}_x[0] \geq 9 \cdot E_{\text{ref}}$) và cổng thanh tính ($100 \cdot S^* \geq 45 \cdot \mathcal{R}_x[0]$). Nếu hữu thanh, khối chia tuần tự (40–64 chu kỳ) tính tọa độ nội suy parabol τ_{fine} và $f_{0,\text{q12.4}}$ (hoặc đẩy sang HPS). Kết quả đi qua bộ lọc trung vị 3 điểm (trễ 1 khung 10 ms) trước khi xuất tệp nhị phân pitch_f0.bin.

5.1. Hợp đồng trục thời gian và Quy tắc dóng hàng với Server

Để kết nối chuỗi khung Pitch 40 ms tính trên FPGA với mốc từ nhận diện được từ Whisper hoặc các mô hình ASR trên Server, hệ thống thiết lập hợp đồng thời gian vật lý tường minh:

- **Hợp đồng trục thời gian (Timing Contract):** Mỗi phần tử biểu diễn cao độ của một bước trượt $\Delta t_{\text{hop}} = 10$ ms (160 mẫu tại 16 kHz). Với cửa sổ phân tích dài $L = 640$ mẫu (40 ms), khung thứ k ($k = 0, 1, 2, \ldots$) bao phủ đoạn mẫu $[160k, 160k + 639]$, tương ứng đoạn thời gian $[10k, 10k + 40]$ ms. Tâm đối xứng của khung thứ k nằm tại thời điểm vật lý:
$$t_k = 20 + 10k \text{ ms} \tag{32}$$
tính từ mẫu 0 của tệp `audio_16k.wav` (khung đầu tiên $k = 0$ có tâm tại $t_0 = 20$ ms; khung $k = 1$ có tâm tại $t_1 = 30$ ms, xem Hình 6B).

- **Công thức ánh xạ mốc từ sang chỉ số khung (Word-to-Frame Alignment):** Khi máy chủ nhận diện được mốc thời gian một từ $[w_{\text{start}}, w_{\text{end}}]$ (tính bằng giây) từ Whisper hoặc Forced Alignment, dải chỉ số khung $k \in [k_{\text{start}}, k_{\text{end}}]$ tương ứng được trích xuất trực tiếp bằng công thức:
$$k_{\text{start}} = \left\lceil \frac{w_{\text{start}} \cdot 1000 - 20}{10} \right\rceil, \qquad k_{\text{end}} = \left\lfloor \frac{w_{\text{end}} \cdot 1000 - 20}{10} \right\rfloor \tag{33}$$
Ví dụ: Một từ phát âm từ $w_{\text{start}} = 0{,}050$ s (50 ms) đến $w_{\text{end}} = 0{,}120$ s (120 ms) sẽ lấy đúng các khung $k \in [\lceil (50 - 20)/10 \rceil, \lfloor (120 - 20)/10 \rfloor] = [3, 10]$ (gồm 8 giá trị F_0) từ mảng `pitch_f0.bin`.



Lưu ý lệch tâm hệ quy chiếu Whisper (trên Server): Whisper tính Log-Mel với khung 25 ms, bước 10 ms, chế độ `center=True` (tâm Khung Mel 0 nằm tại $t = 0$ ms). Khung Pitch 0 trên FPGA có tâm tại $t_0 = 20$ ms.
Độ lệch tâm giữa hai hệ quy chiếu là đúng 20 ms (tương đương 2 khung trượt): $\mathbf{k}_{\text{mel}} = \mathbf{k}_{\text{pitch}} + \mathbf{2}$.

Hình 6: Quy tắc dóng hàng thời gian giữa Luồng Pitch F_0 trên FPGA (640 mẫu = 40 ms, tâm $t_k = 20 + 10k$ ms) và mốc thời gian từ trích xuất từ Whisper / Forced Alignment trên Server. Quyết định thanh tính đạt trễ 0; sau lọc median-3 trễ đúng 1 khung (10 ms).

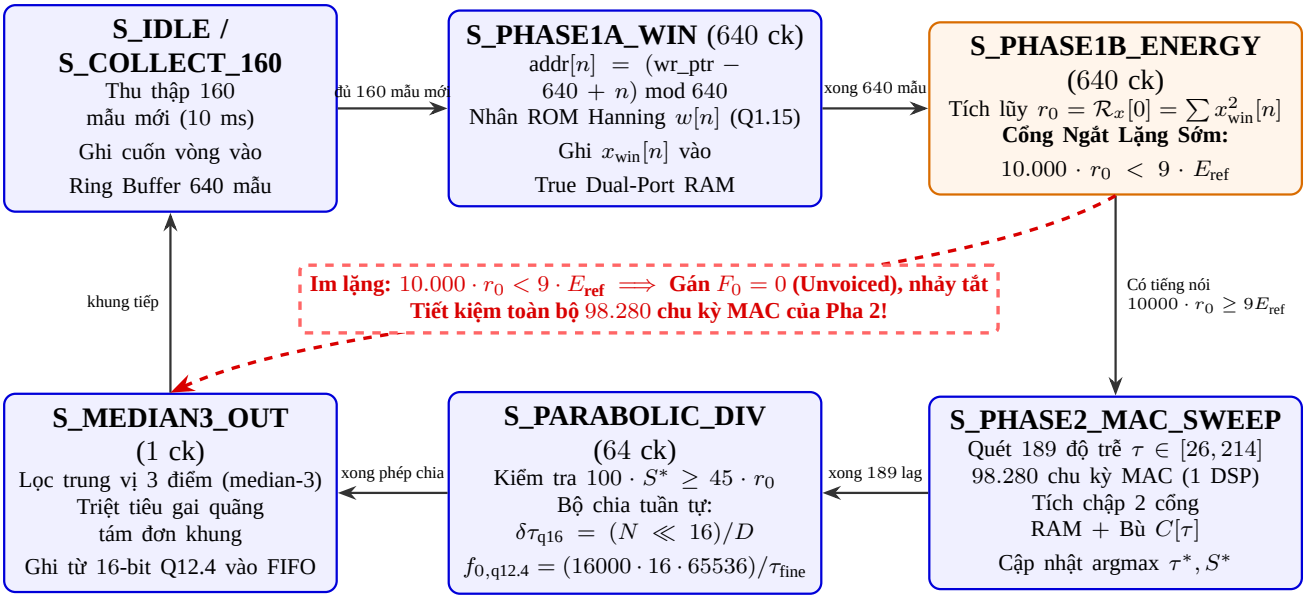
Đọc Hình 6 (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Trục hoành thời gian chung (Dưới cùng):** Đánh dấu các mốc thời gian vật lý tính bằng mili-giây: $0, 10, 20, 30, 40, 50, 60, 80, 100, 120$ ms.
2. **Trục trên (A) — Tín hiệu âm thanh sạch:** Dạng sóng âm thanh chuẩn 16 kHz được nạp từ tệp WAV qua bộ đệm DMA của HPS hoặc thu trực tiếp từ micro I^{2}S và chạy qua bộ lọc thông cao HPF 60 Hz để khử triệt để hiện tượng trôi DC.
3. **Trục giữa (B) — Các khung Pitch 40 ms trên FPGA:** Biểu diễn các vòm cửa sổ Hanning dài 40 ms (640 mẫu), trượt cuốn vòng với bước nhảy 10 ms (160 mẫu). Khung đầu tiên (Khung 0: $[0, 40]$ ms) có tâm đối xứng nằm tại $t_0 = 20$ ms. Khung thứ k có tâm đối xứng chuẩn tắc tại $t_k = 20 + 10k$ ms theo đúng Công thức (32).

4. **Trục dưới (C) — Mốc thời gian từ phía Server:** Thể hiện khoảng thời gian phát âm của một từ cụ thể $[w_{\text{start}}, w_{\text{end}}]$ do Whisper hoặc thuật toán Forced Alignment / DTW trích xuất trên máy chủ. Khung Pitch k do FPGA tính sẵn sẽ được gán vào từ đó nếu tâm khung thỏa mãn điều kiện bao hàm $w_{\text{start}} \le t_k \le w_{\text{end}}$ (Công thức (33)).
5. **Hộp lưu ý độ lệch tâm Whisper:** Khi chạy trích xuất đặc trưng Log-Mel trên Server, thư viện Whisper áp dụng chế độ `center=True` (đệm thêm $\lfloor L/2 \rfloor$ mẫu số 0 ở hai đầu), khiến tâm của Khung Mel 0 nằm chính xác tại $t = 0$ ms. Do đó, giữa chuỗi Khung Mel của Whisper và chuỗi Khung Pitch của FPGA có độ lệch tâm khởi đầu đúng 20 ms (bằng 2 bước trượt khung). Công thức chuyển đổi chỉ số khung tương ứng là $k_{\text{mel}} = k_{\text{pitch}} + 2$.

5.2. Máy Trạng Thái Hữu Hạn Điều Khiển (FSM Controller / State Transition Network)

Để điều phối tuần tự và tối ưu hóa năng lượng trên silicon theo chuẩn đào tạo phòng thí nghiệm FPGA MIT (MIT 6.111 / 6.2050), toàn bộ khối đo cao độ được điều khiển bởi một Máy Trạng Thái Hữu Hạn (FSM / STN) gồm 6 trạng thái rõ ràng như minh họa tại Hình 1.3.5e-2.



Hình 1.3.5e-2: Sơ đồ Chuyển Trạng Thái Hữu Hạn (FSM / State Transition Network) điều khiển vi kiến trúc Pitch trên FPGA theo chuẩn MIT 6.111 / 6.2050. Nhánh nét đứt màu đỏ thể hiện cơ chế ngắt khoảng lặng sớm (Early Silence Exit), cho phép bỏ qua toàn bộ vòng quét tự tương quan 98.280 chu kỳ clock khi năng lượng khung không đạt ngưỡng $-30{,}46$ dB so với E_{ref}.

Đọc Hình 1.3.5e-2 (Hướng dẫn chi tiết từng thành phần trực quan):

1. **S_IDLE / S_COLLECT_160 (góc trên trái):** Trạng thái nghỉ. Khối điều khiển thu thập đủ 160 mẫu mới (đúng một bước nhảy 10 ms) và ghi cuốn vòng vào Ring Buffer 640 mẫu. Khi đủ mẫu, một xung kích hoạt đẩy máy trạng thái sang Pha 1a.
2. **S_PHASE1A_WIN (640 chu kỳ clock):** Nhân cửa sổ Hanning. Địa chỉ đọc được tính cuốn vòng theo công thức $\text{addr}[n] = (\text{wr_ptr} - 640 + n) \bmod 640$; từng mẫu được nhân với hệ số ROM $w[n]$ (Q1.15) rồi ghi kết quả $x_{\text{win}}[n]$ vào True Dual-Port RAM. Xong đủ 640 mẫu thì chuyển sang Pha 1b.
3. **S_PHASE1B_ENERGY (640 chu kỳ clock, khối màu cam):** Tích lũy năng lượng toàn khung $r_0 = \mathcal{R}_x[0] = \sum_n x_{\text{win}}^2[n]$. Đây là nơi đặt **Cổng Ngắt Lặng Sớm**: nếu $10.000 \cdot r_0 < 9 \cdot E_{\text{ref}}$, khung bị kết luận vô thanh ngay. Nhánh nét đứt màu đỏ cho thấy đường nhảy tắt: gán $F_0 = 0$ (Unvoiced) rồi đi thẳng tới S_MEDIAN3_OUT, **tiết kiệm trọn vẹn** 98.280 **chu kỳ MAC của Pha 2**.
4. **Nhánh chuyển tiếp có tiếng nói ($10000 \cdot r_0 \ge 9E_{\text{ref}}$):** Khi năng lượng khung đạt ngưỡng, máy trạng thái đi tiếp sang Pha 2 thay vì nhảy tắt.
5. **S_PHASE2_MAC_SWEEP (góc dưới phải):** Quét 189 độ trễ $\tau \in [26, 214]$ bằng một khối DSP duy nhất, tốn 98.280 chu kỳ MAC. Mỗi lag thực hiện tích chập hai cổng RAM, bù hệ số $C[\tau]$, và cập nhật cực đại τ^*, S^* bằng bộ so sánh thanh ghi. Xong đủ 189 lag thì sang khối chia.

6. **S_PARABOLIC_DIV (64 chu kỳ clock):** Kiểm tra ngưỡng thanh tính $100 \cdot S^* \geq 45 \cdot r_0$; nếu đạt, bộ chia tuần tự tính độ lệch đỉnh dưới mẫu $\delta\tau_{\text{q16}} = (N \ll 16)/D$ và tần số $f_{0,\text{q12.4}} = (16000 \cdot 16 \cdot 65536)/\tau_{\text{fine}}$. Xong phép chia thì sang khối lọc.
7. **S_MEDIAN3_OUT (1 chu kỳ clock, góc dưới trái):** Lọc trung vị 3 điểm dập tắt gai nhảy quãng tám đơn khung, rồi ghi từ 16-bit định dạng Q12.4 vào FIFO. Sau đó máy trạng thái quay về S_IDLE cho khung tiếp theo.

6. Sổ Cái Hạch Toán Tài Nguyên Silicon trên FPGA DE10-Standard

Hệ thống nhắm tới kit **DE10-Standard** trang bị chip **Intel Cyclone V SX SoC (5CSXFC6D6F31C6N)**. Bảng sổ cái hạch toán tài nguyên silicon được kiểm đếm chi tiết từ vi kiến trúc phần cứng:

1. Kiểm đếm số phép tính chính xác trên mỗi khung 10 ms:

- **Vòng quét tự tương quan 189 lag ($\tau \in [26, 214]$):**

$$N_{\text{corr}} = \sum_{\tau=26}^{214} (640 - \tau) = 189 \times \frac{614 + 426}{2} = 189 \times 520 = \textbf{98.280} \text{ phép MAC} \tag{34}$$

  Mỗi chu kỳ xung nhịp, True Dual-Port RAM cung cấp đồng thời 2 mẫu $x_w[n]$ và $x_w[n + \tau]$, khối DSP thực hiện 1 phép MAC: tốn đúng 98.280 chu kỳ clock.

- **Tính năng lượng khung $\mathcal{R}_x[0] = \sum x_w^2[n]$ và Kiểm đếm số chu kỳ với 1 DSP:** Khối xử lý đòi hỏi thêm giá trị năng lượng khung $\mathcal{R}_x[0]$ (640 phép nhân-cộng dồn) để làm mốc so sánh cổng lặng và thanh tính. Thiết kế chuẩn hóa chọn **1 khối DSP duy nhất** (nhằm tối thiểu hóa diện tích silicon), khối DSP thực thi tuần tự: (1) Pha 1a: nhân cửa sổ $x \cdot w$ tốn 640 chu kỳ; (2) Pha 1b: quét lượt $\tau = 0$ tính năng lượng $\mathcal{R}_x[0] = \sum x_w^2$ tốn 640 chu kỳ (tổng Pha 1a & 1b là 1.280 chu kỳ); (3) Pha 2: quét tự tương quan 189 độ trễ tốn 98.280 chu kỳ. Tổng chu kỳ tính toán của DSP cho Pha 1a & 1b và Pha 2 là $1.280 + 98.280 =$ **99.560** chu kỳ clock.

  (Ghi chú: Nếu hệ thống dùng 2 DSP song song, vừa nhân cửa sổ vừa dồn năng lượng $\mathcal{R}_x[0]$, số chu kỳ giảm bớt 640 chu kỳ xuống còn 98.920 chu kỳ MAC, tương đương 1,978 ms).

- **Nội suy parabol và Phép chia số nguyên ngoài vòng quét:** Mỗi khung có đúng 2 phép chia nguyên: $(N \ll 16)//D$ và tử_số$//\tau_{\text{fine, q16}}$, chạy đúng 1 lần mỗi 10 ms sau khi đã tìm ra τ^*. Trên FPGA, một bộ chia số nguyên tuần tự (sequential non-restoring divider) tiêu tốn khoảng **40–64** chu kỳ clock, hoàn toàn không ảnh hưởng tới thời gian thực ($\ll$ 0,002 ms). Nếu muốn tiết kiệm logic, bộ 3 điểm $(S[\tau^* - 1], S[\tau^*], S[\tau^* + 1])$ có thể được xuất qua thanh ghi để nhân ARM HPS thực hiện phép chia.

- **Bù suy hao cửa sổ và tìm cực đại:** Thực hiện gối đầu (pipeline overlap) ngay khi mỗi lag τ hoàn tất phép MAC, nhân với $C[\tau]$ trong bảng tra ROM Q3.13: 189 chu kỳ, hoàn toàn che giấu bên trong đường ống.

2. Định mức thời gian thực và kích thước thanh ghi tích lũy:

- **Hệ số thời gian thực (RTF — Real-Time Factor):** Ở tần số xung nhịp danh định $f_{\text{clk}} = 50$ MHz, toàn bộ 99.560 chu kỳ MAC cộng thêm bộ chia tuần tự (64 chu kỳ) tốn tổng cộng đúng **99.624** chu kỳ xung nhịp, được thực thi hoàn tất trong:

$$T_{\text{calc}} = \frac{99.624}{50.000.000 \text{ Hz}} = \textbf{1,992} \text{ ms} \ll 10 \text{ ms} \implies \text{RTF} \approx \textbf{0,199} \tag{35}$$

  Khối Pitch chỉ tiêu tốn chưa đầy 20% năng lực xử lý của một khối DSP đơn lẻ ở xung nhịp khiêm tốn 50 MHz.

- **Định cỡ thanh ghi tích lũy:** Như đã chứng minh tại Mục 4.0, giá trị tích lũy lớn nhất của 640 tích số Q2.30 đòi hỏi thanh ghi $\mathcal{R}_x$ rộng **41 bit có dấu**. Khi nhân với hệ số bù suy hao $C[\tau]$ (Q3.13), tích số đòi hỏi ≤ 56 **bit có dấu** trước khi dịch $\gg 13$; sau khi dịch, điểm số $S[\tau]$ đòi hỏi ≤ 43 **bit có dấu** và được lưu trong thanh ghi **64-bit có dấu** (`int64_t`, khớp hoàn hảo với Bảng 2). Khối DSP cứng trên Cyclone V hỗ trợ thanh ghi tích lũy 64-bit, bảo đảm an toàn toán học tuyệt đối.

Tài nguyên phần cứng	Tiêu tốn cho Pitch	Khả dụng (Cyclone V)	Tỷ lệ sử dụng	Đánh giá vi kiến trúc
Khối nhớ M10K BRAM	≈ 7 khối	553 khối	1,27%	Ring Buffer (2 khối), RAM x_w (2 khối), ROM $w, C[\tau], F_0$ (3 khối)
Khối DSP (18x18-bit)	1 khối	112 khối	0,89%	1 DSP (99.624 ck gồm 99.560 ck quét + 64 ck chia); RTF $\approx 0{,}199$
Bộ logic ALM / Registers	≈ 480 ALMs	41.910 ALMs	1,14%	Bộ đếm địa chỉ FSM, bộ so sánh cực đại thanh ghi, median-3
Bộ chia số nguyên (Divider)	1 bộ chia tuần tự (hoặc HPS)	IP Core / HPS	$< 0{,}1\%$	Chạy 1 lần mỗi 10 ms ngoài vòng quét (40–64 ck), hoặc đẩy sang HPS

Bảng 6: Sổ cái hạch toán tài nguyên silicon cho khối Pitch trên FPGA DE10-Standard.

7. Tổng Kết và Cầu Nối Chuyển Tiếp

- **Luồng Mel (song song, ngoài phạm vi chi tiết tài liệu này):** Log-Mel 80 chiều đại diện cho khẩu hình và cộng hưởng thanh đạo, phục vụ ASR/Whisper trên Server.
- **Luồng F_0 (tài liệu này):** hoàn thiện thiết kế phần cứng đo cao độ thời gian thực trên FPGA DE10-Standard (Quartus Standard/Lite), xuất `pitch_f0.bin` (uint16 LE Q12.4) và tệp âm thanh sạch sang thư mục chia sẻ.
- **Mô hình phân công Edge-Server:** FPGA gánh trọn vẹn việc tính toán cao độ tức thời và tiền xử lý âm thanh với chi phí chưa đầy 2% silicon, giải phóng hoàn toàn máy chủ khỏi việc chạy Praat C++, cho phép Server tập trung vào mạng nơ-ron Whisper và thuật toán đối sánh Pearson r.

Mục lục

Chương 0. Mục tiêu học tập và bản đồ khối kiến trúc **40**

Chương 1. Nguyên lý Praat từ vật lý đến toán (Boersma 1993) **50**

Chương 2. PLL, MCLK, I2S và CDC mã Gray **72**

Chương 3. DC-blocker IIR (cực/zero, accumulator 32-bit) **80**

Chương 4. FIR Compiler II — hạ mẫu 48→24→16 kHz (polyphase 2/3) **88**

Chương 5. Bộ nhớ trượt M10K (True Dual-Port, Old Data) **95**

Chương 6. Cửa sổ Hanning ROM × mẫu (Q1.15) **101**

Chương 7. Tự tương quan — MAC mặc định / FFT tùy chọn + BFP **107**

Chương 8. Chuẩn hóa C[τ] (ROM nhân thay chia) **112**

Chương 9. Peak, parabolic và $F_0 = f_s/\tau$ (LPM_DIVIDE) **118**

Chương 10. Quyết định voicing, chi phí rút gọn và median-3 trên FPGA **124**

Chương 11. FSM đỉnh và giao thức Avalon-ST (valid/ready/SOP/EOP) **131**

Chương 12. Đóng gói F0 Q12.4 và tệp pitch_f0.bin **138**

Chương 13. Kiểm chứng ba tầng: float / fixed / RTL **145**

Chương 14. Checklist nghiệm thu cuối cùng (kèm lý do) **152**

Chương 0. Mục tiêu học tập và bản đồ khối kiến trúc

▶ MỤC TIÊU HỌC TẬP

Sau khi hoàn thành hệ thống 15 chương trong tập tài liệu này (Chương 0 đến Chương 14), người học sẽ làm chủ trọn vẹn quy trình chuyển đổi giải thuật đo cao độ giọng nói F_0 chuẩn mực phòng thí nghiệm của Paul Boersma (1993, phần mềm Praat) sang một hệ thống phần cứng chuyên dụng chạy theo thời gian thực trên kit FPGA DE10-Standard (chip Intel Cyclone V 5CSXFC6D6F31C6N). Cụ thể, bạn sẽ:

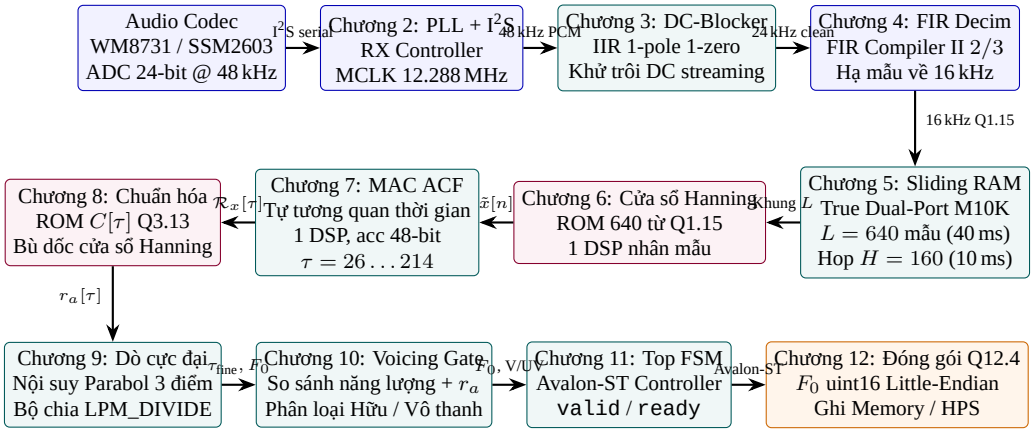
1. Thấu hiểu bản chất vật lý của dao động dây thanh quản, giải thích được từng công thức giải tích trong mã nguồn gốc C++ của Praat (`Sound_to_Pitch.cpp` và `Pitch.cpp`), từ bước khử trôi mức một chiều (DC offset), cửa sổ hóa Hanning, tự tương quan ngắn hạn, chuẩn hóa cửa sổ, đến nội suy đỉnh dưới mẫu và phân loại hữu thanh/vô thanh.
2. Cấu hình chính xác từng khối lõi sở hữu trí tuệ (Intel IP cores) trên phần mềm Quartus Prime Standard / Lite Edition: Altera PLL, On-Chip Memory RAM/ROM 2-PORT, FIR Compiler II, LPM_MULT, LPM_DIVIDE và hiểu rõ cấu tạo vi kiến trúc bên trong các khối DSP blocks và khối nhớ nhúng M10K.
3. Tự tay thiết kế toàn bộ đường truyền dữ liệu (datapath) và máy trạng thái hữu hạn (FSM) bằng ngôn ngữ Verilog HDL chuẩn công nghiệp, tuân thủ chặt chẽ giao thức luồng dữ liệu chuẩn Avalon-ST (`valid`/`ready`).
4. Xây dựng mô hình phần mềm song sinh khớp từng bit (bit-exact Python twin), bảo đảm từng phép cộng, trừ, nhân, dịch bit và bão hòa hai's complement trong phần cứng đều cho kết quả hoàn toàn trùng khớp với phần mềm trên máy tính.
5. Nắm vững phương pháp định lượng tài nguyên silicon (ALM, LUT, Register, DSP, M10K, Fmax, Latency, RTF), bảo đảm thiết kế đạt thông lượng thời gian thực với chi phí dưới 2% diện tích chip Cyclone V.

0.1. Bản đồ đường tín hiệu toàn diện (Complete Signal Path Roadmap)

Trong các hệ thống trí tuệ nhân tạo giọng nói ở biên (Edge Voice AI), tín hiệu âm thanh thu nhận từ micro cần được phân tách thành hai luồng xử lý song song độc lập:

- **Luồng nhận dạng âm học (Log-Mel Frontend):** Gom năng lượng phổ qua 80 bộ lọc tam giác Mel để nhận biết khẩu hình và cộng hưởng ống thanh âm, phục vụ các mô hình nơ-ron nhận dạng giọng nói tự động (ASR như Whisper). Quá trình tích phân dải rộng này cố tình san phẳng và xóa bỏ thông tin cao độ.
- **Luồng đo cao độ (F_0 Stream):** Thực hiện đo chính xác tần số rung cơ học $F_0 = 1/T_0$ của dây thanh quản theo thời gian thực với độ phân giải dưới 1 Hertz, phục vụ nhận diện thanh điệu tiếng Việt (ngang, huyền, sắc, hỏi, ngã, nặng), ngữ điệu câu và chấm điểm phát âm đối sánh tương quan Pearson (r) với chuẩn Praat.

Hình 7 trình bày bản đồ tổng thể đường tín hiệu F_0 trên FPGA từ khi sóng âm chạm vào màng rung micro đến khi xuất dữ liệu cao độ chuẩn định dạng Q12.4 ra thanh ghi giao tiếp.



Hình 7: Bản đồ khối kiến trúc phần cứng ước lượng cao độ F_0 thời gian thực trên kit DE10-Standard (Chương 2 đến 12). Chương 13 thiết lập quy trình kiểm chứng 3 tầng; Chương 14 cung cấp bảng kiểm tra (checklist) nghiệm thu.

0.2. Khung chuẩn mực tám yếu tố của mỗi bài học

Để loại bỏ hoàn toàn tình trạng các chương đọc như bản tóm tắt công thức rời rạc, mỗi bài học trong tài liệu này (từ giải thuật Praat ở Chương 1 đến các khối phần cứng ở Chương 2–12) đều được biên soạn theo một cấu trúc sư phạm chặt chẽ gồm đúng **tám yếu tố bắt buộc**:

1. **Hiện tượng vật lý thật và trực giác cơ bản (Physical Insight):** Phân tích bản chất sinh lý học hoặc điện tử học ngoài đời thực bằng hình ảnh trực quan và liên hệ thực tế; phân định rạch ròi đâu là hiện tượng tự nhiên của âm thanh, đâu chỉ là quy ước toán học hoặc kỹ thuật số.

2. **Toán học từ nguyên lý đầu tiên (First Principles Math):** Dẫn dắt công thức tường minh từ định nghĩa gốc, biến đổi chi tiết từng bước, tuyệt đối không nhảy cóc; định nghĩa rõ ràng đơn vị và ý nghĩa vật lý của mọi ký hiệu trước khi đưa vào biểu thức.

3. **Ví dụ tính tay với số liệu nhỏ (Hand-Worked Example):** Thực hiện tính toán thủ công từng bước với các chuỗi số nhỏ mẫu mực, hiển thị toàn bộ giá trị trung gian. Toàn bộ các con số trong ví dụ đều được kiểm chứng độc lập bằng tập lệnh Python lưu trữ trong thư mục `verify/`.

4. **Vi kiến trúc phần cứng và Intel IP chuẩn (Hardware Architecture):** Phân tích nguyên lý làm việc bên trong các khối IP chuẩn của Intel Quartus (Altera PLL, On-Chip Memory RAM/ROM, FIR Compiler II, LPM_MULT, LPM_DIVIDE) và các khối DSP cứng trên chip Cyclone V. Nêu rõ các ràng buộc silicon thật, ví dụ bộ nhớ M10K không hỗ trợ chế độ đọc-khi-ghi "New Data" trên cùng địa chỉ mà chỉ hỗ trợ "Old Data" hoặc "Don't Care".

5. **Thiết kế RTL Verilog tự viết (Self-Written RTL Alternative):** Cung cấp đoạn mã Verilog HDL tổng hợp được cho khối chức năng tương đương, giải thích chi tiết ý nghĩa kỹ thuật của từng dòng lệnh và cơ chế bắt tay Avalon-ST.

6. **Mô hình song sinh phần mềm khớp bit (Bit-Exact Twin):** Viết hàm Python mô phỏng chính xác hành vi số học dấu phẩy tĩnh (cùng định dạng Q$m.n$, cùng quy tắc làm tròn và bão hòa), đóng vai trò làm thước đo kiểm định khuôn vàng thước ngọc cho phần cứng.

7. **Hộp chi phí silicon chi tiết (Cost Box):** Mở rộng và giải thích đầy đủ mọi thuật ngữ viết tắt ngay lần đầu xuất hiện (ALM, LUT, Register, DSP, M10K, MLAB, Fmax, Latency, RTF) và giải thích cặn kẽ tại sao lựa chọn kiến trúc này là tối ưu cho ngân sách kit DE10-Standard.

8. **Hệ thống bài tập củng cố kèm lời giải chi tiết (Exercises with Explained Solutions):** Các câu hỏi kiểm tra tư duy sâu sắc, có lời giải giải thích nguyên nhân gốc rễ, không chấp nhận câu trả lời một từ.

0.3. Phần cứng mục tiêu và Ngân sách Silicon Kit DE10-Standard

Hệ thống được thiết kế và kiểm chứng thực tế trên kit phát triển **Terasic DE10-Standard**, trang bị vi mạch SoC **Intel Cyclone V SX** (mã linh kiện: `5CSXFC6D6F31C6N`, Speed Grade 6), kết nối với chip giải mã âm thanh analog **Wolfson WM8731** (hoặc SSM2603) thông qua bus dữ liệu nối tiếp I^2S và bus điều khiển I^2C. Nguồn dao động thạch anh chính trên bo mạch có tần số $50\,\text{MHz}$.

Bảng 7 tổng hợp bảng thông số ngân sách tài nguyên vật lý thật trên chip Cyclone V SoC và định mức tối đa dành riêng cho toàn bộ đường xử lý cao độ F_0.

Bảng 7: Bảng phân bổ ngân sách tài nguyên trên chip Intel Cyclone V 5CSXFC6D6F31C6N

Tài nguyên Silicon	**Tổng có sẵn trên Chip**	**Ngân sách luồng F_0**	**Tỷ lệ chiếm dụng (%)**
Logic Modules (ALMs)	41,910 ALMs ($\sim 110\,\text{K}$ LEs)	≈ 850 ALMs	2.03%
Bản ghi logic (Registers)	166,036 Flip-Flops	$\approx 1,200$ Flip-Flops	0.72%
Khối DSP cứng (18×19)	112 khối (hoặc 56 khối 27×27)	3 khối DSP	2.68%
Khối nhớ nhúng M10K	553 khối ($5,530\,\text{Kbit}$)	4 khối M10K ($40\,\text{Kbit}$)	0.72%
Bộ nhớ phân tán MLAB	2,883 khối ($1,845\,\text{Kbit}$)	0 khối (tùy chọn ROM nhỏ)	0.00%
Khối vòng khóa pha (PLL)	6 bộ Fractional PLLs	1 bộ PLL (cho audio)	16.67%

► NHÌN THẤY VẬT LÝ

Toàn bộ thuật toán đo cao độ F_0 chỉ chiếm vỏn vẹn khoảng 2% tài nguyên logic và dưới 3% khối DSP của một chip FPGA tầm trung. Hơn 97% silicon còn lại được giải phóng hoàn toàn để triển khai các mạng nơ-ron sâu nhận dạng giọng nói, xử lý thị giác máy tính hoặc trao đổi dữ liệu tốc độ cao với vi xử lý ARM Cortex-A9 MPCore kép (HPS).

0.4. Từ điển trực giác cốt lõi — Bắt cầu Feynman

Dưới đây là hệ thống định nghĩa sâu sắc về các khái niệm kỹ thuật cốt lõi sẽ xuất hiện xuyên suốt từ Chương 1 đến Chương 14. Mỗi mục được giải thích bằng văn xuôi trôi chảy, trả lời rõ ba câu hỏi: *nó là gì, nó là hiện tượng vật lý thật hay chỉ là công cụ tính toán,* và *tại sao nó ảnh hưởng trực tiếp đến độ chính xác của* F_0.

★ BẮC CẦU FEYNMAN: 1. Tín hiệu liên tục (Continuous-Time), ADC và Quá trình Lấy Mẫu

Sóng âm lan truyền trong không khí là dao động áp suất **liên tục theo thời gian**: tại bất kỳ khoảnh khắc vô cùng nhỏ nào, áp suất khí quyển luôn có một giá trị xác định và biến thiên một cách mượt mà. Màng rung micro chuyển áp suất này thành điện áp tương tự liên tục. Khối **ADC** (*Analog-to-Digital Converter* — Bộ chuyển đổi tương tự sang số) thực hiện phép **lấy mẫu**: cứ sau mỗi chu kỳ thời gian $T_s = 1/f_s$ giây (với f_s là tần số lấy mẫu, ví dụ $48\,\text{kHz}$ hoặc $16\,\text{kHz}$), bộ so sánh điện áp sẽ đo độ lớn tín hiệu và chuyển thành một số nhị phân hữu hạn bit (ví dụ 16 bit hoặc 24 bit), hoàn toàn bỏ qua những biến động xảy ra giữa hai lần đo.

Phép lấy mẫu là một thao tác kỹ thuật nhân tạo, nhưng nó dẫn đến một **hệ quả vật lý thực sự**: phổ tần số của chuỗi số rời rạc bị **lặp lại tuần hoàn với chu kỳ** f_s trên toàn bộ trục tần số, tạo ra vô số các bản sao của phổ gốc tại các vị trí $\pm f_s, \pm 2f_s, \ldots$. Nếu tín hiệu âm thanh ban đầu chứa bất kỳ thành phần tần số nào cao hơn $f_s/2$, các bản sao này sẽ lấn sang và chồng đè lên phổ gốc — gây ra hiện tượng méo chồng phổ (*aliasing*). Đối với việc ước lượng F_0: ta làm việc ở $f_s = 16\,\text{kHz}$, nghĩa là mọi rung động âm học trên $8\,\text{kHz}$ bắt buộc phải được triệt tiêu hoàn toàn trước khi vào bộ tính tương quan; nếu không, năng lượng cao tần sẽ biến thành các chu kỳ giả mạo làm sai lệch cao độ.

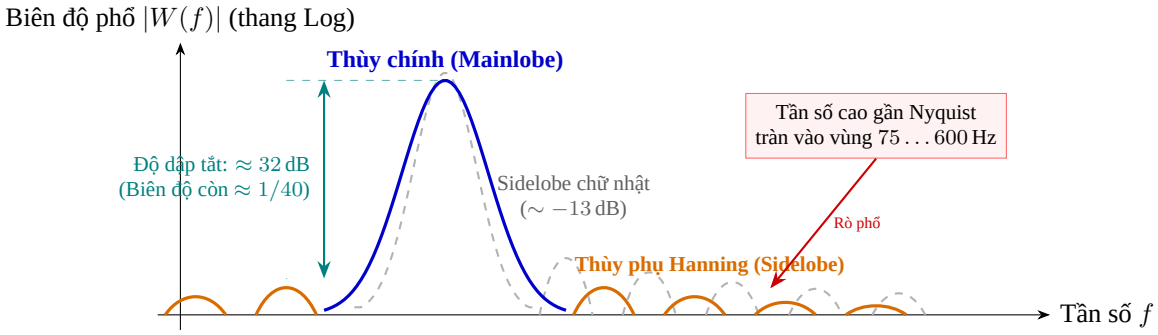
★ BẮC CẦU FEYNMAN: 2. Tần số Nyquist, Hiện tượng Chồng Phổ (Aliasing) và Hình Ảnh Phản Xạ Gương

Tần số Nyquist được định nghĩa là $f_N = f_s/2$, đại diện cho ngưỡng tần số vật lý cao nhất mà một hệ thống rời rạc có thể ghi nhận chính xác theo Định lý lấy mẫu Shannon–Nyquist. **Aliasing** là hiện tượng một tần số cao $f > f_s/2$ bị gập ngược qua ranh giới Nyquist và giả dạng hoàn hảo thành một tần số thấp hơn $f_{\text{alias}} = |f_s - f|$ (hoặc $|k f_s \pm f|$). *Ví dụ số trực quan:* Tại tần số lấy mẫu $f_s = 16\,\text{kHz}$, tần số Nyquist là $f_N = 8\,\text{kHz}$. Một sóng âm có tần số thật là $9\,\text{kHz}$ sau khi qua ADC sẽ tạo ra các điểm mẫu giống hệt như một sóng âm $7\,\text{kHz}$ ($16 - 9 = 7\,\text{kHz}$). Ranh giới $f_s/2$ hoạt động như một **tấm gương phản xạ**: mọi tần số vượt qua mép gương đều bị phản xạ quay ngược vào trong dải tần làm việc. Nếu màng rung micro thu phải tiếng ồn cơ học, tiếng rít quạt gió hay sóng hài bậc cao ở $9\,\text{kHz}$, nó sẽ biến thành một thành phần tần số $7\,\text{kHz}$ sạch sẽ trong dữ liệu số. Thành phần này kết hợp với hiện tượng rò phổ của cửa sổ phân khung sẽ tạo ra các đỉnh tự tương quan giả nằm đúng vào vùng tìm kiếm cao độ con người ($75 \ldots 600\,\text{Hz}$), phá hỏng kết quả đo.

★ BẮC CẦU FEYNMAN: 3. Cắt Khung (Framing), Cửa Sổ (Window), Rò Phổ (Spectral Leakage), Mainlobe, Sidelobe và Đơn Vị dB

Giọng nói con người liên tục thay đổi khẩu hình theo từng âm vị, nên ta không thể xử lý toàn bộ câu nói một lúc mà phải **cắt khung**: trích ra một đoạn ngắn khoảng $40\,\text{ms}$ ($L = 640$ mẫu ở $16\,\text{kHz}$) để phân tích. Phép cắt đột ngột (cửa sổ chữ nhật — Rectangular window) tương đương với việc nhân tín hiệu liên tục với một xung vuông; trong miền tần số, điều này tương đương với phép **tích chập (convolution)** phổ tín hiệu với hàm $\text{sinc}(f) = \sin(\pi f)/(\pi f)$. Hàm sinc sở hữu một búp sóng trung tâm gọi là **thùy chính (mainlobe)** và vô số các gợn sóng lan rộng hai bên gọi là **thùy phụ (sidelobe)**.

Hiện tượng năng lượng của một tần số đơn lẻ bị phát tán lan man sang toàn bộ các tần số khác qua các thùy phụ được gọi là **rò phổ (spectral leakage)**. Thùy phụ đầu tiên của cửa sổ chữ nhật chỉ suy giảm khoảng $-13\,\text{dB}$ so với đỉnh thùy chính. Đơn vị **Decibel (dB)** là thang đo logarit của tỷ số biên độ: $-13\,\text{dB} = 20\log_{10}(A/A_0) \Rightarrow A/A_0 = 10^{-13/20} \approx 0.224 \approx 1/4.5$. Nghĩa là năng lượng rò rỉ vẫn còn tới hơn 22% biên độ gốc! Cửa sổ **Hanning** vuốt hai đầu khung mượt mà về gần 0 bằng một nửa chu kỳ hàm cosin, kéo thùy phụ đầu tiên tụt xuống mức $-32\,\text{dB}$ ($10^{-32/20} \approx 0.025 \approx 1/40$ biên độ, giảm hơn 97% năng lượng rò so với cửa sổ chữ nhật), giúp bảo vệ dải tần F_0 khỏi các bóng ma tần số cao.



Hình 8: So sánh phổ biên độ của cửa sổ Chữ nhật (nét đứt) và cửa sổ Hanning (nét liền). Cửa sổ Hanning chấp nhận thùy chính rộng hơn đôi chút để dập tắt thùy phụ từ mức $-13\,\text{dB}$ xuống $-32\,\text{dB}$, ngăn chặn hiện tượng rò phổ làm bẩn dải đo cao độ F_0.

★ BẮC CẦU FEYNMAN: 4. Phát Ngôn (Utterance), Thuật Toán FFT/IFFT, Mặt Nạ Phổ và Zero-Padding

Trong xử lý ngôn ngữ, một **phát ngôn (utterance)** là một chuỗi âm thanh liên tục tương ứng với một câu nói hoàn chỉnh, ngăn cách bởi các khoảng lặng hít thở. **FFT** (*Fast Fourier Transform*) là thuật toán kinh điển tính nhanh biến đổi Fourier rời rạc (DFT), chuyển chuỗi thời gian $x[n]$ sang các thành phần phổ phức $X[k]$. **IFFT** là phép biến đổi ngược đưa phổ phức trở về miền thời gian.

Mặt nạ phổ (Spectral mask) là một hàm trọng số $G[k]$ được nhân trực tiếp vào từng bin phổ $X[k]$ để triệt tiêu có chủ đích một dải tần số xác định. **Zero-padding** là kỹ thuật chèn thêm các số 0 vào chuỗi dữ liệu. Khi chèn các số 0 vào giữa nửa phổ dương và nửa phổ âm trước khi thực hiện IFFT dài gấp đôi ($2N$), chuỗi thời gian đầu ra sẽ có độ dài tăng gấp hai lần. Đây thực chất là phép nội suy sin(c) lý tưởng: các mẫu mới được tạo ra một cách trơn tru giữa các mẫu cũ mà không tạo ra bất kỳ tần số hài giả nào ngoài dải. Bước 1 của Praat sử dụng tổ hợp FFT cả câu nói $\rightarrow$ nhân mặt nạ hạ dốc từ $0.95 f_N$ về $0 \rightarrow$ zero-padding để upsample $\times 2$ mềm, dọn sạch mép Nyquist.

★ BẮC CẦU FEYNMAN: 5. Hàm Tự Tương Quan (Autocorrelation), Độ Trễ (Lag τ) và Định Lý Wiener–Khinchin

Hàm tự tương quan không vòng của một khung âm thanh $x[n]$ độ dài N mẫu được định nghĩa là:

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{N-1-\tau} x[n] \cdot x[n+\tau], \qquad \tau \geq 0.$$

Đại lượng τ được gọi là **độ trễ (lag)**, tính bằng số mẫu. Hàm tự tương quan đo lường độ giống nhau giữa tín hiệu và bản sao dịch chuyển thời gian của chính nó. Khi dây thanh quản rung tuần hoàn với chu kỳ cơ bản T_0 giây ($\tau_0 = f_s \cdot T_0$ mẫu), sóng âm tại thời điểm n sẽ lặp lại tại $n + \tau_0$. Khi trượt đúng khoảng trễ $\tau = \tau_0$, tích các cặp mẫu cùng pha sẽ đạt giá trị cực đại, tạo nên một đỉnh nổi bật trên đồ thị $\mathcal{R}_x[\tau]$. Đo được vị trí đỉnh τ_0, ta suy ra ngay cao độ: $F_0 = f_s/\tau_0$.

Định lý Wiener–Khinchin là một định lý toán học nền tảng chứng minh rằng: hàm tự tương quan của một tín hiệu chính là biến đổi Fourier ngược của mật độ phổ công suất của nó: $\mathcal{R}_x[\tau] = \mathcal{F}^{-1}\{|X[k]|^2\}$. Định lý này mở ra hai con đường thực thi phần cứng hoàn toàn tương đương về mặt toán học: tính tự tương quan bằng cặp biến đổi FFT $\rightarrow |X|^2 \rightarrow$ IFFT, hoặc tính trực tiếp bằng khối nhân dồn thời gian (MAC).

★ BẮC CẦU FEYNMAN: 6. Cánh Bướm (Butterfly), Hệ Số Xoay (Twiddle Factor) và Định Dạng Khối Chấm Động (BFP)

Trong thuật toán FFT cơ số 2 (Radix-2 Cooley-Tukey), phép tính cơ sở nhỏ nhất lấy hai số phức đầu vào a và b, nhân b với một hệ số pha $W_N^k = e^{-j2\pi k/N}$ (được gọi là **hệ số xoay — twiddle factor**), rồi tạo ra hai kết quả đầu ra: $a + W_N^k b$ và $a - W_N^k b$. Đồ hình đường tín hiệu bắt chéo này trông giống hình đôi cánh bướm nên được gọi là phép tính **cánh bướm (butterfly)**.

Khi tính toán FFT trên phần cứng số cố định bit (ví dụ 16 bit), các phép cộng cánh bướm có thể làm biên độ tăng gấp đôi ở mỗi tầng, gây tràn số. Để khắc phục mà không cần chuyển sang bộ số chấm động (Floating-point) đắt đỏ, Intel FFT IP áp dụng cơ chế **Block Floating Point (BFP — Số chấm động khối)**: toàn bộ các phần tử trong một vector chia sẻ chung một số mũ nguyên e. Mỗi khi một tầng cánh bướm có nguy cơ tràn, toàn bộ vector được dịch phải 1 bit và số mũ e tăng thêm 1. Khi bình phương phổ $|X[k]|^2 = \text{Re}^2 + \text{Im}^2$, độ lớn phổ bị nâng lũy thừa bậc hai, khiến số mũ thực tế của năng lượng bị nhân đôi thành $2e$. Nếu kỹ sư quên không dịch bit bù trừ hệ số 2^{2e} trước khi đưa vào khối IFFT, tỷ lệ biên độ của toàn bộ hàm tự tương quan sẽ bị sai lệch hoàn toàn, làm sụp đổ các ngưỡng so sánh voicing.

★ BẮC CẦU FEYNMAN: 7. Số Dấu Phẩy Tĩnh (Fixed-Point Q$m.n$), Bão Hòa (Saturation) và Khớp Từng Bit (Bit-Exact Twin)

Số **Q**$m.n$ là quy ước biểu diễn số thực có dấu bằng số nguyên hai's complement: dùng m bit cho phần nguyên (bao gồm cả bit dấu) và n bit cho phần thập phân, với trọng số nhỏ nhất (LSB) bằng 2^{-n}. Ví dụ: định dạng **Q1.15** dùng 16 bit, có dải biểu diễn từ -1.0 đến $+1.0 - 2^{-15}$ (tức từ -32768 đến $+32767$), trong đó giá trị số thực x được ánh xạ thành số nguyên $X_{\text{int}} = \text{round}(x \cdot 2^{15})$.

Hiện tượng bão hòa (Saturation): Khi kết quả của một phép tính toán vượt ra ngoài dải biểu diễn của thanh ghi, phép bão hòa sẽ ghim giá trị về ngưỡng cực đại ($+32767$) hoặc cực tiểu (-32768). Nếu để xảy ra hiện tượng tràn vòng tự nhiên (overflow/wrap-around), một số dương cực lớn sẽ đột ngột biến thành một số âm cực lớn, tạo ra tiếng nổ lách cách trong âm thanh và bẻ gãy hoàn toàn hàm tự tương quan. **Mô hình song sinh khớp bit (Bit-exact twin)** là chương trình viết bằng Python/C++ mô phỏng lại chính xác từng phép cộng, trừ, nhân, dịch bit và bão hòa của mạch RTL. Đầu ra của mô hình twin và kết quả mô phỏng ModelSim trên cùng một tệp dữ liệu âm thanh kiểm tra bắt buộc phải giống nhau đến từng bit đơn lẻ (100% bit-match).

★ BẮC CẦU FEYNMAN: 8. Vòng Khóa Pha (PLL), Xung Chủ (MCLK), Giao Thức I²S và Hàng Đợi Bất Đồng Bộ (DCFIFO Mã Gray)

PLL (*Phase-Locked Loop* — Vòng khóa pha) là mạch điện tử analog-số tích hợp bên trong chip FPGA, sử dụng bộ dao động điều khiển bằng điện áp (VCO) để khóa đồng pha với dao động thạch anh ngoại vi $50\,\text{MHz}$, từ đó nhân và chia tần số để tạo ra các xung nhịp đồng hồ cực kỳ ổn định. Chip giải mã âm thanh codec WM8731 yêu cầu một xung nhịp chủ riêng biệt **MCLK** $= 12.288\,\text{MHz}$ (đúng bằng $256 \times 48\,\text{kHz}$) để vận hành bộ lọc oversampling và bộ chuyển đổi delta-sigma bên trong nó.
I²S (*Inter-IC Sound*) là chuẩn giao tiếp nối tiếp 3 dây chuyên dụng truyền âm thanh số: dây chọn kênh `WS` (LRCK, tần số $48\,\text{kHz}$, chọn tai trái/phải), dây xung đồng hồ bit `SCLK` (BCLK, tần số $3.072\,\text{MHz} = 64 \times 48\,\text{kHz}$) và dây dữ liệu nối tiếp `SDAT`. Do miền đồng hồ âm thanh ($12.288\,\text{MHz}$) và miền đồng hồ xử lý tín hiệu lõi ($50\,\text{MHz}$) dao động độc lập không đồng bộ, việc truyền mẫu âm thanh qua lại bắt buộc phải đi qua một bộ đệm **DCFIFO** (*Dual-Clock FIFO*). Các con trỏ ghi/đọc của DCFIFO được mã hóa theo chuẩn **Mã Gray** (mỗi bước tăng chỉ có duy nhất 1 bit thay đổi trạng thái) để triệt tiêu hoàn toàn rủi ro tranh chấp trạng thái giả (*metastability*). Sai lệch tần số MCLK dù chỉ vài chục phần triệu (ppm) cũng sẽ làm trôi chu kỳ lấy mẫu theo thời gian, dẫn đến việc đo lệch cao độ F_0.

★ BẮC CẦU FEYNMAN: 9. Bộ Lọc FIR, Taps, Cấu Trúc Đa Pha (Polyphase Decimation) và Dải Chuyển Tiếp

Bộ lọc FIR (*Finite Impulse Response*) tính toán mẫu đầu ra bằng tổng chập có trọng số của các mẫu đầu vào quá khứ: $y[n] = \sum_{k=0}^{M-1} h[k]x[n-k]$. Số lượng hệ số M được gọi là số **vòi lọc (taps)**. **Hạ mẫu (Decimation)** là thao tác giảm tần số lấy mẫu bằng cách loại bỏ bớt các mẫu thừa (ví dụ từ $24\,\text{kHz}$ xuống $16\,\text{kHz}$, tỷ số $2/3$: cứ nhận 3 mẫu thì tính toán xuất ra 2 mẫu). Trước khi hạ mẫu, bắt buộc phải có bộ lọc thông thấp chặn đứng mọi tần số trên $8\,\text{kHz}$ để ngăn ngừa méo aliasing.
Cấu trúc đa pha (Polyphase) phân rã một bộ lọc FIR dài thành các nhánh lọc con ngắn hơn hoạt động song song ở tần số thấp hơn, chỉ kích hoạt nhân hệ số khi cần xuất mẫu, giúp tiết kiệm đến 66% số phép nhân DSP. **Dải chuyển tiếp (Transition band)** là khoảng tần số nằm giữa dải thông (passband, cho qua nguyên vẹn) và dải chắn (stopband, dập tắt hoàn toàn). Dải chuyển tiếp càng dốc và hẹp thì đòi hỏi số vòi lọc M càng lớn, làm tăng số khối nhân DSP và tài nguyên logic trên FPGA.

★ BẮC CẦU FEYNMAN: 10. Khối Nhớ Nhúng M10K, True Dual-Port, Chế Độ Đọc Khi Ghi (Old Data) và Bộ Nhân DSP

M10K là khối bộ nhớ RAM tĩnh chuyên dụng tích hợp sẵn trên silicon Cyclone V, mỗi khối chứa chính xác $10,240$ bit dữ liệu (khoảng 10 Kbit). Chế độ **True Dual-Port** (hai cổng thực) cho phép hai mạch logic độc lập thực hiện đọc hoặc ghi đồng thời vào hai địa chỉ bất kỳ trong cùng một chu kỳ xung nhịp. Khi một cổng thực hiện ghi dữ liệu mới vào địa chỉ A trong khi cổng kia đọc đúng địa chỉ A tại cùng một cạnh xung nhịp (*read-during-write collision*), phần cứng silicon của Cyclone V **không hỗ trợ chế độ xuất dữ liệu mới (New Data)**, mà chỉ bảo đảm chế độ xuất dữ liệu cũ (**Old Data**) hoặc không xác định (Don't Care). Mọi thuật toán hàng đợi trượt cửa sổ F_0 đều phải được thiết kế để dựa trên dữ liệu Old Data hoặc lệch pha ghi/đọc.
Khối **DSP block** trên Cyclone V là mạch nhân số học silicon cứng tốc độ cao, có khả năng thực hiện 2 phép nhân độc lập 18×19 bit có dấu (hoặc 1 phép nhân 27×27 bit) trong đúng 1 chu kỳ xung nhịp. **ALM** (*Adaptive Logic Module*) là đơn vị logic cơ bản của Cyclone V, bao gồm bảng tra cứu LUT 8 ngõ vào, hai bộ cộng đầy đủ và hai flip-flop, được phần mềm tổng hợp dùng để đếm tài nguyên logic. **Fmax** là tần số xung nhịp làm việc tối đa mà mạch có thể chạy ổn định sau khi đã bố trí và định tuyến dây (Place and Route) mà không bị vi phạm thời gian thiết lập tín hiệu (Setup time violation).

★ BẮC CẦU FEYNMAN: 11. Đỉnh Cực Đại Cục Bộ, Nội Suy Parabol, Nội Suy Sinc, Sai Quãng Tám và Bộ Lọc Median-3

Giá trị cực đại của hàm tự tương quan hiếm khi nằm chính xác tại một số nguyên mẫu trễ τ, mà thường rơi vào khoảng giữa hai mẫu rời rạc. **Nội suy Parabol (Parabolic interpolation)** sử dụng 3 điểm quanh đỉnh để khớp một đường cong bậc hai, tìm ra tọa độ đỉnh thực với độ chính xác dưới mẫu (sub-sample) chỉ bằng 2 phép trừ và 1 phép chia. **Nội suy Sinc** sử dụng khai triển giải tích hàm sinc trên cửa sổ 70 hoặc 700 mẫu (SINC70/SINC700) để tiệm cận lời giải chính xác tuyệt đối, nhưng tốn hàng trăm phép nhân trên mỗi đỉnh. FPGA chọn nội suy Parabol vì nó cân bằng hoàn hảo giữa độ chính xác (sai số < 2 cent) và tài nguyên silicon.
Sai quãng tám (Octave error): Sóng âm tuần hoàn chu kỳ T_0 cũng tuần hoàn chu kỳ $2T_0$, khiến hàm tự tương quan luôn xuất hiện đỉnh phụ tại $\tau = 2T_0$. Nếu biên độ đỉnh phụ bị nhiễu đẩy cao hơn đỉnh chính, thuật toán sẽ nhận nhầm tần số bị tụt xuống một nửa ($F_0/2$). Praat áp dụng **Octave cost** để phạt nhẹ các trễ lớn nhằm triệt tiêu thiên vị này. **Voicing** là phán quyết khung âm thanh là Hữu thanh (dây thanh rung, có F_0) hay Vô thanh / Im lặng ($F_0 = 0$). Để làm sạch đường cong cao độ khỏi các xung nhiễu đơn lẻ mà không phải chạy thuật toán quy hoạch động Viterbi phức tạp, phần cứng sử dụng **Bộ lọc trung vị 3 khung (Median-3 filter)**: lấy giá trị nằm giữa của 3 khung liên tiếp, tạo độ trễ xử lý chỉ đúng $10\,\text{ms}$.

★ BẮC CẦU FEYNMAN: 12. Giao Thức Luồng Avalon-ST (`valid`/`ready`), Điểm Cực và Điểm Không

Avalon-ST (*Avalon Streaming Interface*) là chuẩn giao tiếp truyền dữ liệu điểm-nối-điểm một chiều của Intel dành cho các hệ thống DSP băng thông cao. Giao thức vận hành dựa trên cặp tín hiệu bắt tay cơ bản: bên phát nâng tín hiệu `valid` $= 1$ khi dữ liệu trên bus hợp lệ; bên thu nâng tín hiệu `ready` $= 1$ khi bộ đệm bên trong còn chỗ trống tiếp nhận. Giao dịch chuyển dữ liệu chỉ xảy ra tại cạnh xung nhịp mà cả hai tín hiệu `valid` và `ready` đồng thời bằng 1. Nếu bên thu hạ `ready` $= 0$ (áp lực ngược — *backpressure*), bên phát bắt buộc phải giữ nguyên dữ liệu trên bus; việc lơ là tín hiệu `ready` sẽ làm rơi rớt mẫu âm thanh và phá vỡ cấu trúc khung F_0.
Trong lý thuyết xử lý tín hiệu số, hàm truyền của một bộ lọc rời rạc được biểu diễn dưới dạng phân thức $H(z) = B(z)/A(z)$. Nghiệm của tử số $B(z) = 0$ được gọi là các **điểm không (zeros)**, có tác dụng triệt tiêu hoàn toàn năng lượng tại các tần số tương ứng. Nghiệm của mẫu số $A(z) = 0$ được gọi là các **điểm cực (poles)**, quyết định mức khuếch đại cộng hưởng và tính ổn định của hệ thống. Bộ chặn DC ở Chương 3 đặt một điểm không đúng tại $z = 1$ để triệt tiêu hoàn toàn thành phần một chiều ($0\,\text{Hz}$) và đặt một điểm cực rất gần đó ($z = 0.995$) để duy trì đáp ứng phẳng cho toàn bộ dải âm tần nghe thấy.

★ BẮC CẦU FEYNMAN: 13. Hệ Số Thời Gian Thực (Real-Time Factor – RTF)

Hệ số thời gian thực (RTF) được định nghĩa là tỷ số giữa thời gian phần cứng tiêu tốn để tính toán xong một khung dữ liệu chia cho thời lượng thực tế của khung dữ liệu đó:

$$\text{RTF} = \frac{T_{\text{compute}}}{T_{\text{audio}}}.$$

Một hệ thống được xem là đáp ứng thời gian thực khi và chỉ khi $\text{RTF} \leq 1.0$. Nếu $\text{RTF} > 1.0$, hệ thống tính toán chậm hơn tốc độ âm thanh đi vào, khiến bộ đệm bị tràn và mất dữ liệu. Trong thiết kế này, với bước nhảy khung $H = 160$ mẫu ($10\,\text{ms}$), tại tần số xung nhịp hệ thống $50\,\text{MHz}$, phần cứng mất $120,960$ chu kỳ ($\approx 2.42\,\text{ms}$) để tính xong toàn bộ 189 giá trị tương quan và trích xuất đỉnh cao độ. Hệ số thời gian thực đạt được là:

$$\text{RTF} = \frac{2.42\,\text{ms}}{10\,\text{ms}} = 0.242 \ll 1.0.$$

Hệ số này chứng minh phần cứng chạy nhanh gấp hơn 4.1 lần so với tốc độ thời gian thực, hoàn toàn bảo đảm độ trễ cực thấp cho các ứng dụng tương tác giọng nói trực tiếp.

0.5. Ngân sách học tập và Bản đồ thời gian biểu

CHI PHÍ (LUT / DSP / chu kỳ)

Kế hoạch phân bổ thời gian học tập và thực hành (Tổng thời lượng: ≈ 24 giờ làm việc):

- **Chương 0 & Chương 1 (Nền tảng Praat và Toán giải tích):** Dành từ 4 đến 6 giờ đọc kỹ từng chứng minh toán học, tự tay chạy lại tập lệnh Python `verify/verify_chuong_00_01.py` để kiểm tra từng con số tính tay. *Nguyên lý cốt lõi:* Sai một công thức toán ở Chương 1 thì không một khối DSP hay bộ nhớ M10K nào ở các chương sau có thể cứu vãn được hệ thống.
- **Chương 2 đến Chương 6 (Tiền xử lý âm thanh và Chuẩn bị khung):** Dành từ 5 đến 6 giờ. Nắm vững cấu hình PLL, giao thức I^2S, bộ lọc chặn DC, bộ lọc hạ mẫu FIR $2/3$ và cơ chế đệm khung trượt trên M10K.
- **Chương 7 đến Chương 10 (Trái tim tính toán cao độ):** Dành từ 6 đến 8 giờ. Đây là phần trọng tâm phần cứng gồm bộ nhân dồn MAC tự tương quan, bảng ROM bù dốc cửa sổ, bộ chia nội suy parabol và mạch so sánh voicing.
- **Chương 11 đến Chương 14 (Tích hợp, Đóng gói và Kiểm chứng 3 tầng):** Dành từ 4 đến 5 giờ. Kết nối toàn hệ thống qua FSM Avalon-ST, đóng gói dữ liệu đầu ra Q12.4, chạy mô phỏng ModelSim và đối chiếu sai số với ngưỡng 20 cent của Praat.

0.6. Bài tập tự đánh giá và Lời giải chi tiết

Nhằm giúp người học tự củng cố tư duy kiến trúc và các nguyên lý nền tảng trước khi bước vào chi tiết giải thuật Praat và phần cứng, dưới đây là hệ thống 4 bài tập phân tích sâu kèm lời giải tường minh:

Bài tập 0.1 (Hiện tượng Aliasing và Méo Cao Độ): Một micro thu âm ghi nhận tín hiệu tiếng nói bị lẫn một xung nhiễu điện cơ học tần số cao $f_{\text{noise}} = 9\,\text{kHz}$. Hệ thống xử lý hoạt động tại tần số lấy mẫu $f_s = 16\,\text{kHz}$.

a) Xác định tần số Nyquist của hệ thống và tính tần số phản xạ giả mạo (f_{alias}) của tín hiệu nhiễu sau quá trình lấy mẫu.

b) Tần số giả mạo này có rơi vào vùng tìm kiếm cao độ con người ($75\,\text{Hz} \ldots 600\,\text{Hz}$) hay không? Nếu tín hiệu này tiếp tục đi qua khối hạ mẫu Decimation $2/3$ hoặc bị rò phổ bởi cửa sổ phân khung, hậu quả gì sẽ xảy ra đối với đỉnh tự tương quan?

Lời giải giải thích:

a) Tần số Nyquist của hệ thống là $f_N = f_s/2 = 16\,\text{kHz}/2 = 8\,\text{kHz}$. Do tần số nhiễu $f_{\text{noise}} = 9\,\text{kHz} > f_N$, hiện tượng aliasing chắc chắn xảy ra. Tần số giả mạo xuất hiện trong miền gốc được tính theo công thức phản xạ gương qua ranh giới Nyquist:

$$f_{\text{alias}} = |f_s - f_{\text{noise}}| = |16\,\text{kHz} - 9\,\text{kHz}| = 7\,\text{kHz}.$$

b) Tại tần số $7\,\text{kHz}$, sóng phản xạ chưa rơi trực tiếp vào dải cao độ cơ bản $75 \ldots 600\,\text{Hz}$. Tuy nhiên, trong miền thời gian, một thành phần $7\,\text{kHz}$ có biên độ lớn khi bị nhân với cửa sổ phân khung Hanning dài $40\,\text{ms}$ sẽ sinh ra các thùy phụ lan rộng. Nguy hiểm hơn, nếu hệ thống tiếp tục áp dụng các bước hạ mẫu số mà bộ lọc chống chồng phổ (anti-aliasing) không đạt độ suy giảm tối thiểu $60\,\text{dB}$, thành phần $7\,\text{kHz}$ này sẽ tiếp tục gập qua các ranh giới Nyquist kế tiếp và đổ thẳng vào dải vài trăm Hertz, tạo ra các đỉnh tương quan giả mạo cạnh tranh trực tiếp với tần số dây thanh quản thật. Do đó, việc dọn sạch phổ trước khi phân khung là yêu cầu sinh tử.

Bài tập 0.2 (Định mức Thời gian thực và Hệ số RTF): Hệ thống đo cao độ F_0 hoạt động với xung nhịp đồng hồ hệ thống $f_{\text{clk}} = 50\,\text{MHz}$. Bước nhảy khung thời gian là $H = 160$ mẫu tại $f_s = 16\,\text{kHz}$ ($10\,\text{ms}$). Để quét dải cao độ từ $75\,\text{Hz}$ đến $600\,\text{Hz}$, khối MAC tự tương quan phải tính toán 189 giá trị trễ ($\tau = 26 \ldots 214$), mỗi giá trị trễ tích lũy $L = 640$ tích mẫu.

a) Tính số chu kỳ xung nhịp tối đa mà hệ thống có sẵn trong một bước nhảy khung $10\,\text{ms}$.

b) Giả sử kiến trúc sử dụng 1 khối DSP nhân dồn tuần tự (1 phép tính nhân-cộng trên mỗi chu kỳ đồng hồ). Tính tổng thời gian cần thiết để xử lý xong một khung và suy ra hệ số thời gian thực RTF. Đưa ra nhận xét về biên độ an toàn (margin) của thiết kế.

Lời giải giải thích:

a) Số chu kỳ xung nhịp hệ thống có sẵn trong khoảng thời gian một bước nhảy khung $T_{\text{audio}} = 10\,\text{ms} = 0.01\,\text{s}$ là:

$$N_{\text{cycles}} = f_{\text{clk}} \times T_{\text{audio}} = 50 \times 10^6\,\text{Hz} \times 0.01\,\text{s} = 500,000\,\text{chu kỳ}.$$

b) Tổng số phép nhân dồn MAC cần thực hiện cho 189 giá trị trễ là:

$$N_{\text{MAC}} = 189 \times 640 = 120,960\,\text{phép tính}.$$

Với 1 khối DSP thực thi 1 phép MAC trên 1 chu kỳ, thời gian tính toán tiêu tốn là $T_{\text{compute}} = 120,960/(50\times 10^6\,\text{Hz}) = 2.4192\,\text{ms}$. Hệ số thời gian thực là:

$$\text{RTF} = \frac{T_{\text{compute}}}{T_{\text{audio}}} = \frac{2.4192\,\text{ms}}{10.0\,\text{ms}} \approx 0.2419.$$

Vì $\text{RTF} = 0.2419 \ll 1.0$, phần cứng chỉ dùng chưa đến 25% thời gian có sẵn của mỗi khung. Hệ thống có biên an toàn dồi dào ($> 75\%$ thời gian rảnh rỗi), cho phép xung nhịp có thể hạ xuống $25\,\text{MHz}$ để tiết kiệm năng lượng hoặc dùng chung khối DSP cho các thuật toán khác mà không bao giờ lo tràn bộ đệm.

Bài tập 0.3 (Đặc tính Bộ nhớ M10K và Xung đột Đọc-khi-Ghi): Một kỹ sư đề xuất thiết kế bộ đệm trượt $L = 640$ mẫu âm thanh bằng khối RAM On-Chip Memory 2-PORT trên Cyclone V, trong đó Cổng A liên tục ghi mẫu mới từ ADC vào địa chỉ k trong khi Cổng B đọc mẫu cũ tại chính địa chỉ k đó để chuyển cho khối tự tương quan tính toán. Kỹ sư này cấu hình chế độ ngõ ra là "New Data" để hy vọng đọc được mẫu vừa ghi tức thì.

a) Kiến trúc silicon thật của khối M10K trên Intel Cyclone V có hỗ trợ chế độ "New Data" cho hai cổng độc lập (mixed-port read-during-write) hay không?

b) Điều gì sẽ xảy ra nếu phần mềm tổng hợp Quartus gặp cấu hình này? Giải pháp thiết kế đúng đắn để tránh lỗi dữ liệu rác là gì?

Lời giải giải thích:

a) Không. Cấu tạo vật lý của các khối nhớ nhúng M10K trên dòng vi mạch Intel Cyclone V **hoàn toàn không hỗ trợ** chế độ "New Data" khi đọc và ghi đồng thời vào cùng một địa chỉ từ hai cổng khác nhau. Trong tài liệu chính thức *Cyclone V Device Handbook*, Intel quy định rõ: đối với chế độ True Dual-Port hoặc Simple Dual-Port trên các xung nhịp khác nhau hoặc cùng một xung nhịp nhưng khác cổng, ngõ ra dữ liệu khi xảy ra xung đột địa chỉ chỉ có thể là **"Old Data"** (xuất dữ liệu trước khi ghi) hoặc **"Don't Care"** (dữ liệu không xác định).

b) Nếu cố tình ép chế độ "New Data", Quartus Prime bắt buộc phải chèn thêm hàng chục khối logic ALM ngoại vi để tạo mạch ghép kênh đường vòng (bypass multiplexer logic), làm giảm đáng kể tần số hoạt động cực đại F_{max} và tăng diện tích chip. Giải pháp thiết kế chuẩn mực là: phân định rõ thời gian ghi mẫu mới và đọc khung cũ theo cơ chế lệch pha (ping-pong buffer hoặc quản lý con trỏ modulo vòng tròn với khoảng cách đọc/ghi cách xa nhau ít nhất $H = 160$ địa chỉ), bảo đảm Cổng A và Cổng B không bao giờ truy cập trùng một địa chỉ RAM trong cùng một chu kỳ xung nhịp.

Bài tập 0.4 (Rò Phổ và Đơn Vị Decibel): Biên độ thùy phụ đầu tiên của cửa sổ chữ nhật là $-13\,\text{dB}$ so với thùy chính, trong khi cửa sổ Hanning đạt mức suy giảm $-32\,\text{dB}$.

a) Đổi các mức suy giảm $-13\,\text{dB}$ và $-32\,\text{dB}$ sang tỷ số biên độ tuyến tính tương đối (A/A_0).

b) Giải thích tại sao việc suy giảm năng lượng rò rỉ từ 22.4% xuống 2.5% lại có ý nghĩa sống còn trong việc ngăn chặn hiện tượng nhảy quãng tám (octave error) của hàm tự tương quan.

Lời giải giải thích:

a) Theo định nghĩa đơn vị decibel cho biên độ tín hiệu áp suất âm thanh hoặc điện áp: $L_{\text{dB}} = 20\log_{10}(A/A_0) \Rightarrow A/A_0 = 10^{L_{\text{dB}}/20}$.

- Với cửa sổ chữ nhật: $A/A_0 = 10^{-13/20} = 10^{-0.65} \approx 0.22387$ (tương đương 22.39%, tức biên độ chỉ giảm khoảng 4.47 lần).
- Với cửa sổ Hanning: $A/A_0 = 10^{-32/20} = 10^{-1.60} \approx 0.02512$ (tương đương 2.51%, tức biên độ giảm tới 39.81 lần).

b) Năng lượng sóng âm tỷ lệ với bình phương biên độ. Với cửa sổ chữ nhật, thùy phụ còn giữ tới $0.22387^2 \approx 5.01\%$ năng lượng; khi giọng nói phát ra một nguyên âm có các tần số cộng hưởng formant bậc cao (ví dụ F_1, F_2 ở dải $800 \ldots 2500$ Hz), năng lượng rò rỉ qua các thùy phụ này sẽ tạo nên các gợn sóng nhấp nhô có chu kỳ ngắn đè lên hàm tự tương quan. Các đỉnh phụ này có thể vượt qua đỉnh chu kỳ thật của dây thanh quản, dẫn đến hiện tượng thuật toán bắt nhầm đỉnh phụ và báo sai cao độ lên gấp đôi hoặc xuống một nửa. Cửa sổ Hanning ép năng lượng rò rỉ xuống chỉ còn $0.02512^2 \approx 0.063\%$ (giảm gần 80 lần năng lượng rò so với chữ nhật), giữ cho đáy hàm tương quan giữa các chu kỳ luôn bằng phẳng và sạch sẽ.

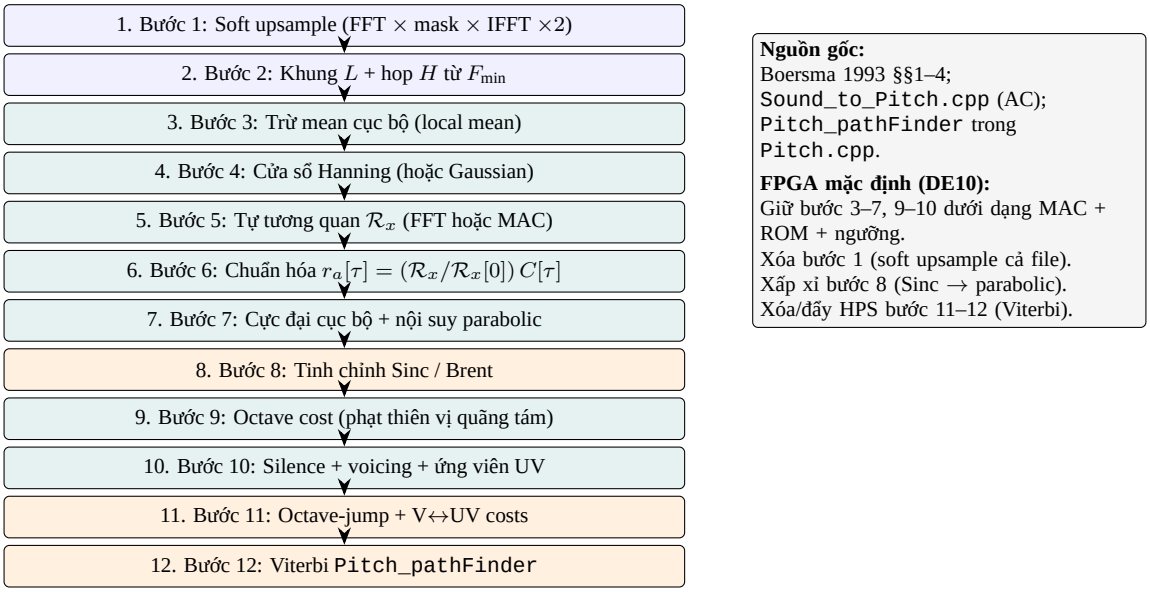
Chương 1. Nguyên lý Praat từ vật lý đến toán (Boersma 1993)

▶ MỤC TIÊU HỌC TẬP

Chương này lần theo đúng mười hai bước của thuật toán đo cao độ F_0 trong phần mềm Praat (Boersma 1993; mã nguồn C++ `Sound_to_Pitch.cpp` và `Pitch.cpp`, nhánh autocorrelation). Với mỗi bước, bạn sẽ: hiểu hiện tượng vật lý mà bước đó bảo vệ; dẫn công thức từ định nghĩa gốc; tính tay một ví dụ số nhỏ đã kiểm bằng tập lệnh `verify/verify_chuong_00_01.py`; thấy ánh xạ silicon trên kit DE10-Standard (giữ, xấp xỉ hay xóa); và nắm mô hình Python khớp bit cùng ngân sách ALM/DSP/M10K/RTF. Các khái niệm đã giải thích ở Chương 0 (ADC, Nyquist, aliasing, framing, mainlobe/sidelobe, Hanning, tự tương quan, Wiener–Khinchin, butterfly, BFP, Q$m.n$, Avalon-ST, RTF,…) chỉ được *tham chiếu*, không định nghĩa lại.

1.0. Bản đồ mười hai bước: giấy Boersma ↔ C++ ↔ silicon

Hình 9 tóm tắt chuỗi đầy đủ Praat AC và ghi chú ngay trên bản đồ những bước mà phần cứng streaming trên Cyclone V sẽ giữ nguyên, xấp xỉ hoặc xóa có chủ đích. Kỹ sư phải hiểu bước đã xóa *trước khi* xóa — đó là nguyên tắc xuyên suốt Chương 1.



Hình 9: Chuỗi mười hai bước Praat autocorrelation. Ô xanh ngọc (`teal!10`): giữ gần bit-exact trên fabric (bước 3–7, 9–10). Ô cam (`orange!12`): xấp xỉ hoặc đẩy sang HPS (bước 8, 11–12). Ô xanh dương nhạt (`blue!6`): soft upsample / framing bị xóa hoặc thay trên streaming (bước 1–2; Bước 1 thay bằng FIR chống alias ở Chương 4).

1.1. Bước 1 — Soft upsampling: dọn mép Nyquist trước khi cắt khung

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: năng lượng sát Nyquist bị cửa sổ "quăng" xuống vùng F_0

Chương 0 đã giải thích lấy mẫu, tần số Nyquist $f_N = f_s/2$, aliasing và rò phổ (mainlobe/sidelobe) của cửa sổ Hanning. Hệ quả thực tế đối với đo cao độ là: nếu một phát ngôn (*utterance*) còn năng lượng gần f_N (tiếng rít quạt, nhiễu cơ học, sóng hài cao), thì khi nhân cửa sổ Hanning dài 40 ms, các thùy phụ khoảng -32 dB sẽ phát tán năng lượng đó xuống đúng dải tìm kiếm $75 \ldots 600$ Hz. Thuật toán sẽ thấy các đỉnh tự tương quan giả — không phải dây thanh tạo ra chúng.

Boersma (1993, Step 1) xử lý cả một utterance trên máy tính như sau: (1) tính FFT toàn chuỗi; (2) nhân phổ với mặt nạ mềm $G[k]$ giảm tuyến tính từ $0{,}95 \cdot f_N$ xuống 0 đúng tại f_N; (3) chèn zero giữa nửa phổ dương và nửa phổ âm rồi IFFT dài $2N$. Kết quả là upsample "mềm" $\times 2$ đồng thời đã dọn sạch mép Nyquist. Đây là thao tác tính toán trên cả câu nói nằm trong RAM, không phải hiện tượng vật lý mới của dây thanh.

Toán từng bước. Đặt $f_N = f_s/2$. Mặt nạ phổ được định nghĩa theo bin tần số k (với N điểm FFT, bin Nyquist là $k = N/2$):

$$G[k] = \begin{cases} 1, & \dfrac{|k|}{N} < 0{,}95 \cdot \dfrac{1}{2}, \\ \dfrac{f_N - |k|\, f_s/N}{f_N - 0{,}95\, f_N}, & 0{,}95\, f_N \le |k|\, f_s/N \le f_N, \\ 0, & |k|\, f_s/N > f_N. \end{cases}$$

Chuỗi sau xử lý là $x_{\text{up}} = \text{IFFT}_{2N}\{\text{pad}(G[k]\, X[k])\}$. Phép zero-pad trong miền tần số rồi IFFT dài gấp đôi chính là nội suy Fourier lý tưởng: lưới thời gian dày gấp hai, phổ hữu ích dưới Nyquist cũ được giữ nguyên (xem lại định nghĩa zero-padding ở Chương 0).

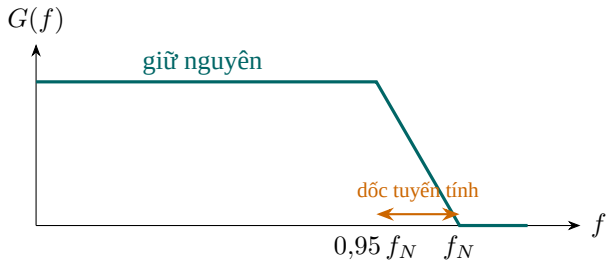
Ví dụ tính tay phổ 8 điểm. Lấy tín hiệu "cứng" đúng tại tần số Nyquist của lưới 8 điểm:

$$x = [1,\ 0,\ -1,\ 0,\ 1,\ 0,\ -1,\ 0].$$

DFT 8 điểm (không chuẩn hóa) cho năng lượng tập trung ở bin gần mép (bin $k = 2$ và đối xứng), còn bin Nyquist $k = 4$ bằng 0 trên tín hiệu này vì chu kỳ khớp lưới:

k	Re $X[k]$	Im $X[k]$	$\lvert X[k]\rvert^2$
0	0	0	0
1	0	0	0
2	4	0	16
3	0	0	0
4 (Nyquist)	0	0	0

Soft filter nhân mỗi bin với $G[k]$: giữ nguyên khi $k/N < 0{,}95/2$, rồi giảm tuyến tính về 0 tại bin Nyquist. Sau đó IFFT dài 16 mẫu — tương đương nội suy dày gấp đôi trên lưới thời gian. (Các số DFT trên đã kiểm bằng `numpy.fft.fft`; xem `verify/verify_chuong_00_01.py`.)



Hình 10: Mặt nạ phổ mềm Boersma Step 1: giữ nguyên dưới $0{,}95\ f_N$, dốc tuyến tính về 0 tại Nyquist, rồi zero-pad và IFFT dài $2N$ để upsample $\times 2$.

Ý nghĩa phần cứng. Trên FPGA streaming, mẫu âm thanh đến từng nhịp đồng hồ: không thể đợi hết cả câu nói rồi FFT toàn utterance. DE10 thay bước 1 bằng (1) lọc chống alias analog trước ADC của codec WM8731, và (2) FIR/decimation sạch từ 48 kHz xuống 16 kHz (Chương 4, FIR Compiler II). Cùng mục tiêu vật lý — đừng để năng lượng gần Nyquist giả thành chu kỳ thấp trong vùng F_0 — khác công cụ.

Twin Python bit-exact. Twin mặc định *bỏ qua* soft upsample và bắt đầu từ PCM đã anti-alias. Lab đối chiếu Praat đầy đủ có thể chạy offline: `np.fft.rfft` → nhân $G[k]$ (cùng dốc $0{,}95\ f_N \to f_N$) → `irfft(n=2N)`, rồi mới đưa vào đường pitch. Sai điểm cắt mask sẽ lệch năng lượng mép phổ so với Praat gốc.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 1 Soft upsample.

- Trên fabric streaming: **0 ALM / 0 DSP / 0 M10K** cho Step 1 (không thực hiện FFT cả utterance).
- Thay thế: anti-alias analog + FIR Compiler II (Chương 4) — vài chục khối DSP và vài M10K hệ số lọc.
- Twin đối chiếu Praat: soft-upsample offline trên Python/HPS trước khi đưa PCM vào đường fixed-point — không tính vào RTF thời gian thực.
- Quên dọn Nyquist: tone $\sim 7 \ldots 8$ kHz giả thành chu kỳ dài trong vùng $75 \ldots 600$ Hz, làm sai F_0 dù khối MAC phía sau hoàn toàn đúng.

Bài tập 1.1. Một utterance tại $f_s = 16$ kHz còn nhiễu 9 kHz. (a) Tần số alias sau ADC là bao nhiêu? (b) Soft upsample của Praat có ngăn được aliasing *đã xảy ra ở* ADC không, hay chỉ dọn năng lượng còn lại dưới Nyquist trước khi cửa sổ hóa? Giải thích.

Lời giải. (a) $f_{\text{alias}} = |16 - 9| = 7$ kHz (đã giải ở Chương 0). (b) Soft upsample *không* đảo ngược aliasing đã xảy ra trong ADC: thành phần 7 kHz đã nằm trong dải gốc. Nó chỉ cắt/làm mềm phần phổ còn lại gần f_N *trước khi* nhân cửa sổ, giảm rò phổ xuống vùng F_0. Trên FPGA, việc chống alias thật sự phải làm bằng lọc analog + FIR trước hạ mẫu (Chương 4).

1.2. Bước 2 — Khung, time step và độ dài cửa sổ từ pitch floor

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: cần thấy ít nhất ba chu kỳ dây thanh trong một khung

Để hàm tự tương quan "nhìn thấy" đỉnh tại $\tau = T_0$ và cả đỉnh phụ tại $\tau = 2T_0$, khung phải dài hơn vài chu kỳ dây thanh. Dây thanh nam có thể chậm tới khoảng 75 Hz, tức $T_0 = 1/75 \approx 13{,}33$ ms. Boersma chọn `periodsPerWindow` $= 3$ với cửa sổ Hanning: ba chu kỳ $= 40$ ms — đủ dài để hai đỉnh nằm trong cửa sổ, nhưng không dài đến mức F_0 đổi nhiều trong một khung (giả định *local-stationary*: cao độ gần như đứng yên trong cửa sổ). Tuỳ chọn Gaussian (`very_accurate`) dùng 6 chu kỳ vì đuôi Gaussian dài hơn trong miền thời gian; mã nguồn nhân đôi `periodsPerWindow` rồi tăng độ sâu nội suy Brent (`Sound_to_Pitch.cpp`, khoảng dòng 405–415).

Toán từng bước. Với $F_{\min}$ (pitch floor, mặc định 75 Hz), f_s (tần số lấy mẫu sau hạ mẫu, 16 kHz trên DE10) và số chu kỳ trong cửa sổ $P = 3$:

$$L_{\text{sec}} = \frac{P}{F_{\min}} = \frac{3}{75} = 0{,}04\,\text{s},$$
$$L = L_{\text{sec}} \cdot f_s = 0{,}04 \times 16000 = 640\,\text{mẫu}.$$

Bước nhảy thời gian mặc định `time_step` $= 0{,}01$ s cho hop

$$H = 0{,}01 \times 16000 = 160\,\text{mẫu}.$$

Tỉ lệ chồng lấn giữa hai khung liên tiếp:

$$\frac{L - H}{L} = \frac{640 - 160}{640} = 0{,}75 = 75\%.$$

Mỗi ước lượng F_0 mới tái sử dụng 480 mẫu đã thấy ở khung trước — đường cong mượt hơn, đổi lại gấp bốn lần số lần tính so với hop $= L$. Nếu `time_step` $= 0$, Praat tự chọn $0{,}75/F_{\min}$ để có khoảng bốn phép đo trên mỗi cửa sổ.

Khoảng trễ tìm kiếm cao độ:

$$\tau_{\min} = \lfloor f_s/F_{\max} \rfloor = \lfloor 16000/600 \rfloor = 26 \quad \text{(lab thường lấy 27 hoặc 32)},$$
$$\tau_{\max} = \lfloor f_s/F_{\min} \rfloor = \lfloor 16000/75 \rfloor = 213.$$

Ngoài đoạn $[\tau_{\min}, \tau_{\max}]$ không có ứng viên F_0 hợp lệ. Trên fabric DE10, vòng MAC quét $\tau = 26 \ldots 214$ (đúng 189 giá trị trễ) để khớp ngân sách đã tính ở Chương 0.

Ví dụ tính tay. $F_{\min} = 75$ Hz, $f_s = 16$ kHz, Hanning 3 chu kỳ: $L = 640$, $H = 160$, chồng lấn 75%, $\tau_{\min} = 26$, $\tau_{\max} = 213$. Đổi $F_{\min} \to 50$ Hz sẽ cho $L = 960$, $\tau_{\max} = 320$ — phải dựng lại toàn bộ ROM cửa sổ và bảng $C[\tau]$, không chỉ sửa một hằng số. (Đã kiểm: `verify_chuong_00_01.py`, hàm `test_hardware_budget`.)

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 2 Frame/hop.

- Giữ cố định $L = 640$, $H = 160$ trên FPGA: khớp đúng 1 khối M10K đệm khung ($640 \times 16 = 10240$ bit) cộng ROM $C[\tau]$ sâu khoảng 189 từ.
- Đổi $F_{\min} \Rightarrow$ đổi L, $\tau_{\max}$, và *bắt buộc rebuild* ROM w lẫn $C[\tau]$.
- RTF: mỗi 10 ms phải xong một khung; tại 50 MHz với 1 MAC/chu kỳ, $640 \times 189 = 120960$ phép nhân mất 2,42 ms (RTF = 0,242), dư sức — xem lại hộp RTF Chương 0.

Bài tập 1.2. Cho $F_{\min} = 100$ Hz, $f_s = 16$ kHz, Hanning 3 chu kỳ, `time_step` $= 0{,}01$ s. Tính L, H, tỉ lệ chồng lấn và $\tau_{\max}$. So sánh số phép MAC mỗi khung với cấu hình $F_{\min} = 75$ Hz (giả sử $\tau_{\min} = 26$ không đổi và τ quét đến $\tau_{\max}$).

Lời giải. $L = 3 \cdot 16000/100 = 480$ mẫu; $H = 160$; chồng lấn $(480 - 160)/480 = 66{,}7\%$; $\tau_{\max} = \lfloor 16000/100 \rfloor = 160$. Số lag $= 160 - 26 + 1 = 135$. MAC $= 480 \times 135 = 64800$ — khoảng một nửa so với 120960 của cấu hình 75 Hz. Khung ngắn hơn và dải τ hẹp hơn đều giảm chi phí, nhưng mất khả năng đo giọng rất trầm dưới 100 Hz.

1.3. Bước 3 — Trừ mean cục bộ (local mean)

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: DC và trôi chậm phá đỉnh $\tau = 0$

Áp suất không khí trung bình, offset mạch mic và rung máy đều cộng một hằng số A vào mọi mẫu. Nhắc lại định nghĩa tự tương quan không vòng trên khung N mẫu (đã nêu ở Chương 0):

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{N-1-\tau} x[n]\, x[n+\tau].$$

Tại $\tau = 0$ chỉ còn tổng bình phương: $\mathcal{R}_x[0] = \sum_{n=0}^{N-1} x[n]^2$ — đúng bằng năng lượng khung. Nếu tín hiệu mang offset, đặt $y[n] = x[n] + A$:

$$\mathcal{R}_y[0] = \sum_{n=0}^{N-1} (x[n] + A)^2 = \sum x[n]^2 + 2A \sum x[n] + NA^2 = \mathcal{R}_x[0] + 2A \sum x[n] + NA^2.$$

Đỉnh tại $\tau = 0$ phình tỉ lệ với A^2 và dễ "nuốt" đỉnh chu kỳ khi so với ngưỡng voicing. Praat trừ mean cục bộ trên khoảng $\pm$ một chu kỳ dài nhất quanh tâm khung (`localMean /= 2*nsamp_period` trong mã nguồn).

Ví dụ tính tay 4 mẫu. Cho $x = [3, 5, 3, 5]$, mean $\mu = 4$, tín hiệu không mean $x_{\text{zm}} = [-1, 1, -1, 1]$. Với $N = 4$:

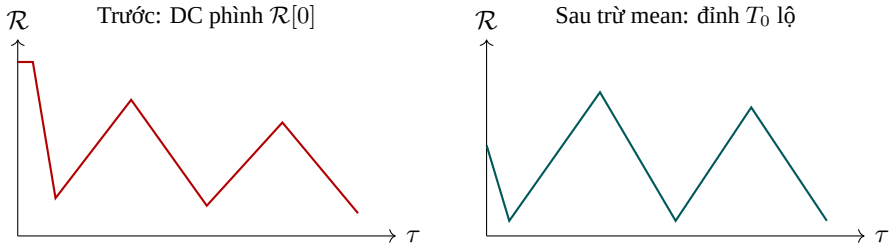
$$\mathcal{R}_{\text{zm}}[0] = (-1)^2 + 1^2 + (-1)^2 + 1^2 = 4,$$
$$\mathcal{R}_{\text{zm}}[2] = (-1) \cdot (-1) + 1 \cdot 1 = 2 \quad \text{(chỉ còn } N - \tau = 2 \text{ cặp)}.$$

Tỉ số thô $\mathcal{R}[2]/\mathcal{R}[0] = 2/4 = 0{,}5$ — *không* bằng 1, dù tín hiệu tuần hoàn chu kỳ 2. Muốn tỉ số về 1 phải chuẩn hóa theo số cặp (unbiased) $\frac{\mathcal{R}[\tau]/(N-\tau)}{\mathcal{R}[0]/N}$, hoặc theo đúng Praat chia thêm tự tương quan cửa sổ $\mathcal{R}_w$ (Bước 6). Với unbiased: $\frac{2/2}{4/4} = 1$.

Nếu quên trừ mean, trên chính $x = [3, 5, 3, 5]$:

$$\mathcal{R}_x[0] = 3^2 + 5^2 + 3^2 + 5^2 = 68,$$
$$\mathcal{R}_x[2] = 3 \cdot 3 + 5 \cdot 5 = 34.$$

Tỉ số thô $34/68 = 0{,}5$; sau unbiased vẫn ra 1 trên tín hiệu tuần hoàn này, nhưng đỉnh $\tau = 0$ đã phình gấp $68/4 = 17$ lần so với bản không mean — đúng bệnh DC làm méo thang đo trước ngưỡng voicing trên tín hiệu thật. (Đã kiểm Python.)



Hình 11: Cùng một tín hiệu có offset DC: trừ mean cục bộ trước cửa sổ là bắt buộc để đỉnh chu kỳ không bị nuốt bởi $\mathcal{R}[0]$.

Ý nghĩa phần cứng / Twin. FPGA tách hai lớp: DC-blocker IIR streaming (Chương 3) nuốt offset chậm; thêm mean khung (cộng dồn L mẫu rồi trừ) trước nhân cửa sổ nếu twin muốn khớp Praat chặt. Twin Python: `x = x - np.mean(x)` trên đúng cửa sổ $\pm$ một chu kỳ dài nhất quanh tâm, hoặc tương đương DC-blocker + mean khung.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 3 Local mean.

- DC-blocker IIR 1-cực 1-zero (Chương 3): vài ALM, 0 DSP, 1 thanh ghi 32-bit.
- Mean khung: 1 adder tích lũy + nhân $1/L$ định dạng Q — dưới 50 ALM.
- Không trừ mean: $\mathcal{R}[0]$ phình, ngưỡng voicing và r_a sai tỉ lệ — hỏng cả chuỗi sau dù MAC đúng.

Bài tập 1.3. Cho $x = [2, 4, 2, 4]$. Tính μ, x_{zm}, $\mathcal{R}_{\text{zm}}[0]$, $\mathcal{R}_{\text{zm}}[2]$, tỉ thô và tỉ unbiased. Tính thêm $\mathcal{R}_x[0]$ khi không trừ mean và hệ số phình $\mathcal{R}_x[0]/\mathcal{R}_{\text{zm}}[0]$.

Lời giải. $\mu = 3$, $x_{\text{zm}} = [-1, 1, -1, 1]$ — trùng ví dụ trên: $\mathcal{R}_{\text{zm}}[0] = 4$, $\mathcal{R}_{\text{zm}}[2] = 2$, thô $0{,}5$, unbiased 1. Không trừ: $\mathcal{R}_x[0] = 4 + 16 + 4 + 16 = 40$, $\mathcal{R}_x[2] = 4 + 16 = 20$, phình $40/4 = 10$ lần.

1.4. Bước 4 — Cửa sổ Hanning (và tuỳ chọn Gaussian)

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: cắt khung đột ngột gây rò phổ; Hanning vuốt biên

Chương 0 đã định nghĩa framing, cửa sổ, mainlobe, sidelobe và đơn vị dB. Nhắc ngắn: framing *không* cắt đứt tín hiệu gốc rồi bỏ phần còn lại, mà nhân toàn bộ $x[n]$ với một cửa sổ trượt $w[n - kH]$ chỉ khác không trên đúng L mẫu của khung k. Cửa sổ chữ nhật nhảy xuống 0 ở hai đầu; phổ sinc của nó có thùy phụ đầu khoảng -13 dB (biên độ còn $\approx 1/4{,}5$). Cửa sổ Hanning vuốt hai đầu về gần 0 bằng nửa chu kỳ cosine, kéo thùy phụ xuống khoảng -32 dB (biên độ $\approx 1/40$), bảo vệ dải F_0 khỏi bóng ma tần số cao.

Toán từng bước — công thức Praat dùng $N + 1$. Praat (`Sound_to_Pitch.cpp`) định nghĩa trên N mẫu của khung:

$$w[i] = 0{,}5 - 0{,}5 \cos \frac{2\pi i}{N + 1}, \qquad i = 1, 2, \ldots, N.$$

Các ký hiệu: i là chỉ số mẫu *trong* khung (chạy $1..N$, không phải $0..N-1$); N là số mẫu khung (640 trên DE10); mẫu số $N + 1$ là lựa chọn có chủ đích của Praat để $w[1]$ và $w[N]$ vẫn > 0 nhỏ — mọi ô trong bộ nhớ đều đóng góp năng lượng, không "chết" hai mẫu biên (giáo trình cổ điển thường dùng $N - 1$ và cho $w[0] = w[N-1] = 0$).

Ví dụ tính tay $N = 4$ (đã kiểm Python).

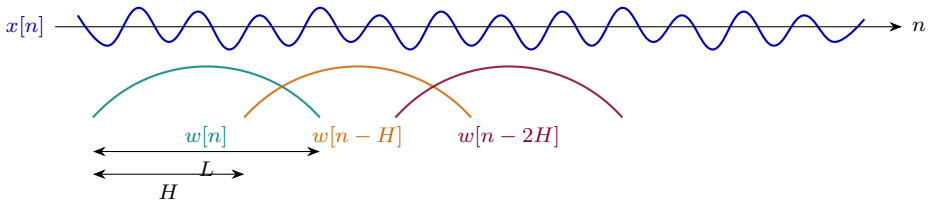
$$w[i] = 0{,}5 - 0{,}5 \cos \frac{2\pi i}{5}, \quad i = 1, 2, 3, 4.$$

$$
\begin{aligned}
i=1:\ & 0{,}5-0{,}5\cos(2\pi/5)=0{,}5-0{,}5(-0{,}309017)=0{,}345491,\\
i=2:\ & 0{,}5-0{,}5\cos(4\pi/5)=0{,}5-0{,}5(-0{,}809017)=0{,}904508,\\
i=3:\ & 0{,}904508, \qquad i=4:\ 0{,}345491.
\end{aligned}
$$

Vậy $w \approx [0{,}3455,\ 0{,}9045,\ 0{,}9045,\ 0{,}3455]$ — đối xứng. Từ đây

$$
\begin{aligned}
\mathcal{R}_w[0] &= \sum_{i=1}^{4} w_i^2 = 1{,}875,\\
\mathcal{R}_w[1] &= \sum_{i=1}^{3} w_i w_{i+1} \approx 1{,}443136,\\
C[1] &= \mathcal{R}_w[0]/\mathcal{R}_w[1] \approx 1{,}299254.
\end{aligned}
$$

Tích $w[n]w[n+\tau]$ suy giảm khi τ tăng — đúng gốc của bảng $C[\tau]$ ở Bước 6 và Chương 8.



Hình 12: Framing = nhân $x[n]$ với cửa sổ trượt $w[n-kH]$. Hop H dịch cửa sổ; chồng lấn 75% khi $L=640$, $H=160$.

Tuỳ chọn Gaussian. Gaussian $w[n] \propto \exp\!\left(-\tfrac{1}{2}(n/\sigma)^2\right)$ hẹp hơn trong tần số (sidelobe thấp hơn) nhưng đuôi dài hơn trong thời gian. Praat khi bật `very_accurate` nhân đôi `periodsPerWindow` ($3 \to 6$) và dùng nội suy SINC700. Trên FPGA điều đó nghĩa là khoảng $2\times$ M10K cửa sổ và $2\times$ MAC. Mặc định silicon: Hanning 3 chu kỳ.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 4 Window.

- ROM Hanning: 640×16-bit (Q1.15) $= 10240$ bit $= 1$ khối M10K; thực tế chỉ cần nửa bảng nhờ đối xứng $w[n] = w[L-1-n]$ (Chương 6).
- Nhân mẫu $\tilde{x}[n] = (x[n]-\mu)\,w[n]$: 1 khối DSP.
- Gaussian 6 chu kỳ: $L \approx 1280$, ROM+MAC roughly $\times 2$ — chỉ bật khi lab đối chiếu “very accurate”.

Bài tập 1.4. Tính $w[1]$ và $w[2]$ theo công thức Praat với $N=8$. Kiểm đối xứng $w[i] = w[N+1-i]$.

Lời giải. $w[i] = 0{,}5 - 0{,}5\cos(2\pi i/9)$. $w[1] = 0{,}5 - 0{,}5\cos(2\pi/9) \approx 0{,}5 - 0{,}5\cdot 0{,}7660 = 0{,}1170$; $w[2] = 0{,}5 - 0{,}5\cos(4\pi/9) \approx 0{,}5 - 0{,}5\cdot 0{,}1736 = 0{,}4132$. Đối xứng: $w[8]=w[1]$, $w[7]=w[2]$,…

1.5. Bước 5 — Tự tương quan: Wiener–Khinchin hoặc MAC trực tiếp

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: đỉnh tại $\tau = T_0$ đo chu kỳ dây thanh

Chương 0 đã định nghĩa hàm tự tương quan $\mathcal{R}_x[\tau]$, độ trễ (lag) và Định lý Wiener–Khinchin: $\mathcal{R}_x = \mathcal{F}^{-1}\{|X|^2\}$. Khi dây thanh rung tuần hoàn với chu kỳ T_0 ($\tau_0 = f_s \cdot T_0$ mẫu), trượt đúng $\tau = \tau_0$ làm các cặp mẫu cùng pha nhân với nhau đạt cực đại. Đo được τ_0 thì $F_0 = f_s/\tau_0$. Định lý Wiener–Khinchin mở hai đường thực thi tương đương toán học: FFT $\to |X|^2 \to$ IFFT, hoặc MAC thời gian trực tiếp.

Toán từng bước. Praat AC path pad zero đến

$$
\texttt{nsampFFT} = 2^{\lceil \log_2(N\cdot(1+\texttt{depth}))\rceil}
$$

với Hanning depth 0,5: $640 \times 1{,}5 = 960 \to 1024$ điểm FFT. Sau IFFT lấy $\mathcal{R}_x[\tau]$ tại các trễ nguyên. Đường FPGA mặc định thay bằng:

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} \tilde{x}[n]\,\tilde{x}[n+\tau], \quad \tau \in [\tau_{\min}, \tau_{\max}],$$

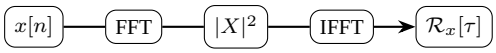
một phép nhân-cộng (MAC) mỗi chu kỳ đồng hồ trên 1 khối DSP.

Ví dụ tính tay DFT 8 điểm + butterfly. Cho $x = [1, 1, -1, -1, 1, 1, -1, -1]$ (chu kỳ 4). Stage 1 radix-2 với twiddle $W = 1$ (cố ý để thấy lõi cộng/trừ; stage sau mới nhân twiddle phức — xem định nghĩa butterfly/twiddle ở Chương 0):

Butterfly	a	b	$a \pm Wb$ ($W = 1$)
pair0	$x[0] = 1$	$x[4] = 1$	2, 0
pair1	1	1	2, 0
pair2	-1	-1	-2, 0
pair3	-1	-1	-2, 0

Sau khi có $X[k]$, bình phương $|X|^2$, rồi IFFT cho $\mathcal{R}[\tau]$ lớn tại $\tau = 0$ và $\tau = 4$ — đúng chu kỳ của x.

Ví dụ Parseval sạch hơn với xung chữ nhật $x = [1, 1, 1, 1, 0, 0, 0, 0]$: $\sum |x|^2 = 4$; DFT cho $X[0] = 4$, $X[1] = 1 - j(1 + \sqrt{2})$, $X[2] = 0$,…và $\frac{1}{N} \sum |X|^2 = 4$ khớp Parseval. (Đã kiểm `test_dft_8_manual`.)



Praat: pad đến $nsampFFT = 1024$ (Hanning depth 0,5).
FPGA mặc định: MAC trực tiếp (cùng $\mathcal{R}$, khác đường sai số).

Hình 13: Đường AC trong `Sound_to_Pitch.cpp`: FFT → power → IFFT. Silicon DE10 thay bằng MAC (Chương 7).

BFP nhắc ngắn. Nếu lab bật FFT IP, nhớ Block Floating Point: khi lấy $|X|^2$ phải nhân lại 2^{2e} trước IFFT (đã giải ở Chương 0). Quên bước này làm sai toàn bộ tỉ lệ $\mathcal{R}[\tau]$.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 5 ACF.

- MAC mặc định: 1 DSP, accumulator 48-bit, 120960 chu kỳ/khung @ 50 MHz $\Rightarrow$ 2,42 ms, RTF = 0,242.
- FFT 1024 tùy chọn: vài nghìn butterfly + twiddle ROM + BFP — đắt hơn khi $\tau_{\max} \ll L$; chỉ lab đối chiếu.
- Twin: vòng `for tau` nhân dồn khớp bit với RTL, hoặc `np.fft` khi so Praat float.

Bài tập 1.5. Với $x_{\text{zm}} = [-1, 1, -1, 1]$, tính $\mathcal{R}[0]$, $\mathcal{R}[1]$, $\mathcal{R}[2]$ theo định nghĩa không vòng. Đỉnh chu kỳ nằm ở τ nào?

Lời giải. $\mathcal{R}[0] = 4$; $\mathcal{R}[1] = (-1)(1) + (1)(-1) + (-1)(1) = -3$; $\mathcal{R}[2] = 2$. Đỉnh dương tại $\tau = 2$ (và $\tau = 0$), đúng chu kỳ 2.

1.6. Bước 6 — Chuẩn hóa cho tự tương quan cửa sổ $\mathcal{R}_w[\tau]$

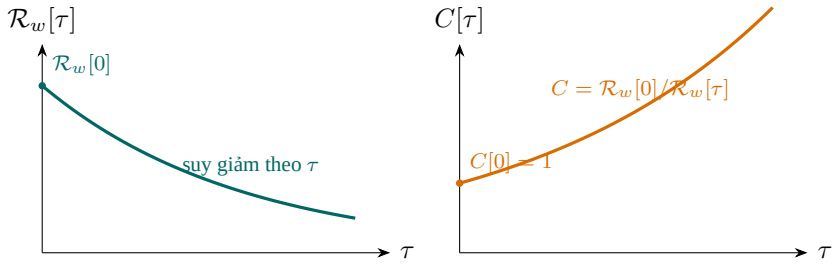
★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng / toán Boersma: hai quả núi Hanning ít chồng khi τ lớn

Sau khi nhân cửa sổ, ngay cả tín hiệu tuần hoàn hoàn hảo cũng có $\mathcal{R}_x[\tau]$ suy giảm khi τ tăng, vì hai bản sao cửa sổ chồng ít hơn. Nếu không bù, thuật toán thiên vị chọn τ nhỏ (cao độ cao giả). Boersma chuẩn hóa:

$$r_a[\tau] = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0] \cdot \mathcal{R}_w[\tau]/\mathcal{R}_w[0]} = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0]} \cdot C[\tau], \quad C[\tau] = \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]}.$$

Mã: `r[i] = ac[i+1] / (ac[1] * windowR[i+1])` (`Sound_to_Pitch.cpp`). $C[\tau]$ chỉ phụ thuộc cửa sổ — tính offline một lần.

Ví dụ tính tay $N = 4$**,** $\tau = 1$**.** Với $w \approx [0{,}3455, 0{,}9045, 0{,}9045, 0{,}3455]$ đã tính ở Bước 4: $\mathcal{R}_w[0] = 1{,}875$, $\mathcal{R}_w[1] \approx 1{,}443$, $C[1] \approx 1{,}299$. Nhân r thô với $1{,}299$ kéo đỉnh gần biên về công bằng với đỉnh giữa. (Đã kiểm Python.)



(a) Tự tương quan cửa sổ (b) Hệ số bù tăng theo τ

Hình 14: Bước 6: hai quả núi Hanning chồng ít hơn khi τ lớn nên $\mathcal{R}_w[\tau]$ giảm (a); hệ số $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ tăng để kéo các đỉnh gần biên về công bằng với đỉnh giữa (b).

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 6 Norm.

- Bảng $C[\tau]$ tính offline từ đúng w đã dùng, lượng tử Q3.13, nạp ROM (Chương 8): $189 \times 16\,\text{bit} = 3024\,\text{bit}$ — chưa đầy 1 M10K.
- Runtime: 1 nhân DSP mỗi τ (nhân r thô với $C[\tau]$). Divider runtime từng τ là phương án khác, tốn ALM/latency — không mặc định.
- Twin: cùng bảng hex $C[\tau]$ với ROM phần cứng; so khớp từng từ Q3.13.

Bài tập 1.6. Với $w = [0{,}5, 1{,}0, 1{,}0, 0{,}5]$, tính $\mathcal{R}_w[0]$, $\mathcal{R}_w[1]$, $\mathcal{R}_w[2]$ và $C[1]$, $C[2]$.

Lời giải. $\mathcal{R}_w[0] = 0{,}25 + 1 + 1 + 0{,}25 = 2{,}5$; $\mathcal{R}_w[1] = 0{,}5 + 1{,}0 + 0{,}5 = 2{,}0$; $\mathcal{R}_w[2] = 0{,}5 + 0{,}5 = 1{,}0$. $C[1] = 2{,}5/2 = 1{,}25$; $C[2] = 2{,}5/1 = 2{,}5$. C tăng theo τ — đúng thiên vị cần bù.

1.7. Bước 7 — Trích cực đại cục bộ và nội suy parabolic

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: nhiều đỉnh = $T_0, 2T_0, \ldots$ cộng nhiễu formant

Tín hiệu tuần hoàn lý tưởng có đỉnh $\mathcal{R}$ tại mọi kT_0. Formant và nhiễu tạo đỉnh giả. Praat lấy tối đa 14 cực đại cục bộ thỏa $r[i] > r[i-1]$, $r[i] \geq r[i+1]$, và $r[i] > 0{,}5 \cdot$ `voicingThreshold`, cộng một ứng viên Unvoiced $\Rightarrow$ `maxnCandidates` $= 15$.
Đỉnh thật thường nằm *giữa* hai mẫu nguyên trên lưới $\tau \in \mathbb{Z}$. Nội suy parabolic khớp ba điểm quanh đỉnh bằng một parabola rồi lấy đỉnh của parabola — rẻ về số học, đủ chính xác cho FPGA (sai số thường dưới vài cent trên sine sạch). Nội suy sinc/Brent tinh hơn nhưng cần nhiều MAC; silicon mặc định dừng ở parabolic (xem lại so sánh ở Chương 0).

Toán từng bước. Đặt ba mẫu quanh đỉnh nguyên τ^*: $r_- = r[\tau^* - 1]$, $r^* = r[\tau^*]$, $r_+ = r[\tau^* + 1]$. Đặt gốc tại τ^* ($x = 0$ ở đỉnh). Parabola $y = ax^2 + bx + c$ đi qua $(-1, r_-)$, $(0, r^*)$, $(1, r_+)$. Đỉnh parabola tại

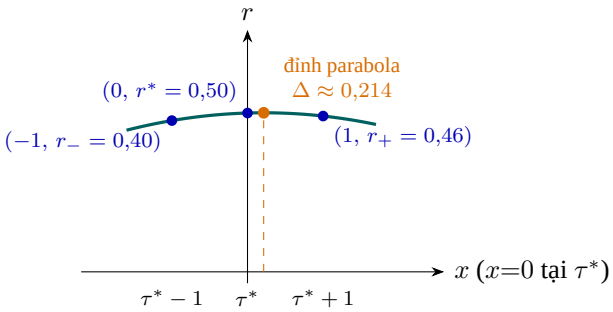
$$\Delta = \frac{1}{2} \cdot \frac{r_- - r_+}{r_- - 2r^* + r_+} = \frac{r_- - r_+}{2(r_- - 2r^* + r_+)}.$$

Trễ tinh: $\tau_{\text{fine}} = \tau^* + \Delta$. Cao độ: $F_0 = f_s / \tau_{\text{fine}}$.

Ví dụ tính tay 3 điểm. $r_- = 0{,}40$, $r^* = 0{,}50$, $r_+ = 0{,}46$ tại $\tau^* = 80$:

$$\Delta = \frac{0{,}40 - 0{,}46}{2\big(0{,}40 - 2 \cdot 0{,}50 + 0{,}46\big)} = \frac{-0{,}06}{2(-0{,}14)} = \frac{-0{,}06}{-0{,}28} = 0{,}214286.$$

$\tau_{\text{fine}} = 80{,}214286$ mẫu; tại $f_s = 16\,\text{kHz}$: $F_0 = 16000/80{,}214286 \approx 199{,}466\,\text{Hz}$. So với ước lượng thô $16000/80 = 200\,\text{Hz}$: lệch khoảng $4{,}63$ cent. (Đã kiểm Python.)



Hình 15: Bước 7: ba điểm (r_-, r^*, r_+) xác định duy nhất một parabola; đỉnh liên tục lệch khỏi mẫu nguyên một lượng $\Delta = (r_- - r_+)/(2(r_- - 2r^* + r_+))$. Ví dụ tay: $\tau^* = 80$, $\Delta \approx 0{,}214$.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 7 Peak/parabolic.

- Argmax một đỉnh (K= 1): comparator quét $\tau_{\text{min}}..\tau_{\text{max}}$, vài ALM, 0 DSP.
- Parabolic: 2 phép trừ, 1 chia (LPM_DIVIDE), 1 nhân $\frac{1}{2}$ — vài chu kỳ, 0–1 DSP (Chương 9).
- Top-14 chỉ khi HPS cần đa ứng viên cho Viterbi; mặc định fabric K= 1 + UV.

Bài tập 1.7. Cho $r = \{0{,}20,\ 0{,}60,\ 0{,}50\}$ tại $\tau^* = 100$. Tính Δ, τ_{fine} và F_0 @ 16 kHz.

Lời giải. $\Delta = (0{,}20 - 0{,}50)/\big(2(0{,}20 - 1{,}20 + 0{,}50)\big) = (-0{,}30)/(2(-0{,}50)) = 0{,}30$. $\tau_{\text{fine}} = 100{,}3$; $F_0 = 16000/100{,}3 \approx 159{,}52\,\text{Hz}$.

1.8. Bước 8 — Nội suy Sinc / Brent (đỉnh phụ mẫu tinh)

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: lưới mẫu rời ⇒ đỉnh liên tục nằm giữa hai mẫu

$\mathcal{R}(\tau)$ liên tục bị lấy mẫu tại $\tau \in \mathbb{Z}$. Nội suy sinc khôi phục tín hiệu band-limited. Praat: lần 1 parabolic; lần 2 `NUMimproveMaximum` với `NUM_PEAK_INTERPOLATE_SINC70` (Hanning) hoặc `SINC700` (Gaussian / very accurate) — `Sound_to_Pitch.cpp` khoảng dòng 247–254. "Depth" là bề rộng cửa sổ sinc quanh đỉnh: SINC70 lấy ± 35 mẫu; SINC700 lấy ± 350.

Ví dụ tính tay sinc 3 điểm. Giá trị tại $x \in \{-1, 0, 1\}$: $y = 0{,}40,\ 0{,}50,\ 0{,}46$. Xấp xỉ sinc cửa sổ 3 tại $t = 0{,}2$:

$$\hat{y}(t) = \sum_{k=-1}^{1} y[k]\,\text{sinc}(t - k), \quad \text{sinc}(u) = \frac{\sin(\pi u)}{\pi u}\ (0 \mapsto 1).$$

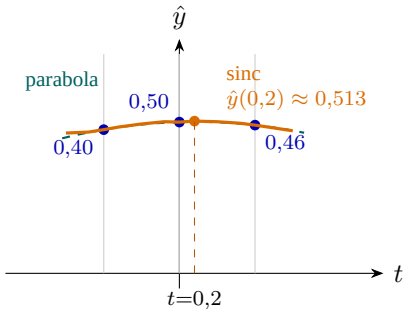
Các mẫu sinc:

$$\begin{aligned}\operatorname{sinc}(1{,}2) &\approx -0{,}155915,\\ \operatorname{sinc}(0{,}2) &\approx 0{,}935489,\\ \operatorname{sinc}(-0{,}8) &\approx 0{,}233872.\end{aligned}$$

Suy ra

$$\begin{aligned}\hat{y} &\approx 0{,}40(-0{,}155915) + 0{,}50(0{,}935489) + 0{,}46(0{,}233872)\\ &\approx 0{,}512960 > y[0].\end{aligned}$$

Đỉnh hơi lệch phải, khớp dấu Δ parabolic dương (đã kiểm `numpy.sinc`).



Hình 16: Bước 8: cùng ba điểm mẫu, nội suy parabolic (nét đứt xanh ngọc) so với xấp xỉ sinc cửa sổ hẹp (nét cam). Sinc nâng đỉnh lên $\approx 0{,}513$ tại $t \approx 0{,}2$, cùng dấu lệch Δ parabolic dương — tinh hơn nhưng tốn MAC.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 8 Sinc/Brent.

- Mặc định fabric: **dừng ở parabolic** (Chương 9) — 0 MAC sinc.
- SINC70: ~ 70 MAC/đỉnh; 14 đỉnh ~ 980 MAC/khung — vẫn nhỏ so với MAC ACF ($\sim 10^5$), nhưng thêm FSM địa chỉ và latency khó đóng timing 10 ms nếu làm naive.
- SINC700: ~ 700 MAC/đỉnh — chỉ optional lab trên HPS, không đưa vào fabric real-time.
- Twin: parabolic bắt buộc khớp bit với RTL; sinc chỉ khi so Praat `very_accurate`.

Bài tập 1.8. Giải thích tại sao FPGA chọn parabolic dù SINC70 chỉ thêm ~ 980 MAC — chưa đến 1% ngân sách MAC ACF.

Lời giải. Chi phí không chỉ là số MAC: cần bộ đệm hàng chục mẫu quanh mỗi đỉnh, FSM địa chỉ, và (nếu đa ứng viên) lặp 14 lần. Trên đường K$=1$ real-time, parabolic cho sai số < 2 cent trên sine sạch — đủ ngưỡng lab 20 cent so Praat (Chương 13) — trong khi thêm Sinc tăng độ phức tạp kiểm chứng mà lợi ích biên nhỏ trên giọng nói thật (nhiễu và formant đã chiếm phần lớn sai số).

1.9. Bước 9 — Octave cost: phạt thiên vị quãng tám

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: $2T_0$ cũng là chu kỳ hoàn hảo — sai octave

Nếu $x(t) = x(t+T_0)$ thì cũng $x(t) = x(t+2T_0)$. Tự tương quan vì thế có đỉnh tại $2T_0$ gần bằng đỉnh tại T_0 sau khi đã bù cửa sổ. Không phạt thì thuật toán thích chọn $F_0/2$ (giọng trầm giả) — đó là **sai octave** (*octave error*): nhầm một quãng tám. Hiện tượng gốc là tính tuần hoàn toán học, không phải dây thanh thật sự rung chậm gấp đôi (đã nêu ở Chương 0).
Boersma trừ một octave cost

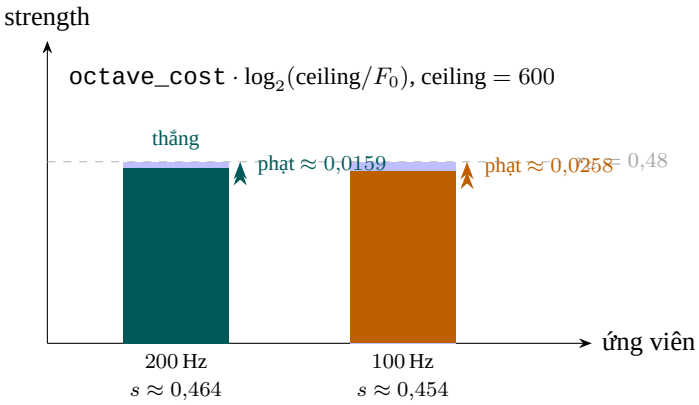
$$\texttt{octave_cost} \cdot \log_2(\text{ceiling}/F_0)$$

khỏi strength của mỗi ứng viên (trong `Pitch_pathFinder`, mảng `delta[]`). Ứng viên tần số thấp bị phạt nặng hơn một chút — đủ để phá thế cân bằng giả giữa T_0 và $2T_0$.

Ví dụ tính tay. `octave_cost` $= 0{,}01$, ceiling $= 600$, hai ứng viên 200 Hz và 100 Hz, cùng $r_a = 0{,}48$:

$$s_{200} = 0{,}48 - 0{,}01 \log_2(600/200) = 0{,}48 - 0{,}01 \cdot 1{,}58496 \approx 0{,}46415,$$
$$s_{100} = 0{,}48 - 0{,}01 \log_2(600/100) = 0{,}48 - 0{,}01 \cdot 2{,}58496 \approx 0{,}45415.$$

200 Hz thắng — đúng F_0, không phải nửa. Tại 500 Hz: phạt chỉ $\approx 0{,}00263$; tại ceiling 600 Hz: phạt $= 0$. (Đã kiểm Python.)



Hình 17: Bước 9: hai ứng viên cùng $r_a = 0{,}48$. Octave cost phạt 100 Hz nặng hơn 200 Hz (thêm một quãng tám so với ceiling), phá thế cân bằng giả $T_0 \,/\, 2T_0$.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 9 Octave cost.

- Fabric K$= 1$: không tính delta đầy đủ (không có đa ứng viên để so). Ceiling/floor vẫn cắt dải τ tìm kiếm — *có* ảnh hưởng fabric qua giới hạn vòng MAC.
- HPS pathFinder: soft-float $\log_2$ hoặc LUT nhỏ.
- Twin: cùng công thức khi bật Viterbi soft; đường median-3 bỏ qua octave cost từng ứng viên nhưng vẫn dùng $\tau_{\min}/\tau_{\max}$.

Bài tập 1.9. Cùng $r_a = 0{,}50$, ceiling $= 600$, `octave_cost` $= 0{,}01$. So s tại 150 Hz và 300 Hz. Ai thắng? Đây có phải bẫy octave không?

Lời giải. $s_{150} = 0{,}50 - 0{,}01 \log_2(600/150) = 0{,}50 - 0{,}02 = 0{,}48$; $s_{300} = 0{,}50 - 0{,}01 \log_2(2) = 0{,}50 - 0{,}01 = 0{,}49$. Ứng viên 300 Hz thắng trên strength đơn khung — đúng thiên vị nhẹ về cao. Nếu 300 Hz là bẫy octave của 150 Hz thật, Bước 11–12 (jump cost + Viterbi) sẽ phạt chuyển $150 \to 300$ giữa các khung bằng $0{,}35$ để kéo đường về 150 Hz.

1.10. Bước 10 — Silence threshold, Voicing threshold và Ứng viên vô thanh

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: ba trạng thái âm thanh và bóng ma nhiễu nền

Tín hiệu tiếng nói của con người gồm ba trạng thái vật lý phân biệt rõ rệt:

1. **Âm hữu thanh (Voiced):** Hai mép dây thanh quản khép sát và rung đóng mở tuần hoàn dưới áp lực luồng khí từ phổi (các nguyên âm /a/, /u/, /i/ hay phụ âm mũi /m/, /n/). Dạng sóng âm trong miền thời gian có tính chu kỳ rất mạnh, tạo nên các đỉnh tự tương quan chuẩn hóa cao vút, thường đạt $r_a[\tau^*] \in [0{,}60;\ 0{,}95]$.
2. **Âm vô thanh (Unvoiced):** Dây thanh quản mở rộng hoàn toàn, không rung; luồng không khí đi qua các khe hẹp trong khoang miệng sinh ra hiện tượng xoáy khí hỗn loạn (các âm xát /s/, /x/, /f/ hay âm bật /t/, /p/, /k/). Tín hiệu vô thanh có bản chất ngẫu nhiên tựa như nhiễu trắng, khiến hàm tự tương quan suy giảm nhanh về 0 và các đỉnh cục bộ chỉ dao động ngẫu nhiên ở mức rất thấp ($r_a[\tau] < 0{,}30$).
3. **Khoảng lặng (Silence):** Người nói ngừng phát âm để lấy hơi hoặc ngắt câu. Màng rung micro chỉ thu được tiếng ồn nền của phòng thu, tiếng quạt gió máy tính hoặc tạp âm nhiệt của mạch tiền khuếch đại điện tử (preamplifier hiss).

Nếu không có cơ chế chặn khoảng lặng, thuật toán chuẩn hóa sẽ chia tự tương quan cho năng lượng khung $\mathcal{R}_x[0]$. Khi năng lượng khung tiến gần về 0, những gợn sóng nhiễu ngẫu nhiên chỉ cỡ micro-volt cũng bị khuếch đại giả tạo thành các đỉnh tương quan chuẩn hóa cao tới $0{,}8$ hoặc $0{,}9$. Hậu quả là bộ ước lượng sẽ xuất ra các giá trị cao độ F_0 giả mạo, nhảy nhót loạn xạ ngay trong những đoạn người nói hoàn toàn im lặng.

Toán từng bước từ định nghĩa gốc. Praat giải quyết triệt để vấn đề này thông qua cặp tham số kép. **Ngưỡng im lặng** (`silenceThreshold`) mặc định $\theta_{\text{sil}} = 0{,}03$ theo biên độ. **Ngưỡng hữu thanh** (`voicingThreshold`) mặc định $\theta_{\text{vox}} = 0{,}45$.

Để có hệ quy chiếu công bằng giữa các khung phát âm to nhỏ khác nhau, Praat quét toàn bộ phát ngôn để tìm năng lượng khung lớn nhất làm mốc tham chiếu toàn cục:

$$E_{\text{ref}} = \max_k \mathcal{R}_x[0]_k.$$

Ngưỡng biên độ $0{,}03$ tương đương với ngưỡng năng lượng (bình phương biên độ):

$$\theta_{\text{energy}} = \theta_{\text{sil}}^2 = (0{,}03)^2 = 0{,}0009 \quad (\approx -30{,}46\,\text{dB so với đỉnh tham chiếu}).$$

Sức mạnh (strength) của ứng viên vô thanh (*unvoiced candidate*) trong khung thứ k được Boersma thiết lập theo hàm phi tuyến:

$$s_{\text{uv}} = \theta_{\text{vox}} + \max\left(0,\ 2 - \frac{\mathcal{R}_x[0]_k / E_{\text{ref}}}{\theta_{\text{sil}}/(1+\theta_{\text{vox}})}\right).$$

Phân tích biểu thức giải tích này cho thấy hai trạng thái biên rõ ràng:

- **Trong khoảng lặng ($\mathcal{R}_x[0]_k \ll E_{\text{ref}}$):** Tỷ số năng lượng trong ngoặc tiến về 0. Giá trị s_{uv} tăng vọt lên mức cực đại:
$$s_{\text{uv}} \to \theta_{\text{vox}} + 2 = 0{,}45 + 2 = 2{,}45.$$
Do đỉnh tự tương quan của bất kỳ sóng âm nào sau chuẩn hóa cũng không bao giờ vượt quá $1{,}0$, giá trị $s_{\text{uv}} = 2{,}45$ này đè bẹp tuyệt đối mọi đỉnh tương quan giả do nhiễu sinh ra, ép khung âm thanh phải được phán quyết là Vô thanh ($F_0 = 0$).
- **Trong khung phát âm rõ ràng ($\mathcal{R}_x[0]_k \geq 2\frac{\theta_{\text{sil}}}{1+\theta_{\text{vox}}} E_{\text{ref}} \approx 0{,}0414 E_{\text{ref}}$):** Số hạng thứ hai bị chặn bằng 0. Khi đó sức mạnh vô thanh trở về đúng bằng ngưỡng danh định:
$$s_{\text{uv}} = \theta_{\text{vox}} = 0{,}45.$$
Lúc này, nếu đỉnh tự tương quan cục bộ mạnh nhất $r_a[\tau^*] \geq 0{,}45$, ứng viên hữu thanh sẽ chiến thắng và hệ thống công nhận khung đó có cao độ $F_0 = f_s/\tau_{\text{fine}}$. Ngược lại, nếu $r_a[\tau^*] < 0{,}45$, khung được coi là vô thanh.

Ví dụ tính tay từng bước (kiểm chứng bằng Python). Xét hai khung âm thanh với $\theta_{\text{sil}} = 0{,}03$, $\theta_{\text{vox}} = 0{,}45$. Mẫu số chuẩn hóa:

$$\text{Mẫu số} = \frac{\theta_{\text{sil}}}{1+\theta_{\text{vox}}} = \frac{0{,}03}{1+0{,}45} = \frac{0{,}03}{1{,}45} \approx 0{,}02068966.$$

- **Trường hợp 1 (Khung thì thầm rất nhỏ, $\mathcal{R}_x[0]/E_{\text{ref}} = 0{,}02$):** Tỷ số năng lượng chuẩn hóa:

$$\text{ratio} = \frac{0{,}02}{0{,}02068966} \approx 0{,}966667.$$

  Sức mạnh ứng viên vô thanh:

$$s_{\text{uv}} = 0{,}45 + \max(0,\ 2 - 0{,}966667) = 0{,}45 + 1{,}033333 = 1{,}483333.$$

  Giả sử nhiễu nền tạo ra một đỉnh tương quan cực đại giả có biên độ thô $r_a = 0{,}70$. Vì $r_a = 0{,}70 < s_{\text{uv}} = 1{,}483333$, ứng viên vô thanh hoàn toàn áp đảo; khung được kết luận là vô thanh ($F_0 = 0$), bảo vệ hệ thống khỏi báo động giả.

- **Trường hợp 2 (Khung phát âm nguyên âm mạnh, $\mathcal{R}_x[0]/E_{\text{ref}} = 0{,}80$):** Tỷ số năng lượng:

$$\text{ratio} = \frac{0{,}80}{0{,}02068966} \approx 38{,}667 \gg 2 \implies 2 - \text{ratio} < 0.$$

  Sức mạnh vô thanh duy trì ở mức sàn: $s_{\text{uv}} = 0{,}45$. Nếu đỉnh tự tương quan đạt $r_a[\tau^*] = 0{,}78$, do $0{,}78 > 0{,}45$, khung được gắn nhãn Hữu thanh (Voiced).

Toàn bộ các giá trị trên đã được kiểm chứng khớp chính xác từng chữ số (tập lệnh trong thư mục `verify/`).

Ý nghĩa phần cứng — dẫn hai cổng số nguyên từ quy tắc float. Trên FPGA streaming không đợi hết utterance để lấy E_{ref} toàn cục; thực tế thay bằng ngưỡng tuyệt đối hoặc *moving peak detector*. Quan trọng hơn: hai phép so sánh số nguyên trong twin/RTL *không* lấy từ s_{uv} mà từ hai quy tắc float riêng — và mỗi bên phải biết rõ đại lượng nào đang bị bình phương / chuẩn hóa.

Silence Gate (năng lượng, có bình phương θ_{sil}). Praat khai báo `silenceThreshold` theo *biên độ*: $\theta_{\text{sil}} = 0{,}03$. Năng lượng khung tỉ lệ với bình phương biên độ, nên ngưỡng năng lượng là

$$\theta_{\text{energy}} = \theta_{\text{sil}}^2 = (0{,}03)^2 = 0{,}0009 = \frac{9}{10{,}000}.$$

Quy tắc float (cứng, abort sớm — khác với cạnh tranh mềm s_{uv} ở trên):

$$\frac{\mathcal{R}_x[0]}{E_{\text{ref}}} < \theta_{\text{energy}} \quad \Longleftrightarrow \quad \frac{\mathcal{R}_x[0]}{E_{\text{ref}}} < \frac{9}{10{,}000}.$$

Nhân hai vế với $10{,}000 \cdot E_{\text{ref}}$ (dương):

$$\mathcal{R}_x[0] \cdot 10{,}000 < 9 \cdot E_{\text{ref}}.$$

Đây chính là Silence Gate số nguyên. Ví dụ bài tập:

$$\begin{aligned} \mathcal{R}_x[0] &= 500{,}000, \quad E_{\text{ref}} = 600{,}000{,}000, \\ \frac{\mathcal{R}_x[0]}{E_{\text{ref}}} &= \frac{500{,}000}{600{,}000{,}000} \approx 0{,}000833 < 0{,}0009, \\ \mathcal{R}_x[0] \cdot 10{,}000 &= 5 \times 10^9 < 5{,}4 \times 10^9 = 9 \cdot E_{\text{ref}}. \end{aligned}$$

Cùng một kết luận "im lặng" trên cả hai dạng float và số nguyên.

Lưu ý tách bạch: công thức s_{uv} dùng mẫu số $\theta_{\text{sil}}/(1+\theta_{\text{vox}})$ trên tỉ số $\mathcal{R}_x[0]/E_{\text{ref}}$ để *nâng sức mạnh ứng viên UV*; Silence Gate cứng dùng θ_{sil}^2 để *cắt sớm* khung quá yếu. Hai cơ chế cùng nguồn θ_{sil} nhưng không cùng bất đẳng thức.

Voicing Gate (chuẩn hóa r_a, *không* bình phương). Quy tắc float Praat trên khung không bị silence cắt: khung hữu thanh khi

$$r_a[\tau^*] \geq \theta_{\text{vox}} = 0{,}45,$$

với r_a *đã chuẩn hóa* (không thứ nguyên):

$$r_a[\tau] = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0]} \cdot C[\tau] = \frac{S[\tau]}{\mathcal{R}_x[0]}, \qquad S[\tau] = \mathcal{R}_x[\tau] \cdot C[\tau].$$

Trên twin/RTL, `best_score` là đúng $S[\tau^*]$ sau nhân ROM $C[\tau]$ (Q3.13, dịch $\gg 13$) — cùng đơn vị năng lượng với $\mathcal{R}_x[0]$, *không* phải r_a đã chia. Do đó

$$\frac{\texttt{best_score}}{\mathcal{R}_x[0]} \geq 0{,}45 = \frac{45}{100} \quad \Longleftrightarrow \quad \texttt{best_score} \cdot 100 \geq 45 \cdot \mathcal{R}_x[0].$$

Nếu nhầm đưa r_a không thứ nguyên $(0{,}78)$ vào chỗ `best_score`, phép so sánh $0{,}78 \cdot 100 \geq 45 \cdot \mathcal{R}_x[0]$ trộn hai thang đo và luôn sai khi $\mathcal{R}_x[0] \gg 1$. Twin `golden/lab_1_3_5.py` giữ `best_score` = lag energy đã bù $C[\tau]$ để khớp bất đẳng thức trên.

Tóm tắt hai cổng số nguyên (không chia, không DSP):

Silence: $\mathcal{R}_x[0] \cdot 10{,}000 < 9 \cdot E_{\text{ref}}$ (từ $\theta_{\text{sil}}^2 = 9/10{,}000$),

Voicing: $\texttt{best_score} \cdot 100 \geq 45 \cdot \mathcal{R}_x[0]$ (từ $S/\mathcal{R}_x[0] \geq 45/100$; S = năng lượng đã $\times C$).

Đoạn mã Verilog tổng hợp dưới đây minh họa khối phán quyết trên FPGA:

Listing 2: cùng đơn vị năng lượng với R[0], không phải r_a chuẩn hóa.]Silence + Voicing Gate số nguyên: best_score là S[tau*] cùng đơn vị năng lượng với R[0], không phải r_a chuẩn hóa.

```
// Silence: R[0]/E_ref < (0.03)^2 = 9/10000
// Voicing: S[tau*]/R[0] >= 0.45 = 45/100  (best_score = S)
wire [63:0] score_scaled = best_score * 64'd100;
wire [63:0] r0_scaled    = r0_energy  * 64'd45;
wire is_silence = (r0_energy * 64'd10000) < (64'd9 * ref_energy);
wire is_voiced  = (!is_silence) && (score_scaled >= r0_scaled);

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        f0_out_reg <= 16'd0;
        vuv_flag   <= 1'b0;
    end else if (frame_done) begin
        vuv_flag   <= is_voiced;
        f0_out_reg <= is_voiced ? f0_interpolated : 16'd0;
    end
end
```

Mô hình song sinh phần mềm (Bit-Exact Twin). Trong mô hình Python tham chiếu (tệp `lab_1_3_5.py` thư mục `golden/`), `best_score` được gán từ `scores[idx] = (r_tau * scoring_rom + 4096) >> 13` — tức $S[\tau]$ — rồi so khớp bit với RTL:

```
# Silence: r0/E_ref < 0.03**2 = 0.0009 = 9/10000
if ref_energy > 0 and r0 * 10000 < 9 * ref_energy:
    return FrameResult(f0=0.0, is_voiced=False, f0_q12_4=0)

# Voicing: best_score/r0 < 0.45  <=>  best_score*100 < 45*r0
# best_score = S[tau*] (C-compensated lag energy), NOT dimensionless r_a
if best_score * 100 < 45 * r0:
    return FrameResult(f0=0.0, is_voiced=False, f0_q12_4=0)
```

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 10 Voicing Gate.

- **Tài nguyên logic (ALM / LUT):** Chiếm khoảng $35 \ldots 45$ ALMs cho hai bộ so sánh 48-bit và các cổng logic bắt cờ.
- **Khối nhân DSP cứng:** 0 khối (nhân hằng số 45 và 100 được tổng hợp tối ưu qua mạng cộng dịch LUT).
- **Độ trễ xử lý (Latency):** Đúng 1 chu kỳ xung nhịp đồng hồ tại thời điểm kết thúc quét mảng tương quan τ.
- **Ảnh hưởng đến RTF:** Độ trễ 1 chu kỳ (20 ns tại 50 MHz) hoàn toàn không đáng kể so với ngân sách 10 ms của bước nhảy khung (0,0002%).

Bài tập 1.10. Một khung âm thanh có năng lượng đo được $\mathcal{R}_x[0] = 500{,}000$, trong khi năng lượng đỉnh tham chiếu là $E_{\text{ref}} = 600{,}000{,}000$. Đỉnh tự tương quan chuẩn hóa đạt $r_a[\tau^*] = 0{,}82$. Khung này có được công nhận là có cao độ F_0 hay không?

Lời giải. Quy tắc float Silence: $\mathcal{R}_x[0]/E_{\text{ref}} < \theta_{\text{sil}}^2 = 0{,}0009$. Tỉ số thực tế $500{,}000/600{,}000{,}000 = 1/1200 \approx 0{,}000833 < 0{,}0009$ — đã im lặng. Dạng số nguyên cùng nguồn ($0{,}0009 = 9/10{,}000$):

$$\mathcal{R}_x[0] \cdot 10{,}000 = 5 \times 10^9, \qquad 9 \cdot E_{\text{ref}} = 5{,}4 \times 10^9.$$

Vì $5 \times 10^9 < 5{,}4 \times 10^9$, Silence Gate kích hoạt, ép $F_0 = 0$ dù $r_a = 0{,}82$ (nhiễu nền bị chuẩn hóa phóng đại). Voicing Gate ($\texttt{best_score} \cdot 100 \geq 45 \cdot \mathcal{R}_x[0]$) không còn được xét: cổng im lặng cắt trước. Nếu giả sử năng lượng đủ lớn và $\texttt{best_score} = S[\tau^*] = 0{,}82 \cdot \mathcal{R}_x[0]$, thì $0{,}82 \cdot \mathcal{R}_x[0] \cdot 100 \geq 45 \cdot \mathcal{R}_x[0]$ đúng — nhưng chỉ sau khi đã qua Silence Gate.

1.11. Bước 11 — Chi phí bước nhảy quãng tám và Chi phí chuyển trạng thái Hữu/Vô thanh

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: Quán tính cơ học của dây thanh quản và sự mượt mà sinh lý

Dây thanh quản của con người cấu tạo từ các lớp cơ vân và màng nhầy (cơ giáp-phễu *thyroarytenoid* và cơ nhẫn-giáp *cricothyroid*). Cơ cấu sinh học này có khối lượng và quán tính vật lý thực sự: sức căng cơ không thể nào biến thiên tức thời gấp đôi hoặc tụt giảm một nửa trong khoảng thời gian chỉ 10 ms giữa hai khung liên tiếp. Trong phát âm tự nhiên, quỹ đạo cao độ F_0 luôn biến thiên một cách mượt mà và liên tục; tốc độ biến đổi cao độ thông thường chỉ dưới $5 \ldots 10\%$ mỗi khung (< 1 bán âm).
Nếu chỉ dựa vào cực đại tự tương quan của từng khung đơn lẻ, hiện tượng nhiễu dải tần hoặc cộng hưởng formant có thể đẩy một đỉnh hài phụ lên cao hơn đỉnh chính, gây ra hiện tượng nhảy vọt cao độ gấp đôi ($2F_0$) hoặc tụt một nửa ($F_0/2$) chỉ trong một khung rồi lập tức quay về giá trị cũ. Tương tự, tại các ranh giới chuyển tiếp giữa nguyên âm và phụ âm, trạng thái Hữu thanh và Vô thanh có thể bị chập chờn (flickering: V–UV–V–UV) nếu năng lượng khung mấp mé ngưỡng so sánh. Để ngăn chặn các bước nhảy phi tự nhiên này, giải thuật Praat áp dụng hai hàm phạt chuyển trạng thái (*transition costs*) giữa hai khung thời gian liên tiếp.

Toán từng bước từ định nghĩa gốc. Praat định nghĩa hai thành phần chi phí chuyển trạng thái từ ứng viên i ở khung $t-1$ sang ứng viên j ở khung t:

1. **Chi phí bước nhảy quãng tám (Octave-jump cost):** Áp dụng khi cả hai khung liên tiếp đều là âm Hữu thanh ($F_0 > 0$), tỷ lệ thuận với khoảng cách quãng tám logarit giữa hai tần số:

$$C_{\text{jump}}\big(F_0(t-1),\, F_0(t)\big) = \texttt{octaveJumpCost} \times \left|\log_2 \frac{F_0(t-1)}{F_0(t)}\right|.$$

   Trong phần mềm Praat, tham số mặc định là $\texttt{octaveJumpCost} = 0{,}35$.

2. **Chi phí chuyển đổi trạng thái Hữu/Vô thanh (Voiced/Unvoiced cost):** Phạt mỗi khi đường cao độ chuyển từ trạng thái Hữu thanh sang Vô thanh hoặc ngược lại:

$$C_{\text{vuv}} = \texttt{voicedUnvoicedCost} = 0{,}14.$$

   Đối với quá trình giữ nguyên trạng thái Vô thanh liên tục (UV $\to$ UV), chi phí chuyển đổi bằng 0: $C_{\text{uv}\to\text{uv}} = 0$.

Ví dụ tính tay chi tiết (đã kiểm chứng bằng Python).

- **Biến thiên mượt tự nhiên ($150\,$Hz $\to 160\,$Hz):** Khoảng cách quãng tần:

$$\Delta_{\text{oct}} = \left|\log_2 \frac{150}{160}\right| = \left|\frac{\ln(150/160)}{\ln(2)}\right| \approx 0{,}093109\,\text{octave}.$$

  Chi phí chuyển trạng thái:

$$C_{\text{jump}} = 0{,}35 \times 0{,}093109 \approx 0{,}032588.$$

  Mức phạt chỉ khoảng $0{,}033$, rất nhỏ so với sức mạnh đỉnh tương quan ($0{,}6 \ldots 0{,}9$), hoàn toàn cho phép đường cong cao độ uốn lượn tự nhiên.

- **Bẫy nhảy quãng tám giả mạo ($150\,$Hz $\to 300\,$Hz):** Khoảng cách quãng tần:

$$\Delta_{\text{oct}} = \left|\log_2 \frac{150}{300}\right| = |\log_2(0{,}5)| = 1{,}0000\,\text{octave}.$$

  Chi phí chuyển trạng thái:

$$C_{\text{jump}} = 0{,}35 \times 1{,}0000 = 0{,}3500.$$

  Mức phạt $0{,}35$ là cực kỳ nặng: nó làm giảm độ ưu tiên của ứng viên $300\,$Hz tương đương với việc hạ sức mạnh đỉnh từ $0{,}80$ xuống chỉ còn $0{,}45$, chặn đứng hoàn toàn hiện tượng nhảy quãng tám giả do nhiễu.

Ý nghĩa phần cứng và Bộ lọc Trung vị 3 điểm (Median-3 Filter). Trên fabric FPGA streaming thời gian thực, việc duy trì một ma trận đồ hình chuyển trạng thái đa chiều đòi hỏi phải đệm hàng chục khung vào RAM ngoài, gây trễ lớn. Thay vào đó, kiến trúc phần cứng chuyên dụng trên kit DE10-Standard sử dụng một khối xử lý phi tuyến cực kỳ thanh lịch và hiệu quả: **Bộ lọc trung vị 3 điểm (Median-3 Filter)** trực tiếp trên dòng dữ liệu F_0:

$$F_{0,\text{filt}}[t] = \text{median}\big(F_0[t-1],\, F_0[t],\, F_0[t+1]\big).$$

Bộ lọc trung vị có đặc tính toán học xuất sắc: triệt tiêu tuyệt đối mọi xung gai nhọn đơn lẻ (như khung bị nhảy quãng tám đột ngột lên $2F_0$ hoặc rớt về 0 do mất khung) trong khi bảo toàn hoàn hảo độ dốc và biên dạng của các đoạn chuyển tiếp cao độ thực. Độ trễ phần cứng tạo ra đúng bằng 1 bước nhảy khung ($10\,$ms).

Đoạn mã Verilog tổng hợp được dưới đây mô tả cấu trúc phần cứng của bộ lọc trung vị 3 điểm:

Listing 3: Thiết kế RTL khối lọc trung vị 3 điểm cho dòng cao độ F0 trên FPGA.

```
module median3_f0 (
    input  wire        clk,
    input  wire        rst_n,
    input  wire        valid_in,
    input  wire [15:0] f0_in,       // Dinh dang Q12.4
    output reg         valid_out,
    output reg  [15:0] f0_out
);
    reg [15:0] d0, d1, d2;

    // Mach so sanh 3 cap so de tim trung vi:
    // median(a,b,c) = a > b ? (b > c ? b : (a > c ? c : a))
    //                       : (a > c ? a : (b > c ? c : b))
    function [15:0] get_median;
        input [15:0] a, b, c;
        begin
            if (a > b) begin
                if (b > c)      get_median = b;
                else if (a > c) get_median = c;
                else            get_median = a;
            end else begin
                if (a > c)      get_median = a;
                else if (b > c) get_median = c;
                else            get_median = b;
            end
        end
```

```
    endfunction

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            d0 <= 16'd0; d1 <= 16'd0; d2 <= 16'd0;
            valid_out <= 1'b0; f0_out <= 16'd0;
        end else if (valid_in) begin
            d0 <= f0_in;
            d1 <= d0;
            d2 <= d1;
            valid_out <= 1'b1;
            f0_out    <= get_median(f0_in, d0, d1);
        end else begin
            valid_out <= 1'b0;
        end
    end
endmodule
```

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 11 Transition Costs / Median-3.

- **Tài nguyên logic (ALM / LUT):** Bộ lọc Median-3 chỉ tốn khoảng $22 \ldots 28$ ALMs gồm 3 thanh ghi 16-bit và mạng so sánh phân nhánh 3 ngõ vào.
- **Khối nhân DSP cứng:** 0 DSP (không dùng phép toán số học nào ngoài phép so sánh độ lớn nguyên).
- **Bộ nhớ M10K:** 0 khối RAM (dữ liệu lưu hoàn toàn trong các flip-flop nội của ALM).
- **Độ trễ thời gian (Latency):** Đúng 1 khung thời gian âm thanh ($10\,\text{ms}$). Với các hệ thống tương tác giọng nói, độ trễ $10\,\text{ms}$ nằm sâu dưới ngưỡng phát hiện của tai người ($\approx 30 \ldots 50\,\text{ms}$).

Bài tập 1.11. Chuỗi giá trị cao độ thô xuất ra từ bộ dò cực đại qua 5 khung liên tiếp (bước nhảy $10\,\text{ms}$) có giá trị là: $148\,\text{Hz}$, $150\,\text{Hz}$, $300\,\text{Hz}$ (bẫy hài bậc hai do nhiễu), $152\,\text{Hz}$, $151\,\text{Hz}$.

a) Xác định chuỗi cao độ ngõ ra sau khi đi qua bộ lọc trung vị 3 điểm Median-3.

b) Tính chi phí bước nhảy quãng tám C_{jump} giữa khung 2 ($150\,\text{Hz}$) và khung 3 ($300\,\text{Hz}$) theo công thức Praat và giải thích tại sao bộ lọc trung vị lại triệt tiêu được lỗi này mà không cần giải thuật quy hoạch động phức tạp.

Lời giải.

a) Áp dụng cửa sổ trung vị trượt 3 điểm trên từng khung (bỏ qua điều kiện biên đầu/cuối):

- Khung 2 (nhìn 3 mẫu $148, 150, 300$): Trung vị của $\{148, 150, 300\}$ là $150\,\text{Hz}$.
- Khung 3 (nhìn 3 mẫu $150, 300, 152$): Sắp xếp $\{150, 152, 300\}$, giá trị ở giữa là $152\,\text{Hz}$.
- Khung 4 (nhìn 3 mẫu $300, 152, 151$): Sắp xếp $\{151, 152, 300\}$, giá trị ở giữa là $152\,\text{Hz}$.

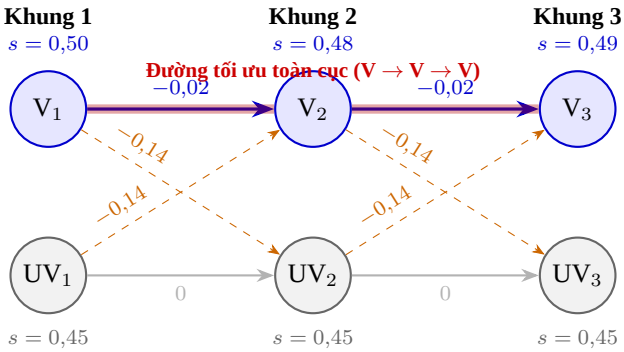
Giá trị gai nhiễu $300\,\text{Hz}$ đã bị gạt phăng hoàn toàn ra khỏi đường cong; chuỗi đầu ra duy trì mượt mà quanh $150 \ldots 152\,\text{Hz}$.

b) Chi phí bước nhảy: $C_{\text{jump}} = 0{,}35 \times |\log_2(150/300)| = 0{,}3500$. Trong khi Viterbi phải tính tổng tích lũy qua nhiều khung và duyệt ngược, bộ lọc trung vị tận dụng tiên đề thống kê rằng một xung đột biến chỉ tồn tại trong 1 khung thì luôn là phần tử ngoại lai (outlier) nằm ở đầu mút của tập 3 điểm, do đó không bao giờ được chọn làm giá trị trung vị. Mạch RTL đạt hiệu quả tương đương Viterbi đối với các xung nhiễu cô lập mà chỉ tiêu tốn chưa đầy 30 ALMs.

1.12. Bước 12 — Thuật toán tìm đường tối ưu toàn cục Viterbi (`Pitch_pathFinder`)

★ NGUYÊN LÝ ĐẦU TIÊN: Hiện tượng thật: Sự đánh đổi giữa cực đại cục bộ và sự mượt mà toàn cục

Trong thực tế, quyết định cao độ tối ưu không thể chỉ nhìn vào một thời điểm duy nhất. Xét trường hợp một người đang nói câu có ngữ điệu trầm, tại một khung âm thanh nào đó, đỉnh tự tương quan ở quãng tám cao ($2F_0$) tình cờ cao hơn đỉnh ở tần số cơ bản (F_0) chỉ $0{,}02$ đơn vị do một cộng hưởng vòm họng. Nếu chỉ chọn theo cực đại cục bộ (greedy search), đường cao độ sẽ bị gãy một nốt nhô cao rồi thụt xuống.
Thuật toán Viterbi (`Pitch_pathFinder` trong mã nguồn `Pitch.cpp` của Praat) giải quyết bài toán này dưới dạng **quy hoạch động (dynamic programming)**: tìm đường đi xuyên qua lưới không gian trạng thái (trellis) sao cho tổng điểm tích lũy toàn cục là lớn nhất, hài hòa giữa sức mạnh đỉnh của từng khung và sự liên tục tự nhiên theo thời gian.



Hình 18: Đồ hình lưới Trellis 3 khung thời gian của giải thuật Viterbi trong Praat. Đường màu đỏ thể hiện quỹ đạo có tổng điểm tích lũy lớn nhất sau khi đã trừ các chi phí chuyển trạng thái.

Toán từng bước: Hệ phương trình truy hồi Viterbi. Tại mỗi khung thời gian $t \in \{1, 2, \ldots, T\}$, giả sử ta có K ứng viên cao độ (tối đa 14 ứng viên hữu thanh và 1 ứng viên vô thanh). Đặt:

- $s_t(j)$: Sức mạnh đơn khung của ứng viên thứ j tại khung t.
- $C(i, j)$: Chi phí chuyển đổi từ ứng viên i ở khung $t-1$ sang ứng viên j ở khung t.
- $\delta_t(j)$: Điểm số tích lũy lớn nhất của đường đi kết thúc tại ứng viên j ở khung t.
- $\psi_t(j)$: Con trỏ ngược (backpointer) lưu chỉ số ứng viên ở khung $t-1$ đã mang lại điểm số tích lũy tối đa cho ứng viên j.

Phương trình quy hoạch động Viterbi được xác lập theo hai pha:

1. **Pha lan truyền thuận (Forward recursion):**

$$\begin{aligned} &\text{Khởi tạo } (t=1): && \delta_1(j) = s_1(j), \qquad \forall j \in \{1, \ldots, K\}, \\ &\text{Quy nạp } (t \geq 2): && \delta_t(j) = s_t(j) + \max_{1 \leq i \leq K} \big[\delta_{t-1}(i) - C(i, j)\big], \\ & && \psi_t(j) = \arg\max_{1 \leq i \leq K} \big[\delta_{t-1}(i) - C(i, j)\big]. \end{aligned}$$

2. **Pha truy hồi ngược (Backtracking):** Tại khung kết thúc T, chọn trạng thái có điểm tích lũy cao nhất:

$$j_T^* = \arg\max_{1 \leq j \leq K} \delta_T(j).$$

Truy hồi ngược lần lượt từ $t = T-1$ lùi dần về $t = 1$:

$$j_t^* = \psi_{t+1}(j_{t+1}^*).$$

Ví dụ tính tay lưới 3 khung (đã kiểm chứng trong `verify_chuong_00_01.py`). Xét bài toán 3 khung thời gian với hai trạng thái: Hữu thanh (V) và Vô thanh (UV):

- Sức mạnh ứng viên: Khung 1: $s_1(\text{V}) = 0{,}50$, $s_1(\text{UV}) = 0{,}45$. Khung 2: $s_2(\text{V}) = 0{,}48$, $s_2(\text{UV}) = 0{,}45$. Khung 3: $s_3(\text{V}) = 0{,}49$, $s_3(\text{UV}) = 0{,}45$.
- Chi phí chuyển trạng thái: $C_{\text{v}\to\text{v}} = 0{,}02$ (độ lệch cao độ nhỏ); $C_{\text{v}\to\text{uv}} = C_{\text{uv}\to\text{v}} = 0{,}14$; $C_{\text{uv}\to\text{uv}} = 0{,}00$.

  Tại Khung 1 ($t = 1$):

  $$\delta_1(\text{V}) = 0{,}50, \qquad \delta_1(\text{UV}) = 0{,}45.$$

  Tại Khung 2 ($t = 2$):

  - Điểm đến trạng thái V_2: Từ V_1: $\delta_1(\text{V}) - C_{\text{v}\to\text{v}} = 0{,}50 - 0{,}02 = 0{,}48$. Từ UV_1: $\delta_1(\text{UV}) - C_{\text{uv}\to\text{v}} = 0{,}45 - 0{,}14 = 0{,}31$. Chọn cực đại: $\max(0{,}48,\, 0{,}31) = 0{,}48$ (đến từ V_1). $\Longrightarrow$ $\delta_2(\text{V}) = s_2(\text{V}) + 0{,}48 = 0{,}48 + 0{,}48 =$ **0**,**96**, con trỏ $\psi_2(\text{V}) = \text{V}$.
  - Điểm đến trạng thái UV_2: Từ V_1: $\delta_1(\text{V}) - C_{\text{v}\to\text{uv}} = 0{,}50 - 0{,}14 = 0{,}36$. Từ UV_1: $\delta_1(\text{UV}) - C_{\text{uv}\to\text{uv}} = 0{,}45 - 0{,}00 = 0{,}45$. Chọn cực đại: $\max(0{,}36,\, 0{,}45) = 0{,}45$ (đến từ UV_1). $\Longrightarrow$ $\delta_2(\text{UV}) = s_2(\text{UV}) + 0{,}45 = 0{,}45 + 0{,}45 =$ **0**,**90**, con trỏ $\psi_2(\text{UV}) = \text{UV}$.

  Tại Khung 3 ($t = 3$):

  - Điểm đến trạng thái V_3: Từ V_2: $\delta_2(\text{V}) - C_{\text{v}\to\text{v}} = 0{,}96 - 0{,}02 = 0{,}94$. Từ UV_2: $\delta_2(\text{UV}) - C_{\text{uv}\to\text{v}} = 0{,}90 - 0{,}14 = 0{,}76$. $\Longrightarrow$ $\delta_3(\text{V}) = s_3(\text{V}) + \max(0{,}94,\, 0{,}76) = 0{,}49 + 0{,}94 =$ **1**,**43**, con trỏ $\psi_3(\text{V}) = \text{V}$.
  - Điểm đến trạng thái UV_3: Từ V_2: $\delta_2(\text{V}) - C_{\text{v}\to\text{uv}} = 0{,}96 - 0{,}14 = 0{,}82$. Từ UV_2: $\delta_2(\text{UV}) - C_{\text{uv}\to\text{uv}} = 0{,}90 - 0{,}00 = 0{,}90$. $\Longrightarrow$ $\delta_3(\text{UV}) = s_3(\text{UV}) + \max(0{,}82,\, 0{,}90) = 0{,}45 + 0{,}90 =$ **1**,**35**, con trỏ $\psi_3(\text{UV}) = \text{UV}$.

  Pha truy hồi ngược: Tại khung cuối $t = 3$, so sánh hai điểm tích lũy:

  $$\delta_3(\text{V}) = 1{,}43 > \delta_3(\text{UV}) = 1{,}35 \implies j_3^* = \text{V}.$$

Lần ngược theo con trỏ: $j_2^* = \psi_3(\text{V}) = \text{V}$; $j_1^* = \psi_2(\text{V}) = \text{V}$. Đường tối ưu được chọn là $\{\text{V}_1 \to \text{V}_2 \to \text{V}_3\}$ với tổng điểm $1{,}43$.

Phân bổ kiến trúc hệ thống: Fabric FPGA và HPS ARM Cortex-A9. Trên chip vi mạch Cyclone V SX của kit DE10-Standard, kiến trúc được phân định rạch ròi:

- **Fabric FPGA:** Đảm trách toàn bộ đường xử lý tín hiệu thông lượng cao theo thời gian thực (streaming datapath từ Chương 2 đến 10) gồm bộ lọc IIR, FIR decimation, bộ đệm khung trượt M10K, khối nhân dồn MAC tính tự tương quan, bộ chuẩn hóa ROM và bộ chia nội suy parabol. Chế độ mặc định sử dụng bộ lọc Median-3 thời gian thực với độ trễ chỉ 10 ms, đạt hệ số RTF $= 0{,}242 \ll 1{,}0$.
- **HPS (Hard Processor System):** Nếu ứng dụng cần phân tích âm học ngôn ngữ chuẩn mực phòng thí nghiệm với đầy đủ 15 ứng viên và duyệt Viterbi toàn câu nói, luồng dữ liệu ứng viên từ FPGA sẽ được DMA đẩy lên bộ nhớ DDR3 của bộ xử lý ARM Cortex-A9 kép chạy Linux để thực thi hàm `Pitch_pathFinder` nguyên bản của Boersma.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — Bước 12 Viterbi.

- **Quyết định thiết kế cốt lõi: Tắt Viterbi trên fabric FPGA streaming**.
- **Lý do kỹ thuật:** Triển khai Viterbi trên fabric đòi hỏi hàng chục khối M10K để lưu bảng con trỏ và làm trễ tín hiệu hàng giây, vi phạm hoàn toàn tiêu chí thời gian thực biên (Edge Real-Time).
- **Phương án trên fabric:** Dùng khối lọc trung vị 3 điểm Median-3 (chỉ tốn 25 ALMs, 0 DSP, 0 M10K) hoặc xuất trực tiếp cặp giá trị $\{F_0,\, \text{is_voiced}\}$ ra thanh ghi giao tiếp.

Bài tập 1.12. Giả sử tại Khung 2, sức mạnh của ứng viên hữu thanh bị nhiễu làm sụt giảm nghiêm trọng xuống mức $s_2(\mathrm{V}) = 0{,}20$, trong khi các thông số khác giữ nguyên như ví dụ tính tay trên. Tính lại $\delta_2(\mathrm{V})$, $\delta_2(\mathrm{UV})$, $\delta_3(\mathrm{V})$, $\delta_3(\mathrm{UV})$ và cho biết đường tối ưu toàn cục có còn giữ trạng thái Hữu thanh liên tục hay không.

Lời giải.

- Tại Khung 2: $\delta_2(\mathrm{V}) = s_2(\mathrm{V}) + \max(0{,}50 - 0{,}02,\, 0{,}45 - 0{,}14) = 0{,}20 + 0{,}48 = 0{,}68$ (đến từ V_1). $\delta_2(\mathrm{UV}) = 0{,}45 + 0{,}45 = 0{,}90$ (đến từ UV_1).
- Tại Khung 3: $\delta_3(\mathrm{V}) = 0{,}49 + \max\big(0{,}68 - 0{,}02,\, 0{,}90 - 0{,}14\big) = 0{,}49 + \max(0{,}66,\, 0{,}76) = 0{,}49 + 0{,}76 = 1{,}25$ (đến từ UV_2!). $\delta_3(\mathrm{UV}) = 0{,}45 + \max\big(0{,}68 - 0{,}14,\, 0{,}90 - 0{,}00\big) = 0{,}45 + \max(0{,}54,\, 0{,}90) = 0{,}45 + 0{,}90 = 1{,}35$ (đến từ UV_2).

Tại khung 3: $\delta_3(\mathrm{UV}) = 1{,}35 > \delta_3(\mathrm{V}) = 1{,}25 \implies$ trạng thái kết thúc là UV_3. Truy hồi ngược: $\psi_3(\mathrm{UV}) = \mathrm{UV}_2$, $\psi_2(\mathrm{UV}) = \mathrm{UV}_1$. Đường tối ưu toàn cục chuyển hoàn toàn thành $\{\mathrm{UV}_1 \to \mathrm{UV}_2 \to \mathrm{UV}_3\}$. Khi một mắt xích bị sụt giảm quá sâu dưới ngưỡng, Viterbi tự động tái định tuyến toàn bộ để tránh một khung hữu thanh đơn lẻ bị cô lập.

1.13. Bảng giữ / xấp xỉ / xóa trên FPGA

Trước khi mở Quartus, kỹ sư phải biết khối nào *không* có trên fabric — tránh tìm IP không tồn tại và tránh im lặng lệch Praat rồi đổ lỗi twin.

Giữ (gần bit-exact)	**Xấp xỉ**	**Xóa trên fabric**
L=640, H=160, Hanning	Silence: IIR/max-hold thay `globalPeak` cả file	Soft upsample FFT cả utterance (Bước 1)
Mean/DC-blocker, MAC ACF	K= 1 thay 14 ứng viên	Sinc/Brent (Bước 8)
ROM $C[\tau]$, parabolic, f_s/τ	Median-3 thay Viterbi	`Pitch_pathFinder` đầy đủ (Bước 12)
Voicing gate, đóng gói Q12.4	—	CC method (`to_Pitch_cc`)

Mỗi ô "xóa/xấp xỉ" đã có lý do chi phí ở mục silicon tương ứng. Ngân sách sai số lab: $|\Delta F| < 20$ cent trên sine sạch so Praat (Chương 13). Mỗi bước xóa phải ghi trong báo cáo kiểm chứng — không im lặng.

CHI PHÍ (LUT / DSP / chu kỳ)

Vì sao bảng này kết thúc Chương 1? Để bản đồ "giấy Boersma → C++ → silicon" khép lại trước khi Chương 2 mở PLL và I^{2}S. Từ đây trở đi, mỗi khối phần cứng chỉ còn việc hiện thực những ô "Giữ" và "Xấp xỉ".

1.14. Mười tham số `Sound_to_Pitch` — ý nghĩa vật lý từng cái

Tham số	Default	Ý nghĩa vật lý / thiết kế một câu
time step	$0 \to$ auto	Khoảng giữa hai đo; auto $= 0{,}75/F_{\text{min}}$
pitch floor F_{min}	75 Hz	Chu kỳ dài nhất $\Rightarrow L$ và τ_{max}
very accurate	off	Hanning/SINC70 vs Gaussian/SINC700
silence threshold	0,03	Tỉ local/global dưới mức $\to$ nghỉ lặng
voicing threshold	0,45	Sàn strength cho ứng viên UV
octave cost	0,01	Phạt $\log_2$ thiên vị $F_0/2$
octave jump cost	0,35	Phạt $\lvert\log_2 f_1/f_2\rvert$ giữa khung
voiced/unvoiced cost	0,14	Phạt đổi trạng thái V/UV
ceiling	600 Hz	F_{max} + mốc octave cost
maxn candidates	15	14 đỉnh + UV

1.15. Tóm tắt ký hiệu dùng xuyên suốt các chương sau

Ký hiệu	Đơn vị / kiểu	Định nghĩa
f_s	Hz	Tần số lấy mẫu sau hạ mẫu (16 kHz trên DE10)
F_0	Hz	Cao độ $= 1/T_0$ dây thanh
T_0, τ_0	s hoặc mẫu	Chu kỳ cơ bản
τ	mẫu	Trễ (lag) trong tự tương quan
$\mathcal{R}_x[\tau]$	năng lượng	$\sum x[n]x[n+\tau]$ (không vòng)
$r_a[\tau]$	1	Autocorr chuẩn hóa Boersma
$C[\tau]$	1	$\mathcal{R}_w[0]/\mathcal{R}_w[\tau]$
L, H	mẫu	Độ dài cửa sổ và hop khung
Q$m.n$	—	Fixed-point: m bit nguyên (kể dấu), n fraction

1.16. Bài tập tổng hợp và Lời giải chi tiết

Bài tập T1 (Chuỗi số từ mean đến r_a). Cho $x = [3, 5, 3, 5]$ và cửa sổ Praat $N = 4$ đã tính

$$w \approx [0{,}3455,\ 0{,}9045,\ 0{,}9045,\ 0{,}3455], \qquad C[2] \approx \mathcal{R}_w[0]/\mathcal{R}_w[2].$$

a) Trừ mean, nhân cửa sổ, tính $\mathcal{R}_{\tilde{x}}[0]$ và $\mathcal{R}_{\tilde{x}}[2]$.

b) Tính $C[2]$ rồi $r_a[2] = (\mathcal{R}[2]/\mathcal{R}[0]) \cdot C[2]$. Nhận xét so với tỉ thô $0{,}5$.

Lời giải giải thích.

a) $x_{\text{zm}} = [-1, 1, -1, 1]$; $\tilde{x}[i] = x_{\text{zm}}[i-1] \cdot w[i]$ (chỉ số $i = 1..4$):

$$\tilde{x} \approx [-0{,}3455,\ 0{,}9045,\ -0{,}9045,\ 0{,}3455].$$

Vì $|x_{\text{zm}}| = 1$ mọi mẫu nên $\mathcal{R}[0] = \sum \tilde{x}^2 = \mathcal{R}_w[0] = 1{,}875$.

$$\mathcal{R}[2] = (-0{,}3455)(-0{,}9045) + (0{,}9045)(0{,}3455) = 2 \cdot 0{,}3455 \cdot 0{,}9045 \approx 0{,}6249.$$

b) $\mathcal{R}_w[2] = w_1 w_3 + w_2 w_4 \approx 0{,}6249$, nên $C[2] = 1{,}875/0{,}6249 \approx 3{,}000$.

$$r_a[2] = (0{,}6249/1{,}875) \cdot 3{,}000 = 0{,}3333 \cdot 3 = 1{,}000.$$

Chuẩn hóa Boersma kéo tỉ số về đúng 1 trên tín hiệu tuần hoàn sau cửa sổ — đó là lý do bắt buộc có bảng $C[\tau]$ trên ROM.

Bài tập T2 (Ngân sách RTF và quyết định giữ/xóa). Tại 50 MHz, hop 10 ms, MAC 120960 chu kỳ. Giả sử thêm SINC70 cho 1 đỉnh (70 MAC) và Viterbi soft trên HPS (không đụng fabric).

a) RTF fabric sau khi thêm SINC70 còn bao nhiêu?

b) Vì sao tài liệu vẫn khuyến cáo *không* đưa SINC70 vào fabric mặc định?

Lời giải giải thích.

a) Tổng chu kỳ $\approx 120960 + 70 = 121030$; $T_{\text{compute}} = 121030/(50 \cdot 10^6) = 2{,}4206$ ms; RTF $= 0{,}24206$ — gần như không đổi.

b) Lý do không phải RTF mà là độ phức tạp kiểm chứng, FSM địa chỉ, và lợi ích biên nhỏ trên speech thật so với parabolic (đã bàn ở Bài tập 1.8). Nguyên tắc: mỗi khối thêm phải chứng minh cải thiện sai số cent trên bộ kiểm thử Chương 13, không chỉ "còn chỗ trên chip".

Bài tập T3 (Checklist Feynman trước khi cắt bước Praat). Với mỗi bước trong {1, 8, 12}, trả lời năm câu: (1) Bảo vệ khỏi hiện tượng vật lý nào? (2) Sai số khi xóa trên sine? trên speech? (3) Silicon: LUT/DSP/M10K/latency? (4) Twin còn so được Praat không? (5) Có đẩy HPS thay vì xóa hoàn toàn không?

Lời giải giải thích (tóm tắt).

- **Bước 1:** Bảo vệ khỏi rò phổ mép Nyquist; xóa trên fabric thay bằng FIR — sai số kiểm soát được nếu stopband FIR đủ sâu; twin offline vẫn soft-upsample được; không cần HPS real-time.
- **Bước 8:** Bảo vệ khỏi bias lưới mẫu; xóa $\rightarrow$ parabolic: lệch vài cent trên sine, thường nhỏ hơn nhiễu speech; 0 DSP thêm; twin parabolic khớp RTL; HPS optional Sinc.
- **Bước 12:** Bảo vệ khỏi nhảy octave giữa khung; xóa $\rightarrow$ median-3: mất tối ưu đường dài nhưng đủ cho lab; $<$ 200 ALM; twin median-3 khớp fabric, Viterbi soft trên HPS khi cần "Praat đầy đủ".

▶ NHÌN THẤY VẬT LÝ

Sai một công thức ở Chương 1 thì không một khối DSP hay bộ nhớ M10K nào ở các chương sau có thể cứu vãn được hệ thống. Hãy chạy lại toàn bộ `verify/verify_chuong_00_01.py` trước khi sang Chương 2.

Chương 2. PLL, MCLK, I²S và CDC mã Gray

▶ MỤC TIÊU HỌC TẬP

Sau Chương 2, người học tạo được đúng xung nhịp âm thanh từ thạch anh 50 MHz trên DE10-Standard, thu mẫu PCM từ codec WM8731 qua I²S, và đưa mẫu sang miền đồng hồ xử lý 50 MHz bằng DCFIFO mã Gray mà không gây metastability. Các khái niệm PLL, MCLK, I²S, DCFIFO và mã Gray đã được định nghĩa trực giác ở mục 0.4 (ô Feynman 8) của Chương 0 — chương này đi sâu vào cấu hình, tính tay, nội tại IP, RTL từng dòng và twin Python.

2.1. Từ thạch anh 50 MHz đến MCLK 12,288 MHz

▶ NHÌN THẤY VẬT LÝ

Thạch anh trên bo mạch DE10 dao động cơ–điện rất ổn định (sai số thường chỉ vài phần triệu — ppm). Chip codec **Wolfson WM8731** không dùng trực tiếp 50 MHz: nó cần một xung chủ riêng gọi là **MCLK** đúng 12,288 MHz (bằng 256×48 kHz) để chạy bộ lọc oversampling và ADC delta-sigma bên trong. Nếu MCLK lệch vài chục ppm so với giá trị danh định, chu kỳ lấy mẫu sẽ trôi chậm theo thời gian; thuật toán F_0 vẫn "đúng công thức" nhưng đo trên một trục thời gian bị kéo giãn, tạo ra sai số cao độ giả. Ví dụ đời thường: một đồng hồ đeo tay chạy nhanh 50 ppm sẽ lệch khoảng 4,3 giây mỗi ngày — với audio 16 kHz, cùng tỷ lệ đó làm trôi 48 mẫu mỗi phút, đủ để lệch một phần bước nhảy khung $H = 160$.

Khối **PLL** (*Phase-Locked Loop* — vòng khóa pha) trên Cyclone V tạo ra MCLK bằng cách *khóa* một dao động nội chip vào thạch anh ngoài. Trong vòng đó có ba khối vật lý cần gọi đúng tên. **VCO** (*Voltage-Controlled Oscillator* — dao động điều khiển bằng điện áp) là mạch LC/ring trên silicon: điện áp vào cao hơn thì tần số ra cao hơn, giống vặn núm radio nhưng bằng volt. **PFD** (*Phase–Frequency Detector* — bộ so pha–tần số) không phải phép trừ số trên giấy: mỗi lần cạnh xung tham chiếu (f_{in}) và cạnh feedback (sau chia M) lệch nhau, PFD mở một khóa bơm điện tích (*charge pump*) vào **loop filter** (tụ và điện trở lọc). Điện áp trên tụ kéo VCO tới khi hai cạnh khớp — đó mới gọi là *khóa*. Chỉ khi đã khóa, bộ chia C mới cho ra MCLK sạch. Counter mềm bằng flip-flop không đạt jitter mức audio (jitter lớn làm méo bit I²S và làm ADC delta-sigma "run" không đều); vì thế trên Quartus Standard/Lite bắt buộc dùng IP **Altera PLL** (ALTPLL / PLL Intel FPGA IP), không tự viết PLL bằng RTL.

Toán từng bước — integer-N. Trong chế độ integer-N mọi hệ số đều nguyên:

$$f_{\text{VCO}} = f_{\text{in}} \cdot \frac{M}{N}, \qquad f_{\text{out}} = \frac{f_{\text{VCO}}}{C} = f_{\text{in}} \cdot \frac{M}{N \cdot C}.$$

Với $f_{\text{in}} = 50$ MHz và $f_{\text{out}} = 12{,}288$ MHz:

$$\frac{M}{N \cdot C} = \frac{12{,}288}{50} = 0{,}24576.$$

Tỷ số $0{,}24576$ không phân tích thành bộ (M, N, C) nguyên nhỏ nằm trong dải VCO Cyclone V (600–1600 MHz) với sai số đúng 0 ppm. Có thể tìm bộ nguyên lớn cho sai số khoảng 0,2 ppm — ví dụ $M = 1507$, $N = 73$, $C = 84$:

$$f_{\text{VCO}} = 50 \cdot \frac{1507}{73} \approx 1032{,}19178 \text{ MHz} \quad \text{(nằm trong 600–1600)},$$
$$f_{\text{out}} = \frac{f_{\text{VCO}}}{84} = 50 \cdot \frac{1507}{73 \cdot 84} \approx 12{,}28799739 \text{ MHz}.$$

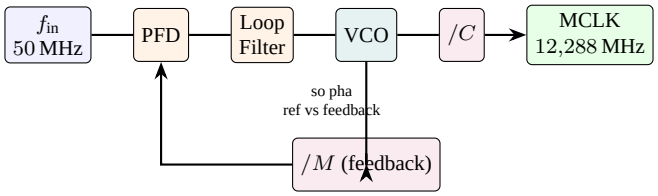
Sai số tuyệt đối so với 12,288 MHz là khoảng 2,61 Hz, tương đương

$$\varepsilon_{\text{ppm}} = \frac{|f_{\text{out}} - 12{,}288|}{12{,}288} \cdot 10^6 \approx 0{,}212 \text{ ppm}.$$

Fractional PLL trên DE10. MegaWizard/IP Catalog cho phép đặt ngõ ra đúng 12,288 MHz; phần mềm chọn hệ số phân số (fractional-M hoặc tương đương tùy family) và ghi M, N, C vào báo cáo Compilation. Một nghiệm phân số minh họa trên giấy: $N = 5$, $M = 61{,}44$, $C = 50$ cho

$$f_{\text{VCO}} = 50 \cdot \frac{61{,}44}{5} = 614{,}4 \text{ MHz}, \qquad f_{\text{out}} = \frac{614{,}4}{50} = 12{,}288 \text{ MHz}$$

đúng tuyệt đối theo số học. Trên silicon, "$61{,}44$" được hiện thực bằng bộ đếm phân số trung bình theo thời gian; jitter dư rất nhỏ so với yêu cầu audio nếu loop filter được cấu hình đúng trong IP.



Hình 19: Luồng tín hiệu trong PLL: PFD so sánh cạnh tham chiếu với feedback sau chia M; khi khóa, chia C cho ra MCLK dùng cho codec.

Ví dụ tính tay — trôi mẫu theo ppm. Giả sử sai số MCLK là $\varepsilon = 50\,\text{ppm} = 50 \cdot 10^{-6}$. Trong $T = 60\,\text{s}$ ở tần số lấy mẫu xử lý $f_s = 16\,\text{kHz}$, số mẫu bị trôi tuyệt đối là

$$\Delta N = \varepsilon \cdot T \cdot f_s = 50 \cdot 10^{-6} \cdot 60 \cdot 16000 = 48 \text{ mẫu.}$$

Bốn mươi tám mẫu ở 16 kHz tương đương 3 ms — đủ để lệch một phần đáng kể của bước nhảy khung $H = 160$ mẫu (10 ms). Với sai số integer-N ở trên ($\approx 0{,}212$ ppm):

$$\Delta N \approx 0{,}212 \cdot 10^{-6} \cdot 60 \cdot 16000 \approx 0{,}204 \text{ mẫu/phút,}$$

không đáng kể so với 48. Mục tiêu lab: giữ sai số audio dưới 5 ppm ($\Delta N \approx 4{,}8$ mẫu/phút), và luôn chờ tín hiệu `locked` của PLL trước khi bật FSM I^{2}S. Nếu bỏ qua `locked`, vài trăm miligiây đầu sau reset MCLK chưa ổn định — codec ghi rác vào FIFO và đỉnh tự tương quan nhảy lung tung.

IP Altera PLL — tham số thật. Trong IP Catalog chọn PLL Intel FPGA IP (hoặc ALTPLL trên tool cũ): Reference Clock 50 MHz; Output $c0 = 12{,}288$ MHz (đưa chân MCLK codec); có thể thêm $c1 = 50$ MHz hệ thống nếu muốn cùng nguồn khóa pha với audio (giảm drift tương đối giữa datapath và MCLK). Bật cổng `locked`; nối `areset` về nút reset toàn cục đã đồng bộ. Báo cáo PLL List ghi rõ M, N, C, f_{VCO}, ppm residual — lab phải chép các số này vào sổ, không đoán.

RTL và twin. RTL chỉ được *chia* sau khi đã có MCLK (ví dụ tạo SCLK $=$ MCLK$/4$ cho frame 64 bit), không thay VCO bằng counter. Mô hình twin Python không mô phỏng jitter analog; nó giả lập trôi thời gian lấy mẫu

$$t_n = \frac{n}{f_s}\left(1 + \varepsilon_{\text{ppm}} \cdot 10^{-6}\right)$$

rồi nội suy sóng hình sin F_0 biết trước lên lưới t_n lệch và quan sát độ lệch cao độ ước lượng khi $\varepsilon \neq 0$. Script `verify/verify_chuong_02.py` kiểm mọi số ppm/drift nêu trong chương.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — PLL (mọi viết tắt).

- **PLL** cứng: 1 / 6 trên Cyclone V 5CSXFC6D6F31C6N ($\approx 16{,}7\%$ ngân sách PLL bảng Chương 0).
- **ALM** (*Adaptive Logic Module* — ô logic cơ bản của Cyclone V, gồm LUT và flip-flop): vài ALM cho logic `locked`/reset. **LUT** (*Look-Up Table*) là bảng chân lý nhỏ trong ALM.
- **DSP** (*Digital Signal Processing* block — khối nhân cứng 18×19): 0.
- **M10K** (block RAM ≈ 10 kbit): 0. **MLAB** (*Memory Logic Array Block* — RAM nhỏ trong LAB): 0.
- **Fmax** (*maximum frequency* — tần số đồng hồ lớn nhất vẫn đạt timing): không phải nút thắt (PLL cứng chạy VCO hàng trăm MHz).
- **Latency**: thời gian từ lúc cấp clock đến khi `locked`=1 (hàng trăm μs đến vài ms tùy IP) — FSM I^{2}S phải chờ.
- **RTF** (*real-time factor* = thời gian xử lý / thời gian audio): khối PLL không đếm RTF theo mẫu; nó chỉ cung cấp nhịp.
- Vì sao bắt buộc IP: soft-counter không đạt ppm/jitter của VCO silicon cho MCLK audio.

2.2. Giao thức I²S với codec WM8731

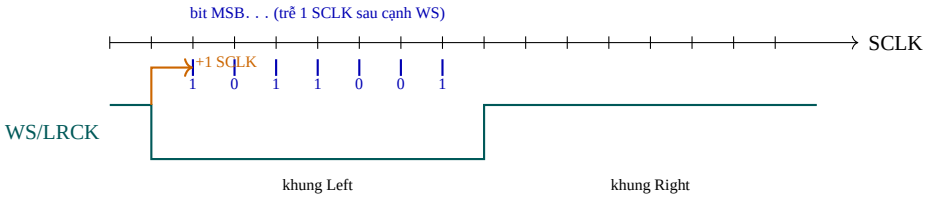
▶ NHÌN THẤY VẬT LÝ

Sau khi ADC trong WM8731 đã lượng tử hóa sóng âm, dữ liệu không nhảy sang FPGA dưới dạng mảng số nguyên song song, mà đi trên ba dây nối tiếp theo chuẩn **I²S** (*Inter-IC Sound* — đã giới thiệu ở Chương 0): dây **SCLK**/BCLK chốt từng bit, dây **WS**/LRCK chọn kênh trái/phải, dây **SDAT** mang dòng bit. Đây là quy ước giao diện điện tử, không phải hiện tượng dây thanh; nhưng đếm sai một bit hoặc trễ sai một cạnh SCLK sẽ làm hỏng mẫu Q1.15 trước khi tín hiệu kịp vào DC-blocker. Cấu hình thanh ghi codec (định dạng I²S, độ dài word, master/slave clock) đi qua bus **I²C** riêng — lab phải khởi tạo I²C trước khi kỳ vọng SDAT có nghĩa.

Quan hệ đồng hồ. Với cấu hình stereo phổ biến trên DE10 (mỗi slot 32 bit, hai kênh = 64 bit/frame) tại $f_s = 48\,\text{kHz}$:

$$\text{MCLK} = 256 \cdot f_s = 256 \cdot 48000 = 12{,}288{,}000\,\text{Hz},$$
$$\text{SCLK} = 64 \cdot f_s = 64 \cdot 48000 = 3{,}072{,}000\,\text{Hz}.$$

Nếu lab chỉ lấy 16 bit/kênh (32 bit/frame tổng cộng L+R) thì SCLK $= 32 \cdot f_s = 1{,}536{,}000$ Hz. Chuẩn Philips I²S: MSB đi trước, dữ liệu hợp lệ bắt đầu sau đúng một cạnh SCLK kể từ cạnh đổi của WS. Trên đường ước lượng F_0 mono, FSM chỉ chốt kênh Left rồi bỏ Right — đủ thông tin cao độ, tiết kiệm băng thông FIFO.



Hình 20: Timing I²S đơn giản hóa: cạnh WS mở khung Left; dữ liệu trên SDAT hợp lệ bắt đầu sau đúng một cạnh SCLK (chuẩn Philips).

Ví dụ tính tay — đóng gói nibble rồi word Q1.15. Bốn cạnh SCLK liên tiếp nhận các bit $1, 0, 1, 1$ (MSB trước). Thanh ghi dịch trái lần lượt:

$$0 \to 1 \to 2 \to 5 \to 11 \quad (0\text{b}1011 = 11_{10}).$$

Mười sáu bit MSB của slot Left mang mẫu có dấu. Ví dụ bit-stream $0100\,0000\,0000\,0000$ (MSB trước) đóng thành

$$0\text{b}0100\,0000\,0000\,0000 = 16384_{10},$$

đúng mức $0{,}5$ trong Q1.15 (ô Feynman 7, Chương 0). Lab lấy 16 bit cao của slot 32 bit (bỏ 16 bit đệm thấp) rồi đẩy vào DCFIFO.

Hai lộ trình phần cứng. (1) **University Program Audio Core**: IP Avalon bọc cấu hình I²C của codec và FIFO L/R — gắn nhanh Platform Designer nhưng che mất đường mẫu, khó đối chiếu bit-exact. (2) **RTL tự viết (default dạy)**: FSM Idle→WaitWS→Skip1→Shift16→Push, đồng bộ cạnh WS sang miền SCLK bằng hai flip-flop, dịch 16 bit hữu ích, nâng `wrreq` mỗi mẫu Left. Twin phần mềm: tệp `.wav` int16 little-endian chính là đầu ra bit-exact sau khi bỏ padding slot.

RTL giải thích từng nhóm lệnh.

```
// Đồng bộ WS (async từ codec) sang miền SCLK bằng 2FF
reg ws_ff1, ws_ff2, ws_prev;
always @(posedge sclk) begin
  ws_ff1 <= ws_async;   // FF1: hấp thụ metastability
  ws_ff2 <= ws_ff1;     // FF2: bit đã ổn định
```

```
  ws_prev <= ws_ff2;    // nhớ cạnh trước để phát hiện rising
end
wire ws_rise = ws_ff2 & ~ws_prev;  // rising = start Left

// FSM rút gọn: sau ws_rise chờ 1 SCLK (Philips), rồi dịch 16 bit
// reg [4:0] bit_cnt; reg [15:0] shreg;
// khi bit_cnt==15: data_left <= {shreg[14:0], sdat}; wrreq_fifo <= 1;
```

Dòng `ws_ff1 <= ws_async` là chỗ metastability được phép xảy ra — chỉ trên *một* bit điều khiển. Dòng `ws_ff2` cho bit thời gian ổn định trước khi dùng. `ws_rise` phát hiện cạnh; sau đúng một SCLK (trễ Philips) mới lấy MSB. Khi đủ 16 bit, ghi DCFIFO phía `wrclk = sclk`. Phía `rdclk = 50 MHz` đọc ra Avalon-ST (`valid`/`ready`) sang DC-blocker.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — I²S RX (mọi viết tắt).

- **ALM/LUT**: khoảng 50–100 ALM (shift 16–32 bit + cạnh WS + FSM). 0 **DSP**, 0 **M10K/MLAB**.
- **Fmax**: miền SCLK 3,072 MHz rất dư so với silicon tốc độ 6.
- **Latency**: từ cạnh WS đến mẫu Left hợp lệ $\approx 1 + 16$ cạnh SCLK (Philips + 16 bit) $\approx 5{,}5\ \mu$s.
- **RTF**: mỗi frame audio 64 cạnh SCLK mang đúng một mẫu Left hữu ích; FSM không bỏ mẫu $\Rightarrow$ RTF thu thập $= 1$ (xử lý kịp thời gian thật).
- Vì sao mono Left: F_0 không cần stereo; stereo nhân đôi băng thông CDC mà không làm sắc đỉnh $\mathcal{R}[\tau]$.

2.3. CDC: DCFIFO bất đồng bộ và mã Gray

▶ NHÌN THẤY VẬT LÝ

Miền đồng hồ âm thanh (MCLK 12,288 MHz, SCLK 3,072 MHz) và miền xử lý lõi (50 MHz) dao động từ các nguồn độc lập — chúng **bất đồng bộ**. Nếu nối một thanh ghi 16 bit từ miền SCLK sang miền 50 MHz bằng dây thô, một số bit có thể chốt đúng lúc tín hiệu đang ở vùng điện áp "không phải 0 cũng không phải 1". Hiện tượng đó gọi là **metastability** (bán ổn định): flip-flop đích đứng lình xình ở mức giữa trong một khoảng thời gian không đoán trước được, rồi mới đổ về 0 hoặc 1 — giống quả bóng cân bằng trên đỉnh tường rồi rơi sang một bên. Hậu quả trên đường F_0 là mẫu rác đột biến, đỉnh tự tương quan nhảy lung tung, đôi khi FSM tưởng FIFO full/empty sai.

Giải pháp chuẩn là bộ đệm **DCFIFO** (*Dual-Clock FIFO* — đã nêu ở Chương 0): cổng ghi chạy theo clock nguồn (SCLK), cổng đọc chạy theo clock đích (50 MHz). Dữ liệu mẫu nằm trong RAM dual-port nội bộ; cái bắt chéo miền clock *không* phải là word dữ liệu 16 bit mà là **con trỏ** ghi/đọc. Con trỏ khi bắt chéo miền phải được mã hóa **mã Gray**: mỗi lần tăng địa chỉ chỉ đúng một bit thay đổi. Nhờ đó, dù bên kia lấy mẫu lúc con trỏ đang chuyển, nó chỉ có thể đọc nhầm tối đa một bước địa chỉ liền kề — không bao giờ đọc phải từ bit-pattern trung gian vô nghĩa của mã nhị phân thường.

Toán mã Gray. Với số nguyên không dấu n, mã Gray phản xạ là

$$G(n) = n \oplus \lfloor n/2 \rfloor = n \oplus (n \gg 1).$$

Bảng tính tay $n = 0 \ldots 7$ (3 bit):

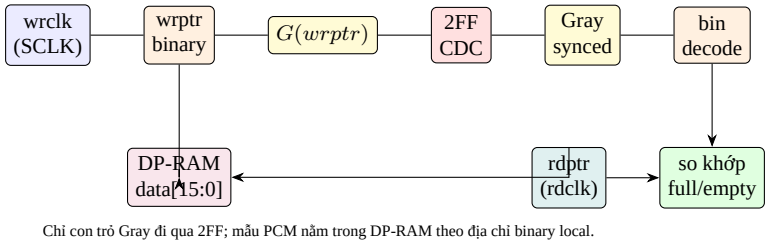
n (thập phân)	Nhị phân	Gray $G(n)$
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

Hai mã Gray liên tiếp luôn khác đúng một bit: ví dụ $G(3) = 010$ và $G(4) = 110$ chỉ khác bit cao. Đảo ngược Gray→binary trên giấy: $b_2 = g_2$, $b_1 = g_2 \oplus g_1$, $b_0 = g_2 \oplus g_1 \oplus g_0$.

Nội tại DCFIFO Intel — đồng bộ con trỏ. IP FIFO (dcfifo) thực hiện đúng chuỗi sau mỗi lần ghi/đọc:

1. Tăng con trỏ nhị phân phía local clock ($wrptr$ hoặc $rdptr$).
2. Đổi sang Gray: $wrptr_gray = G(wrptr)$.
3. Đưa $wrptr_gray$ qua chuỗi **hai flip-flop** sang miền clock bên kia (đúng chuẩn 2FF CDC cho từng bit Gray — vì mỗi bit Gray chỉ đổi đơn lẻ, metastability trên một bit không kéo theo bit khác).
4. Ở miền đích, đổi Gray→binary rồi so với con trỏ local để tính `wrfull`/`rdempty`/`rdusedw`.

Word dữ liệu 16 bit *không* đi qua 2FF: nó nằm trong dual-port RAM, được ghi theo địa chỉ binary phía write và đọc theo địa chỉ binary phía read; chỉ có "vé địa chỉ" (con trỏ Gray) là tín hiệu bất đồng bộ cần đồng bộ hóa.



Hình 21: Luồng CDC trong DCFIFO: mã hóa Gray → đồng bộ 2FF từng bit → giải mã binary → so full/empty; dữ liệu mẫu không bắt chéo thô.

Ví dụ tính tay — vì sao nhị phân thường nguy hiểm. Con trỏ nhị phân từ 3 (011) lên 4 (100) đổi *cả ba bit* cùng lúc. Bên đọc bắt chéo miền có thể chốt thành 000, 111, 001,…— địa chỉ giả cách xa hàng đợi thật, đọc nhầm mẫu cũ hoặc đè cờ full/empty. Với Gray, bước $010 \to 110$ chỉ một bit; sai lệch tối đa là ± 1 ô.

Ví dụ tính tay — chọn depth. Tốc độ ghi mono $= 48$ kHz. Giả sử jitter tương đối và burst đọc phía hệ thống tạo độ trễ hấp thụ tối đa $100\,\mu$s. Số mẫu cần đệm:

$$N_{\text{depth}} \geq 48000 \cdot 100 \cdot 10^{-6} = 4{,}8 \quad \Rightarrow \quad \text{chọn depth } 8 \text{ hoặc } 16.$$

Depth 8 từ $\times 16$ bit $= 128$ bit — vừa một MLAB; không cần M10K. Lab mặc định depth 16 để dư margin khi FSM phía đọc thỉnh thoảng `ready`=0.

Twin Python (mô hình logic, không mô phỏng metastability analog).

```
def bin_to_gray(n):
    return n ^ (n >> 1)

# Kiểm bảng 0..7 và tính chất một bit
grays = [bin_to_gray(i) for i in range(8)]
assert [format(g, "03b") for g in grays] == [
```

```
    "000","001","011","010","110","111","101","100"]
for a, b in zip(grays, grays[1:]):
    assert bin(a ^ b).count("1") == 1
```

Twin mức hệ thống: hàng đợi `collections.deque` với hai "clock tick" độc lập; assert không đọc khi empty và không ghi khi full — tương ứng cờ IP.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — CDC / DCFIFO (mọi viết tắt).

- **MLAB/M10K**: depth 8–16 từ $\times 16$ bit $=$ 128–256 bit — thường 1 MLAB (hoặc mảnh M10K nếu tool gộp).
- **ALM/LUT**: vài chục ALM cho Gray + full/empty + chuỗi 2FF. 0 **DSP**.
- **Latency** CDC: khoảng 2–4 chu kỳ clock đích (50 MHz) $\approx$ 40–80 ns — so với hop 10 ms (500 000 chu kỳ) là $< 0{,}001\%$.
- **Fmax**: phía `rdclk`=50 MHz dễ đạt; phía `wrclk`=3,072 MHz cũng dư.
- **RTF**: FIFO không "tính toán" mẫu; nó chỉ chuyển miền. RTF bị phá khi `wrfull`=1 (mất mẫu) — depth 16 chính là bảo hiểm RTF$= 1$.
- Vì sao không 2FF cho cả word 16 bit: 2FF chỉ an toàn từng bit điều khiển; word cần FIFO + Gray để mẫu nhất quán theo thứ tự ghi.

2.4. Chi phí tổng hợp và chỗ đứng trong đường tín hiệu

Khối Chương 2 nằm ngay sau codec WM8731 và ngay trước DC-blocker trên bản đồ đường tín hiệu Chương 0: thạch anh $\rightarrow$ PLL $\rightarrow$ MCLK/SCLK $\rightarrow$ I^{2}S RX $\rightarrow$ DCFIFO $\rightarrow$ Avalon-ST 50 MHz. Nhiệm vụ của cả cụm là *đưa đúng một mẫu PCM mỗi chu kỳ lấy mẫu sang miền xử lý*, không phải tự ước lượng F_0. Vì vậy ngân sách silicon phải cố tình rẻ: dành gần như toàn bộ **DSP** và phần lớn **M10K** cho FIR (Chương 4) và MAC tự tương quan (Chương 7).

► NHÌN THẤY VẬT LÝ

Gom ba ô chi phí mục 2.1–2.3 lại thành một bức tranh. **ALM** (*Adaptive Logic Module*) là "gạch" logic của Cyclone V; mỗi ALM chứa vài **LUT** (*Look-Up Table*) và flip-flop. **DSP** là khối nhân cứng. **M10K** là block RAM khoảng 10 240 bit; **MLAB** là RAM nhỏ hơn nằm trong logic array. **Fmax** là tần số đồng hồ lớn nhất còn thỏa timing. **Latency** là trễ mẫu/chu kỳ từ vào đến ra. **RTF** (*real-time factor*) $=$ thời gian xử lý / thời gian audio: RTF≤ 1 nghĩa là kịp thời gian thật, không bỏ mẫu.

Tổng hợp số liệu lab. Trên 5CSXFC6D6F31C6N có khoảng 41 910 ALM và 6 PLL cứng. Cụm Chương 2 tiêu thụ:

- 1 PLL Fractional (MCLK 12,288 MHz),
- $<$ 150 ALM cho I^{2}S RX + Gray/2FF/cờ FIFO (tức $< 0{,}4\%$ ALM chip),
- 0 DSP,
- tối đa 1 MLAB (hoặc mảnh M10K) cho DCFIFO depth 8–16.

Fmax phía `sysclk`=50 MHz dễ vượt 150 MHz trên silicon tốc độ 6 — điểm nghẽn *không* phải diện tích hay Fmax, mà là kỷ luật CDC và việc AND tín hiệu `locked` vào enable FSM.

Ví dụ tính tay — RTF và phần trăm hop. Giả sử phía đọc DCFIFO luôn `ready`=1. Mỗi mẫu Left vào ở 48 kHz đi ra đúng một mẫu trên Avalon-ST 50 MHz: thời gian audio mỗi mẫu là $1/48000 \approx 20{,}833\,\mu$s, trong khi latency CDC chỉ khoảng 3 chu kỳ $\times 20$ ns $= 60$ ns. Tỷ số

$$\mathrm{RTF}_{\mathrm{CDC}} = \frac{60 \cdot 10^{-9}}{20{,}833 \cdot 10^{-6}} \approx 0{,}0029 \ll 1.$$

Trong một hop xử lý 10 ms ở 50 MHz có $500\,000$ chu kỳ; 3 chu kỳ CDC chiếm

$$\frac{3}{500\,000} = 6 \cdot 10^{-6}$$

(tức $0{,}0006\%$) — không bao giờ là lý do trễ F_0. Ngược lại, nếu FSM quên `locked` hoặc depth FIFO quá nông khiến `wrfull`=1, RTF "vỡ" theo nghĩa mất mẫu: một mẫu mất ở đây không sửa được bằng thêm DSP ở Chương 7.

Vì sao thiết kế này khớp bài toán. Ước lượng F_0 cần MAC hàng trăm nghìn phép nhân-cộng mỗi hop; cụm thu thập chỉ cần *không làm hỏng mẫu*. Giữ < 150 ALM và 0 DSP ở Chương 2 là lựa chọn có chủ đích, không phải "chưa tối ưu". Lab nghiệm thu cụm: (1) PLL List ghi đúng $12{,}288$ MHz và `locked`=1 trước khi thu; (2) Fitter báo 0 DSP cho khối này; (3) không có warning timing CDC xuyên miền thô; (4) script `verify_chuong_02.py` xanh trên mọi số ppm/Gray/SCLK.

2.5. Bài tập và lời giải giải thích

Bài tập 2.1 (Tỷ số PLL và sai số ppm). Từ thạch anh $f_{\text{in}} = 50$ MHz, kỹ sư cần MCLK $= 12{,}288$ MHz.

a) Tính tỷ số $M/(N \cdot C)$ lý thuyết.

b) Nếu dùng integer-N với $M = 1507$, $N = 73$, $C = 84$, tần số thực tế là bao nhiêu và sai số so với $12{,}288$ MHz tính bằng ppm?

c) Trong 60 giây ở 16 kHz, sai số đó làm trôi bao nhiêu mẫu? So với trường hợp 50 ppm thuần.

Lời giải giải thích.

a) $M/(N \cdot C) = 12{,}288/50 = 0{,}24576$.

b)

$$f_{\text{out}} = 50 \cdot \frac{1507}{73 \cdot 84} \approx 12{,}28799739\,\text{MHz}, \quad \varepsilon_{\text{ppm}} \approx 0{,}212.$$

(Script `verify_chuong_02.py`.)

c) $\Delta N \approx 0{,}204$ mẫu/phút — không đáng kể so với 48 mẫu/phút khi sai 50 ppm. Fractional PLL vẫn tiện hơn vì MegaWizard điền đúng $12{,}288$.

Bài tập 2.2 (SCLK và nibble I²S). Codec chạy $f_s = 48$ kHz, frame stereo 64 bit.

a) Tính SCLK.

b) Dịch bốn bit $1, 0, 1, 1$ (MSB trước) — giá trị thập phân?

c) Bit-stream 16 bit $0100\,0000\,0000\,0000$ đóng thành số Q1.15 nào?

d) Vì sao lab mono Left đủ cho F_0?

Lời giải giải thích.

a) SCLK $= 64 \cdot 48000 = 3{,}072{,}000$ Hz.

b) $0\text{b}1011 = 11_{10}$.

c) 16384 — đúng mức $0{,}5$ trong Q1.15.

d) Cao độ là thuộc tính nguồn kích thích dây thanh, gần như đồng nhất trên hai kênh micro gần nhau; lấy thêm Right chỉ nhân đôi băng thông CDC.

Bài tập 2.3 (Gray và metastability). Con trỏ FIFO 3 bit tăng từ 3 lên 4.

a) Viết mã nhị phân và mã Gray của hai giá trị.

b) Giải thích tại sao bắt chéo miền clock trên mã nhị phân có thể đọc ra địa chỉ xa, còn Gray thì không.

Lời giải giải thích.

a) Nhị phân: $3 = 011$, $4 = 100$ (đổi 3 bit). Gray: $G(3) = 010$, $G(4) = 110$ (đổi 1 bit).

b) Nhiều bit đổi cùng lúc cho phép tổ hợp trung gian bất kỳ (địa chỉ xa). Gray bảo đảm mọi thời điểm trung gian chỉ sai tối đa một bước liền kề — lý do DCFIFO Intel gray-code hóa con trỏ xuyên miền.

Bài tập 2.4 (Depth FIFO và 2FF).

a) Với mono 48 kHz và jitter hấp thụ $100\,\mu$s, depth tối thiểu lý thuyết là bao nhiêu? Lab chọn 8 hay 16 thì vì sao?

b) Vì sao chuỗi 2FF áp dụng cho từng bit Gray chứ không cho cả word mẫu 16 bit?

Lời giải giải thích.

a) $48000 \cdot 100 \cdot 10^{-6} = 4{,}8$ — tối thiểu 5, chọn 8 hoặc 16 để dư margin khi `ready`=0 phía đọc. Depth 16 an toàn hơn khi FSM FIR/DC thỉnh thoảng dừng.

b) 2FF chỉ đảm bảo một bit điều khiển thoát metastability; 16 bit dữ liệu có thể mỗi bit ổn định ở giá trị khác nhau tạo word rác. FIFO giữ data trong RAM và chỉ đồng bộ con trỏ Gray.

Bài tập 2.5 (`locked` và trôi 5 ppm).

a) Tính ΔN trong 60 s ở 16 kHz nếu sai số còn 5 ppm.

b) FSM I^{2}S có được phép chạy khi `locked`=0 không? Giải thích hậu quả.

Lời giải giải thích.

a) $\Delta N = 5 \cdot 10^{-6} \cdot 60 \cdot 16000 = 4{,}8$ mẫu/phút — mục tiêu lab chấp nhận được.

b) Không. Khi chưa `locked`, MCLK lệch mạnh hoặc dừng; codec xuất bit vô nghĩa, FIFO đầy mẫu rác, đỉnh $\mathcal{R}[\tau]$ không ổn định. Phải AND `locked` vào tín hiệu enable của FSM thu.

Chương 3. DC-blocker IIR (cực/zero, accumulator 32-bit)

▶ MỤC TIÊU HỌC TẬP

Sau Chương 3, người học giải thích được vì sao offset một chiều phá đỉnh tự tương quan, viết đúng hàm truyền một cực–một zero với $\alpha = 1 - 2^{-8}$, hiện thực RTL accumulator 32 bit theo công thức $y_{\text{acc}} \leftarrow y_{\text{acc}} + x_\Delta - (y_{\text{acc}} \ggg 8)$ rồi $\text{sat}_{16}(y_{\text{acc}} \ggg 15)$, đối chiếu bit-exact với twin Python, và tính tay từng chu kỳ y_{acc}. Khái niệm điểm cực/điểm không và Avalon-ST đã có ở ô Feynman 12 của Chương 0.

3.1. Hiện tượng: offset DC nuốt đỉnh chu kỳ

▶ NHÌN THẤY VẬT LÝ

Micro, codec WM8731 và đường dây analog luôn cộng thêm một thành phần gần đứng yên: điện áp offset một chiều (DC), cộng thêm trôi chậm do nhiệt. Trong miền mẫu, đó là một hằng A cộng vào mọi $x[n]$. Phép tự tương quan (ô Feynman 5, Chương 0) không phân biệt năng lượng hữu ích và năng lượng offset. Nếu A lớn, đỉnh tại độ trễ $\tau = 0$ phình theo A^2 và có thể át hoàn toàn đỉnh chu kỳ T_0 của dây thanh — thuật toán sẽ báo vô thanh hoặc treo ở cao độ sai. Đời thường: đo chiều cao sóng trên mặt biển khi mực nước thủy triều đang dâng — nếu không trừ mực nước trung bình, "đỉnh sóng" chỉ là mực nước chứ không phải sóng.

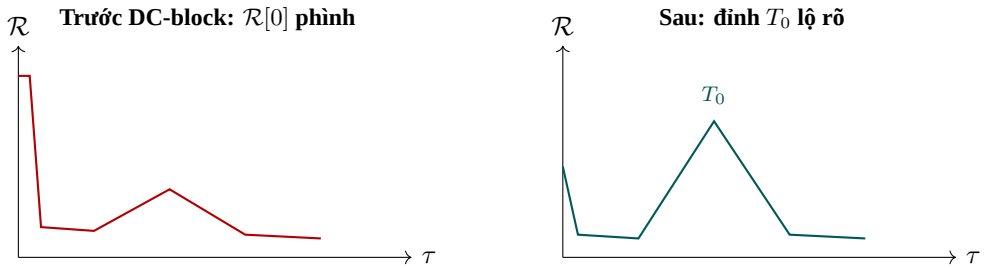
Viết tường minh với tín hiệu $x[n] + A$:

$$\begin{aligned}\mathcal{R}_{x+A}[0] &= \sum_{n=0}^{N-1} (x[n] + A)^2 \\ &= \sum_{n=0}^{N-1} x[n]^2 + 2A \sum_{n=0}^{N-1} x[n] + NA^2 = \mathcal{R}_x[0] + 2A \sum_n x[n] + NA^2.\end{aligned}$$

Số hạng NA^2 luôn dương và tỉ lệ với bình phương offset. Với độ trễ $\tau \neq 0$:

$$\mathcal{R}_{x+A}[\tau] = \mathcal{R}_x[\tau] + A \sum_n x[n] + A \sum_n x[n+\tau] + NA^2,$$

nên toàn bộ hàm tự tương quan bị nâng một "bệ" DC — đỉnh chu kỳ bị che. Praat trừ mean cục bộ từng khung; trên FPGA streaming ta dùng bộ lọc IIR liên tục gọi là **DC-blocker**: một bộ thông cao rất nhẹ, ổn định, rẻ silicon, đặt ngay sau I²S/CDC và trước FIR hạ mẫu (hoặc ngay sau CDC ở 48 kHz, tùy lab — cutoff vẫn dưới $F_{0,\text{min}}$).



Hình 22: Cùng một tín hiệu có offset: bên trái đỉnh $\tau = 0$ nuốt chu kỳ; bên phải sau DC-blocker đỉnh T_0 hiện ra để Chương 9 nội suy.

Ví dụ tính tay — DC thuần và offset cộng vào sóng. Chuỗi $x = [1, 1, 1, 1]$ (toàn DC) cho $\mathcal{R}[0] = \sum x_n^2 = 4$. Sau khi trừ mean (mọi mẫu về 0) thì $\mathcal{R}[0] = 0$: DC thuần phải đi tiếp như tín hiệu vô thanh. Chuỗi $x_0 = [1, -1, 1, -1]$ cộng $A = 2$ thành $[3, 1, 3, 1]$:

$$\sum (x_0 + A)^2 = 9 + 1 + 9 + 1 = 20, \qquad \sum x_0^2 = 4.$$

Offset đã nhân năng lượng $\tau = 0$ lên năm lần — đủ để ngưỡng voicing và so sánh đỉnh T_0 lệch hoàn toàn nếu không chặn DC.

3.2. Toán: cực, zero và chọn $\alpha = 1 - 2^{-8}$

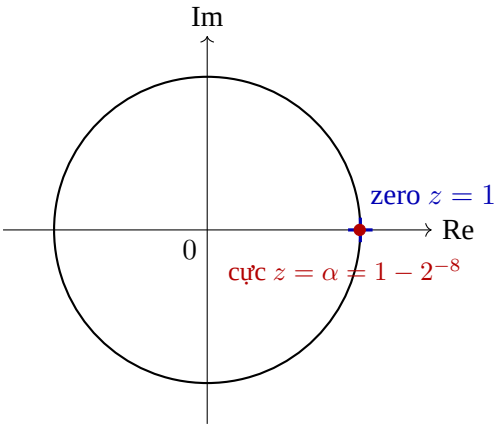
Hàm truyền chuẩn của DC-blocker bậc nhất:

$$H(z) = \frac{1 - z^{-1}}{1 - \alpha z^{-1}}, \qquad 0 < \alpha < 1.$$

Zero (nghiệm tử số) tại $z = 1$ — đúng tần số 0 Hz — nên $H(1) = 0$: mọi tín hiệu hoàn toàn đứng yên bị triệt tiêu. **Cực** (nghiệm mẫu số) tại $z = \alpha$, nằm sát zero nhưng vẫn trong vòng đơn vị nên bộ lọc ổn định. Phương trình sai phân tương đương:

$$y[n] = x[n] - x[n-1] + \alpha\, y[n-1].$$

Đáp ứng tần số gần DC: $|H(e^{j\omega})|$ tăng gần tuyến tính với ω khi ω nhỏ; hằng thời gian xấp xỉ $1/(1-\alpha)$ mẫu.



Zero trên vòng đơn vị tại DC: $H(1) = 0$.
Cực gần zero (trong vòng): HPF nhẹ, ổn định.

Hình 23: DC-blocker trên mặt phẳng z: zero tại $z = 1$, cực tại $\alpha = 1 - 2^{-8} = 0{,}99609375$.

Trên silicon ta chọn $\alpha = 1 - 2^{-8} = 0{,}99609375$ vì phép nhân $\alpha \cdot y$ thu gọn thành một phép dịch và trừ:

$$\alpha y = y - (y \ggg 8) = y \cdot (1 - 2^{-8}),$$

không tốn khối DSP. Hằng thời gian $1/(1-\alpha) = 256$ mẫu. Tại $f_s = 16$ kHz (sau FIR), cutoff xấp xỉ

$$f_c \approx \frac{1-\alpha}{2\pi} f_s = \frac{2^{-8}}{2\pi} \cdot 16000 \approx 9{,}95\,\text{Hz},$$

nằm dưới $F_{0,\min} = 75$ Hz và trên DC — đúng chỗ cần chặn. Ở 48 kHz ngay sau codec, $f_c \approx 29{,}84$ Hz, vẫn an toàn trước khi hạ mẫu.

Ví dụ tính tay — đáp ứng với $\alpha = 0{,}5$ (cực xa để thấy nhanh). Đầu vào hằng $x = [1, 1, 1, 1, 1]$, $y[-1] = 0$:

n	$x[n]$	$y[n] = (x[n] - x[n-1]) + \alpha y[n-1]$	giá trị
0	1	$1 - 0 + 0$	1
1	1	$0 + 0{,}5 \cdot 1$	0,5
2	1	$0 + 0{,}5 \cdot 0{,}5$	0,25
3	1	$0 + 0{,}5 \cdot 0{,}25$	0,125
4	1	$0 + 0{,}5 \cdot 0{,}125$	0,0625

Năng lượng bị nuốt theo cấp số nhân về 0. Với $\alpha = 0{,}75$ trên cùng đầu vào hằng: 1, 0,75, 0,5625, 0,421875, 0,31640625, …— chậm hơn $\alpha = 0{,}5$ nhưng vẫn về 0. Với $\alpha = 0{,}99609375$ suy giảm chậm khoảng 256 mẫu cho một hằng số thời gian.

3.3. Phần cứng: accumulator 32-bit, ví dụ chu kỳ và RTL

FIR Compiler II không thay thế được IIR có phản hồi: kiến trúc FIR không có vòng $y[n-1]$. Viết khoảng mười dòng Verilog với accumulator 32 bit là lựa chọn dạy và lab.

Nếu giữ y chỉ 16 bit Q1.15, mỗi lần dịch $\ggg 8$ sẽ cắt mất phần thập phân khi $|y|$ nhỏ, tạo vùng chết (deadband) hoặc dao động limit-cycle. Giải pháp: giữ thanh ghi phản hồi `y_acc` signed 32 bit; mẫu vào Q1.15 được kéo lên cùng thang bằng dịch trái 15 bit trước khi trừ; ngõ ra lấy lại Q1.15 bằng dịch phải 15 bit rồi bão hòa.

Phương trình bit-exact.

$$\begin{aligned} x_\Delta &= (x[n] \ll 15) - (x[n-1] \ll 15), \\ y_{\text{acc}}[n] &= y_{\text{acc}}[n-1] + x_\Delta - (y_{\text{acc}}[n-1] \ggg 8), \\ y_{\text{out}} &= \text{sat}_{16}(y_{\text{acc}}[n] \ggg 15), \end{aligned}$$

với sat_{16} ghim vào $[-32768, +32767]$. Trên số có dấu, Python/NumPy `>>` và Verilog `>>>` cùng là arithmetic floor shift.

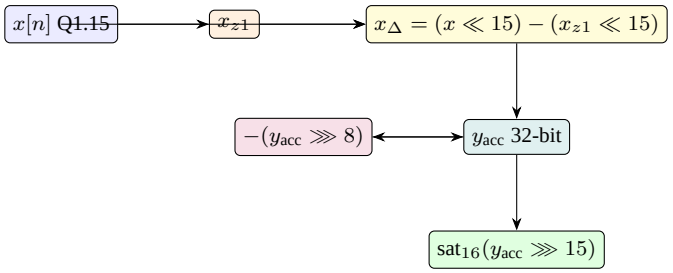
Ví dụ tính tay — từng chu kỳ sau bước Q1.15. Đầu vào $[0, 0, 0]$ rồi bước DC 16384 ($0{,}5$ trong Q1.15). Bảng trung gian (đã chạy script):

n	x	x_Δ	y_{acc} trước	y_{acc} sau	y_{out}
0..2	0	0	0	0	0
3	16384	536870912	0	536870912	16384
4	16384	0	536870912	534773760	16320
5	16384	0	534773760	532684800	16256
6	16384	0	532684800	530604000	16192
7	16384	0	530604000	528531329	16129
8	16384	0	528531329	526466754	16066

Tại $n = 3$: $x_\Delta = 16384 \ll 15 = 536870912$, $y_{\text{acc}} = 536870912$, $y_{\text{out}} = 536870912 \ggg 15 = 16384$. Tại $n = 4$:

$$y_{\text{acc}} \leftarrow 536870912 + 0 - (536870912 \ggg 8) = 534773760, \quad y_{\text{out}} = 16320.$$

Mỗi bước đầu $|y_{\text{out}}|$ giảm khoảng 64 — đúng $(y \ggg 8) \ggg 15 = y_{\text{out}} \ggg 8$ khi chưa bão hòa. Sau khoảng $5 \cdot 256$ mẫu, đầu ra tụt dưới 50 LSB.



Hình 24: Datapath DC-blocker: sai phân kéo thang $\ll 15$, phản hồi 32 bit với $\ggg 8$, ngõ ra bão hòa Q1.15.

RTL giải thích từng nhóm.

```
module dc_blocker (
  input  wire               clk, rst_n,
  input  wire               in_valid,
  output wire               in_ready,
  input  wire signed [15:0] x_in,       // Q1.15
  output reg                out_valid,
  input  wire               out_ready,
  output reg  signed [15:0] y_out
);
  reg signed [15:0] x_z1;               // trễ 1 mẫu
```

```
  reg signed [31:0] y_acc;              // phản hồi 32-bit (KHÔNG cắt 16)
  assign in_ready = out_ready | ~out_valid;  // Avalon-ST backpressure

  wire signed [31:0] x_delta =
      ($signed({x_in, 15'b0})) - ($signed({x_z1, 15'b0}));
  // {x,15'b0} = x<<15 trong cùng bề rộng 31+1 bit có dấu
  wire signed [31:0] y_next =
      y_acc + x_delta - (y_acc >>> 8);  // >>> = arithmetic shift
  wire signed [31:0] y_shift = y_next >>> 15;

  function automatic signed [15:0] sat16;
    input signed [31:0] v;
    begin
      if (v > 32'sd32767)       sat16 = 16'sd32767;
      else if (v < -32'sd32768) sat16 = -16'sd32768;
      else                      sat16 = v[15:0];
    end
  endfunction

  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      x_z1 <= 0; y_acc <= 0; y_out <= 0; out_valid <= 0;
    end else if (in_valid && in_ready) begin
      y_acc <= y_next;     // update feedback
      x_z1  <= x_in;
      y_out <= sat16(y_shift);
      out_valid <= 1'b1;
    end else if (out_valid && out_ready) out_valid <= 1'b0;
  end
endmodule
```

`in_ready = out_ready | ~out_valid` cho phép nhận mẫu mới khi đầu ra trống hoặc bên dưới đã lấy. `x_delta` dùng nối bit `{x,15'b0}` thay vì dịch có dấu để tránh mở rộng sai. `y_acc >>> 8` là chỗ hiện thực $\alpha = 1 - 2^{-8}$. `sat16` bắt buộc: nếu bỏ bão hòa, tràn 32→16 tạo tiếng click giả đỉnh F_0.

3.3b. Hiện tượng limit-cycle và chọn α trên silicon

▶ NHÌN THẤY VẬT LÝ

Trên giấy, bộ lọc IIR tuyến tính với $|\alpha| < 1$ luôn đưa trạng thái về 0 khi đầu vào tắt. Trên silicon, mỗi phép nhân/dịch bị *lượng tử hóa*: sai số làm tròn không còn là nhiễu nhỏ thụ động mà bị khuếch đại quanh cực sát vòng đơn vị. Kết quả có thể là một dao động biên độ nhỏ *bền vững* — gọi là **limit cycle** (chu trình giới hạn). Hình dung quả lắc đồng hồ: nếu mỗi bước chỉnh kim chỉ được nhảy nguyên "một vạch" thô, kim có thể mãi mãi nhảy qua lại giữa hai vạch quanh vị trí nghỉ thay vì dừng hẳn. DC-blocker với α sát 1 (cực rất gần $z = 1$) đặc biệt nhạy khi thanh ghi y chỉ rộng 16 bit.

Hai mặt của cùng một lượng tử. (1) **Vùng chết (deadband):** với cập nhật $y \leftarrow y - (y \ggg 8)$ trên số nguyên, khi $|y| < 256$ thì $y \ggg 8 = 0$, nên y *đứng im* dù vẫn còn residual — offset nhỏ đọng lại, nuôi $\mathcal{R}[0]$ giả. (2) **Limit cycle lưỡng cực:** nếu phép làm tròn kèm đổi dấu (mô hình toy của nhánh phản hồi lượng tử), trạng thái có thể nhảy $+\ell \leftrightarrow -\ell$ mãi mãi. Cả hai đều là hệ quả của lưới số nguyên quá thô so với cực gần vòng đơn vị.

Ví dụ tính tay — deadband trên lưới dịch 8 bit. Giả sử sau khi DC lớn đã suy giảm, residual còn $y = 255$ và đầu vào đã về 0. Mỗi bước

$$y \leftarrow y - (y \ggg 8).$$

Vì $255 \ggg 8 = 0$, ta được $y = 255$ *ở mọi* bước tiếp theo. Cùng công thức với $y = 256$:

$$\begin{aligned} y_0 &= 256, \\ y_1 &= 256 - (256 \ggg 8) = 256 - 1 = 255, \\ y_2 &= 255 - (255 \ggg 8) = 255. \end{aligned}$$

Residual kẹt ở 255 LSB — đó chính là vùng chết. Trên Q1.15, $255/32768 \approx 0{,}0078$ vẫn đủ làm phình nhẹ $\mathcal{R}[0]$ nếu đọng xuyên khung.

Ví dụ tính tay — limit cycle lưỡng cực (mô hình làm tròn). Xét ánh xạ minh họa tối giản $y \leftarrow -\text{round}(0{,}875 \cdot y)$ (minh họa phản hồi lượng tử có đổi dấu; không phải RTL lab). Xuất phát $y_0 = 5$:

$$\begin{aligned} y_1 &= -\text{round}(0{,}875 \cdot 5) = -\text{round}(4{,}375) = -4, \\ y_2 &= -\text{round}(0{,}875 \cdot (-4)) = +4, \\ y_3 &= -\text{round}(0{,}875 \cdot 4) = -4. \end{aligned}$$

Từ bước 1 trở đi dãy là $-4, +4, -4, +4, \ldots$ (biên độ không về 0): đó là limit cycle chu kỳ 2.

Script `verify_chuong_03.py` khẳng định cả deadband 255 và dao động ± 4.

Vì sao accumulator 32 bit triệt cả hai. RTL lab giữ y_{acc} signed 32 bit; phép $y_{\text{acc}} \leftarrow y_{\text{acc}} + x_\Delta - (y_{\text{acc}} \ggg 8)$ làm việc trên lưới mịn hơn LSB ngõ ra Q1.15 khoảng 2^{15} lần. Nhiễu lượng tử bị đẩy xuống dưới LSB của mẫu 16 bit lấy ra bằng $y_{\text{acc}} \ggg 15$ rồi bão hòa — người nghe và khối MAC không còn thấy vòng dao động hay residual đứng im ở thang 16 bit. Đó là lý do kỹ thuật, không phải "thói quen 32 bit".

Chọn α trên silicon. Có thể lấy $\alpha = 1 - 2^{-7}$ (cutoff gần gấp đôi) hoặc $1 - 2^{-9}$ (chậm hơn) chỉ bằng đổi hằng dịch trong RTL; không cần DSP. Lab khóa 2^{-8} để mọi twin và mọi hình dùng một số. Đổi α phải đổi cả bảng f_c và bài tập.

Ví dụ tính tay — đáp ứng xung $\alpha = 0{,}5$ (thấy nhanh cực xa). Đặt $x = [1, 0, 0, 0, 0]$, $y[-1] = 0$, $\alpha = 0{,}5$:

$$\begin{aligned} y[0] &= (1 - 0) + 0{,}5 \cdot 0 = 1, \\ y[1] &= (0 - 1) + 0{,}5 \cdot 1 = -0{,}5, \\ y[2] &= (0 - 0) + 0{,}5 \cdot (-0{,}5) = -0{,}25, \\ y[3] &= -0{,}125, \\ y[4] &= -0{,}0625. \end{aligned}$$

Đáp ứng đổi dấu rồi về 0 — đúng HPF: tổng mẫu tiến về 0 ($H(1) = 0$). Trên Q1.15, xung 16384 cho dãy tương tự sau khi nhân thang; với accumulator 32 bit, đuôi không kẹt deadband.

3.4. Twin Python bit-exact

```
import numpy as np

def sat16(v):
    return int(np.clip(v, -32768, 32767))

def dc_blocker_q15(x, alpha_shift=8):
    """Bit-exact với RTL: y_acc int32, x Q1.15 int16."""
    y_acc = np.int32(0)
    x_z1 = np.int32(0)
    out = np.empty(len(x), dtype=np.int16)
    for i, xi in enumerate(np.asarray(x, dtype=np.int32)):
        dx = np.int32((xi << 15) - (x_z1 << 15))
        y_acc = np.int32(y_acc + dx - (y_acc >> alpha_shift))
```

```
        x_z1 = xi
        out[i] = sat16(int(y_acc >> 15))
    return out
```

Vòng lặp trên phải khớp từng LSB với dump simulation RTL. Đổi `np.int32` thành `int` Python không giới hạn bề rộng sẽ *sai* khi gần tràn 32 bit — twin phải giữ đúng modulo hai's complement 32 bit như silicon. Script `verify/verify_chuong_03.py` khẳng định $y[3{:}6] = [16384, 16320, 16256]$, $f_c(16\,\text{kHz}) \approx 9{,}947\,\text{Hz}$, và sau 1600 mẫu bước thì $|y| < 50$.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — DC-blocker (mọi viết tắt).

- **ALM** (*Adaptive Logic Module*) / **LUT** (*Look-Up Table*): khoảng 80–120 ALM cho adder/sub 32 bit + dịch số học + bão hòa.
- **DSP** (*Digital Signal Processing* block): 0 — nhờ $\alpha = 1 - 2^{-8}$ hiện thực bằng dịch, không nhân.
- **M10K** / **MLAB**: 0 (không cần đệm mẫu trong khối này).
- **Latency**: 1 mẫu (pipeline Avalon-ST một nhịp) $\approx 20{,}8\,\mu$s ở 48 kHz hoặc $62{,}5\,\mu$s ở 16 kHz.
- **Fmax** (*maximum frequency*): dễ > 200 MHz trên tốc độ 6.
- **RTF** (*real-time factor*): một mẫu vào / một mẫu ra mỗi lần bắt tay `valid`/`ready` $\Rightarrow$ RTF$= 1$ khi không back-pressure.
- Vì sao không cắt 16 bit sớm: tiết kiệm ~ 30 ALM nhưng gây deadband/limit-cycle — đắt hơn mọi LUT trên đường F_0. Vì sao không FIR Compiler II: IIR phản hồi không map vào kiến trúc FIR taps.

Đạo hàm cutoff từ cực. Với $H(z) = (1 - z^{-1})/(1 - \alpha z^{-1})$, trên vòng đơn vị $z = e^{j\omega}$ và $\omega \ll 1$ ta xấp xỉ $1 - e^{-j\omega} \approx j\omega$ và $1 - \alpha e^{-j\omega} \approx (1 - \alpha) + j\alpha\omega$. Biên độ

$$|H| \approx \frac{\omega}{\sqrt{(1-\alpha)^2 + (\alpha\omega)^2}}.$$

Đặt $|H| = 1/\sqrt{2}$ và $\alpha \approx 1$ cho $\omega_c \approx 1 - \alpha$, tức $f_c \approx (1 - \alpha) f_s/(2\pi)$ — đúng công thức đã dùng. Sai số của xấp xỉ này ở $\alpha = 1 - 2^{-8}$ dưới vài phần trăm so với nghiệm đúng phương trình $|H| = 1/\sqrt{2}$; đủ cho chọn tham số lab, không đủ thay đo đạc nếu đổi α lớn.

Vị trí trên datapath và quan hệ với FIR. Có hai chỗ hợp lệ: (1) ngay sau CDC ở 48 kHz rồi mới vào chuỗi hạ mẫu; (2) sau FIR 16 kHz. Chỗ (1) triệt DC sớm, MAC và cửa sổ không bao giờ thấy offset; chỗ (2) rẻ chu kỳ hơn một chút vì chạy chậm hơn nhưng FIR/half-band phải "chở" DC qua. Tài liệu này mặc định **sau CDC, trước FIR** — khớp bản đồ Chương 0. DC-blocker *không* thay high-pass 60 Hz kiểu telephone nếu lab cần cắt rumble mạnh; $\alpha = 1 - 2^{-8}$ chỉ là HPF rất nhẹ.

Tràn 32-bit và chọn bề rộng. Với $|x| \le 32767$, $|x_\Delta| \le 32767 \cdot 2^{15} \cdot 2 < 2^{31}$, và nhánh $-(y \ggg 8)$ cùng Magnitut, tổng $y_{\text{acc}} + x_\Delta - (y \ggg 8)$ trong vận hành bình thường (sau reset, có Avalon-ST) không vượt signed 32 bit nếu đầu vào không phải xung full-scale xen kẽ dấu liên tục ở tần số cực đại. Lab vẫn giữ `sat16` ở ngõ ra; nếu muốn chắc hơn nữa có thể bão hòa y_{acc} vào $\pm(2^{31} - 1)$ trước khi ghi thanh ghi — hiếm khi kích hoạt trên tiếng nói thật.

So sánh với trừ mean theo khung. Trừ mean Praat: $\tilde{x}[n] = x[n] - \frac{1}{L}\sum_{k=0}^{L-1} x[k]$ trên từng khung $L = 640$. Cách đó cần cộng 640 mẫu rồi chia (hoặc nhân $1/L$) mỗi hop — tốn MAC và trễ một khung. IIR DC-blocker cập nhật mỗi mẫu với hai phép cộng/trừ và một dịch, latency một mẫu, phù hợp streaming FPGA. Sai khác: IIR có "đuôi" bộ nhớ y_{acc} xuyên khung; trừ mean không. Trên tiếng nói, đuôi 256 mẫu (16 ms ở 16 kHz) chấp nhận được so với hop 10 ms.

3.4b. Avalon-ST quanh DC-blocker và đo latency

Trên đường tín hiệu, DC-blocker là khối streaming mẫu-đơn: mỗi khi `in_valid & in_ready` nhận đúng một $x[n]$, đúng một nhịp sau (hoặc cùng nhịp pipeline tổ hợp + latch) phát `out_valid` với $y[n]$. Không dùng `sop`/`eop` vì không có biên khung ở đây — biên khung chỉ xuất hiện khi sliding RAM (Chương 5) phát burst L mẫu. Backpressure: nếu FIR phía dưới kéo `out_ready`=0, DC-blocker hạ `in_ready`, DCFIFO phía trên dần đầy; depth 16 đã tính ở Chương 2 hấp thụ vài trăm microgiây. Đo latency lab: đếm chu kỳ từ cạnh lên `in_valid` đến cạnh lên `out_valid` trên ModelSim/Questa với testbench twin — phải ra đúng 1 (theo RTL trên) hoặc đúng số pipeline đã chèn.

Kiểm chứng năng lượng sau khi chặn DC. Lấy chuỗi step Q1.15 dài 1600 mẫu như script: năng lượng $\sum y^2$ tập trung ở đoạn đầu (đáp ứng HPF với bậc thang) rồi đuôi gần 0. Nếu quên `sat16` và tràn, năng lượng đuôi có thể bùng — đó là tín hiệu debug. So sánh với twin float $\alpha = 0{,}99609375$ trên cùng tín hiệu chuẩn hóa: sai số từng mẫu sau đoạn transient phải nằm trong vài LSB khi lượng tử đúng.

Người chấm lab đối chiếu dump RTL với bảng y_{acc} trong mục 3.3: lệch một LSB là chưa đạt. Người học chỉ được chuyển Chương 4 khi script `verify_chuong_03.py` chạy OK và RTL khớp dump.

3.5. Bài tập và lời giải giải thích

Bài tập 3.1 (Phình $\mathcal{R}[0]$.) Cho $x_0 = [1, -1, 1, -1]$ và offset $A = 2$. Tính $\sum (x_0[n] + A)^2$ và giải thích vì sao DC-blocker phải đứng trước MAC.

Lời giải. $\sum (x_0 + A)^2 = 20$, trong khi $\sum x_0^2 = 4$. Offset nhân năng lượng $\tau = 0$ lên năm lần; không chặn DC thì ngưỡng voicing và đỉnh T_0 đều lệch.

Bài tập 3.2 (Đệ quy $\alpha = 0{,}5$ và $\alpha = 0{,}75$).

a) Với $x = [1, 1, 1, 1, 1]$, $\alpha = 0{,}5$, lập bảng $y[0..4]$.

b) Lặp lại với $\alpha = 0{,}75$; nhận xét tốc độ về 0.

Lời giải. (a) $1, 0{,}5, 0{,}25, 0{,}125, 0{,}0625$. (b) $1, 0{,}75, 0{,}5625, 0{,}421875, 0{,}31640625$ — α lớn hơn (cực sát vòng đơn vị hơn) suy giảm chậm hơn; $\alpha = 1 - 2^{-8}$ còn chậm hơn nữa (~ 256 mẫu).

Bài tập 3.3 (Bit-exact step từng chu kỳ). Dùng công thức $y_{\text{acc}} \mathrel{+}= x_\Delta - (y_{\text{acc}} \ggg 8)$, tính y_{out} tại $n = 3, 4, 5$ của bước 16384 (sau ba số 0). Viết cả y_{acc} trung gian ở $n = 4$.

Lời giải. $y_{\text{out}}[3] = 16384$, $y_{\text{acc}}[4] = 534773760$, $y_{\text{out}}[4] = 16320$, $y_{\text{out}}[5] = 16256$ (script). Sai một LSB đồng nghĩa twin và RTL lệch thang.

Bài tập 3.4 (Cutoff và $F_{0,\text{min}}$). Tính f_c tại 16 kHz và 48 kHz với $\alpha = 1 - 2^{-8}$. Vì sao cả hai đều chấp nhận được trước $F_{0,\text{min}} = 75$ Hz?

Lời giải. $f_c \approx 9{,}95$ Hz và $29{,}84$ Hz — cả hai dưới 75 Hz nên không ăn vào dải cao độ; vẫn trên DC nên triệt offset.

Bài tập 3.5 (Deadband 16-bit). Giả sử ai đó giữ phản hồi chỉ 16 bit và làm $y \leftarrow y - (y \ggg 8)$ trên Q1.15. Khi $|y| < 256$, giải thích hiện tượng vùng chết.

Lời giải. $y \ggg 8 = 0$ khi $|y| < 256$, nên y dừng cập nhật dù vẫn còn offset nhỏ — residual DC đọng lại, nuôi $\mathcal{R}[0]$ giả. Accumulator 32 bit giữ phần thập phân dưới ngưỡng 16 bit nên không chết.

Bài tập 3.6 (Trừ mean vs IIR). So sánh chi phí: trừ mean trên khung $L = 640$ mỗi hop $H = 160$ ở 50 MHz cần tối thiểu bao nhiêu phép cộng mỗi hop? DC-blocker cần bao nhiêu phép cộng/trừ mỗi mẫu? Trong 10 ms có bao nhiêu mẫu ở 48 kHz?

Lời giải. Trừ mean: ≥ 640 phép cộng + 1 phép nhân/chia mỗi hop (cộng thêm L phép trừ để tách mean). DC-blocker: 2–3 phép cộng/trừ mỗi mẫu. Trong $10\,\text{ms}$ ở $48\,\text{kHz}$ có 480 mẫu — khoảng $480 \times 3 \approx 1440$ phép, vẫn rẻ hơn nhiều so với MAC tự tương quan ($\sim 189 \times 640$ nhân-cộng mỗi hop, Chương 0), và latency một mẫu thay vì một khung.

Chương 4. FIR Compiler II — hạ mẫu $48 \to 24 \to 16$ kHz (polyphase $2/3$)

▶ MỤC TIÊU HỌC TẬP

Sau Chương 4, người học hạ mẫu đúng chuỗi $48\,\text{kHz} \to 24\,\text{kHz} \to 16\,\text{kHz}$: lọc chống alias trước mỗi lần bỏ mẫu, cấu hình FIR Compiler II fractional rate $I = 2, D = 3$ (polyphase E_0/E_1), giải thích folding/pre-adder và cách IP chia sẻ DSP theo thời gian với số liệu cụ thể, viết twin Python cùng hệ số Q1.15. Thuật ngữ FIR, taps, polyphase, transition band đã có ở ô Feynman 9 của Chương 0; aliasing ở ô 2.

4.1. Hiện tượng: bỏ mẫu không lọc sẽ nuốt F_0

▶ NHÌN THẤY VẬT LÝ

Codec WM8731 xuất PCM ở $48\,\text{kHz}$. Ngân sách Praat/FPGA làm việc ở $16\,\text{kHz}$ (Nyquist $8\,\text{kHz}$ đủ cho F_0 và vài formant thấp). **Hạ mẫu (decimation)** nghĩa là giữ lại một phần mẫu và bỏ phần còn lại. Nếu bỏ mẫu mà chưa lọc thông thấp, mọi thành phần trên nửa tần số mới sẽ **alias** (gương qua Nyquist — Chương 0) đổ xuống vùng $75 \ldots 600\,\text{Hz}$ đúng chỗ ta tìm cao độ. Ví dụ: một formant hoặc nhiễu ở $9\,\text{kHz}$ khi hạ thẳng xuống $16\,\text{kHz}$ sẽ hiện ra như thành phần $7\,\text{kHz}$ hoặc thấp hơn tùy phép gập — đủ để tạo đỉnh giả trên phổ và làm nhiễu tự tương quan.

Lộ trình thực tế trên đường pitch:

1. $48\,\text{kHz} \xrightarrow{\downarrow 2} 24\,\text{kHz}$: hạ nguyên lần 2 (FIR half-band hoặc decimate có lọc), Nyquist mới $12\,\text{kHz}$. Half-band tận dụng gần nửa hệ số bằng 0 xen kẽ — tiết kiệm DSP.

2. $24\,\text{kHz} \xrightarrow{2/3} 16\,\text{kHz}$: tỉ số hữu tỉ — dùng FIR Compiler II Fractional Rate $I = 2, D = 3$ (mỗi 3 mẫu vào tạo 2 mẫu ra).

CIC (*Cascaded Integrator–Comb*) chỉ thắng khi tỷ lệ nguyên $R \gg 1$; với R hiệu dụng $1{,}5$, CIC méo sinc^N và cần bù — loại khỏi lộ trình này.

Ví dụ tính tay — alias số học đơn giản. Tín hiệu $x[n] = \cos(2\pi \cdot 9000 \cdot n/48000)$ tại $48\,\text{kHz}$. Nếu lấy mỗi mẫu thứ ba mà không lọc (minh họa thô cho ý gập phổ), thành phần cao gập vào dải thấp; sau lọc thông thấp cắt trên $\sim 8\,\text{kHz}$ rồi mới bỏ mẫu, biên độ thành phần $9\,\text{kHz}$ bị suy giảm mạnh trước khi gập — đúng vai trò anti-alias.

4.1b. Ngân sách phổ trước và sau hạ mẫu

Trước hạ mẫu, phổ hữu ích của tiếng nói cho bài toán F_0 tập trung dưới khoảng $1\,\text{kHz}$ cho bản thân cao độ, nhưng formant và hài có thể kéo lên vài kHz. Sau bước $\downarrow 2$, Nyquist còn $12\,\text{kHz}$ (tần số lấy mẫu $24\,\text{kHz}$) — vẫn rộng. Sau bước xuống $16\,\text{kHz}$, Nyquist còn $8\,\text{kHz}$: đó là ngưỡng cuối. Mọi năng lượng từ $8\,\text{kHz}$ đến $12\,\text{kHz}$ (và phần còn sót từ miền $48\,\text{kHz}$ trước đó) trên tín hiệu trung gian phải bị dải chặn của FIR $2/3$ đè xuống *trước* khi bỏ mẫu.

Stopband là gì trong thực tế. Bộ lọc số chia phổ thành ba vùng tên gọi chuẩn: **passband** (dải thông — tần số đi qua gần như nguyên vẹn), **transition band** (dải chuyển — suy giảm dần), và **stopband** (dải chặn — nơi biên độ phải bị đè xuống dưới một ngưỡng, ví dụ $-60\,\text{dB}$). Stopband không phải "chỗ không có tín hiệu"; nó là chỗ ta *ép* năng lượng đủ nhỏ để khi gập phổ (alias) xuống dải thấp, phần còn lại không đủ mạnh để tạo đỉnh giả trên tự tương quan. Nếu chỉ "lấy 2 mẫu bỏ 1" mà không lọc, tỷ lệ $2/3$ vẫn đúng về số mẫu nhưng sai về phổ.

Ví dụ tính tay — tone $10\,\text{kHz}$ gập vào đâu. Tại miền $24\,\text{kHz}$, một tone $f = 10\,\text{kHz}$ biên độ $0\,\text{dBFS}$ (biên độ tuyến tính 1). Khi hạ xuống $f_{s,\text{out}} = 16\,\text{kHz}$ mà *không* lọc, tần số alias gần nhất là

$$f_{\text{alias}} = |10\,\text{kHz} - 16\,\text{kHz}| = 6\,\text{kHz},$$

nằm gọn trong Nyquist 8 kHz — đúng vùng formant/cạnh phổ có thể nhiễu đỉnh T_0. Sau FIR có stopband -60 dB, biên độ tuyến tính còn

$$A_{-60} = 10^{-60/20} = 10^{-3} = 0{,}001$$

(tức công suất còn 10^{-6} so với 0 dBFS). Tự tương quan của nhiễu đó nhỏ hơn tín hiệu tiếng nói thông thường hàng chục dB — đủ để đỉnh T_0 không bị át. Nếu stopband chỉ -20 dB:

$$A_{-20} = 10^{-20/20} = 0{,}1,$$

công suất còn $0{,}01$ — nhiễu vẫn lớn, đỉnh phụ có thể thắng ngưỡng voicing. Lab khóa mục tiêu stopband ≤ -60 dB trước 8 kHz.

4.2. Toán FIR, folding và lượng tử hệ số

Bộ lọc **FIR** (*Finite Impulse Response* — đáp ứng xung hữu hạn) là phép tích chập với một dãy hệ số dài hữu hạn N. Định nghĩa gốc, chưa rút gọn:

$$y[n] = \sum_{k=0}^{N-1} h[k]\, x[n-k].$$

Mỗi hệ số $h[k]$ gọi là một **tap** ("vòi" lấy mẫu trễ k). Trên phần cứng, mỗi tap ứng với một phép nhân và một phép cộng vào accumulator — trừ khi ta khai thác đối xứng.

Từ tổng đầy đủ đến folding. Khi thiết kế pha tuyến tính (trễ nhóm không đổi theo tần số — quan trọng để không méo dạng xung dây thanh), hệ số đối xứng:

$$h[k] = h[N-1-k] \qquad (k = 0, 1, \ldots, N-1).$$

Viết lại tổng bằng cách ghép từng cặp $(k,\ N-1-k)$. Với N chẵn:

$$\begin{aligned} y[n] &= \sum_{k=0}^{N/2-1} \Big(h[k]\, x[n-k] + h[N-1-k]\, x[n-(N-1-k)] \Big) \\ &= \sum_{k=0}^{N/2-1} h[k] \Big(x[n-k] + x[n-(N-1-k)] \Big), \end{aligned}$$

vì $h[N-1-k] = h[k]$. Đây là **symmetric folding**: chỉ còn $N/2$ phép nhân thay vì N. (Với N lẻ còn thêm một tap giữa nhân một lần.)

Pre-adder là gì trên silicon. Khối **DSP** 18×19 của Cyclone V có sẵn một bộ cộng cứng ngay trước nhân, gọi là **pre-adder**: nó tính $x_a + x_b$ trong cùng chu kỳ rồi đưa tổng vào nhân với hệ số h. Đúng khớp công thức folding ở trên — vì thế FIR đối xứng map gọn vào silicon mà không tốn ALM để cộng tay trước nhân. Nếu bỏ folding, mỗi cặp tốn hai nhân DSP thay vì một nhân + một pre-adder.

Ví dụ tính tay — folding $N = 4$ từng bước. Cho $h = [h_0, h_1, h_2, h_3] = [1, 2, 2, 1]$ (đối xứng) và cửa sổ mẫu $[x_0, x_1, x_2, x_3] = [a, b, c, d] = [3, 4, 5, 6]$. Tổng thẳng, chưa folding:

$$\begin{aligned} y &= h_0 a + h_1 b + h_2 c + h_3 d \\ &= 1 \cdot 3 + 2 \cdot 4 + 2 \cdot 5 + 1 \cdot 6 \\ &= 3 + 8 + 10 + 6 = 27 \end{aligned}$$

(bốn phép nhân). Cùng y theo folding ($N/2 = 2$ nhân):

$$\begin{aligned} y &= h_0(a + d) + h_1(b + c) \\ &= 1 \cdot (3 + 6) + 2 \cdot (4 + 5) \\ &= 1 \cdot 9 + 2 \cdot 9 \\ &= 9 + 18 = 27. \end{aligned}$$

Hai đường trùng nhau; pre-adder cứng chính là chỗ tính $(3 + 6)$ và $(4 + 5)$ trước khi nhân.

Ví dụ tính tay — convolution 3-tap từng số hạng. $h = [1, 2, 1]$, $x = [3, 4, 5, 0, 0]$, quy ước $x[m] = 0$ khi $m < 0$:

$$y[0] = 1 \cdot x[0] + 2 \cdot x[-1] + 1 \cdot x[-2] = 1 \cdot 3 + 2 \cdot 0 + 1 \cdot 0 = 3,$$
$$y[1] = 1 \cdot x[1] + 2 \cdot x[0] + 1 \cdot x[-1] = 1 \cdot 4 + 2 \cdot 3 + 1 \cdot 0 = 10,$$
$$y[2] = 1 \cdot x[2] + 2 \cdot x[1] + 1 \cdot x[0] = 1 \cdot 5 + 2 \cdot 4 + 1 \cdot 3 = 16.$$

Với $h = [1, 2, 1]/4$ và $x = [4, 0, 4, 0, 4]$: $y = [1, 2, 2, 2, 2]$ — thông thấp làm phẳng thành phần gần Nyquist (script kiểm).

Lượng tử hệ số Q1.15 từng bước. Đưa hệ số thực $h \in [-1, 1)$ sang số nguyên có dấu 16 bit:

$$h_q[k] = \text{sat}_{16}\big(\text{round}(h[k] \cdot 2^{15})\big), \qquad 2^{15} = 32768.$$

Ví dụ $h = [0{,}25, 0{,}5, 0{,}25]$:

$$0{,}25 \cdot 32768 = 8192, \quad 0{,}5 \cdot 32768 = 16384 \quad \Rightarrow \quad h_q = [8192, 16384, 8192].$$

Ví dụ dài hơn $h = [0{,}1, 0{,}2, 0{,}4, 0{,}2, 0{,}1]$:

$$0{,}1 \cdot 32768 = 3276{,}8 \to 3277,$$
$$0{,}2 \cdot 32768 = 6553{,}6 \to 6554,$$
$$0{,}4 \cdot 32768 = 13107{,}2 \to 13107,$$

nên $h_q = [3277, 6554, 13107, 6554, 3277]$ và $\sum h_q = 32769$ (lệch 1 LSB khỏi 32768 do làm tròn — lab ghi nhận và dùng đúng vector này trong IP lẫn twin). Twin, ROM và FIR IP phải dùng *cùng* h_q.

4.3. Polyphase $2/3$: từ $H(z)$ đến commutator E_0, E_1

Viết $k = mM + r$ với $r = 0, \ldots, M - 1$:

$$H(z) = \sum_{r=0}^{M-1} z^{-r} E_r(z^M), \qquad E_r(z) = \sum_m h[mM + r] z^{-m}.$$

Noble identity: sau khi hạ mẫu $\downarrow M$, các nhánh E_r chạy ở tốc độ thấp hơn — tiết kiệm tới hệ số M số MAC so với FIR thẳng rồi bỏ mẫu.

Với fractional $I/D = 2/3$: upsample $\uparrow 2$ (chèn 0) $\to$ lọc $\to$ downsample $\downarrow 3$, được gộp thành polyphase. **Chiều đúng:** mỗi 3 mẫu vào tạo 2 mẫu ra. Ví dụ tách $h = [h_0, \ldots, h_5]$, $M = 3$:

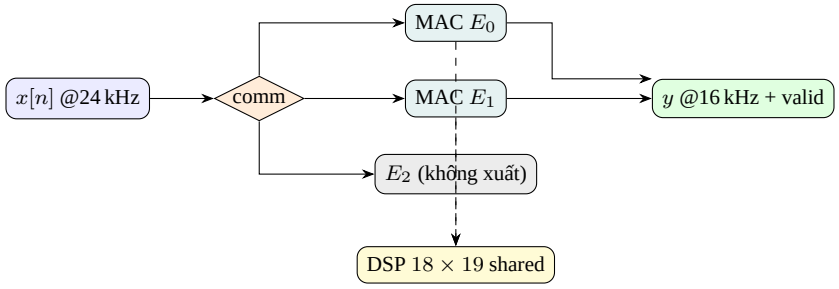
$$E_0 = [h_0, h_3], \quad E_1 = [h_1, h_4], \quad E_2 = [h_2, h_5].$$

Với số nguyên $h = [1, 2, 3, 4, 5, 6]$: $E_0 = [1, 4]$, $E_1 = [2, 5]$, $E_2 = [3, 6]$. Commutator phân phối mẫu vào các nhánh; trên đường $24 \to 16$ kHz chỉ xuất hai pha mỗi ba chu kỳ vào.

Ví dụ lịch mẫu và convolution đầy đủ. Sáu mẫu vào $x = [1, 2, 3, 4, 5, 6]$ qua $h = [1, 2, 3, 4, 5, 6]$ cho

$$y_{\text{full}} = [1, 4, 10, 20, 35, 56, 70, 76, 73, 60, 36]$$

(mode `full`). Lịch $2/3$ giữ 2 trong mỗi 3 chỉ số ra sau khi lọc (chi tiết lịch phụ thuộc phase của commutator trong IP). Tỉ số độ dài hữu ích: sáu vào $\to$ bốn ra cho đoạn ổn định tỷ lệ $4/6 = 2/3$.



Commutator = bộ đếm pha + mux. Chỉ E_0, E_1 xuất theo lịch $2/3$;
một DSP time-mux nhiều tap nhờ folding và clock dư.

Hình 25: Polyphase fractional $I = 2, D = 3$: ba nhánh pha, hai nhánh xuất; DSP chia sẻ theo thời gian.

4.3b. Half-band $48 \to 24$ trước bước $2/3$

Trước fractional $2/3$, bước $\downarrow 2$ từ $48 \to 24$ kHz nên dùng FIR half-band: gần một nửa hệ số đúng bằng 0 (các tap xen kẽ), transition band đối xứng quanh $f_s/4$. Số nhân logic gần còn một nửa so với FIR thông thấp tổng quát cùng độ dài. FIR Compiler II có tùy chọn half-band decimator; lab có thể tách thành hai IP nối tiếp (half-band $\downarrow 2$ rồi fractional $2/3$) để báo cáo DSP từng khối rõ ràng. Twin Python: cùng h_q half-band, `convolve` rồi lấy mẫu chẵn (hoặc lẻ — khớp phase IP).

Ngân sách chu kỳ toàn chuỗi. Một mẫu ra 16 kHz "ăn" trung bình $48/16 = 3$ mẫu vào gốc. Ở 50 MHz, mỗi mẫu ra có

$$\frac{50 \cdot 10^6}{16000} = 3125 \text{ chu kỳ}$$

để toàn bộ half-band + $2/3$ + DC + MAC chạy xong trước mẫu tiếp theo nếu muốn pipeline sâu theo mẫu; thực tế hệ thống làm theo hop 10 ms (500 000 chu kỳ) nên FIR streaming chỉ cần không bỏ `valid` — RTF phần FIR $\ll 1$.

Ví dụ tính tay — đếm MAC polyphase. Cho h dài 6 như trên, mỗi mẫu ra từ nhánh E_0 cần $|E_0| = 2$ nhân, nhánh E_1 cần 2 nhân; nhánh E_2 không xuất nên 0 nhân cho mẫu ra. Trung bình mỗi mẫu vào "gánh" ít MAC hơn FIR 6-tap thẳng (6 nhân) rồi bỏ $1/3$ kết quả. Đúng tinh thần Noble identity: lọc sau khi đã giảm tốc.

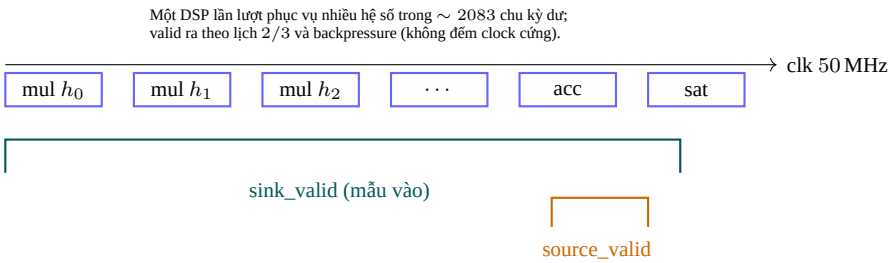
4.4. Intel FIR Compiler II — folding, chia sẻ DSP và số liệu

FIR Compiler II (UG 683208) hỗ trợ Distributed Arithmetic, Mult-add hoặc Systolic. Trên Cyclone V map vào khối DSP Variable Precision: mode 18×19 (đủ Q1.15×Q1.15) hoặc 27×27 khi hệ số/accum rộng hơn. Cổng Avalon-ST: `data`/`valid`/`ready`; `sop`/`eop` là biên *kênh* multi-channel — **không** phải biên khung pitch 40 ms. Lab mono liên tục chỉ cần data/valid/ready.

Cách IP gấp và chia sẻ DSP (số liệu lab). Giả sử thiết kế FIR đối xứng $N = 48$ taps cho nhánh $24 \to 16$ kHz. Folding còn $N/2 = 24$ phép nhân "logic". Polyphase $M = 3$ với $I = 2$ chỉ xuất 2 nhánh — số MAC hữu ích mỗi mẫu ra thấp hơn FIR thẳng. Clock hệ thống 50 MHz và tốc độ mẫu vào 24 kHz cho

$$\frac{50 \cdot 10^6}{24000} \approx 2083 \text{ chu kỳ clock mỗi mẫu vào.}$$

FIR Compiler II khai thác dư địa này: một khối DSP 18×19 được **time-multiplex** qua nhiều cặp (h_k, x) trong cùng một khoảng giữa hai mẫu audio. Báo cáo generate điển hình cho order lab (vài chục tap, fractional $2/3$) ghi khoảng vài đến vài chục DSP — *phải đọc Fitter*, không đoán. Nếu bật "Fully Parallel", số DSP ≈ số nhân sau folding và tăng vọt; lab chọn chế độ cho phép folding/sharing để giữ ngân sách DSP toàn hệ thống khoảng 3 khối (bảng Chương 0) hoặc nới có kiểm soát nếu đo Fmax/RTF vẫn đạt.



Hình 26: Chia sẻ DSP theo thời gian giữa hai mẫu vào 24 kHz: IP gấp nhiều nhân vào một khối 18×19; `source_valid` theo lịch fractional.

Cấu hình dạy. Fractional Rate $I = 2$, $D = 3$; ghi rõ rounding (truncate hoặc convergent) và saturation vào twin. Coefficient bit-width quá hẹp làm stopband nông (alias vào F_0); quá rộng lãng phí DSP. FIR $2/3$ là anti-alias hạ mẫu, *không* thay high-pass 60 Hz — DC-blocker Chương 3 lo offset chậm.

RTL thay thế. Khi không gọi IP: systolic MAC + coeff ROM, folding tay nếu $N \leq 32$. Không tự viết Distributed Arithmetic trừ khi tối ưu diện tích cực đoan — DA khó bit-exact với IP. Luôn giữ cùng h_q giữa ROM, IP và Python.

Twin bit-exact.

```
import numpy as np
# h_q: int16 đã lượng tử giống FIR IP / ROM
acc = np.convolve(x.astype(np.int64), h_q.astype(np.int64), mode="full")
# áp dụng đúng round/sat rồi lấy mẫu theo lịch 2/3 của IP
# so dump Avalon-ST beat-by-beat; thêm test ready=0 vài chu kỳ
```

Phải mô phỏng `ready`=0: IP dừng tiến mẫu vào nhưng không được làm twin "tự đếm clock".

Stopband và nguy cơ alias vào F_0. Thông số thiết kế tối thiểu: passband đến khoảng 5–6 kHz (giữ formant thấp phục vụ phân tích), transition xuống stopband trước 8 kHz (Nyquist 16 kHz). Độ sâu stopband khuyến nghị ≤ -60 dB trong lab để nhiễu alias sau hạ mẫu nằm dưới sàn tự tương quan điển hình. Hệ số Q1.15 với N quá ngắn không đạt -60 dB; đó là lúc tăng N hoặc dùng half-band dài hơn chứ không phải "tăng bit hệ số vô hạn".

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — FIR $2/3$ (mọi viết tắt).

- **DSP** (*Digital Signal Processing* block 18×19, có **pre-adder**): số DSP $\propto$ (taps sau folding)/(hệ số chia sẻ theo thời gian) — *đọc Fitter*, không đoán. Mục tiêu vài DSP để toàn hệ ≈ 3 DSP (bảng Chương 0), hoặc ghi rõ nếu lab nới.
- **M10K** (block RAM ≈ 10 kbit): hệ số + delay line, thường 1–2 khối. **MLAB**: có thể dùng cho delay ngắn; depth lớn nên ưu tiên M10K.
- **ALM/LUT**: vài trăm cho control + Avalon-ST (không kể soft-multiplier nếu hết DSP).
- **Latency**: độ trễ nhóm FIR + pipeline IP (ví dụ ~ 40 chu kỳ) — cộng vào hop nhưng $\ll 500\,000$.
- **Fmax**: thường > 150 MHz khi dùng DSP cứng; sụp nếu Fitter nhét nhân mềm vào ALM.
- **RTF**: với ≈ 2083 chu kỳ dư mỗi mẫu @ 24 kHz/50 MHz, sharing giữ RTF$\ll 1$ miễn không bỏ `valid`.
- Vì sao polyphase: bỏ MAC của pha bị discard, tiết kiệm tới $\sim 66\%$ nhân. Vì sao không CIC: $2/3$ không phải R nguyên lớn.

4.4b. Từ báo cáo Generate đến bảng chi phí điền số

Sau khi Generate FIR Compiler II, mở file báo cáo (hoặc tab Resource Utilization trong IP GUI) và chép tay vào sổ lab các mục: Number of DSP blocks, Number of memory bits / M10K, Latency (clock cycles), Input/output data widths, Coefficient storage (logic/M10K). Ví dụ minh họa (số giả định để luyện đọc — *thay bằng số Fitter thật của bạn*):

Mục báo cáo	Giá trị ví dụ lab $N \approx 48$, $I/D = 2/3$
DSP (18×19)	4
Memory bits	~ 2000 (coeff + delay)
Latency	12–40 chu kỳ (phụ thuộc pipeline)
Avalon-ST	sink/source valid/ready

Latency IP phải cộng vào ngân sách hop: 40 chu kỳ không đáng kể so với $500\,000$, nhưng FSM nối tiếp phải chờ `valid` thật, không trừ latency bằng đếm cứng trừ phi đã đo. Coefficient storage chọn M10K khi N lớn để khỏi ăn ALM; với N nhỏ logic đủ.

So sánh Fully Parallel vs Multicycle. Fully Parallel: mỗi nhân một DSP (sau folding) — Fmax cao, diện tích lớn, dễ vượt ngân sách 3 DSP của bảng Chương 0. Multicycle/sharing: ít DSP, cần nhiều chu kỳ giữa hai mẫu; với 2083 chu kỳ dư ở 24 kHz thì sharing là lựa chọn mặc định dạy. Distributed Arithmetic: ít/không DSP, dùng LUT — khó khớp bit với twin nhân trực tiếp; chỉ dùng khi hết DSP và chấp nhận viết lại twin theo DA.

Twin mở rộng — lịch $2/3$ giả lập.

```
def decimate_2_over_3(y_full):
    """Giữ 2 mẫu, bỏ 1 theo chu kỳ cố định (minh họa).
    Twin khớp IP phải theo đúng phase report của FIR II."""
    out = []
    for i, yi in enumerate(y_full):
        if i % 3 != 2:   # giữ i mod 3 in {0,1}
            out.append(yi)
    return out
assert len(decimate_2_over_3(range(6))) == 4
```

Đoạn trên chỉ minh họa tỷ lệ; phase thật (mod $3 = 0, 1$ hay $1, 2$) phải khớp dump IP. Sai phase làm lệch toàn bộ thời gian đến sliding RAM.

Điều kiện qua chương: script `verify_chuong_04.py` OK và đọc được số DSP từ báo cáo Generate (không bịa).

4.5. Bài tập và lời giải giải thích

Bài tập 4.1 (Folding). Với $h = [1, 2, 2, 1]$, $x = [3, 4, 5, 6]$, chứng minh folding cho cùng kết quả và đếm số nhân.

Lời giải. Thẳng: $3 + 8 + 10 + 6 = 27$ (4 nhân). Folding: $1 \cdot (3 + 6) + 2 \cdot (4 + 5) = 27$ (2 nhân). Pre-adder DSP làm đúng phép cộng trước nhân.

Bài tập 4.2 (Polyphase split). $h = [0, 1, 2, 3, 4, 5]$, $M = 3$. Viết E_0, E_1, E_2. Tỉ số ra/vào của lịch $2/3$ trên 6 mẫu vào là bao nhiêu?

Lời giải. $E_0 = [0, 3]$, $E_1 = [1, 4]$, $E_2 = [2, 5]$. Sáu vào $\rightarrow$ bốn ra: $4/6 = 2/3$.

Bài tập 4.3 (Valid không đều). Vì sao FSM sau FIR II fractional không được đếm clock cố định "cứ 3 ck lấy 2"?

Lời giải. Backpressure Avalon-ST (`ready`=0) làm giãn khoảng cách mẫu vào; lịch valid ra phụ thuộc trạng thái IP. Chỉ bắt mẫu khi `valid`=1.

Bài tập 4.4 (Chu kỳ dư và DSP share).

a) Tính số chu kỳ clock $50\,\text{MHz}$ giữa hai mẫu vào $24\,\text{kHz}$.

b) Với $N = 48$ đối xứng, sau folding còn bao nhiêu nhân logic? Vì sao một DSP vẫn có thể đủ nếu IP time-mux?

Lời giải. (a) $50 \cdot 10^6 / 24000 \approx 2083$ chu kỳ. (b) 24 nhân logic; $2083 \gg 24$ nên một DSP có thể lần lượt thực hiện nhiều nhân trong khoảng giữa hai mẫu — đúng cơ chế sharing của FIR Compiler II ở chế độ không Fully Parallel.

Bài tập 4.5 (Lượng tử hệ số). Cho $h = [0{,}1, 0{,}2, 0{,}4, 0{,}2, 0{,}1]$, tính $h_q = \text{round}(h \cdot 2^{15})$ và $\sum h_q$. Vì sao twin phải dùng đúng vector này?

Lời giải. $h_q = [3277, 6554, 13107, 6554, 3277]$, tổng 32769 (lệch 1 LSB khỏi 32768 do làm tròn). Sai một hệ số làm lệch stopband — alias có thể xuyên vào dải F_0.

Bài tập 4.6 (Half-band và chu kỳ ở $16\,$kHz).

a) Vì sao half-band phù hợp bước $48 \to 24$ hơn CIC?

b) Mỗi mẫu ra $16\,$kHz tương ứng bao nhiêu chu kỳ clock $50\,$MHz? So với $500\,000$ chu kỳ/hop thì FIR streaming chiếm tỷ lệ nào nếu latency IP $= 40$ chu kỳ mỗi mẫu ra và hop có 160 mẫu ra?

Lời giải. (a) Half-band tận dụng hệ số 0 xen kẽ và transition quanh $f_s/4$ đúng cho $\downarrow 2$; CIC cần $R \gg 1$ và méo sinc. (b) 3125 chu kỳ/mẫu ra. $160 \times 40 = 6400$ chu kỳ latency tích lũy theo pipeline mẫu (thực tế pipeline chồng nên wall-clock gần latency một mẫu + thời gian streaming); dù đếm tối đa $6400/500000 \approx 1{,}3\%$ — FIR không phải nút thắt so với MAC.

Ghi chú triển khai Platform Designer. Khi kéo FIR Compiler II vào Platform Designer, nối Avalon-ST từ DC-blocker (hoặc từ half-band) sang `avalon_streaming_sink`, và từ `avalon_streaming_source` sang khối sau. Đừng nối MM (memory-mapped) trừ khi cấu hình hệ số động — lab hệ số tĩnh dùng ROM trong IP. Reset của IP phải đồng bộ miền $50\,$MHz; không reset bằng tín hiệu async thô từ nút bấm.

Lỗi thường gặp khi nối FIR II. (1) `ready` phía sink luôn gắn 1 nhưng quên kiểm `valid` nguồn — mất mẫu khi IP tạm dừng. (2) Dùng `sop`/`eop` như biên khung pitch — sai ngữ nghĩa multi-channel. (3) Hệ số nạp Q2.14 trong khi twin Q1.15 — lệch 6 dB toàn cục. (4) Fully Parallel trên $N = 128$ làm hết DSP kit, Fitter nhét soft-multiplier vào ALM, Fmax sụp. Mỗi lỗi trên đều có triệu chứng riêng trên waveform: học viên nên ghi checklist trước khi hỏi "sao F_0 sai".

Ôn lại tỷ lệ hữu tỉ. $16/24 = 2/3$ là phân số tối giản; mọi lịch valid ra dài hạn phải có đúng 2 nhịp có dữ liệu trên mỗi 3 mẫu vào (khi ready luôn 1). Đo trên simulation: đếm N_{in} và N_{out} trong $0{,}1\,$s lý thuyết — N_{out}/N_{in} phải $\approx 2/3$ với sai số biên pipeline. Sai lệch lớn báo phase commutator hoặc backpressure không mô phỏng đúng trong twin.

Chương 5. Bộ nhớ trượt M10K (True Dual-Port, Old Data)

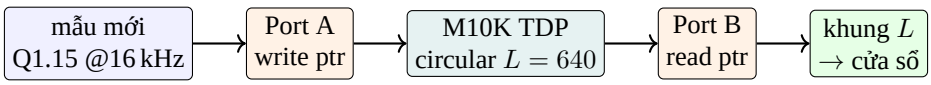
▶ MỤC TIÊU HỌC TẬP

Sau Chương 5, người học dựng bộ đệm vòng $L = 640$ mẫu trên M10K True Dual-Port, chọn đúng mixed-port read-during-write = Old Data, khớp latency đọc 1 hoặc 2 chu kỳ với twin và FSM trên sơ đồ timing chu kỳ-cụ-thể, và tính ngân sách bit/M10K. Thuật ngữ M10K, True Dual-Port, Old Data, ALM, Fmax đã có ở ô Feynman 10 và bài tập 0.3 của Chương 0 — chương này triển khai datapath.

5.1. Hiện tượng: vì sao phải nhớ L mẫu trên chip

▶ NHÌN THẤY VẬT LÝ

Giọng nói là sóng theo thời gian liên tục. Praat cắt khung L mẫu (ví dụ $L = 640$ tại 16 kHz với $F_{0,\min} = 75$ Hz, khoảng 40 ms) rồi trượt theo bước nhảy $H = 160$ (10 ms). Trên FPGA, mỗi mẫu từ FIR chỉ đến một lần rồi biến mất khỏi thanh ghi vào — không thể "tua lại" micro. **Bộ nhớ trượt** (circular buffer) giữ đúng L mẫu gần nhất: cổng A ghi mẫu mới mỗi beat audio hợp lệ, cổng B đọc burst L mẫu khi FSM cần một khung để nhân cửa sổ và MAC. Đời thường: hộp đen máy bay chỉ giữ L giờ ghi gần nhất; mỗi phút mới đè lên phút cũ nhất — đúng phép modulo trên địa chỉ.



Hình 27: Datapath sliding window: ghi một mẫu mỗi beat audio, đọc burst L mẫu khi FSM yêu cầu khung.

Công thức chọn L theo tinh thần Praat: $L \approx 3 \cdot f_s / F_{0,\min}$. Với $f_s = 16$ kHz, $F_{0,\min} = 75$ Hz:

$$L \approx 3 \cdot \frac{16000}{75} = 640 \text{ mẫu.}$$

Nếu lab nâng $F_{0,\min}$ lên 100 Hz thì $L \approx 480$ — tiết kiệm RAM nhưng giảm độ bền đỉnh với giọng trầm. Không chọn L tùy hứng: quá ngắn làm đỉnh $\mathcal{R}[\tau]$ nông; quá dài tốn M10K và tăng MAC.

Vì sao on-chip, không SDRAM. Đọc 640 mẫu liên tiếp mỗi hop 10 ms đòi băng thông và latency Deterministic. SDRAM/HPS có refresh, arbitration, cache — RTF và jitter địa chỉ phá deadline 500 000 chu kỳ. Một khối M10K cho đúng 10240 bit là vừa khít và hoàn toàn Deterministic.

5.1b. So sánh với FIFO tuyến tính và với bộ đệm phần mềm

FIFO tuyến tính (chỉ đẩy mẫu mới, đọc tuần tự từ đầu) không đủ cho khung trượt chồng lấp: sau khi đọc hết L mẫu một lần, dữ liệu cũ mất; trong khi hop chỉ tiến $H < L$ nên cần giữ lại $L - H$ mẫu. Circular buffer giải quyết bằng cách *không* xóa khi đọc — đọc không phá nội dung, chỉ ghi mới đè mẫu cũ nhất. Trong phần mềm Praat, mảng vòng hoặc copy L mẫu mỗi hop làm cùng việc; trên FPGA, copy 640 mẫu mỗi 10 ms vào BRAM khác sẽ nhân đôi M10K và thêm 640 chu kỳ chép — lãng phí so với đọc tại chỗ từ circular.

An toàn đa khung. Nếu FSM chậm và yêu cầu khung mới khi khung trước chưa đọc xong, cần hoặc chặn `frame_req` (backpressure) hoặc double-buffer hai vùng L (tốn 2 M10K). Lab mặc định backpressure: một burst tại một thời điểm, đơn giản và đủ vì $642 \ll 500000$. Double-buffer chỉ cần khi pipeline MAC quá sâu đến mức đọc khung chồng nhau — không phải trường hợp thiết kế DE10 ở Chương 0.

Liên hệ RTF. **RTF** (*real-time factor*) = thời gian xử lý / thời gian audio. Thời gian đọc khung $642/(50 \times 10^6) \approx 12{,}84\ \mu$s trên 10 ms audio cho RTF riêng phần nhớ $\approx 0{,}00128$. Toàn hệ thống RTF bị chi phối bởi MAC (Chương 7), không bởi M10K — đúng chỗ đặt ngân sách silicon.

5.2. Toán con trỏ vòng và ví dụ tính tay

Con trỏ ghi $n_w \in \{0, \ldots, L-1\}$ chạy modulo L: mỗi beat `valid` ghi $\text{mem}[n_w] \leftarrow x_{\text{in}}$ rồi $n_w \leftarrow (n_w + 1) \bmod L$. Khi FSM cần khung (sau khi đã có đủ mẫu mới theo hop H), đọc oldest-first:

$$x_{\text{frame}}[k] = \text{mem}\big[(n_w + k) \bmod L\big], \qquad k = 0, \ldots, L-1,$$

với quy ước n_w đang trỏ ô trống tiếp theo (mẫu cũ nhất nằm đúng tại n_w).

Ví dụ tính tay — circular 8 ô. $L = 8$, ghi lần lượt $1, 2, 3, 4, 5, 6, 7, 8, 9$ (mẫu 9 đè ô 0):

addr	0	1	2	3	4	5	6	7
sau 8 ghi	1	2	3	4	5	6	7	8
sau ghi 9	**9**	2	3	4	5	6	7	8

Khi $n_w = 1$ (vừa ghi 9 vào addr 0 rồi tăng ptr), khung oldest→newest lấy tuần tự addr 1,2,3,4,5,6,7,0, tức dãy mẫu 2,3,4,5,6,7,8,9.

Ví dụ tính tay — quan hệ hop và số mẫu mới. Mỗi hop $H = 160$ mẫu ở 16 kHz tương ứng 10 ms. Bộ đệm $L = 640$ giữ 4 hop lịch sử. Sau mỗi hop, chỉ 160 ô bị ghi đè; 480 ô còn lại vẫn là quá khứ hữu ích cho cửa sổ chồng lấp (overlap) — đúng tinh thần phân tích khung trượt Praat.

5.2b. Overlap, hop và số lần đọc mỗi giây

Với $H = 160$, $f_s = 16$ kHz, số khung mỗi giây là $16000/160 = 100$ khung/s (đúng 10 ms/khung). Mỗi khung đọc $L = 640$ mẫu từ M10K $\Rightarrow 64000$ phép đọc/giây — với clock 50 MHz là tầm thường. Overlap $L - H = 480$ mẫu (75% overlap) làm liền mạch đường F_0 theo thời gian: hai khung liên tiếp chia sẻ 480 mẫu, nên đỉnh cao độ ít nhảy hơn so với hop không chồng. Trên silicon, overlap *không* tốn thêm RAM: circular buffer vốn đã giữ L; chỉ cần phát `frame_req` thường hơn.

Ví dụ tính tay — địa chỉ sau nhiều hop. $L = 8$, $H = 2$ (minh họa thu nhỏ). Bắt đầu $n_w = 0$ sau khi đã đầy 1..8. Sau một hop ghi thêm 2 mẫu (giá trị 9,10): $n_w = 2$, mem$= [9, 10, 3, 4, 5, 6, 7, 8]$, khung $= [3, 4, 5, 6, 7, 8, 9, 10]$. Sau hop nữa (11,12): $n_w = 4$, khung $= [5, 6, 7, 8, 9, 10, 11, 12]$. Quan sát: mỗi hop khung trượt đúng H mẫu trên dòng thời gian — đúng định nghĩa hop Praat.

Đọc đồng thời với ghi — định mức chu kỳ. Trong 10 ms có 160 mẫu cần ghi và một burst 640 đọc. Tổng giao dịch bộ nhớ ≈ 800 mỗi hop, vẫn $\ll 500\,000$. True Dual-Port cho phép ghi và đọc cùng chu kỳ trên hai địa chỉ khác nhau mà không mất mẫu; chỉ khi trùng địa chỉ mới cần quy tắc Old Data đã phân tích.

5.3. Old Data, latency 1/2 và timing chu kỳ-cụ-thể

Trên Cyclone V, khối **M10K** True Dual-Port cho phép Port A ghi và Port B đọc *cùng lúc*. Khi hai cổng chạm *cùng một địa chỉ* trong cùng chu kỳ (mixed-port read-during-write), cần một quy tắc: silicon trả về giá trị nào? **Old Data** nghĩa là cổng đọc nhận đúng nội dung ô *trước* cạnh ghi — như nhìn vào ngăn kéo trước khi ai đó thay đồ trong đó. **New Data** nghĩa là đọc ngay giá trị vừa ghi; trên mixed-port Cyclone V chế độ này *không được bảo đảm* (chỉ Don't Care hoặc Old Data). Twin Python phải trả về giá trị cũ trong cùng "delta-cycle"; nếu không, testbench xanh trên mô hình tưởng tượng nhưng đỏ trên chip thật. Chương 0 đã cảnh báo điều này ở bài tập 0.3 — đây là chỗ hiện thực.

Cấu hình IP. On-Chip Memory (RAM: 2-PORT), True Dual-Port, width 16 (Q1.15), depth $\geq L$ (có thể làm tròn 1024). Mixed-port RDW = **Old Data**. Không dùng MLAB cho $L = 640$ (depth lớn, timing kém). Single-port + time-mux ghi/đọc sẽ buộc FSM không ghi trong lúc đọc khung — phức tạp hóa và dễ mất mẫu audio.

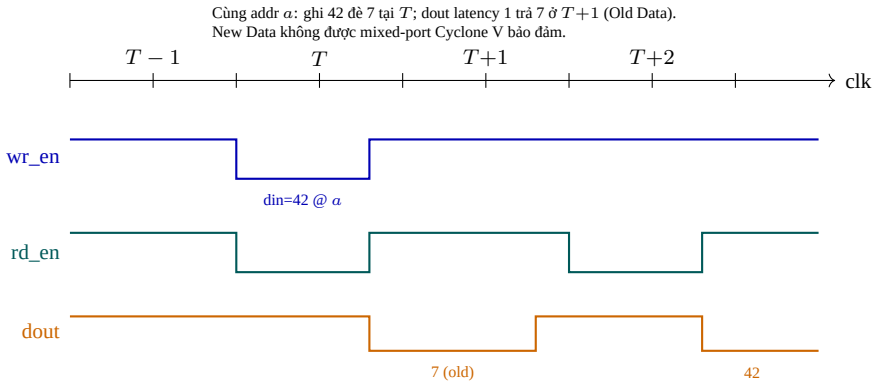
Latency đọc — điểm dễ lệch khung.

- RTL inferred `dout <= mem[rd_ptr]`: latency **1 chu kỳ** sau `rd_en`.
- MegaWizard bật Output Registers: thường **2 chu kỳ** (block RAM register + output register).

FSM Chương 11 và twin **phải** cùng model. Lệch một chu kỳ làm lệch cả 640 mẫu cửa sổ và toàn bộ MAC.

Sơ đồ timing — Old Data cùng địa chỉ. Giả sử tại chu kỳ T Port A ghi giá trị mới 42 vào địa chỉ a đang giữ 7, đồng thời Port B đọc a với `rd_en`=1, cấu hình Old Data, latency 1:

Chu kỳ	Port A (write)	Port B (read addr)	`dout` (lat 1)
$T-1$	—	—	(giá trị cũ hơn)
T	ghi 42 vào a (cũ = 7)	đọc a	— (chưa ra)
$T+1$	—	—	7 (Old Data)
$T+2$	—	đọc a lại	42 (đã là nội dung mới)



Hình 28: Timing chu kỳ-cụ-thể mixed-port read-during-write = Old Data, latency đọc 1:dout ở $T+1$ là giá trị trước khi ghi.

Nếu IP bật output register (latency 2), mọi cạnh `dout` trong hình dịch thêm một chu kỳ — FSM phải chờ đúng 2 nhịp sau `rd_en` đầu trước khi lấy mẫu đầu khung.

RTL inferred (latency 1) giải thích từng dòng.

```
localparam int L = 640;
reg signed [15:0] mem [0:L-1];  // M10K via ramstyle
reg [$clog2(L)-1:0] wr_ptr, rd_ptr;
reg signed [15:0] dout;
always @(posedge clk) begin
  if (valid_i) begin
    mem[wr_ptr] <= din;              // Port A: ghi mẫu audio mới
    wr_ptr <= (wr_ptr == L-1) ? '0 : wr_ptr + 1'b1;
  end
  if (rd_en) begin
    dout <= mem[rd_ptr];         // Port B Old Data
    rd_ptr <= (rd_ptr == L-1) ? '0 : rd_ptr + 1'b1;
  end
end
```

Dòng `dout <= mem[rd_ptr]` tạo thanh ghi ngõ ra — latency 1. Hai tiến trình trong cùng `always` là True Dual-Port khi công cụ suy diễn được; để chắc, lab dùng IP 2-PORT và nối tay hai con trỏ. Không ghi `dout = mem[...]` (blocking tổ hợp) nếu muốn khớp M10K đồng bộ.

Twin địa chỉ khớp Old Data.

```
L = 640
mem = [0] * L
wr = 0
def write(x):
    global wr
    old = mem[wr]       # giá trị trước khi đè --- Old Data
    mem[wr] = int(x)
    wr = (wr + 1) % L
    return old
def read_frame():
    return [mem[(wr + k) % L] for k in range(L)]  # oldest-first
```

Hàm `write` trả về `old` khớp dout silicon khi đọc trùng địa chỉ đang ghi. Script `verify_chuong_05.py` kiểm khung $[2, \ldots, 9]$ ($L = 8$) và ngân sách $640 \times 16 = 10\,240$ bit.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách — sliding M10K (mọi viết tắt).

- **M10K** (block RAM ≈ 10 kbit): $L \times 16 = 640 \times 16 = 10\,240$ bit — vừa khít **1 khối**. Depth 1024 từ vẫn khoảng 1–2 M10K. Không dùng **MLAB** cho $L = 640$ (depth lớn, timing kém).
- **ALM/LUT**: khoảng 50–100 cho hai con trỏ + FSM đọc. 0 **DSP**.
- **Latency** đọc: 1 hoặc 2 chu kỳ (cấu hình IP) $\Rightarrow$ đọc hết khung 641 hoặc 642 chu kỳ.
- **Fmax**: dễ > 150 MHz khi ép `ramstyle=M10K`; RAM logic làm Fmax và hành vi Old Data lệch twin.
- **RTF**: $642/500\,000 \approx 0{,}00128$ trên hop 10 ms @ 50 MHz — nhớ không phải nút thắt; MAC Chương 7 mới là nút thắt.
- Vì sao True Dual-Port + Old Data: ghi/đọc song song mỗi chu kỳ; New Data không bảo đảm trên mixed-port Cyclone V.

5.3b. Khởi động ấm máy và đồng bộ hop

Sau reset, bộ đệm chứa 0 (hoặc don't care). Nếu FSM phát khung ngay, MAC sẽ nhân cửa sổ với chuỗi toàn 0 hoặc rác — đỉnh F_0 vô nghĩa. Quy trình ấm máy: chờ ít nhất $L = 640$ mẫu `valid` từ FIR (khoảng 40 ms ở 16 kHz) trước khung đầu tiên; sau đó mỗi $H = 160$ mẫu phát một khung mới. Cờ `warm` trong FSM: đếm mẫu ghi cho tới L, rồi cho phép `frame_req`. Twin Python phải có cùng quy tắc nếu so sánh bit-exact từng khung.

Xung đột ghi/đọc khi burst dài. Trong lúc Port B đọc 640 chu kỳ, Port A vẫn có thể nhận mẫu audio mới mỗi ≈ 3125 chu kỳ hệ thống (một mẫu 16 kHz). Xác suất trùng địa chỉ đúng lúc đang đọc địa chỉ đó là có thật: đó chính là lúc Old Data quyết định đúng/sai. New Data "hy vọng đọc giá trị vừa ghi" sẽ làm một mẫu trong khung mang giá trị tương lai — lệch cửa sổ một cách không lặp lại theo seed, cực khó debug. Old Data bảo đảm khung là ảnh nhất quán của quá khứ tại thời điểm bắt đầu burst (plus các ô chưa bị đè trong lúc đọc — đúng mô hình circular đang trôi).

Hai con trỏ đọc: oldest và tuần tự. Có thể khởi rd_ptr $\leftarrow n_w$ khi bắt burst (oldest), rồi tăng mỗi chu kỳ; hoặc giữ rd_base cố định trong burst và rd_ptr = rd_base + k. Cách hai tránh lệch nếu n_w bị ghi tăng trong lúc đọc — khuyến nghị lab: **chốt** rd_base $= n_w$ **lúc nhận** `frame_req`, rồi đọc rd_base + k mod L, trong khi n_w vẫn tăng trên Port A. Twin:

```
def read_frame_stable(wr_at_req):
    return [mem[(wr_at_req + k) % L] for k in range(L)]
```

Thuộc tính `ramstyle` và suy diễn.

```
(* ramstyle = "M10K" *) reg signed [15:0] mem [0:L-1];
```

Ép map M10K, tránh công cụ phân tán MLAB khi depth lớn. Kiểm Fitter: 1 M10K, không cảnh báo inferred ROM/RAM conflict. Nếu thấy "RAM logic", latency và Old Data có thể không còn đúng như block RAM — phải sửa attribute hoặc dùng IP.

Điều kiện qua chương: script `verify_chuong_05.py` OK, Fitter báo 1 M10K, và waveform RDW khớp Old Data.

5.4. Bài tập và lời giải giải thích

Bài tập 5.1 (Ngân sách bit). $L = 640$, width 16: bao nhiêu bit? Cần tối thiểu bao nhiêu M10K? Nếu $L = 480$ ($F_{0,\min} = 100$ Hz) thì sao?

Lời giải. $640 \times 16 = 10\,240$ bit $= 1$ M10K vừa khít. $480 \times 16 = 7680$ bit vẫn 1 M10K (không chia nhỏ khối) nhưng dư chỗ — có thể dùng dual-purpose lưu thêm hệ số cửa sổ nếu cần.

Bài tập 5.2 (Old Data timing). Cùng địa chỉ: Port A ghi 42 vào ô đang giữ 7, Port B đọc cùng ô trong cùng chu kỳ, latency 1, Old Data. Dout ở chu kỳ sau là gì? Ở hai chu kỳ sau nếu đọc lại?

Lời giải. Chu kỳ $T{+}1$: dout $= 7$. Chu kỳ $T{+}2$ nếu đọc lại addr đó: dout $= 42$. New Data không được mixed-port Cyclone V bảo đảm.

Bài tập 5.3 (Đọc khung). Sau khi ghi mẫu 9 vào $L = 8$ như bảng trên ($n_w = 1$), liệt kê khung oldest-first.

Lời giải. $[2, 3, 4, 5, 6, 7, 8, 9]$ — khớp script `verify/verify_chuong_05.py`.

Bài tập 5.4 (Latency 1 vs 2). Vì sao lab phải ghi rõ "inferred 1 ck" hay "IP output register 2 ck" trước khi viết FSM? Hậu quả lệch một chu kỳ lên MAC là gì?

Lời giải. Mỗi mẫu cửa sổ lệch một chu kỳ làm lệch toàn bộ tích $\mathcal{R}[\tau]$; sai số là dịch cả khung — đỉnh F_0 lệch hệ thống, khódebug vì "có vẻ có đỉnh" nhưng sai cao độ.

Bài tập 5.5 (Ngân sách chu kỳ hop).

a) Trong 10 ms ở 50 MHz có bao nhiêu chu kỳ?

b) Đọc $L = 640$ với latency 2 mất bao nhiêu chu kỳ? Tỷ lệ so với ngân sách hop?

Lời giải. (a) $500\,000$ chu kỳ. (b) 642 chu kỳ $\approx 0{,}128\%$ ngân sách hop — đọc khung gần như "miễn phí" so với MAC 189×640.

Bài tập 5.6 (Warm-up và chốt rd_base**).**

a) Sau reset, cần tối thiểu bao nhiêu mẫu `valid` trước khung đầu? Tương đương bao nhiêu ms ở 16 kHz?

b) Vì sao phải chốt rd_base $= n_w$ lúc `frame_req` thay vì luôn đọc theo n_w đang chạy trong lúc burst?

Lời giải. (a) 640 mẫu $= 40$ ms. (b) Trong burst 640 chu kỳ, n_w có thể tăng khi có mẫu mới; nếu rd_ptr bám n_w động, tập địa chỉ bị trượt trong lúc đọc, khung không còn là một cửa sổ nhất quán tại thời điểm yêu cầu. Chốt rd_base đóng băng ảnh circular tại thời điểm req.

Bài tập 5.7 (`ramstyle` và Fitter). Fitter báo RAM suy diễn thành logic thay vì M10K. Hậu quả lên Old Data/latency? Cách ép?

Lời giải. RAM logic không còn hành vi block mixed-port Old Data chuẩn; timing và latency đổi, twin M10K không còn vàng. Ép `(* ramstyle = "M10K" *)` hoặc dùng IP On-Chip Memory 2-PORT, rồi đọc lại báo cáo Fitter xác nhận 1 M10K.

5.5. Liên kết Chương 0 và chương sau

Bài tập 0.3 của Chương 0 đã cảnh báo New Data trên Cyclone V: Chương 5 là chỗ hiện thực câu trả lời đó bằng IP + timing. Chương 6 (cửa sổ) sẽ nhân từng mẫu đọc từ Port B với hệ số Hanning/Hamming từ ROM; lệch latency đọc sẽ lệch luôn tích cửa sổ. Chương 7 (MAC) giả định khung L đã ổn định trên dây Avalon-ST — mọi sai Old Data ở đây biến thành sai $\mathcal{R}[\tau]$ không sửa được bằng "thêm DSP".

Checklist nghiệm thu Chương 5. (1) Fitter: đúng 1 M10K cho buffer 640×16. (2) Waveform: trùng địa chỉ ghi/đọc cho dout = Old Data. (3) Latency đọc khớp ghi chú lab (1 hoặc 2). (4) Warm-up ≥ 640 mẫu. (5) Khung oldest-first khớp twin trên vector $L = 8$ và trên dump $L = 640$ rút gọn. (6) Trong một hop, số chu kỳ đọc + ghi $\ll 500\,000$. Đủ sáu mục mới được coi là xong khối nhớ trượt.

Định mức overlap 75%. Với $L = 640$, $H = 160$, tỷ lệ chồng $480/640 = 0{,}75$. Đây là lựa chọn cân bằng giữa mượt đường F_0 và chi phí MAC mỗi giây (100 khung/s). Giảm overlap (tăng H) làm thưa điểm cao độ; tăng overlap (giảm H) nhân tải MAC gần tuyến tính.

Chương 6. Cửa sổ Hanning ROM × mẫu (Q1.15)

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn giải thích được vì sao khung âm thanh phải nhân cửa sổ Hanning trước khi tự tương quan; viết đúng công thức Praat ($N+1$) và đối chiếu dạng cổ điển ($N-1$); lượng tử hóa hệ số sang định dạng Q1.15; sinh tệp `hanning_rom.hex` khớp bit với twin Python; hiện thực phép nhân ROM×mẫu bằng một khối DSP trên Cyclone V; đọc latency đăng ký của On-Chip Memory; và đọc được ngân sách M10K / DSP / ALM của khối cửa sổ.

6.1. Hiện tượng thật: cắt khung không được "cắt đứt" sóng

Giọng nói là một dòng sóng liên tục trong không khí và trong ống thanh âm. Để đo chu kỳ rung dây thanh T_0, thuật toán Boersma/Praat không nhìn cả câu nói một lần; nó chỉ lấy một đoạn ngắn khoảng 40 ms — tương đương $L = 640$ mẫu ở tần số lấy mẫu $f_s = 16$ kHz — đúng như bản đồ khối ở Chương 0 đã ghi. Đoạn đó gọi là **khung phân tích (analysis frame)**. Mỗi 10 ms (hop $H = 160$ mẫu) khung trượt đi một bước, tạo ra một ước lượng F_0 mới.

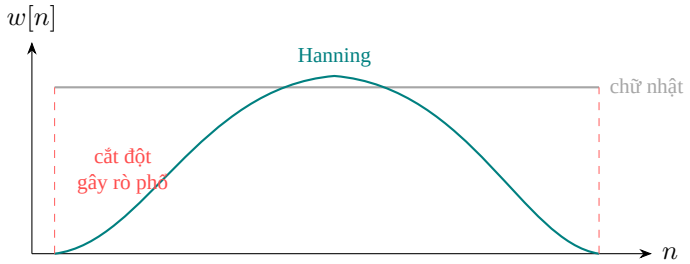
Nếu lấy đoạn đó đúng kiểu cắt bằng kéo — giữ nguyên biên độ đến mẫu cuối rồi nhảy đột ngột về 0 ngoài khung — thì đoạn đó, nhìn từ miền tần số, giống một xung chữ nhật nhân với tín hiệu thật. Phép nhân trong miền thời gian tương đương **tích chập (convolution)** phổ tín hiệu với phổ của xung chữ nhật. Phổ xung chữ nhật là hàm sinc: có một **thùy chính (mainlobe)** rộng và nhiều **thùy phụ (sidelobe)** cao khoảng -13 dB so với đỉnh. Chương 0 đã đổi -13 dB sang tỷ số biên độ $\approx 0{,}224$: nghĩa là mỗi thùy phụ còn khoảng một phần tư biên độ thùy chính — đủ lớn để "rò" năng lượng của hài $2F_0$, $3F_0$ sang đúng vùng tìm cao độ $75 \ldots 600$ Hz. Năng lượng giả đó gọi là **rò phổ (spectral leakage)**.

Hệ quả với cao độ không phải trừu tượng: đỉnh tự tương quan $\mathcal{R}_x[\tau]$ bị méo vì các tần số láng giềng tràn sang. Trực giác đời thường: bạn đang nghe một nốt đàn sạch, nhưng mỗi lần cắt đoạn lại "vỗ" hai đầu đoạn — tiếng vỗ đó tạo thêm tiếng lách cách phổ, không có trong bản gốc. Trên silicon, nếu quên cửa sổ, MAC ở Chương 7 vẫn tính đúng công thức, nhưng đỉnh τ có thể lệch vài mẫu chỉ vì hình dạng cắt khung.

Cửa sổ **Hanning** (còn gọi Hann) làm hai đầu khung về gần 0 một cách mượt, giống vuốt dần biên độ thay vì cắt đứt. Phổ Hanning có thùy phụ khoảng -32 dB (biên độ còn khoảng $1/40$ so với thùy chính — Chương 0). Thùy chính rộng hơn một chút (độ phân giải tần số kém hơn một ít so với chữ nhật cùng độ dài), nhưng với bài đo F_0 bằng tự tương quan thời gian thì đánh đổi này rất đáng: ta cần đỉnh $\mathcal{R}_x$ sạch hơn là phổ sắc như dao. Hamming gần Hanning nhưng nâng hai đầu lên $0{,}08$; gói này theo Praat nên chọn đúng Hanning $N+1$.

▶ NHÌN THẤY VẬT LÝ

Nhân cửa sổ $x[n] \cdot w[n]$ trên silicon chỉ là một phép nhân mỗi mẫu. Toàn bộ "phép màu" giảm rò phổ đã được nướng sẵn vào bảng hệ số $w[n]$. Không cần bộ sinh hàm lượng giác lúc chạy (CORDIC — thuật toán chỉ dùng dịch/cộng để xoay góc), cũng không cần tính cos mỗi mẫu — đó là lý do ROM thắng mọi cách tạo cửa sổ động trên đường pitch streaming.



Hình 29: Cửa sổ chữ nhật giữ biên độ đến sát biên rồi cắt đứt; Hanning vuốt hai đầu về 0, hạ thùy phụ từ khoảng -13 dB xuống khoảng -32 dB.

6.2. Toán từng bước — mọi ký hiệu trước khi dùng

Gọi L (hoặc N) là số mẫu trong một khung phân tích. Với sàn cao độ 75 Hz ở $f_s = 16$ kHz, một chu kỳ tối đa dài $f_s/75 \approx 213{,}3$ mẫu. Praat chọn khung dài khoảng ba chu kỳ của sàn đó: $L = 640$. Sau khi đã trừ mean cục

bộ (Chương 3), tín hiệu cửa sổ là

$$\tilde{x}[n] = (x[n] - \mu)\, w[n], \qquad n = 0, 1, \ldots, L-1.$$

Ký hiệu μ là trung bình số học của L mẫu trong khung; trừ μ trước khi nhân w để đỉnh $\mathcal{R}_x[0]$ không bị DC đẩy lên giả tạo.

Hai định nghĩa Hanning hay gặp — phải chọn một và ghi vào twin, vì khác nhau từng LSB:

1. **Praat ($N{+}1$), dùng trong golden `lab_1_3_5.py`:**

$$w[i] = 0{,}5 - 0{,}5\cos\Big(\frac{2\pi i}{N+1}\Big), \qquad i = 1, 2, \ldots, N.$$

   Hai đầu lớn hơn 0 một chút (không đúng bằng 0). Với $N = 640$, mẫu số là 641. Chỉ số i chạy $1 \ldots N$ rồi ánh xạ sang địa chỉ ROM $0 \ldots N-1$.

2. **Cổ điển / scipy ($N{-}1$):**

$$w[n] = \frac{1}{2}\Big(1 - \cos\frac{2\pi n}{N-1}\Big), \qquad n = 0, 1, \ldots, N-1.$$

   Hai đầu đúng bằng 0; đối xứng $w[n] = w[N-1-n]$ sạch — tiện lưu nửa bảng ROM (chỉ $N/2$ từ).

Gói FPGA này lấy dạng **Praat** $N{+}1$ làm hợp đồng bit-exact với `hanning_rom.hex` (đầy đủ 640 từ). Dạng cổ điển chỉ dùng trong ví dụ tính tay nhỏ và khi muốn minh họa tối ưu nửa bảng.

Lượng tử **Q1.15** (đã định nghĩa ở Chương 0: một bit dấu/phần nguyên, 15 bit phần thập phân; giá trị $1{,}0$ ánh xạ thành $2^{15} = 32768$, nhưng số có dấu 16-bit bão hòa tại $+32767$):

$$w_q[n] = \text{sat}_{16}\big(\text{round}(w[n] \cdot 32768)\big) \in [0, 32767]$$

(với hệ số Hanning không âm). Mẫu vào x_q cũng Q1.15. Tích rồi kéo về Q1.15 bằng làm tròn gần nhất:

$$\begin{aligned} y_{\text{acc}} &= x_q \cdot w_q && \text{(Q2.30, 32-bit có dấu),} \\ y_q &= \text{sat}_{16}\big((y_{\text{acc}} + 2^{14}) \ggg 15\big). \end{aligned}$$

Cộng $2^{14} = 16384$ trước khi dịch phải 15 bit chính là round-to-nearest trên số học hai's complement khi dùng arithmetic shift (`>>>` trong Verilog). Nếu quên cộng 16384 thì mọi tích bị thiên lệch xuống khoảng nửa LSB của Q1.15 — đủ để lệch twin vài chục LSB trên cả khung.

Năng lượng cửa sổ (dùng ở Chương 8 để chuẩn hóa):

$$\mathcal{R}_w[0] = \sum_{n=0}^{N-1} w_q[n]^2$$

tính offline trên int64; twin in giá trị này vào header hex để đối chiếu. Trên $N = 640$ Praat, script xác nhận $\mathcal{R}_w[0] = 258\,100\,276\,868$.

★ NGUYÊN LÝ ĐẦU TIÊN: Vì sao nhân rồi dịch 15, không giữ 32-bit suốt đường

Giữ y dạng Q2.30 suốt MAC sẽ phình bộ nhớ khung và DSP. Hợp đồng gói này: sau cửa sổ, mẫu vẫn Q1.15 — cùng định dạng với đầu vào — để MAC Chương 7 tái sử dụng cùng kiểu nhân 18×19. Mọi bit-growth được quản lý ở accum 41-bit của MAC, không ở cửa sổ.

6.3. Ví dụ tính tay — mọi số đã chạy Python

Mọi con số dưới đây được kiểm chứng bởi `verify/verify_chuong_06_09.py`.

A. Dạng cổ điển $N = 8$ (đối xứng nửa bảng). Công thức $w[n] = \frac{1}{2}\big(1 - \cos(2\pi n/7)\big)$, $n = 0..7$. Từng bước cho $n = 2$:

$$\cos\Big(\frac{4\pi}{7}\Big) \approx -0{,}222521, \quad w[2] = \tfrac{1}{2}\big(1 - (-0{,}222521)\big) = 0{,}611260.$$

Lượng tử minh họa bằng round$(w \cdot 32767)$ (hệ số dương tối đa vừa khít int16 dương):

n	$w[n]$ float	w_q
0	0.000000	0
1	0.188255	6169
2	0.611260	20029
3	0.950484	31145

Đối xứng: $w_q[7] = w_q[0] = 0$, $w_q[6] = w_q[1] = 6169$, $w_q[5] = w_q[2] = 20029$, $w_q[4] = w_q[3] = 31145$ — ROM vật lý chỉ cần 4 từ. Nhân tay: $x_q = 10000$, $w_q = 20029$:

$$\begin{aligned} \text{prod} &= 10000 \times 20029 = 200\,290\,000, \\ \text{rounded} &= 200\,290\,000 + 16\,384 = 200\,306\,384, \\ y_q &= 200\,306\,384 \ggg 15 = 6112. \end{aligned}$$

Đối chiếu float $10000 \times 0{,}611260 = 6112{,}6$ — khớp trong 1 LSB. Script assert `q15_mul(10000, 20029) == 6112`.

B. Dạng Praat $N = 8$ (khớp tinh thần golden). $w[i] = 0{,}5 - 0{,}5\cos(2\pi i/9)$, $i = 1..8$, lượng tử round$(w \cdot 32768)$ rồi bão hòa:

i	w float	w_q
1	0.116978	3833
2	0.413176	13539
3	0.750000	24576
4	0.969846	31780

Hai đầu không về 0 tuyệt đối — đúng hành vi Praat. Đối xứng quanh giữa: $w[i] = w[N + 1 - i]$ với cách đánh số Praat.

C. Khung thật $N = 640$ Praat. Các mốc Q1.15 (script assert): $w_q[0] = 1$, $w_q[1] = 3$, $w_q[319] = w_q[320] = 32767$, $w_q[638] = 3$, $w_q[639] = 1$. Tổng bình phương $\mathcal{R}_w[0] = \sum w_q^2 = 258\,100\,276\,868$ (int64). Giữa khung bão hòa tại $+32767$ (không phải 32768) vì giới hạn signed 16-bit — đó là chỗ twin và RTL phải cùng `clip`.

D. Dấu âm — cùng quy tắc làm tròn. $x_q = -10000$, $w_q = 20029$:

$$\begin{aligned} \text{prod} &= -200\,290\,000, \\ \text{rounded} &= -200\,290\,000 + 16\,384 = -200\,273\,616, \\ y_q &= -200\,273\,616 \ggg 15 = -6112. \end{aligned}$$

Script xác nhận `q15_mul(-10000, 20029) == -6112`. Arithmetic shift phải giữ dấu; logical shift sẽ phá số âm.

6.4. Ý nghĩa phần cứng — ROM On-Chip Memory + DSP

Cửa sổ Hanning *không* cần FIR Compiler II: hệ số bất biến với L cố định. Chọn IP **On-Chip Memory** (Quartus: `altsyncram` / ROM 1-PORT) khởi tạo bằng `.hex` hoặc `.mif`:

- **Width** $= 16$ (Q1.15 unsigned-looking nhưng đọc ra signed khi nhân với x_q).
- **Depth** $= 640$ (đầy đủ, khớp Praat) hoặc 320 nếu dùng dạng cổ điển nửa bảng.

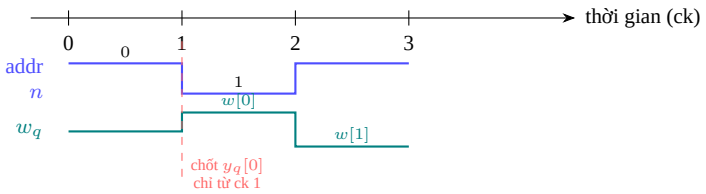
- **Registered output**: bắt buộc bật — trên Cyclone V, đọc M10K có latency đăng ký 1 chu kỳ đồng hồ. FSM phải lấy w_q *một cạnh sau* khi đưa địa chỉ n.
- **Init file**: đường dẫn tới `hanning_rom.hex`; mỗi dòng một từ hex 4 chữ số (`04X`).
- Một khối **DSP** (*Digital Signal Processing* block) trên Cyclone V nhân 18×19 mỗi mẫu: $x_q \times w_q$. Có thể suy diễn thành `*` trong Verilog; Quartus map vào DSP cứng nếu bật "Auto DSP".

Không gọi FIR Compiler chỉ để nhân cửa sổ: lãng phí logic cấu hình systolic và latency hàng chục chu kỳ không cần thiết. Đổi L hoặc công thức w thì **bắt buộc rebuild hex** và nạp lại ROM; quên bước này thì F_0 sai im lặng — testbench vẫn "pass" nếu chỉ kiểm handshake.

Bảng timing chu kỳ — ROM registered + nhân tổ hợp (minh họa). Giả sử nhân suy diễn tổ hợp (0 pipeline DSP), registered ROM:

Chu kỳ k	Địa chỉ ROM	w_q hợp lệ	y_q / handshake
0	đặt n=0, `in_valid`	— (chưa)	chờ
1	n=0 vẫn giữ	$w_q[0]$ ra	$y_q[0]$ chốt nếu `in_ready`
2	đặt n=1	$w_q[0]$ (cũ) rồi $w_q[1]$	$y_q[1]$
⋮	⋮	⋮	⋮
640	n=639 đã đặt từ k=639	$w_q[639]$	mẫu cuối khung

Nếu DSP nhân cũng đăng ký 1 chu kỳ (IP LPM_MULT pipeline $= 1$), thêm một hàng trễ: tổng latency địa chỉ$\rightarrow y_q$ thành 2 chu kỳ. Lab phải ghi rõ "inferred 1 ck" hay "IP output register 2 ck" trước khi viết FSM — lệch một mẫu làm lệch toàn bộ tích MAC $\mathcal{R}[\tau]$ (bài tập Chương 5 đã cảnh báo).



Hình 30: ROM On-Chip Memory bật registered output: địa chỉ đặt ở cạnh 0, w_q chỉ ổn định từ cạnh 1. Nhân/chốt y_q trước cạnh 1 là đọc nhầm hệ số.

Ví dụ tính tay — latency 1 ck trên 3 mẫu. Đặt $x_q = [10000, 20000, 30000]$, $w_q = [20029, 31145, 0]$ (cổ điển $N = 8$). Nếu FSM đúng latency: y lần lượt `q15_mul` cho $(10000, 20029) \rightarrow 6112$, $(20000, 31145) \rightarrow 19009$, $(30000, 0) \rightarrow 0$. Nếu bỏ chờ 1 ck và nhân với w trễ một mẫu, cặp thành $(10000, X)$, $(20000, 20029)$, $(30000, 31145)$ — sai hoàn toàn. Script assert hai tích đầu; tích với 0 là hệ quả trực tiếp của định nghĩa.

RTL datapath — giải thích từng dòng.

```
module window_hanning #(
  parameter N = 640
)(
  input  wire               clk, rst_n,
  input  wire               in_valid,
  output wire               in_ready,
  input  wire signed [15:0] x_q,        // Q1.15
  output reg                out_valid,
  input  wire               out_ready,
  output reg  signed [15:0] y_q         // Q1.15 windowed
);
  (* ram_init_file = "hanning_rom.hex" *)
  reg [15:0] rom [0:N-1];
```

```
  reg [9:0]  n;                          // 0..N-1
  // Lưu ý: rom[n] đọc tổ hợp ở đây; trên silicon thật
  // nên tách reg w_q_r <= rom[n] (latency 1) rồi nhân.
  wire signed [15:0] w_q = rom[n];
  wire signed [31:0] prod = x_q * w_q;  // Q2.30
  wire signed [31:0] rounded = prod + 32'sd16384;
  wire signed [31:0] shifted = rounded >>> 15;
  function automatic signed [15:0] sat16;
    input signed [31:0] v;
    begin
      if (v > 32'sd32767) sat16 = 16'sd32767;
      else if (v < -32'sd32768) sat16 = -16'sd32768;
      else sat16 = v[15:0];
    end
  endfunction
  assign in_ready = out_ready | ~out_valid;
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin n <= 0; out_valid <= 0; y_q <= 0; end
    else if (in_valid && in_ready) begin
      y_q <= sat16(shifted);
      out_valid <= 1'b1;
      n <= (n == N-1) ? 10'd0 : n + 10'd1;
    end else if (out_valid && out_ready) out_valid <= 1'b0;
  end
endmodule
```

Handshake Avalon-ST dùng `in_ready`/`out_ready` (Chương 0): downstream chưa sẵn sàng thì upstream giữ `valid`. Hàm `sat16` bão hòa (không wrap). `ram_init_file` bảo Quartus nạp hex lúc cấu hình.

Nếu muốn tiết kiệm một nửa BRAM với dạng cổ điển: địa chỉ `(n < 320) ? n : (639-n)` và ROM depth 320. Twin phải cùng công thức — không trộn Praat hex với địa chỉ nửa bảng cổ điển.

6.5. Twin Python bit-exact

Twin không phải “code tham khảo đẹp”; nó là hợp đồng số học. Mọi thay đổi round/sat phải sửa **cùng chỗ** RTL và twin.

```
import numpy as np
Q15 = 1 << 15

def hanning_praat_q15(N=640):
    i = np.arange(1, N + 1, dtype=np.float64)
    w = 0.5 - 0.5 * np.cos(2.0 * np.pi * i / (N + 1))
    return np.clip(np.round(w * Q15), 0, Q15 - 1).astype(np.int64)

def apply_window(xq, wq):
    # Khớp RTL: (x*w + 2^14) >>> 15, rồi sat ngoài nếu cần
    prod = xq.astype(np.int64) * wq.astype(np.int64)
    return ((prod + (1 << 14)) >> 15).astype(np.int64)

wq = hanning_praat_q15(640)
with open("hex/hanning_rom.hex", "w") as f:
    for v in wq:
        f.write(f"{int(v) & 0xFFFF:04X}\n")
print("Rw0 =", int(np.sum(wq * wq)))
```

Giải thích từng khối: `arange(1,N+1)` khớp chỉ số Praat; `round` rồi `clip` khớp sat dương; `apply_window`

không dùng float nhân. Script kiểm số tay `verify/verify_chuong_06_09.py` — chạy trước mọi lần commit hex.

6.6. Chi phí silicon — viết tắt mở rộng lần đầu trong chương

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách khối cửa sổ.

- **M10K** (khối BRAM nhúng $10\,240$ bit trên Cyclone V, Chương 0): ROM đầy đủ $640 \times 16 = 10\,240$ bit = đúng 1 M10K. Nửa bảng cổ điển $320 \times 16 = 5\,120$ bit vẫn chiếm 1 M10K (không chia nhỏ hơn một block) — tiết kiệm độ sâu địa chỉ, không tiết kiệm số block nếu chỉ một ROM.
- **DSP**: 1 nhân 18×19 mỗi mẫu cửa sổ (có thể dùng chung với MAC ở Pha 2 nếu FSM tuần tự chia sẻ một DSP theo thời gian).
- **ALM** (*Adaptive Logic Module* — đơn vị logic cơ bản Cyclone V, gồm LUT look-up table + flip-flop): con trỏ n + sat + handshake $\lesssim 80$ ALM.
- **Fmax** (tần số đồng hồ tối đa sau place-and-route): $\gg 50$ MHz — đường nhân một chu kỳ không phải nút thắt.
- **Latency**: 1 chu kỳ đọc ROM (+0/1 chu kỳ nhân pipeline). Trên hop 10 ms $= 500\,000$ chu kỳ @ 50 MHz, cửa sổ 640 chu kỳ chiếm $0{,}128\%$ ngân sách.
- Vì sao ROM: $w[n]$ bất biến — nướng offline một lần rẻ hơn CORDIC/sinh cos lúc chạy; FIR Compiler (Chương 4) dành cho lọc đa tốc độ, không phải bảng 640 từ cố định.

6.7. Bài tập và lời giải giải thích

Bài tập 6.1. Sinh hex Praat $N = 8$; đối chiếu bốn từ đầu với bảng mục 6.3.B.

Bài tập 6.2. RTL sim: khung toàn mẫu 32767 nhân Hanning — mô tả biên độ y ở hai đầu và giữa khung. Giải thích vì sao.

Bài tập 6.3. Cố ý dùng hex Praat nhưng địa chỉ nửa bảng cổ điển: $\mathcal{R}_x[\tau]$ lệch twin thế nào?

Bài tập 6.4. Tính tay $x_q = -10000$, $w_q = 20029$ với cùng quy tắc $+16384$ rồi $\ggg 15$. Kết quả?

Bài tập 6.5. On-Chip Memory bật registered output. Vẽ timeline 3 chu kỳ đầu: khi nào `address`, khi nào `q` ổn định, khi nào được phép chốt y_q. Nếu FSM bỏ qua 1 chu kỳ chờ, mẫu cửa sổ nào bị lệch?
Lời giải giải thích.

1. Kỳ vọng bốn từ đầu 3833, 13539, 24576, 31780 (script đã assert).
2. Giữa khung $w \approx 1$ nên $y \approx 32767$; hai đầu w nhỏ nên y nhỏ — đúng hình quả núi Hanning. Đây là hệ quả trực tiếp của công thức Praat, không phải lỗi bão hòa.
3. Hệ số sai đối xứng làm méo $\mathcal{R}_x$, đỉnh τ lệch vài mẫu, dẫn đến sai vài chục cent khi đối chiếu Praat ở Chương 13.
4. Tích $-200\,290\,000$; cộng 16384 rồi arithmetic $\ggg 15$ cho -6112 (đối xứng với ví dụ dương ở mục 6.3.A). Script xác nhận `q15_mul(-10000, 20029) == -6112`.
5. Chu kỳ 0: đặt address$= 0$. Chu kỳ 1: $q = w_q[0]$ ổn định — chỉ lúc này mới nhân với $x[0]$. Chu kỳ 2: address$= 1$, q chuyển sang $w_q[1]$. Bỏ chờ 1 ck khiến $x[0]$ nhân nhầm w cũ (hoặc X) — cả khung lệch một mẫu, MAC $\mathcal{R}[\tau]$ dịch hệ thống.

Chương 7. Tự tương quan — MAC mặc định / FFT tùy chọn + BFP

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn viết được định nghĩa tự tương quan không vòng từ nguyên lý đầu tiên; tính tay từng giá trị $\mathcal{R}[\tau]$ trên chuỗi ngắn và trên khung cosin chu kỳ 8; thiết kế datapath MAC tuần tự bằng một khối DSP và bộ tích lũy rộng kèm bảng timing chu kỳ; giải thích FFT IP radix-4 / BFP từng tầng cánh bướm và barrel shift 2^{2e} trước IFFT; viết twin Python bit-exact; và tính được ngân sách chu kỳ / RTF so với bước nhảy 10 ms.

7.1. Hiện tượng thật: đo mức "giống chính mình"

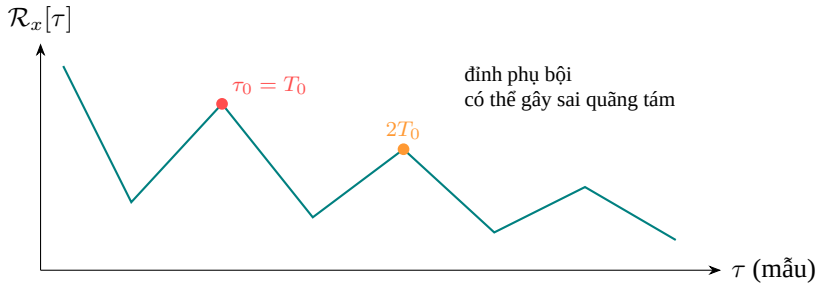
Khi dây thanh quản rung gần tuần hoàn với chu kỳ cơ bản T_0 giây, sóng áp suất tại thời điểm t gần trùng với sóng tại $t + T_0$. Trên lưới mẫu $f_s = 16$ kHz, chu kỳ đó là $\tau_0 = f_s \cdot T_0$ mẫu. Ví dụ giọng nam khoảng 100 Hz cho $\tau_0 = 160$; giọng nữ khoảng 200 Hz cho $\tau_0 = 80$. Phép **tự tương quan (autocorrelation)** đo mức giống nhau đó bằng cách trượt một bản sao đi τ mẫu (**độ trễ — lag**), nhân từng cặp mẫu chồng nhau và cộng dồn. Tại $\tau = \tau_0$, các cặp cùng pha cho tích dương lớn, tạo đỉnh nổi trên đồ thị $\mathcal{R}_x[\tau]$. Đo được τ_0 thì suy ra ngay $F_0 = f_s/\tau_0$.

Đây là hiện tượng vật lý thật của sóng tuần hoàn, không chỉ là quy ước toán học. Định lý **Wiener–Khinchin** (Chương 0) chứng minh rằng $\mathcal{R}_x$ cũng bằng biến đổi Fourier ngược của mật độ phổ công suất $|X[k]|^2$, nên có hai đường silicon tương đương về toán: nhân–cộng trực tiếp (MAC) hoặc FFT $\to |X|^2 \to$ IFFT. Với dải trễ hẹp $\tau = 26 \ldots 214$ (189 giá trị, khớp `lab_1_3_5.py` và bản đồ Chương 0), đường mặc định trên Cyclone V là **MAC** (*Multiply-ACcumulate*): chỉ tính đúng những τ cần thiết, không tính toàn bộ phổ rồi mới cắt. Mỗi hop 10 ms chỉ có 500 000 chu kỳ ở 50 MHz; 98 280 phép MAC chiếm khoảng một phần năm ngân sách đó và vẫn để lại biên cho cửa sổ, chuẩn hóa và chia F_0.

Tại sao không lấy đỉnh phổ trực tiếp? Vì phổ biên độ của nguyên âm có đỉnh formant (cộng hưởng ống thanh) có thể cao hơn đỉnh F_0; tự tương quan thời gian "nhìn" chu kỳ lặp của toàn sóng, ít bị formant đánh lừa hơn — đó là lý do Praat chọn ACF (sau chuẩn hóa) làm lõi đo cao độ.

▶ NHÌN THẤY VẬT LÝ

Tự tương quan không "tạo ra" cao độ. Nó chỉ đo độ trùng pha giữa tín hiệu và bản sao trễ của chính nó. Nếu khung đã bị rò phổ hoặc còn DC (Chương 3, 6), đỉnh giả sẽ cạnh tranh với đỉnh thật — MAC tính đúng công thức vẫn báo sai F_0.



Hình 31: Hàm tự tương quan của khung hữu thanh: đỉnh toàn cục tại $\tau = 0$ (năng lượng), đỉnh cục bộ đầu tiên tại $\tau_0 = T_0$ là ứng viên cao độ; đỉnh $2T_0$ là ứng viên sai quãng tám.

7.2. Toán từng bước — định nghĩa gốc trước mọi phím tắt

Hàm tự tương quan **không vòng (linear / non-circular)** của khung $x[n]$ độ dài L được định nghĩa từ tích vô hướng của hai đoạn chồng:

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} x[n] \cdot x[n+\tau], \qquad \tau \geq 0.$$

Từng bước: (1) chọn τ; (2) lấy cặp $(x[0], x[\tau])$, $(x[1], x[\tau+1])$, …, $(x[L-1-\tau], x[L-1])$; (3) nhân từng cặp; (4) cộng tất cả tích. Không được viết tắt thành "chỉ một hạng $x[0]x[\tau]$".

Mỗi ký hiệu:

- $x[n]$: mẫu đã trừ mean và đã nhân cửa sổ Hanning (Chương 3, 6), định dạng Q1.15 trên silicon.
- τ: độ trễ tính bằng số mẫu. Dải quét phần cứng: $\tau \in [26, 214]$ (189 giá trị) để đỉnh tìm trong $[27, 213]$ luôn có láng giềng cho nội suy parabolic (Chương 9). Sàn $75\,\text{Hz} \leftrightarrow \tau \approx 213$; trần $600\,\text{Hz} \leftrightarrow \tau \approx 27$.
- Số hạng trong tổng đúng bằng $L - \tau$: trễ càng lớn, càng ít cặp chồng — đây là thiên vị hình học sẽ được bù ở Chương 8.
- Tại $\tau = 0$: $\mathcal{R}_x[0] = \sum_n x[n]^2$ là năng lượng khung (cực đại toàn cục).

  Định lý Wiener–Khinchin cho phép đường tương đương:

$$\mathcal{R}_x[\tau] = \mathcal{F}^{-1}\{|X[k]|^2\}.$$

FFT mặc định tính tự tương quan **vòng (circular)**: mẫu cuối "quấn" sang mẫu đầu. Muốn khớp định nghĩa không vòng ở trên phải zero-pad ít nhất thêm L số 0 trước FFT (thường pad lên $N_{\text{FFT}} = 1024$ hoặc 2048). Đó là một trong những lý do đường mặc định pitch chọn MAC.

Trên silicon, mỗi tích $x[n] \cdot x[n+\tau]$ là Q1.15×Q1.15 = Q2.30 (32-bit). Cộng dồn đến 640 hạng tử đòi hỏi bộ tích lũy rộng: thiết kế dùng **41-bit có dấu** (khớp stub `rtl/mac_autocorr.v` và comment trong `lab_1_3_5.py`). Ước lượng thô: $|x| \le 2^{15}$, $|x \cdot x| \le 2^{30}$, $640 \cdot 2^{30} < 2^{40}$ — 41 bit có dấu vừa đủ biên an toàn một bit.

7.3. Ví dụ tính tay — mọi số đã chạy Python

A. Chuỗi cực ngắn $x = [1, 2, 1, 2]$. Từng cặp:

$$\begin{aligned}
\mathcal{R}[0] &= 1\cdot 1 + 2\cdot 2 + 1\cdot 1 + 2\cdot 2 = 1 + 4 + 1 + 4 = 10,\\
\mathcal{R}[1] &= 1\cdot 2 + 2\cdot 1 + 1\cdot 2 = 2 + 2 + 2 = 6,\\
\mathcal{R}[2] &= 1\cdot 1 + 2\cdot 2 = 1 + 4 = 5,\\
\mathcal{R}[3] &= 1\cdot 2 = 2.
\end{aligned}$$

Tỉ số thô $\mathcal{R}[2]/\mathcal{R}[0] = 0{,}5$. Sau khi nhân $C[\tau]$ (Chương 8) và chia theo số hạng, đỉnh chu kỳ được kéo về gần 1.

B. Khung cosin chu kỳ 8 mẫu, $N = 16$ (Q1.15). $x[n] = \text{sat}_{16}(\text{round}(\cos(2\pi n/8) \cdot 32768))$, $n = 0..15$. Từng bước cho $n = 0$: $\cos 0 = 1$, $\text{round}(32768) = 32768$, sat $\to 32767$. Cho $n = 4$: $\cos \pi = -1$, $\text{round}(-32768) = -32768$ (vừa khít min int16).

n	0	1	2	3	4	5	6	7
x	32767	23170	0	−23170	−32768	−23170	0	23170
n	8	9	10	11	12	13	14	15
x	32767	23170	0	−23170	−32768	−23170	0	23170

Tự tương quan không vòng (script):

$$\begin{aligned}
\mathcal{R}[0] &= 8\,589\,627\,426,\\
\mathcal{R}[8] &= 4\,294\,813\,713 \approx \tfrac{1}{2}\mathcal{R}[0],\\
\mathcal{R}[4] &= -6\,442\,220\,568 \quad \text{(chống pha, không phải đỉnh dương).}
\end{aligned}$$

Tỉ số thô $\mathcal{R}[8]/\mathcal{R}[0] = 0{,}5$ vì chỉ còn 8 cặp; sau chuẩn hóa unbiased $(\mathcal{R}[8]/8)/(\mathcal{R}[0]/16) = 1$. Đây là lý do Chương 8 tồn tại.

C. Ngân sách chu kỳ trên đường MAC thật. Với $\tau = 26 \ldots 214$, tổng số phép nhân–cộng đúng bằng

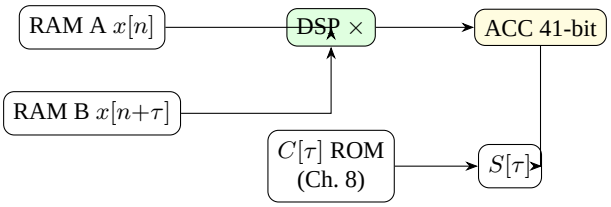
$$N_{\text{MAC}} = \sum_{\tau=26}^{214} (640 - \tau).$$

Tính tổng cấp số: số hạng 189; số hạng đầu $640 - 26 = 614$; số hạng cuối $640 - 214 = 426$; tổng $= 189 \cdot (614 + 426)/2 = 98\,280$. Không phải $189 \times 640 = 120\,960$ — công thức naïve đếm thừa. Tại $f_{\text{clk}} = 50\,\text{MHz}$:

$$T_{\text{compute}} = \frac{98\,280}{50 \times 10^6} = 1{,}9656\,\text{ms}, \qquad \text{RTF} = \frac{1{,}9656}{10} = 0{,}19656 \ll 1.$$

7.4. Ý nghĩa phần cứng — MAC mặc định

Hai con trỏ đọc khung trong BRAM True Dual-Port (Chương 5): một trỏ tới $x[n]$, một tới $x[n+\tau]$. Mỗi chu kỳ một khối DSP nhân 18×18 (hoặc 18×19), cộng vào bộ tích lũy 41-bit có dấu. Quét τ tuần tự. Sau $(L - \tau)$ bước, thanh ghi accum chính là $\mathcal{R}_x[\tau]$.



Hình 32: Datapath MAC: hai toán hạng từ TDP RAM, một nhân DSP, một tích lũy — khớp vòng `for tau` trong `lab_1_3_5.py`.

Bảng timing chu kỳ — một τ cố định (ví dụ $\tau = 2$, $L = 4$).

ck	Trạng thái	n	xa=$x[n]$, xb=$x[n+\tau]$	`acc` sau cạnh
0	IDLE→LOAD	—	—	clear 0
1	RUN	0	$x[0], x[2]$	$x[0]x[2]$
2	RUN	1	$x[1], x[3]$	$x[0]x[2] + x[1]x[3]$
3	DONE	—	—	$= \mathcal{R}[2]$; `done_tau`=1

Với $\tau = 26$, $L = 640$: pha RUN kéo dài đúng 614 chu kỳ. FSM ngoài tăng τ và lặp. Không được clear `acc` giữa các n của cùng một τ.

RTL lõi (ý tưởng, khớp stub `mac_autocorr.v`).

```
module mac_autocorr (
  input  wire                clk, rst_n, start,
  input  wire signed [15:0] xa, xb,   // x[n], x[n+tau] Q1.15
  input  wire [9:0]          n_pairs,  // = L - tau
  output reg  signed [40:0] acc,      // 41-bit signed
  output reg                 done_tau
);
  // FSM: IDLE -> RUN (n_pairs cycles: acc <= acc + xa*xb) -> DONE
  // xa, xb phải ổn định đúng cạnh khi acc cập nhật (đồng bộ với
  // latency đọc TDP RAM --- thường 1 ck registered).
endmodule
```

Không dùng `numpy.correlate` float làm chuẩn silicon: twin phải nhân–cộng int cùng độ rộng accum.

7.5. Đường tùy chọn: FFT IP + BFP + barrel shift trước IFFT

Chỉ bật sau khi MAC đã xanh bit-exact. Dùng **FFT Intel FPGA IP** (User Guide 683374):

- Kiến trúc **radix-4** (hoặc radix-2): mỗi tầng cánh bướm (**butterfly**) kết hợp 4 (hoặc 2) điểm với twiddle $W_N^k = e^{-j2\pi k/N}$ lấy từ ROM nội bộ. Với $N = 1024$ radix-4 cần $\log_4 1024 = 5$ tầng; radix-2 cần 10 tầng.
- Streaming hoặc buffered; tín hiệu SOP/EOP đánh dấu một block N điểm trên Avalon-ST.
- **Block Floating Point (BFP — số chấm động khối)**: mọi bin chia sẻ một số mũ nguyên e; mantissa dịch sao cho max $|m|$ vừa khít bit. Sau mỗi tầng butterfly, nếu có nguy cơ tràn thì toàn vector dịch phải 1 bit và e tăng 1. IP xuất `source_exp`.
- BFP **không** có cổng nhập exponent vào IFFT — kỹ sư phải scale bên ngoài trước khi đưa mantissa vào IFFT.

BFP từng tầng — ví dụ mantissa 8-bit, 2 tầng. Bắt đầu stage 0: $m = [40, -30, 10, -5]$, $e = 0$. Ngưỡng tràn minh họa $= 32$ (giả sử còn 1 bit đầu). Vì max $|m| = 40 > 32$:

Sự kiện	e	Mantissa	Ghi chú
Vào stage 0	0	$[40, -30, 10, -5]$	max $= 40 > 32$
Sau scale stage 0	1	$[20, -15, 5, -2]$	dịch phải 1
Sau stage 1 (không tràn)	1	$[20, -15, 5, -2]$	giữ e
Power bin 0 thô $\|m\|^2$	—	400	quên 2^{2e}
Power đúng $\|m\|^2 \cdot 2^{2e}$	—	1600	$= 40^2$

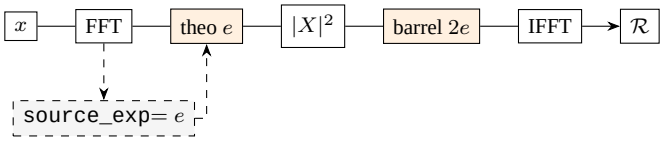
★ NGUYÊN LÝ ĐẦU TIÊN: Sau $|X|^2$, số mũ nhân đôi thành $2e$

Mantissa $y = 3$, exponent $e = 2$ có giá trị thật $3 \cdot 2^{-2}$. Bình phương:

$$(3 \cdot 2^{-2})^2 = 9 \cdot 2^{-4}$$

— số mũ thành $2e$. Đẩy mantissa $|y|^2$ thẳng vào IFFT (IP kỳ vọng fixed-point với exponent $= 0$ ở lối vào) mà quên barrel-shift theo $2e$ sẽ làm cả dải động $\mathcal{R}[\tau]$ sai tỷ lệ, phá ngưỡng voicing.

Barrel shift trước IFFT. RTL bắt buộc: nếu `source_exp`$= e$ sau FFT, sau khi tính $|X[k]|^2$ trên mantissa, dịch trái (hoặc nhân) kết quả theo $2e$ trước khi đưa vào IFFT — hoặc dịch mantissa IFFT input tương đương. Twin phải dùng model `_model` của variation FFT IP, không đoán e.



Hình 33: Đường FFT/Wiener–Khinchin: hai khối cam (scale theo e và barrel 2^{2e}) là RTL bắt buộc. Quên chúng gần như chắc fail bit-exact — đó là lý do mặc định vẫn là MAC.

Vì sao MAC thắng khi $\tau_{\max} \ll L$. MAC: 98 280 nhân thực/khung. FFT 1024 radix-4: 5 tầng, mỗi tầng 256 butterfly phức (mỗi butterfly vài nhân thực) — cộng IFFT tương tự cộng $|X|^2$, rồi vẫn phải cắt τ. Thêm rủi ro bit-reverse và BFP. Chỉ mở FFT khi lab yêu cầu so khớp đường Praat FFT hoặc khi RTF MAC không đủ.

Bit-reverse, zero-pad — hai bẫy của đường FFT. **Bit-reverse** là thứ tự địa chỉ khi đảo từng bit của chỉ số bin (ví dụ bin $6 = 110_2$ thành $011_2 = 3$). FFT IP có thể xuất natural order hoặc bit-reverse tùy cấu hình; twin và RTL sau $|X|^2$ phải cùng một thứ tự trước IFFT, nếu không đỉnh τ_0 sẽ "nhảy chỗ". **Zero-pad** nghĩa là thêm số 0 vào cuối khung trước FFT để tự tương quan vòng không "quấn" đuôi vào đầu: khung $L = 640$ cần pad ít nhất thêm 640 số 0 (thường lên $N_{\text{FFT}} = 2048$). MAC không bao giờ gặp hai bẫy này vì nó chỉ cộng đúng $L - \tau$ hạng thật.

7.6. Twin Python bit-exact

```
import numpy as np

def acf_linear_int(x, tau_min, tau_max):
    x = np.asarray(x, dtype=np.int64)
    L = len(x)
    out = {}
    for tau in range(tau_min, tau_max + 1):
        # Khớp MAC: cộng dồn int, không float
        out[tau] = int(np.dot(x[: L - tau], x[tau:]))
    return out

# Khớp lab: tau 26..214, accum int64 (>= 41-bit signed trên RTL)
R = acf_linear_int(xw_q15, 26, 214)
```

```
# So dump R[tau] với ModelSim: sai lệch 0 bit trên đường MAC
```

Giải thích: `np.dot` trên `int64` tương đương vòng RUN của FSM; không lấy `numpy.correlate` float. Trên nhánh FFT, twin phải gồm cả barrel 2^{2e} giống RTL và so với `_model` của IP.

Ví dụ tính tay — ngân sách RTF an toàn. Với $N_{\text{MAC}} = 98\,280$ đã tính ở mục 7.3.C, thêm giả sử 640 chu kỳ cửa sổ + 189 chu kỳ nhân C + 300 chu kỳ peak/chia $\approx 99\,409$ chu kỳ $\approx 1{,}988$ ms @ 50 MHz. Lab dành biên 3 ms (RTF $0{,}3$) để chứa thêm latency đọc RAM đăng ký và handshake Avalon-ST. Script khóa các số lõi; biên 3 ms là quy ước nghiệm thu, không phải assert tuyệt đối.

7.7. Chi phí silicon

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách MAC autocorr (default).

- **DSP**: 1 khối MAC/chu kỳ; 98 280 phép/khung $\approx 1{,}97$ ms @ 50 MHz.
- **ALM**: ~ 200 cho FSM con trỏ τ, n, handshake Avalon-ST.
- **M10K**: đệm khung dùng chung Chương 5; MAC không thêm BRAM.
- **RTF** $= 0{,}197 \ll 1$ (còn $> 80\%$ thời gian rảnh mỗi hop 10 ms $= 500\,000$ chu kỳ). Vì sao vừa khít: ngay cả khi thêm cửa sổ + $C[\tau]$ + parabolic + chia, tổng vẫn dưới khoảng 3 ms, RTF $< 0{,}3$.
- **FFT optional**: hàng nghìn ALM + nhiều M10K twiddle/buffer + DSP theo variation; barrel shifter vài chục ALM. Chỉ bật khi twin đã khóa và lab yêu cầu.
- Vì sao MAC mặc định: 0 rủi ro BFP/bit-reverse, 0-bit với golden `lab_1_3_5.py`.

7.8. Bài tập và lời giải giải thích

Bài tập 7.1. Với $x = [1, 2, 1, 2]$, tính $\mathcal{R}[1]$ và $\mathcal{R}[2]$ từ định nghĩa gốc. Có được phép viết $\mathcal{R}[2] = x[0]x[2]$ một hạng duy nhất không?

Bài tập 7.2. Trên khung cosin mục 7.3.B, vì sao $\mathcal{R}[8]/\mathcal{R}[0] = 0{,}5$ dù tín hiệu lặp hoàn hảo? Làm thế nào để tỉ số về gần 1?

Bài tập 7.3. Mantissa sau FFT $m = 20$, $e = 1$. Tính power $|m|^2 \cdot 2^{2e}$. Nếu quên nhân 2^{2e}, sai bao nhiêu lần?

Bài tập 7.4. Tính N_{MAC} đúng bằng $\sum_{\tau=26}^{214}(640 - \tau)$ và RTF @ 50 MHz. So với ước lượng naïve 189×640.

Bài tập 7.5. Vẽ bảng timing 4 chu kỳ cho $\tau = 1$, $L = 4$, $x = [1, 2, 1, 2]$: mỗi cạnh `acc` mang giá trị gì? `done_tau` lên khi nào?
Lời giải giải thích.

1. $\mathcal{R}[1] = 6$, $\mathcal{R}[2] = 5$. Không được chỉ lấy một hạng: định nghĩa cộng mọi cặp chồng, với $\tau = 2$ còn đúng hai hạng $1 \cdot 1 + 2 \cdot 2$.
2. Vì chỉ còn $N - \tau = 8$ cặp trong tổng không vòng, trong khi $\mathcal{R}[0]$ có 16 hạng. Chuẩn hóa unbiased chia cho số hạng, hoặc nhân $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ (Chương 8), kéo tỉ số về gần 1.
3. Power $= 400 \cdot 4 = 1600$. Quên 2^{2e} còn 400 — sai đúng $4 = 2^{2e}$ lần (khoảng $+6$ dB nếu đo năng lượng sai chiều ngược), đủ phá ngưỡng voicing.
4. $N_{\text{MAC}} = 98\,280$, $T = 1{,}9656$ ms, RTF $\approx 0{,}197$. Naïve 120 960 lớn hơn khoảng 23% vì đếm thừa các hạng không tồn tại khi $\tau > 0$.
5. ck1: `acc`$= 1 \cdot 2 = 2$; ck2: $2 + 2 \cdot 1 = 4$; ck3: $4 + 1 \cdot 2 = 6 = \mathcal{R}[1]$; `done_tau` sau đúng $L - \tau = 3$ bước RUN.

Chương 8. Chuẩn hóa $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ (ROM nhân thay chia)

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn giải thích được vì sao phải chia cho tự tương quan của cửa sổ; dẫn ra $C[\tau]$ từ nguyên lý đầu tiên từng bước; tính tay và kiểm Python một bảng nhỏ cùng các mốc trên khung $L = 640$; cấu hình ROM M10K mang $C[\tau]$ dạng Q3.13 kèm latency đăng ký; đối chiếu với LPM_DIVIDE / bộ chia non-restoring khi $\mathcal{R}_w$ đổi lúc chạy; viết twin sinh `.hex` bit-exact; và đọc được hộp chi phí — tại sao nhân thay chia runtime.

8.1. Hiện tượng thật: đỉnh τ lớn bị "đạp" vì hai quả núi ít chồng

Sau khi nhân cửa sổ Hanning (Chương 6), mỗi khung trở thành một "quả núi": biên độ lớn ở giữa, nhỏ ở hai đầu. Nếu nhìn dạng sóng sau cửa sổ trên máy hiện sóng, hai đầu gần im lặng còn giữa khung vẫn giữ chu kỳ dây thanh — đúng ý đồ: giảm discontinuity mà không xóa thông tin T_0 ở phần năng lượng chính. Khi tính $\mathcal{R}_x[\tau]$, ta đang đo độ chồng giữa quả núi đó và bản sao trễ τ mẫu. Trễ càng lớn, hai quả núi chồng nhau càng ít — năng lượng chồng đó chính là tự tương quan của *chính cửa sổ*:

$$\mathcal{R}_w[\tau] = \sum_{n=0}^{L-1-\tau} w[n]\, w[n+\tau].$$

Nếu lấy $\mathcal{R}_x[\tau]$ thô làm điểm số, đỉnh ở τ lớn (giọng trầm, T_0 dài) bị đạp xuống chỉ vì hình học cửa sổ, không phải vì tín hiệu kém tuần hoàn. Ví dụ khung cosin ở Chương 7 cho $\mathcal{R}[8]/\mathcal{R}[0] = 0{,}5$ dù sóng lặp hoàn hảo — đó đúng là hiệu ứng này: tám cặp chồng so với mười sáu hạng ở $\tau = 0$.

Trực giác đời thường: bạn so hai tờ giấy có hình quả núi in mực; kéo lệch chúng theo chiều ngang thì vùng mực chồng nhau nhỏ dần. Không phải vì hình vẽ xấu đi — chỉ vì hình học chồng. $C[\tau]$ chính là hệ số "kéo độ đậm" lại cho công bằng giữa các vị trí lệch.

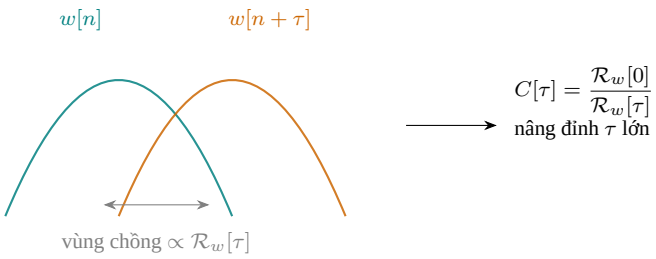
Praat/Boersma nhân hệ số bù

$$C[\tau] = \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]}$$

để kéo mọi đỉnh về cùng thước gần $[0, 1]$. $C[\tau]$ **không phải hiện tượng dây thanh**: nó là hiệu chỉnh hình học của cửa sổ. Trên FPGA, $\mathcal{R}_w$ chỉ phụ thuộc w và L (bất biến lúc chạy), nên tính offline, nạp ROM, rồi *nhân* thay vì chia mỗi τ. Chia runtime 189 lần mỗi khung vừa tốn latency vừa khó khớp bit với round cố định.

▶ NHÌN THẤY VẬT LÝ

Quên nhân $C[\tau]$ thiên lệch hệ thống về τ nhỏ (giọng cao giả): đỉnh trầm bị đạp, thuật toán thích đỉnh ngắn hơn. Đây là lỗi kiến trúc, không phải nhiễu ngẫu nhiên — cent-error sẽ lệch một phía trên mọi câu nói trầm.



Hình 34: Hai quả núi cửa sổ chồng ít dần khi τ tăng. Hệ số $C[\tau]$ bù đúng độ mất chồng đó.

8.2. Toán từng bước — mọi ký hiệu

Bắt đầu từ định nghĩa Boersma sau khi đã có $\mathcal{R}_x[0]$:

$$r_a[\tau] = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0]} \cdot C[\tau], \qquad C[\tau] = \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]}.$$

Từng bước dẫn C: (1) tính $\mathcal{R}_w[0] = \sum_n w[n]^2$; (2) với mỗi τ, tính $\mathcal{R}_w[\tau] = \sum_{n=0}^{L-1-\tau} w[n]w[n+\tau]$; (3) chia $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ (float offline); (4) lượng tử Q3.13.

- $\mathcal{R}_x[\tau]$: tự tương quan tín hiệu đã cửa sổ (Chương 7), int rộng trên silicon.
- $\mathcal{R}_w[\tau]$: tự tương quan cửa sổ — mẫu số "độ chồng", tính offline từ w_q.
- $C[\tau] \geq 1$ và tăng theo τ (vì $\mathcal{R}_w$ giảm).
- Lượng tử silicon: $C_q[\tau] = \text{sat}_{u16}\big(\text{round}(C[\tau] \cdot 2^{13})\big)$ — định dạng **Q3.13** (3 bit phần nguyên gồm đầu, 13 bit thập phân; $1{,}0 \mapsto 8192$; dải đến gần $4{,}0$). Chọn Q3.13 vì trên $L = 640$, $C[214] \approx 2{,}13$ vượt quá 1 (Q1.15 sẽ bão hòa sai tại $1{,}0$).
- Bit-growth: R_x (vd Q2.30 / accum) $\times C$ (Q3.13) rồi làm tròn về thang điểm số. Twin và RTL dùng cùng quy tắc:
$$S[\tau] = (R_x[\tau] \cdot C_q[\tau] + 2^{12}) \gg 13$$
(làm tròn gần nhất bằng cộng $2^{12} = 4096$ trước khi dịch 13, khớp `lab_1_3_5.py`).

Dải ROM phần cứng: 189 từ cho $\tau = 26 \ldots 214$. Tệp `hex/c_tau_rom.hex` trong gói này còn nhân thêm hệ số octave-cost của Praat (xem `scoring_rom_q13` trong golden) — đó là tiện ích gộp ROM, không đổi bản chất $C[\tau]$ bù cửa sổ. Twin kiểm tra cửa sổ thuần dùng `compensation_rom_q13`; twin kiểm tra điểm số đầy đủ dùng scoring hex.

★ NGUYÊN LÝ ĐẦU TIÊN: Nhân nghịch đảo offline thay chia runtime

Chia số nguyên trên FPGA (LPM_DIVIDE) có latency hàng chục chu kỳ và thương/dư phụ thuộc làm tròn IP. Vì mẫu số $\mathcal{R}_w[\tau]$ biết trước, nghịch đảo một lần trên Python rồi nhân là đủ — 189 nhân cố định latency, bit-exact với twin.

8.3. Ví dụ tính tay — mọi số đã chạy Python

A. $L = 4$, $w = [1, 2, 2, 1]$ **— từng hạng.**

$$\begin{aligned}
\mathcal{R}_w[0] &= 1 \cdot 1 + 2 \cdot 2 + 2 \cdot 2 + 1 \cdot 1 = 1 + 4 + 4 + 1 = 10, \\
\mathcal{R}_w[1] &= 1 \cdot 2 + 2 \cdot 2 + 2 \cdot 1 = 2 + 4 + 2 = 8, \\
\mathcal{R}_w[2] &= 1 \cdot 2 + 2 \cdot 1 = 2 + 2 = 4, \\
\mathcal{R}_w[3] &= 1 \cdot 1 = 1.
\end{aligned}$$

Vậy $C = [10/10,\ 10/8,\ 10/4,\ 10/1] = [1,\ 1{,}25,\ 2{,}5,\ 10]$. Q3.13 từng bước cho $C = 1{,}25$:

$$1{,}25 \cdot 8192 = 10\,240 \quad (\texttt{0x2800}).$$

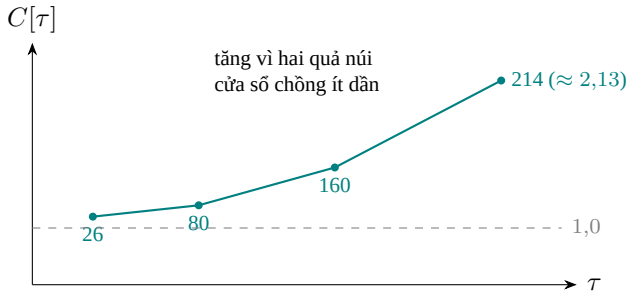
Giả sử $\mathcal{R}_x = [20, 12, 6, 1]$ thì sau nhân C (float): $[20,\ 15,\ 15,\ 10]$ — đỉnh phụ τ được nâng tương đối. Trên int: $S[1] = (12 \cdot 10240 + 4096) \gg 13 = 15$ (khớp).

B. $w = [1, 2, 1]$**,** $\tau = 2$**.** $\mathcal{R}_w[0] = 1 + 4 + 1 = 6$, $\mathcal{R}_w[2] = 1 \cdot 1 = 1$, $C[2] = 6$.

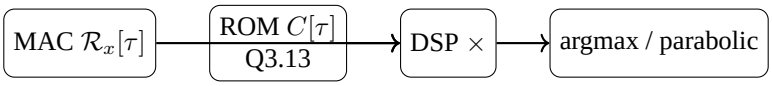
C. Khung thật $N = 640$ **Praat (bù cửa sổ thuần).**

τ	C float	C_q Q3.13	hex
26	1.010885	8281	`2059`
80	1.108152	9078	`2376`
160	1.515087	12412	`307C`
214	2.127577	17429	`4415`

Hex triển khai (`scoring_rom`, có octave-cost): $C_q[26] = 8283$, $C_q[214] = 17070$ — lệch nhẹ so với bảng thuần, đúng thiết kế gộp phạt quãng tám vào cùng một lần nhân. Script `verify_chuong_06_09.py` assert cả hai bảng.



Hình 35: $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ tăng theo τ: càng trễ, vùng chồng cửa sổ càng nhỏ, hệ số bù càng lớn. Các mốc khớp bảng 8.3.C.



Hình 36: Sau nhân C: đưa $S[\tau]$ (hoặc $r_a[\tau]$) vào khối dò đỉnh (Chương 9).

8.4. Ý nghĩa phần cứng — ROM mặc định, chia chỉ khi bắt buộc

Đường mặc định: IP **On-Chip Memory** kiểu ROM (hoặc RAM khởi tạo `.hex`/`.mif`).

- Depth $= 189$ (hoặc $\tau_{\max} + 1$ nếu lưu từ 0); width $= 16$ bit mang Q3.13.
- Registered output: 1 chu kỳ — FSM Chương 11 chờ đúng latency trước khi nhân.
- Một khối DSP nhân $R_x \times C_q$ mỗi τ (có thể dùng chung nhịp với MAC).
- Init file: `c_tau_rom.hex` (scoring) hoặc `compensation` thuần — ghi rõ trong README lab.

Bảng timing — đọc ROM C rồi nhân.

ck	Địa chỉ	C_q	S
0	$\tau - 26$ đặt	—	—
1	giữ	C_q ổn định	bắt đầu nhân (hoặc chốt nếu DSP 0-pipe)
2	τ kế	C mới	S của τ trước hợp lệ nếu DSP 1-pipe

Khi nào mới chia runtime? Chỉ khi $\mathcal{R}_w$ *đổi lúc chạy* (cửa sổ adaptive). Dùng **LPM_DIVIDE** hoặc bộ chia **non-restoring** tự viết bằng ALM (gặp lại ở Chương 9 cho $F_0 = f_s/\tau$). Đường pitch mặc định **không** chia mỗi τ: ROM đã nướng nghịch đảo. IP "ALTDIV_TENT không tồn tại.

Đổi L hoặc công thức w thì **rebuild file hex bắt buộc**; nạp ROM cũ sẽ sai im lặng và đỉnh F_0 thiên lệch.

RTL (ý tưởng) — từng dòng.

```
(* ram_init_file = "c_tau_rom.hex" *)
reg [15:0] C_rom [0:188];          // tau 26..214
reg [15:0] C_q;
reg signed [42:0] S;               // score after *C
always @(posedge clk) begin
  C_q <= C_rom[tau - 26];          // 1-cycle registered read
  // Nhân signed: zero-extend C_q thành số không âm
  S   <= (R_x * $signed({1'b0,C_q}) + 43'sd4096) >>> 13;
end
```

Dòng `C_q <= C_rom[...]` là latency 1; dòng `S <= ...` dùng C_q của chu kỳ trước nếu cùng `always`. Lab phải pipeline đúng: tách hai `always`, hoặc chấp nhận S trễ thêm 1 ck so với R_x mới.

8.4b. LPM_DIVIDE nội bộ khi bắt buộc chia $\mathcal{R}_w$

Khi lab cố tình đổi cửa sổ theo khung (ví dụ adaptive window), ROM cố định không còn đúng. Khi đó cấu hình **LPM_DIVIDE**:

- `LPM_WIDTHN`: độ rộng numerator — đủ chứa $\mathcal{R}_w[0]$ (thường 32–48 bit int).
- `LPM_WIDTHD`: độ rộng denominator $\mathcal{R}_w[\tau]$.
- `LPM_PIPELINE`: số tầng pipeline L (mỗi tầng một hàng thanh ghi). Latency đúng L chu kỳ sau khi `numer`/`denom` ổn định.
- `LPM_NREPRESENTATION` / `LPM_DREPRESENTATION`: unsigned hoặc signed; khớp twin.

Timing LPM_DIVIDE (pipeline $L = 4$ minh họa).

ck	Sự kiện	Ghi chú
0	nạp $\mathcal{R}_w[0]$, $\mathcal{R}_w[\tau]$	start ngầm theo data
1–3	pipeline nội bộ	không đọc quotient
4	`quotient`$\approx C$ thô	mới được nhân với $\mathcal{R}_x$

So với ROM 1 ck: chia chậm hơn và khó bit-exact vì IP có thể cắt thương khác `round` Python. Vì vậy tài liệu giữ ROM làm mặc định.

Non-restoring tự viết — giải thích thuật ngữ. **Non-restoring division** là thuật toán chia số nguyên từng bit: mỗi chu kỳ đồng hồ dịch thanh ghi dư một bit, rồi *trừ* hoặc *cộng* số chia tùy dấu dư hiện tại, và ghi một bit thương. Khác restoring (trừ thử rồi cộng lại nếu âm), non-restoring không hoàn tác — ít multiplex hơn, dễ viết bằng ALM thuần. Với thương 16-bit cần khoảng 16 chu kỳ. Vẫn chậm hơn đọc ROM một chu kỳ, nhưng khi lab bắt buộc chia $\mathcal{R}_w$ lúc chạy và muốn so từng bước với twin, non-restoring dễ quan sát hơn LPM_DIVIDE (IP "đen"). Chương 9 sẽ dùng lại đúng thuật toán này cho $F_0 = f_s/\tau$.

8.4c. Octave-cost gộp vào cùng ROM

Praat phạt ứng viên gần bội của đỉnh mạnh (sai quãng tám) bằng hệ số < 1 nhân thêm vào điểm số. Gói này nhân octave-cost vào cùng từ hex với $C[\tau]$ (scoring_rom) để FSM chỉ một lần nhân DSP. Hệ quả: bảng "thuần C" và bảng "scoring" lệch vài LSB như mục 8.3.C — không phải lỗi làm tròn. Lab phải chọn đúng tệp: so khớp Praat đầy đủ dùng scoring; chứng minh công thức bù cửa sổ dùng compensation thuần.

Ví dụ số: tại $\tau = 214$, thuần $C_q = 17429$, scoring 17070. Tỉ số $17070/17429 \approx 0{,}979$ — octave-cost khoảng vài phần trăm, đủ kéo đỉnh bội xuống dưới đỉnh chính trên nhiều khung hữu thanh.

8.5. Twin Python bit-exact

Twin ở đây là mô hình Python nhân–cộng/lượng tử giống hệt RTL: cùng công thức, cùng làm tròn, cùng bão hòa. **Hợp đồng bit-exact** nghĩa là dump thanh ghi trên FPGA phải trùng từng bit với số twin in ra trên cùng vector vào — không "gần đúng". Mọi thay đổi round/sat phải sửa cùng một chỗ ở twin và RTL, rồi sinh lại hex từ twin (không sửa hex tay).

```
import numpy as np
Q13 = 1 << 13

def build_C(w, tau_min, tau_max):
    w = np.asarray(w, float)
    L = len(w)
    Rw0 = float(np.dot(w, w))
    out = []
    for t in range(tau_min, tau_max + 1):
        Rwt = float(np.dot(w[: L - t], w[t:]))
```

```
        out.append(int(round(Rw0 / max(Rwt, 1e-30) * Q13)))
    return np.clip(out, 0, 65535).astype(np.uint16)

def apply_C(R_x, C_q):
    return (R_x.astype(np.int64) * C_q.astype(np.int64) + (1 << 12)) >> 13

# assert: build_C([1,2,2,1], 0, 3)[1] == 10240  # C=1.25
```

Giải thích: `max(Rwt, 1e-30)` tránh chia 0 khi cửa sổ suy biến; `round` rồi `clip` khớp sat u16; `apply_C` cộng 2^{12} trước $\gg 13$ khớp RTL. Mọi thay đổi công thức octave-cost phải sửa cùng chỗ golden và hex.

8.5b. Quy trình sinh hex và nghiệm thu

1. Chạy twin `build_C` trên đúng w_q Praat $N = 640$ (cùng hex cửa sổ Chương 6).
2. Ghi `compensation_rom_q13.hex` (thuần) và/hoặc nhân octave-cost $\rightarrow$ `c_tau_rom.hex`.
3. So 4 mốc $\tau \in \{26, 80, 160, 214\}$ với bảng mục 8.3.C — sai một mốc là dừng, không nạp bitstream.
4. Trên ModelSim: kích τ tuần tự, so $S[\tau]$ với `apply_C` trên cùng R_x dump từ MAC.
5. Chỉ khi dump xanh mới nối argmax Chương 9.

Nếu đổi Hanning sang Hamming, toàn bộ $\mathcal{R}_w$ đổi — hex cửa sổ và hex C phải rebuild cùng commit. Sai một LSB công thức (quên octave-cost hoặc dùng nhầm Q1.15) lệch hàng trăm đơn vị trên C_q, đủ đổi argmax; vì vậy nghiệm thu theo bốn mốc hex mục 8.3.C, không chỉ theo "trông có vẻ tăng theo τ".

8.6. Chi phí silicon

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách chuẩn hóa $C[\tau]$.

- **M10K**: $189 \times 16 = 3\,024$ bit < 1 M10K ($10\,240$ bit) — dư chỗ ghép thêm bảng nhỏ nếu cần.
- **DSP**: 1 nhân $R_x \times C$ mỗi τ (dùng chung hoặc ngay sau MAC). 189 nhân/khung $\ll 98\,280$ MAC.
- **ALM**: địa chỉ + đăng ký — vài chục.
- So với LPM_DIVIDE mỗi τ: tiết kiệm 189 phép chia/khung và latency không đều — lý do chọn ROM. Một LPM_DIVIDE 32/16 pipeline 16 ck sẽ thêm 189×16 chu kỳ nếu tuần tự — phá RTF.
- Vì sao Q3.13: $C[214] \approx 2{,}13$ vừa trong 3 bit nguyên; Q1.15 sẽ bão hòa tại $1{,}0$ và phá bù. Vì sao không Q4.12: đủ nhưng mất 1 bit thập phân không cần thiết khi $C < 4$.

Trên hop 10 ms, khối C chỉ thêm khoảng 189 chu kỳ nhân (+ latency ROM). Tổng Pha 2 vẫn dễ nằm dưới 3 ms nếu MAC là nút thắt chính. Lab nghiệm thu: dump $S[\tau]$ sau nhân C phải khớp twin trên cùng hex.

Ví dụ tính tay — ba lớp sai số (cùng một khung). Giả sử $\tau^* = 80$ đúng trên twin. (1) Hex $C_q[80]$ lệch $+200$ LSB so twin: $S[80]$ bị nâng giả, có thể thắng τ láng giềng — đỉnh lệch có hệ thống theo τ. (2) FSM đọc ROM sớm 1 chu kỳ: $S[\tau]$ mang $C_q[\tau - 1]$; đỉnh lệch đúng 1 mẫu. (3) Dùng compensation thuần trong khi twin so scoring: lệch vài phần trăm như $17070/17429 \approx 0{,}979$ ở $\tau = 214$, dễ thành sai quãng tám trên nguyên âm ổn định. Ba lớp này cho triệu chứng khác nhau; đừng gộp thành "F_0 sai".

Trên đường mặc định không có khối chia $\mathcal{R}_w$: thanh ghi accum của MAC mang $\mathcal{R}_x[\tau]$, ROM mang $C_q[\tau]$, một DSP nhân rồi dịch 13 bit có làm tròn. IP tên "ALTDIV_TENT" không tồn tại. Phép chia thật chỉ xuất hiện khi $\mathcal{R}_w$ đổi lúc chạy (LPM_DIVIDE / non-restoring ở trên) hoặc khi Chương 9 tính $F_0 = f_s/\tau_{\text{fine}}$ — một lần mỗi khung, không phải 189 lần. So sánh voicing với ngưỡng cũng làm bằng nhân nguyên (best $\cdot\, 100 < 45 \cdot R_x[0]$), không chia float — twin phải cùng quy tắc.

8.7. Bài tập và lời giải giải thích

Bài tập 8.1. $w = [1, 2, 1]$: tính $C[2]$.

Bài tập 8.2. Hex Q3.13 cho $C = 1{,}5$: giá trị nguyên và mã hex?

Bài tập 8.3. Twin dùng floor thay round khi lượng tử C: lệch tối đa bao nhiêu LSB trước khi cắt?

Bài tập 8.4. Quên nhân C: đỉnh τ lớn bị thiên lệch thế nào?

Bài tập 8.5. Liệt kê ba tham số IP ROM bắt buộc: width, depth, init file. Khi nào cân nhắc LPM_DIVIDE thay ROM? Vẽ timeline 2 chu kỳ đọc registered.

Lời giải giải thích.

1. $\mathcal{R}_w[0] = 1 + 4 + 1 = 6$, $\mathcal{R}_w[2] = 1 \cdot 1 = 1$, $C[2] = 6$.
2. $1{,}5 \cdot 8192 = 12288 =$`0x3000`.
3. Floor thay round lệch tối đa $0{,}5$ LSB của thang Q3.13 trước khi ép kiểu nguyên.
4. Quên C: đỉnh τ lớn bị đạp bởi hình học cửa sổ, thuật toán thiên về τ nhỏ (giọng cao giả).
5. Width 16, depth 189 (hoặc $\tau_{\max} - \tau_{\min} + 1$), init `c_tau_rom.hex`. LPM_DIVIDE chỉ khi $\mathcal{R}_w$ đổi lúc chạy; đường mặc định pitch không chia mỗi τ. Timeline: ck0 đặt địa chỉ; ck1 C_q hợp lệ — mới được nhân với R_x.

Chương 9. Peak, parabolic và $F_0 = f_s/\tau$ (LPM_DIVIDE)

▶ MỤC TIÊU HỌC TẬP

Sau chương này bạn giải thích được vì sao đỉnh τ trên lưới mẫu nguyên chưa đủ cho độ phân giải cao độ; dẫn ra và tính tay nội suy parabolic ba điểm với mọi trung gian; cấu hình LPM_DIVIDE (pipeline từng tầng) hoặc divider non-restoring tự viết để lấy $F_0 = f_s/\tau_{\text{fine}}$; đóng gói kết quả dạng Q12.4; viết twin chờ đúng latency chia; và đọc được từng dòng hộp chi phí silicon kèm lý do chọn parabolic thay sinc.

9.1. Hiện tượng thật: đỉnh tự tương quan = chu kỳ đóng/mở thanh môn

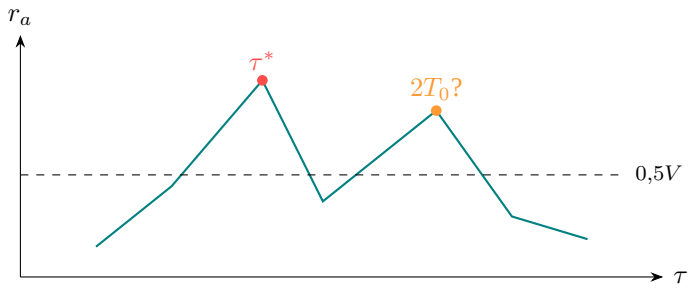
Sau chuẩn hóa Boersma (Chương 8), điểm số $S[\tau]$ (hay $r_a[\tau]$) gần thang $[0, 1]$ đo mức "khung giống chính mình" tại trễ τ. Với dây thanh gần tuần hoàn, đỉnh lớn nhất trong $[\tau_{\min}, \tau_{\max}]$ ước lượng T_0 tính bằng số mẫu. **Argmax** là phép tìm chỉ số τ^* cho $S[\tau^*]$ lớn nhất — thuật ngữ tính toán, nhưng nó bám sát hiện tượng vật lý: khoảng thời gian giữa hai lần đóng thanh môn.

Trên thiết kế này, mảng điểm số có 189 ô cho $\tau = 26 \dots 214$, nhưng argmax bị hạn chặt vào $[27, 213]$ (187 ứng viên) để τ^* luôn còn hai láng giềng $\tau^* \pm 1$ phục vụ nội suy. Ngưỡng voicing Praat mặc định $V = 0{,}45$: nếu đỉnh tốt nhất $< 0{,}45 \cdot \mathcal{R}_x[0]$ (so sánh nguyên: best $\cdot\, 100 < 45 \cdot R_x[0]$) thì khung vô thanh, $F_0 = 0$. So sánh nguyên tránh chia float trên silicon và khớp twin bit-exact.

Vì sao không dừng ở τ^* nguyên? Vì $F_0 = f_s/\tau$: một mẫu lệch ở $\tau = 80$ đổi F_0 khoảng $16000/80 - 16000/81 \approx 2{,}5$ Hz — với giọng nữ đó đã là vài chục cent, đủ fail nghiệm thu Chương 13. Nội suy dưới mẫu kéo sai số xuống dưới 1 Hz điển hình với chi phí vài phép cộng/trừ. Parabolic giả định đỉnh đơn cục bộ sắc; nếu sóng bão hòa cắt ngọn, đỉnh $\mathcal{R}_x$ phẳng và Δ nhảy loạn — khi đó kiểm tra cửa sổ (Chương 6) trước khi đổ lỗi cho divider.

▶ NHÌN THẤY VẬT LÝ

Đỉnh tại $\tau = 2T_0$ luôn tồn tại vì sóng chu kỳ T_0 cũng chu kỳ $2T_0$. Nếu biên độ đỉnh bội bị nhiễu đẩy cao hơn đỉnh chính, thuật toán báo $F_0/2$ — **sai quãng tám (octave error)**. Octave-cost trong ROM điểm số (Chương 8) và cổng voicing (Chương 10) giảm rủi ro này; argmax một mình không đủ. Trên tiếng Việt, sai quãng tám biến thanh sắc thành "trầm giả" và phá tương quan Pearson với Praat.



Hình 37: Argmax chọn đỉnh cao nhất trên $[\tau_{\min}, \tau_{\max}]$; đỉnh bội có thể còn nhưng thường thấp hơn sau chuẩn hóa và octave-cost.

9.2. Nội suy parabolic — từng ký hiệu, không nhảy bước

$\mathcal{R}(\tau)$ (hay $r_a(\tau)$) là hàm của biến liên tục τ, nhưng phần cứng chỉ có mẫu tại số nguyên. Gần một đỉnh đơn, đồ thị trông giống một **parabola** $y = ax^2 + bx + c$. Khớp ba điểm r_-, r^*, r_+ tại $x = -1, 0, +1$:

$$
\begin{aligned}
r_- &= a - b + c, \\
r^* &= c, \\
r_+ &= a + b + c.
\end{aligned}
$$

Trừ hai phương trình biên: $r_- - r_+ = -2b \Rightarrow b = (r_+ - r_-)/2$. Cộng rồi trừ $2r^*$: $r_- + r_+ - 2r^* = 2a \Rightarrow a = (r_- - 2r^* + r_+)/2$. Đỉnh parabola tại $x = -b/(2a)$:

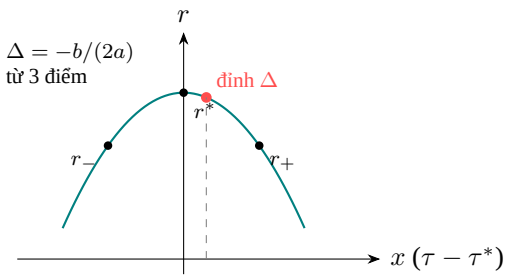
$$\Delta = \frac{1}{2} \cdot \frac{r_- - r_+}{r_- - 2r^* + r_+}, \qquad \tau_{\text{fine}} = \tau^* + \Delta, \qquad F_0 = \frac{f_s}{\tau_{\text{fine}}}.$$

- $r_- = r_a[\tau^* - 1]$, $r^* = r_a[\tau^*]$, $r_+ = r_a[\tau^* + 1]$.
- Mẫu số $r_- - 2r^* + r_+$ xấp xỉ đạo hàm hai (độ cong); phải < 0 tại đỉnh thật (cong xuống). Nếu > 0 đó là đáy — RTL phải bắt.
- $\Delta > 0$ nghĩa là đỉnh hơi lệch phải; $\Delta < 0$ lệch trái.
- Trên silicon fixed-point (Mode B trong golden): $N = S_- - S_+$, $D = 2(S_- - 2S^* + S_+)$,

  $$\tau_{\text{fine,Q16}} = (\tau^* \ll 16) + ((N \ll 16)//D),$$

  rồi chia F_0 bằng số nguyên có làm tròn.

Sai số điển hình $\ll 0{,}5$ mẫu trên đỉnh sắc — đủ cho lab pitch trước khi nghĩ đến sinc (Chương 0 đã so sánh Sinc70/Sinc700 đắt hàng trăm nhân).



Hình 38: Nội suy parabolic: khớp ba điểm r_-, r^*, r_+ bằng $y = ax^2 + bx + c$, lấy đỉnh tại $x = \Delta$. $\Delta > 0$ nghĩa là chu kỳ thật hơi dài hơn τ^* nguyên.

9.3. Ví dụ tính tay — mọi số đã chạy Python

A. Argmax có ngưỡng — từng quyết định. Các cực đại cục bộ $(80, 0{,}50)$, $(160, 0{,}47)$, $(40, 0{,}30)$; $V = 0{,}45$. So $0{,}30$ với $0{,}45$: loại 40. Còn 80 và 160: $0{,}50 > 0{,}47$ nên $\tau^* = 80$. Nếu hai đỉnh bằng nhau, quy ước lab lấy τ nhỏ hơn (cao độ cao hơn) — ghi rõ trong twin.

B. Parabolic và Q12.4 — mọi trung gian. Với $r_- = 0{,}40$, $r^* = 0{,}50$, $r_+ = 0{,}46$, $\tau^* = 80$, $f_s = 16\,000$:

$$\begin{aligned} r_- - r_+ &= 0{,}40 - 0{,}46 = -0{,}06, \\ r_- - 2r^* + r_+ &= 0{,}40 - 1{,}00 + 0{,}46 = -0{,}14, \\ \Delta &= \tfrac{1}{2} \cdot \frac{-0{,}06}{-0{,}14} = \tfrac{1}{2} \cdot \frac{3}{7} = \frac{3}{14} \approx 0{,}214285714. \end{aligned}$$

$\tau_{\text{fine}} = 80 + 3/14 \approx 80{,}2142857$.

$$F_0 = \frac{16000}{80{,}2142857} \approx 199{,}4657\,\text{Hz}.$$

Đóng gói **Q12.4** (12 bit phần nguyên, 4 bit thập phân; $1\,\text{Hz} \mapsto 16$):

$$F_{0,\text{Q12.4}} = \text{round}(199{,}4657 \cdot 16) = \text{round}(3191{,}451) = 3191.$$

Bảng quét thêm (script `verify/verify_chuong_06_09.py`):

y_-	y_*	y_+	Δ	τ_{fine}	F_0 @16 kHz / Q12.4
0.40	0.50	0.46	0.214286	80.214286	199.4657 / 3191
0.30	0.55	0.50	0.333333	100.333333	159.4684 / 2551
0.48	0.50	0.40	−0.333333	119.666667	133.7047 / 2139
0.20	0.60	0.20	0.000000	64.000000	250.0000 / 4000

C. Phép chia $13 \div 3$ (minh họa non-restoring 4-bit) — từng bước. Dividend $D = 13 = 1101_2$, divisor $V = 3 = 0011_2$. Non-restoring 4 bước (ý tưởng):

Bước k	Thao tác	Dư gần đúng	Bit thương
1	dịch, thử trừ V	dương $\rightarrow$ giữ trừ	1
2	dịch, trừ/cộng theo dấu	…	…
3	…	…	…
4	kết thúc	dư $= 1$	thương $= 4$

Vì $3 \cdot 4 + 1 = 13$. Testbench bắt buộc: assert $13/3 = 4$, $16000/80 = 200$.

9.4. Ý nghĩa phần cứng — LPM_DIVIDE pipeline và non-restoring

Trên laptop, $16000/80$ là một lệnh. Trên Cyclone V, chia nguyên thường làm bằng IP **LPM_DIVIDE** (pipeline cố định L chu kỳ) hoặc FSM **non-restoring** tự viết (khoảng W chu kỳ cho thương W bit). Latency là hợp đồng: twin và testbench phải chờ đúng số cạnh clock trước khi đọc quotient. Lệch một chu kỳ gắn F_0 khung trước vào khung sau — lỗi thời gian, không phải lỗi toán.

LPM_DIVIDE — tham số và pipeline từng tầng.

- `LPM_WIDTHN`: numerator (vd 32-bit mang f_s đã scale, hoặc $16000 \cdot 16 \cdot 65536$ theo Mode B).
- `LPM_WIDTHD`: denominator τ_{fine} (vd 16–32 bit Q-format).
- `LPM_PIPELINE`$= L$: IP chèn L hàng thanh ghi; mỗi chu kỳ dữ liệu tiến một tầng.
- `LPM_NREPRESENTATION`/`LPM_DREPRESENTATION`: unsigned khuyến nghị cho F_0 dương.

Bảng timing LPM_DIVIDE ($L = 8$ minh họa).

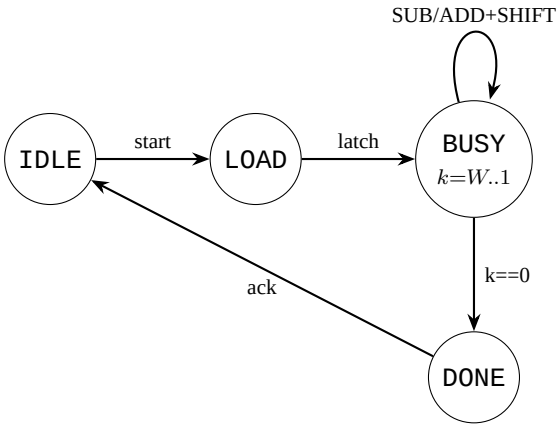
ck	Sự kiện	Ghi chú
0	nạp `numer`=fs_scaled, `denom`=τ_q	sau parabolic xong
1–7	dữ liệu trong pipeline	**không** đọc quotient
8	`quotient` $= F_{0,\text{Q12.4}}$ thô	chốt vào thanh ghi xuất
9	`valid` lên Avalon-ST	Chương 11/12 đóng gói

Golden Mode B:

$$f0_{\text{Q12.4}} = (16000 \cdot 16 \cdot 65536 + (\tau_{\text{fine,Q16}} \gg 1))//\tau_{\text{fine,Q16}}$$

(chia có làm tròn bằng cộng nửa mẫu số).

Non-restoring — trực giác và FSM. **Restoring division** trừ số chia khỏi dư; nếu kết quả âm thì cộng lại ("restore") rồi ghi bit thương 0. **Non-restoring** bỏ bước hoàn tác: nếu dư âm thì ở chu kỳ sau *cộng* số chia và ghi bit 0; nếu dư dương thì trừ và ghi bit 1. Ít multiplex hơn, phù hợp RTL thuần ALM trên Cyclone V. FSM bốn trạng thái: IDLE (chờ start) $\rightarrow$ LOAD (chốt dividend/divisor) $\rightarrow$ BUSY (W vòng dịch + trừ/cộng, mỗi vòng một bit thương) $\rightarrow$ DONE (giữ quotient đến khi ack).



Hình 39: FSM divider non-restoring: IDLE→LOAD→BUSY (W vòng)→DONE. LPM_DIVIDE ẩn FSM này trong IP, nhưng latency vẫn phải khớp twin.

RTL bọc LPM_DIVIDE — từng dòng ý tưởng.

```
// Sau tau_fine_q16 và fs=16000:
// start_div pulse -> chờ L chu kỳ -> f0_quot hợp lệ (Q12.4)
lpm_divide #(.LPM_WIDTHN(32), .LPM_WIDTHD(16), .LPM_PIPELINE(L))
  u_div (.clock(clk), .numer(fs_scaled), .denom(tau_q),
         .quotient(f0_quot));
// FSM ngoài: đếm L cạnh sau start; chỉ khi cnt==L mới assert out_valid
```

Không tồn tại IP `ALTDIV_TENT`. Chỉ LPM_DIVIDE hoặc RTL non-restoring.

Bảng timing argmax tuần tự (187 ứng viên).

ck	Việc làm	Thanh ghi
0	$\tau = 27$, `best`=S[27], $\tau^* = 27$	khởi tạo
1	so S[28] với best; cập nhật nếu lớn hơn	…
⋮	quét đến 213	…
186	xong argmax	τ^*, best
187+	đọc S[$\tau^* \pm 1$], parabolic, start chia	

9.4b. Mode B fixed-point — từng phép trên silicon

Golden `lab_1_3_5.py` không chia float trên đường chính. Sau khi có S_-, S^*, S_+ (int sau nhân C):

1. $N \leftarrow S_- - S_+$ (có dấu).
2. $D \leftarrow 2(S_- - 2S^* + S_+)$ — mẫu số cong; bắt buộc $D < 0$ tại đỉnh.
3. $\Delta_{\text{Q16}} \leftarrow (N \ll 16)//D$ (chia nguyên có dấu, làm tròn tùy golden).
4. $\tau_{\text{fine,Q16}} \leftarrow (\tau^* \ll 16) + \Delta_{\text{Q16}}$.
5. $f0_{\text{Q12.4}} \leftarrow (16000 \cdot 16 \cdot 65536 + (\tau_{\text{fine,Q16}} \gg 1))//\tau_{\text{fine,Q16}}$.

Mỗi phép $//$ trên RTL là một lần LPM_DIVIDE hoặc một vòng non-restoring. Có thể tái sử dụng cùng IP chia hai lần tuần tự (parabolic rồi F_0) nếu FSM cho phép — tiết kiệm một instance IP với giá latency cộng dồn.

Ví dụ số Mode B minh họa với thang đơn giản. Giả sử sau scale $S_- = 400$, $S^* = 500$, $S_+ = 460$, $\tau^* = 80$ (cùng tỉ lệ với ví dụ float 0.40/0.50/0.46):

$$N = 400 - 460 = -60,$$
$$D = 2(400 - 1000 + 460) = 2(-140) = -280,$$
$$\Delta_{\text{float}} = 0{,}5 \cdot (-60)/(-140) = 3/14.$$

Twin float và Mode B phải hội tụ trong sai số làm tròn cố định — script khóa Q12.4 $= 3191$.

9.4c. So sánh parabolic với sinc (nhắc Chương 0)

Sinc nội suy lý tưởng cần nhiều hệ số (Sinc70, Sinc700 trong Praat đầy đủ). Trên streaming FPGA với Viterbi tắt, chi phí hàng trăm nhân/đỉnh không đáng khi parabolic đã giữ sai số dưới khoảng 2 cent trên đỉnh sắc. Lab nâng cao có thể thêm FIR nội suy sau này; đường mặc định gói này dừng ở parabolic ba điểm + một chia.

Khi đỉnh phẳng (ba điểm gần bằng nhau), mẫu số $D \to 0$ — parabolic không ổn định. RTL phải: (a) nếu $|D|$ dưới ngưỡng thì lấy $\Delta = 0$; (b) hoặc dựa voicing để đánh vô thanh. Twin phải cùng ngưỡng — đây là chỗ hay lệch bit nếu quên.

9.5. Twin Python bit-exact

```
import numpy as np

def parabolic_delta(ym1, y0, yp1):
    denom = ym1 - 2.0 * y0 + yp1
    return 0.5 * (ym1 - yp1) / denom

def f0_q12_4(fs, tau_fine):
    return int(round(fs / float(tau_fine) * 16))

d = parabolic_delta(0.40, 0.50, 0.46)
assert abs(d - 3/14) < 1e-12
assert f0_q12_4(16000, 80 + d) == 3191
# TB: sau đúng L posedge, so quotient với twin
```

Giải thích: mẫu số `denom` phải khác 0 và âm tại đỉnh; twin float chỉ để khóa công thức — Mode B int trên RTL mới là hợp đồng silicon. Định dạng xuất (Chương 12): F_0 Q12.4 đóng gói `uint16` little-endian; 0 = vô thanh; trần biểu diễn $4095{,}9375$ Hz.

9.5b. Hợp đồng testbench với latency chia

Một testbench tối thiểu phải:

1. Nạp ba điểm S đã biết $\rightarrow$ kỳ vọng τ_{fine} và $F_{0,\text{Q12.4}}$ từ twin.
2. Pulse `start_div`; đếm đúng L cạnh `posedge clk`.
3. Chỉ khi `cnt==L` mới so `quotient` với twin — so sớm hơn là fail giả.
4. Thêm case vô thanh: best dưới ngưỡng $\rightarrow$ `f0==0`, không cần chờ chia (hoặc chia nhưng bỏ quotient).
5. Case mẫu số $|D|$ nhỏ: $\Delta = 0$, $F_0 = \text{round}(f_s/\tau^* \cdot 16)$.

Trên board DE10-Standard, đo latency thực bằng SignalTap: trigger `start_div`, probe `out_valid`. Nếu khoảng cách cạnh khác L đã khai báo, sửa RTL và twin cùng lúc — không vá delay tùy hứng trong FSM top.

Khi đóng gói Q12.4 ra HPS (Chương 12), nhớ **little-endian** uint16: byte thấp ghi trước (ví dụ $3191 = 0x0C77$ thành các byte `77 0C`). Đọc nhầm big-endian đảo thành $0x770C = 30476$ — divider đúng nhưng file sai. Twin xuất `pitch_f0.bin` cùng endian để so dump.

9.6. Chi phí silicon

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách peak + parabolic + divider.

- **Argmax** quét ≈ 187 giá trị: vài ALM + comparator, 0 DSP, 0 M10K — ≈ 187 chu kỳ.
- **Parabolic**: 2 trừ + 1 chia ngắn + nhân $1/2$ — 0–1 DSP hoặc soft logic, vài chu kỳ.
- **LPM_DIVIDE** 32/16 pipeline: IP dùng ALM (và tùy chọn DSP); *nút thắt là latency L*, không phải diện tích (chỉ 1 phép/khung). Với $L = 16$, thêm 16 chu kỳ — không đáng kể so với 98 280 MAC.
- **Non-restoring** tự viết W-bit: $\approx W$ chu kỳ, thuần ALM, dễ twin từng bước.
- **M10K**: 0 cho khối này (trừ khi đệm kết quả nhiều khung).
- Vì sao parabolic + 1 chia/khung: sai số $<$ 2 cent điển hình (Chương 0); Sinc70/Sinc700 đắt hàng trăm phép nhân/đỉnh trong khi Viterbi đã tắt trên đường streaming. Vì sao không chia float mềm: một LPM_DIVIDE cố định latency dễ FSM hơn FPU.

Tổng Pha peak+div thường < 300 chu kỳ/khung — dưới $0{,}1\%$ hop 10 ms @ 50 MHz. Nút thắt vẫn là MAC Chương 7. Lab nghiệm thu: (1) parabolic float khớp bảng 9.3; (2) Mode B int khớp golden; (3) testbench chờ đúng L ck trước khi đọc quotient.

Ví dụ tính tay — debug theo lớp, không theo tai. Giả sử dump Q12.4 $= 3191$ ($199{,}4375$ Hz) nhưng Praat báo ≈ 100 Hz. Đừng nghe tai trước: kiểm tra lần lượt (a) τ^* có phải ≈ 160 thay vì 80 không (sai quãng tám / quên C); (b) τ_{fine} có khớp parabolic twin không; (c) quotient có được chốt đúng sau L cạnh không; (d) file `pitch_f0.bin` có little-endian uint16 không — đảo byte biến $3191 = 0x0C77$ thành $0x770C = 30476$ "trông hợp lệ" trong hex viewer nhưng vô nghĩa. Script khóa các số (a)–(b); (c)–(d) là hợp đồng testbench và đóng gói Chương 12.

9.7. Bài tập và lời giải giải thích

Bài tập 9.1. $r = [0{,}2,\ 0{,}6,\ 0{,}5]$ tại $\tau = 100, 101, 102$: tính Δ, τ_{fine}, F_0 @ 16 kHz, mã Q12.4.

Bài tập 9.2. Testbench: $16000/80 = 200$. Cố ý đọc quotient sớm 1 chu kỳ — quan sát triệu chứng gắn nhầm khung.

Bài tập 9.3. Với bộ $(0{,}40, 0{,}50, 0{,}46)$, so Δ parabolic với ý nghĩa "đỉnh lệch phải". Nếu mẫu số $r_- - 2r^* + r_+ > 0$, đó có còn là đỉnh không?

Bài tập 9.4. LPM_DIVIDE pipeline $L = 8$. Vẽ timeline từ khi parabolic xong đến `out_valid`. Nếu Fmax tụt buộc tăng L lên 16, RTF đổi bao nhiêu trên hop 10 ms?

Bài tập 9.5. Giải thích vì sao argmax chỉ chạy $[27, 213]$ dù ROM có $\tau = 26$ và 214.
Lời giải giải thích.

1. Đỉnh nguyên tại $\tau^* = 101$ (0,6 lớn nhất). $\Delta = 0{,}5 \cdot (0{,}2 - 0{,}5)/(0{,}2 - 1{,}2 + 0{,}5) = 0{,}5 \cdot (-0{,}3)/(-0{,}5) = 0{,}3$. $\tau_{\text{fine}} = 101{,}3$, $F_0 \approx 157{,}9467$ Hz, Q12.4 $= 2527$ (script assert).
2. Đọc sớm 1 ck lấy thương của lần chia trước (hoặc giá trị chưa ổn định). Triệu chứng điển hình: đường F_0 "trễ một hop" so với twin, cent error nhảy có hệ thống theo thời gian chứ không theo biên độ.
3. $\Delta = +3/14 > 0$: đỉnh thật hơi bên phải τ^*. Nếu mẫu số > 0 thì độ cong hướng lên — đó là đáy (cực tiểu), không được lấy làm chu kỳ dây thanh; RTL phải kiểm tra mẫu số < 0 hoặc dựa vào argmax đã chọn cực đại.
4. Thêm 8 chu kỳ pipeline: $+8/500000$ trên hop — RTF tăng không đáng kể ($\ll 0{,}001$). Tăng L không phải lý do fail thời gian thực; fail thường do quên chờ L.
5. Cần $\tau^* \pm 1$ luôn nằm trong mảng điểm số. Nếu cho phép $\tau^* = 26$, thiếu r_-; $\tau^* = 214$ thiếu r_+. Vì vậy biên argmax thụt vào một mẫu mỗi phía.

Chương 10. Quyết định voicing, chi phí rút gọn và median-3 trên FPGA

▶ MỤC TIÊU HỌC TẬP

Sau khi học xong Chương 10, người học sẽ:

1. Giải thích được hiện tượng vật lý khi dây thanh quản ngừng rung (im lặng, phụ âm vô thanh, hơi thở) và vì sao vẫn báo F_0 trong các khung đó sẽ phá đường cong cao độ.
2. Viết và tính tay được cổng voicing/silence từ $r_a[\tau^*]$ và tỷ số năng lượng $P_{\text{local}}/P_{\text{global}}$, cùng ứng viên Unvoiced strength s_{UV} đúng công thức Praat trong `Pitch.cpp`.
3. Triển khai bộ lọc trung vị ba khung (median-3) có thanh ghi xuất trễ đúng một bước nhảy $H = 160$ mẫu (≈ 10 ms), và chứng minh nó đập được một xung nhiễu đơn mà không cần quy hoạch động Viterbi.
4. Tính tay chi phí chuyển trạng thái octave-jump / V↔UV và một bảng Viterbi 2×2 (hoặc 3 khung) để hiểu `Pitch_pathFinder`, rồi nêu rõ vì sao mặc định trên fabric Cyclone V **tắt** Viterbi và chỉ giữ soft-float trên HPS khi cần đường cong "Praat đầy đủ".
5. Đọc hộp chi phí silicon (ALM, DSP, M10K, latency, RTF) của khối voicing + median-3 và đối chiếu với ngân sách Chương 0.

10.1. Hiện tượng thật: dây thanh không rung liên tục

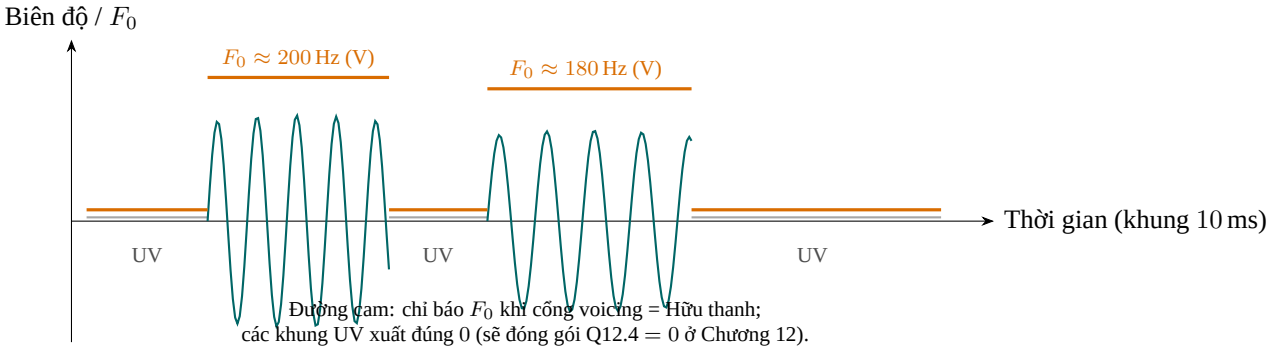
★ NGUYÊN LÝ ĐẦU TIÊN: Khi nào cao độ phải bằng không?

Trong Chương 0 và Chương 1 ta đã thấy $F_0 = 1/T_0$ là tần số đóng/mở thanh môn khi dây thanh rung tuần hoàn. Nhưng trong một câu nói thật, dây thanh chỉ rung trong các đoạn **hữu thanh** (nguyên âm, phụ âm hữu thanh). Ba tình huống sau *không* có chu kỳ dây thanh ổn định:

- **Im lặng / hơi thở:** năng lượng khung rất nhỏ; hàm tự tương quan gần như nhiễu trắng, không có đỉnh r_a nổi.
- **Phụ âm vô thanh** (ví dụ $/s/$, $/f/$): có năng lượng cao tần nhưng không có chu kỳ T_0 của thanh môn.
- **Khoảng chuyển tiếp ngắn:** đỉnh r_a yếu, dễ bị nhiễu đẩy sang đỉnh giả.

Nếu phần cứng vẫn xuất một số Hz "bịa" trong các khung đó, đường cong cao độ sẽ nhảy lung tung và mọi chỉ số đối sánh với Praat (GPE, VDE ở Chương 13) đều xấu. Praat giải quyết bằng hai tầng: (1) cổng silence/voicing + ứng viên Unvoiced ngay trên từng khung; (2) tuỳ chọn thuật toán quy hoạch động `Pitch_pathFinder` (Viterbi) với các chi phí octave và V↔UV. Trên FPGA DE10-Standard, thiết kế mặc định giữ tầng (1) cộng bộ lọc median-3 trên fabric, và **tắt** tầng (2) khỏi silicon logic — chuyển sang HPS khi thật sự cần.

Hình 40 minh họa một đoạn sóng có nguyên âm xen khoảng lặng: chỉ các khung hữu thanh mới được phép mang $F_0 > 0$.



Hình 40: Trực giác voicing: sóng có đoạn rung và đoạn im. Cổng voicing buộc $F_0 = 0$ trên khung vô thanh/im lặng trước khi median-3 và đóng gói.

10.2. Toán từng bước: cổng voicing và ứng viên Unvoiced

Sau Chương 8–9 ta đã có đỉnh chuẩn hóa $r_a[\tau^*] \in [0, 1]$ và (tuỳ chọn) F_0 nội suy. Praat không chỉ nhìn r_a: nó còn so sánh **đỉnh biên độ cục bộ** của khung với một **đỉnh toàn cục chậm**.

Định nghĩa ký hiệu.

- $r_a[\tau^*]$: độ mạnh tự tương quan tại đỉnh tốt nhất sau khi nhân ROM $C[\tau]$ (Chương 8).
- $V = \texttt{voicingThreshold} \approx 0{,}45$: ngưỡng sàn để một đỉnh còn được coi là ứng viên hữu thanh.
- $S = \texttt{silenceThreshold} \approx 0{,}03$: ngưỡng tỷ số năng lượng để nghi ngờ im lặng.
- $P_{\text{local}} = \max_n |x[n]|$ trên khung hiện tại (hoặc $\sqrt{\mathcal{R}_x[0]/L}$ — lab dùng max-abs cho rẻ silicon).
- P_{global}: ước lượng đỉnh biên độ của cả phát ngôn. Offline Praat lấy max cả file; streaming FPGA dùng **max-hold** hoặc IIR chậm vì không nhìn trước cả câu (Chương 0 đã nhấn mạnh ràng buộc streaming).

Cổng hữu thanh rút gọn trên fabric.

$$\text{voiced} = \big(r_a[\tau^*] \geq V\big) \ \wedge \ \big(P_{\text{local}} \geq S \cdot P_{\text{global}}\big). \tag{36}$$

Hai điều kiện đọc bằng lời: (1) đỉnh tương quan đủ cao; (2) khung không quá nhỏ so với đỉnh toàn cục. Nếu một trong hai sai, xuất $F_0 = 0$.

Ứng viên Unvoiced strength (đầy đủ Praat). Trong `Pitch.cpp`, Praat còn tạo một "ứng viên vô thanh" cạnh tranh với mọi đỉnh hữu thanh:

$$s_{\text{UV}} = V + \max\!\Big(0,\ 2 - \frac{P_{\text{local}}/P_{\text{global}}}{S/(1+V)}\Big). \tag{37}$$

Khi khung gần im lặng, phân số năng lượng nhỏ, hạng $\max(0, \ldots)$ dương lớn, nên s_{UV} thắng mọi đỉnh r_a yếu và pathFinder (nếu bật) chọn Unvoiced. Trên fabric mặc định, công thức (37) được dùng để *hiểu* và để twin Python đối chiếu Praat; RTL tối thiểu chỉ cần (36).

▶ NHÌN THẤY VẬT LÝ

V và S là **ngưỡng quy ước tính toán**, không phải hằng số vật lý của dây thanh. Đổi V từ $0{,}45$ xuống $0{,}30$ sẽ khiến nhiều khung nhiễu bị gắn nhầm hữu thanh (VDE tăng ở Chương 13). Đổi S quá cao sẽ nuốt cả nguyên âm nhỏ lúc đầu câu khi P_{global} chưa kịp warmup.

10.3. Ví dụ tính tay — voicing và s_{UV} (đã kiểm Python)

Ngưỡng Q1.14. Lab lưu r_a dạng Q1.14 (15 bit có dấu dùng 14 bit frac). Đổi $V = 0{,}45$ sang số nguyên:

$$V_{\text{int}} = \lfloor 0{,}45 \cdot 2^{14} \rfloor = \lfloor 0{,}45 \cdot 16384 \rfloor = 7372.$$

(Tập lệnh `verify/verify_chuong_10_14.py` khẳng định đúng 7372.)

Khung sine hữu thanh. Cho $r_a = 12000$ (tức $12000/16384 \approx 0{,}732$), $P_{\text{local}} = 3277$, $P_{\text{global}} = 30000$, $S = 0{,}03$:

$$S \cdot P_{\text{global}} = 0{,}03 \cdot 30000 = 900.$$

Vì $12000 \geq 7372$ và $3277 \geq 900$, cổng (36) cho **voiced** $= 1$.

Khung gần im lặng. Cùng $r_a = 12000$ (nhiễu có thể tạo đỉnh giả cao!) nhưng $P_{\text{local}} = 200$:

$$200 < 900 \quad \Rightarrow \quad \text{voiced} = 0.$$

Đây là lý do bắt buộc có nhánh silence: chỉ nhìn r_a sẽ bị đánh lừa.

Tính s_{UV}. Đặt $P_{\text{local}} = 0{,}02$, $P_{\text{global}} = 1$, $S = 0{,}03$, $V = 0{,}45$:

$$\frac{S}{1+V} = \frac{0{,}03}{1{,}45} \approx 0{,}0206897, \qquad \frac{P_{\text{local}}/P_{\text{global}}}{S/(1+V)} = \frac{0{,}02}{0{,}0206897} \approx 0{,}96667.$$

$$s_{\text{UV}} = 0{,}45 + \max(0,\ 2 - 0{,}96667) = 0{,}45 + 1{,}03333 = 1{,}48333.$$

Một đỉnh $r_a = 0{,}30$ thua hoàn toàn ứng viên Unvoiced $\Rightarrow F_0 = 0$. Nếu $P_{\text{local}} = P_{\text{global}}$ thì hạng max tắt và $s_{\text{UV}} = V = 0{,}45$: silence không kích.

10.4. Median-3: làm sạch xung đơn với trễ đúng một hop

★ NGUYÊN LÝ ĐẦU TIÊN: Vật lý / trực giác: lấy giá trị "ở giữa" ba khung

Giọng người không đổi cao độ một quãng tám rồi quay lại trong đúng 10 ms. Một xung F_0 đơn (do nhiễu thắng argmax một khung) gần như chắc là lỗi. Bộ lọc **trung vị ba điểm (median-3)** lấy giá trị nằm giữa của ba mẫu liên tiếp: xung lệch bị loại, còn đường cong đơn điệu được giữ. Đây là thao tác thống kê trên chuỗi F_0, không phải hiện tượng khí động học mới — nhưng nó bám sát ràng buộc vật lý "F_0 người biến thiên chậm".

Với ba số a, b, c, một công thức chỉ dùng min / max (thuận RTL, không cần sắp xếp đầy đủ) là:

$$\text{med}(a,b,c) = \max\big(\min(a,b),\ \min\big(\max(a,b),c\big)\big). \tag{38}$$

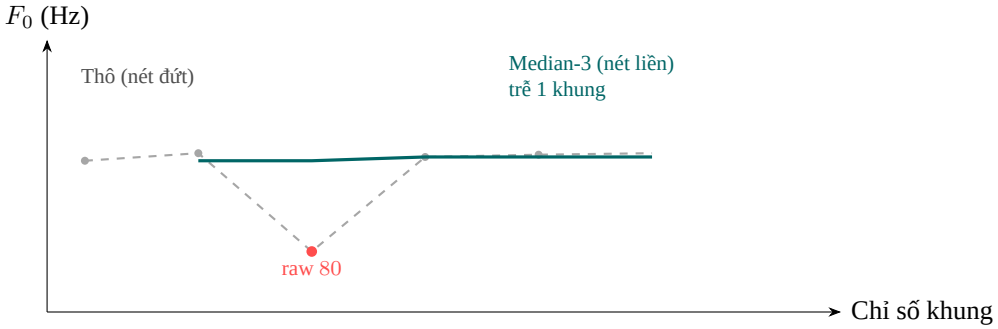
Trên silicon, ba thanh ghi trượt $f_1 \leftarrow f_2 \leftarrow f_3 \leftarrow F_0^{(\text{mới})}$; mỗi khi `in_valid`, tính med rồi **chốt vào** `m_reg`. Ngõ ra `f_out` chỉ cập nhật từ `m_reg` ở nhịp `in_valid` *kế tiếp* — tức trễ nhóm đúng một bước nhảy $H = 160$ mẫu ≈ 10 ms (đúng như Feynman box 11 ở Chương 0).

Ví dụ tính tay. Chuỗi cao độ thô (Hz): $200,\ 210,\ 80,\ 205,\ 208$. Các cửa sổ ba điểm:

Cửa sổ (a,b,c)	Sắp xếp	med	Ý nghĩa
$(200, 210, 80)$	$80, 200, 210$	200	Đập spike 80
$(210, 80, 205)$	$80, 205, 210$	205	
$(80, 205, 208)$	$80, 205, 208$	205	

Spike 80 Hz (thường là bắt nhầm $2T_0$ hoặc nhiễu) biến mất khỏi chuỗi xuất. Kiểm chứng bằng script `verify/verify_chuong_10_14.py`.

Bài tập nhỏ trong bài. Với một cửa $(100, 400, 110)$, med $= 110$ (sắp $100, 110, 400$). Xung 400 Hz (octave nhảy giả) bị loại; kết quả *không* phải 100 — đó là min, không phải median.



Hình 41: Median-3 đập một xung thấp đơn; đường xuất trễ đúng một khung so với quyết định voicing tức thời.

10.5. Chi phí rút gọn và Viterbi — hiểu để biết vì sao tắt trên fabric

Praat gắn mỗi ứng viên một độ mạnh $\delta(t,j)$ (đã trừ `octaveCost`), rồi tìm đường cực đại:

$$V(t,j) = \delta(t,j) + \max_i \big[V(t-1,i) - c_{ij}\big], \tag{39}$$

trong đó chi phí chuyển c_{ij} gồm:

- `octaveJumpCost`$\cdot|\log_2(f_j/f_i)|$ khi cả hai hữu thanh (mặc định $0{,}35$);
- `voicedUnvoicedCost` khi đổi V$\leftrightarrow$UV (mặc định $0{,}14$);
- nhân `timeStepCorrection`$= 0{,}01/\Delta t$ nếu đổi hop.

Ví dụ chi phí chuyển trạng thái.

Từ $\rightarrow$ Sang	f_1	f_2	c_{ij}
V$\rightarrow$V	150	160	$0{,}35 \cdot \lvert\log_2(150/160)\rvert \approx 0{,}03259$
V$\rightarrow$V	150	300	$0{,}35 \cdot 1 = 0{,}35$ (đúng một quãng tám — đắt)
V$\rightarrow$UV	150	0	$0{,}14$

Ví dụ Viterbi 2 khung $\times$ 2 ứng viên. Cho $\delta(t{=}1) = [V{:}0{,}50,\ \text{UV}{:}0{,}40]$, $\delta(t{=}2) = [V{:}0{,}45,\ \text{UV}{:}0{,}48]$, chi phí V–V $= 0{,}05$, V–UV $= 0{,}14$:

$$\text{cum}(V_2) = 0{,}45 + \max(0{,}50 - 0{,}05,\ 0{,}40 - 0{,}14) = 0{,}45 + 0{,}45 = 0{,}90,$$
$$\text{cum}(\text{UV}_2) = 0{,}48 + \max(0{,}50 - 0{,}14,\ 0{,}40 - 0) = 0{,}48 + 0{,}40 = 0{,}88.$$

Đường thắng là V$\rightarrow$V dù $\delta(\text{UV}_2)$ cao hơn $\delta(V_2)$ một chút — đúng tinh thần pathFinder: phạt chuyển trạng thái. (Phiên bản 3 khung với jump $0{,}02$ cho cum$(V_2) = 0{,}96 >$ cum$(\text{UV}_2) = 0{,}90$; xem script verify.)

Vì sao fabric tắt Viterbi? Độ phức tạp $O(TK^2)$ với K ứng viên/khung (Praat tới 15), cần lưu path metric và backpointer — tốn M10K và logic so sánh rộng. Lab chỉ giữ $K{=}1$ (argmax) + UV qua cổng (36) + median-3. Hai lý do không port `pathFinder` lên fabric ở lab này: (1) median-3 đã đập đủ spike đơn cho đo thanh điệu/Pearson trên giọng sạch; (2) HPS ARM Cortex-A9 có thể chạy soft-float pathFinder offline trên chuỗi F_0 thô nếu bài yêu cầu "Praat đầy đủ", không chiếm ALM thời gian thực.

10.6. Phần cứng: thuần ALM, không IP voicing

Không có Intel IP cho voicing hay median. Toàn bộ là so sánh và min / max trên Adaptive Logic Module (ALM).

```
// Voicing gate + registered median-3 (tre dung 1 hop)
// Gia su f0_in da la Hz nguyen hoac Q12.4 thô truoc pack.
reg [15:0] f1, f2, f3, m_reg, f_out;
reg        voiced_r, out_valid;
wire       voiced = (ra_q14 >= V_Q14) && (p_local >= silence_floor);
wire [15:0] med = max(min(f1,f2), min(max(f1,f2),f3));

always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    f1<=0; f2<=0; f3<=0; m_reg<=0; f_out<=0; out_valid<=0;
  end else if (in_valid) begin
    f1<=f2; f2<=f3;
    f3<= voiced ? f0_in : 16'd0;   // UV ep 0 truoc median
    m_reg <= med;                  // tinh tren cua so cu
    f_out <= m_reg;                // xuat ban chot truoc (= tre 1 khung)
    out_valid <= 1'b1;
  end else out_valid <= 1'b0;
end
```

Giải thích từng ý: (1) ép UV về 0 *trước* median để một khung vô thanh không bị "kéo" bởi hai hàng xóm hữu thanh thành một giá trị giả; (2) `m_reg` rồi `f_out <= m_reg` tạo pipeline trễ đúng một `in_valid`; (3) không dùng DSP, không dùng M10K.

10.7. Twin Python khớp hành vi

```
import numpy as np

def med3(a, b, c):
    return max(min(a, b), min(max(a, b), c))

def voicing_gate(ra, V, P_local, P_global, S=0.03):
    return (ra >= V) and (P_local >= S * P_global)

def s_uv(P_local, P_global, S=0.03, V=0.45):
    return V + max(0.0, 2.0 - (P_local / P_global) / (S / (1.0 + V)))

def median3_stream(f0, voiced, delay=True):
    """Ep UV=0, median-3, tre 1 khung neu delay=True."""
    x = [float(f if v else 0.0) for f, v in zip(f0, voiced)]
    y = []
    for i in range(len(x)):
        if i < 2:
            m = x[i]  # warmup: xuat thô / 0 tuy chinh lab
        else:
            m = med3(x[i-2], x[i-1], x[i])
        y.append(m)
    if delay and y:
        y = [0.0] + y[:-1]  # thanh ghi xuat tre 1
    return np.asarray(y)
```

Mọi số trong mục 10.3–10.5 được khẳng định bởi `verify/verify_chuong_10_14.py`.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — khối voicing + median-3 (giải thích từng mục).

- **ALM** (*Adaptive Logic Module*): đơn vị logic Cyclone V. Cổng voicing (hai so sánh + AND) $\approx$ 20–40 ALM; median-3 (min / max trên 16 bit + thanh ghi) $\approx$ 60–100 ALM; max-hold P_{global} vài ALM. Tổng khối này $\ll 150$ ALM — chỉ một phần nhỏ ngân sách ≈ 850 ALM của cả luồng F_0 ở Chương 0.
- **DSP / M10K:** 0. Không nhân, không bảng lớn.
- **Latency:** đúng 1 hop $= 10$ ms do thanh ghi xuất median. Không tăng RTF tính MAC (RTF vẫn do Chương 7 quyết định, $\approx 0{,}24$).
- **Viterbi trên fabric:** 0 ALM mặc định. Nếu bật đủ $K{=}15$, ước tính hàng nghìn ALM + vài M10K path metric — phá ngân sách 2% và không cần cho lab median-3.
- **Vì sao chọn median thay Viterbi trên fabric:** đập spike đơn đủ cho đo thanh điệu tiếng Việt và Pearson với Praat trên giọng sạch; giữ RTF và diện tích; vẫn để cửa HPS chạy pathFinder khi bài yêu cầu.

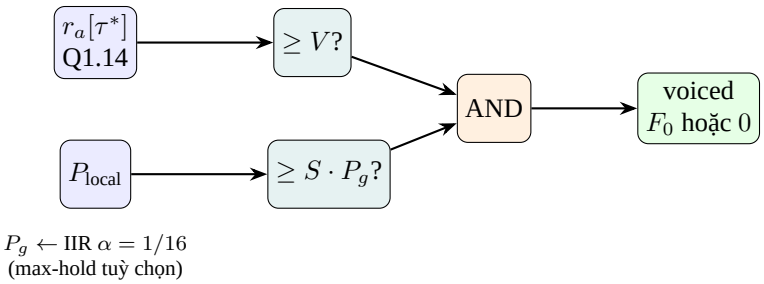
Ví dụ tính tay — warmup P_{global} bằng IIR chậm. Trên streaming FPGA không nhìn được cả câu nói trước, nên P_{global} được ước bằng bộ lọc IIR (*Infinite Impulse Response* — phản hồi vô hạn: lối ra phụ thuộc cả lối vào hiện tại và trạng thái nội đã nhớ). Ở đây dùng IIR bậc một với hệ số $\alpha = 1/16$ (dịch phải bốn bit, rất rẻ trên ALM):

$$P_g[n] = (1-\alpha)\,P_g[n-1] + \alpha\,P_{\text{local}}[n]. \tag{40}$$

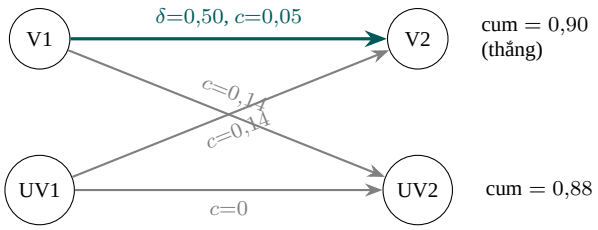
Cho chuỗi đỉnh biên độ cục bộ $P_{\text{local}} = \{100, 200, 500\}$ và $P_g[0] = 0$:

$$\begin{aligned}
P_g[1] &= \tfrac{15}{16}\cdot 0 + \tfrac{1}{16}\cdot 100 = 6{,}25,\\
P_g[2] &= \tfrac{15}{16}\cdot 6{,}25 + \tfrac{1}{16}\cdot 200 = 18{,}359375,\\
P_g[3] &= \tfrac{15}{16}\cdot 18{,}359375 + \tfrac{1}{16}\cdot 500 = 48{,}4619140625.
\end{aligned}$$

Sàn silence $S \cdot P_g$ lúc đó lần lượt $0{,}1875$, $0{,}5508$, $1{,}4539$ — vẫn nhỏ hơn P_{local}, nên cổng voicing *không* nuốt nguyên âm đang lên dần. Nếu α quá lớn (ví dụ $1/2$), P_g nhảy theo từng khung nhiễu và sàn silence dao động, dễ tạo VDE giả. Mọi số trên khớp `verify/verify_chuong_10_14.py`.

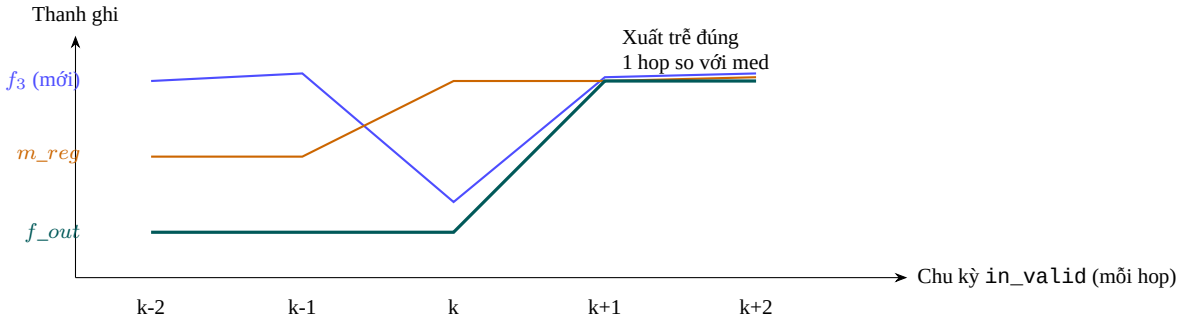


Hình 42: Khối cổng voicing trên fabric: hai so sánh + AND; P_g cập nhật chậm; UV ép $F_0 = 0$ trước median-3.

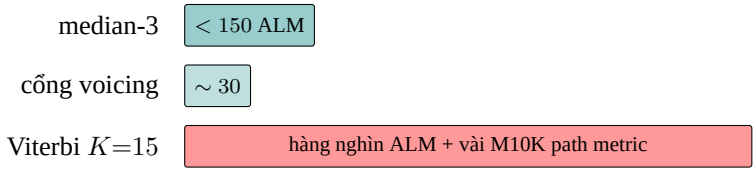


Trellis Viterbi 2×2: đường V→V thắng dù $\delta(\text{UV}_2) > \delta(V_2)$.

Hình 43: Lưới chuyển trạng thái rút gọn của `Pitch_pathFinder`: chi phí c_{ij} phạt nhảy V↔UV và octave.



Hình 44: Pipeline median-3 đăng ký: `f_out` lấy giá trị `m_reg` của nhịp trước, khớp Feynman box 11 (trễ nhóm 10 ms).



So sánh chi phí silicon: giữ median trên fabric; chuyển pathFinder sang HPS nếu cần.

Hình 45: Trực giác ngân sách: median-3 + cổng voicing nằm gọn trong phần nhỏ của ≈ 850 ALM; Viterbi đa ứng viên phá ngân sách lab.

Giải thích RTL từng dòng (mở rộng). Dây `voiced` là tổ hợp thuần (chỉ so sánh và AND, không có xung nhịp bên trong): so sánh r_a với ngưỡng V đã lượng tử Q1.14 thành $V_{\text{Q14}} = 7372$ (tức $0{,}45 \cdot 2^{14}$) và so P_{local} với sàn silence $S \cdot P_g$; không cần khối DSP. Ba thanh ghi f_1, f_2, f_3 tạo cửa sổ trượt ba khung. Trong Verilog, phép gán không chặn (*non-blocking*, toán tử `<=`) trong cùng khối `always` có nghĩa: mọi vế phải đọc giá trị *cũ* của thanh ghi trước khi cập nhật — vì vậy `med` nhìn đúng cửa sổ vừa hoàn tất, rồi mới dịch cửa. Ép khung vô thanh (UV) thành 0 ngay khi vào f_3 để một khung im không bị hai hàng xóm hữu thanh "kéo" thành cao độ giả qua median. Cặp `m_reg` ← med, rồi `f_out` ← `m_reg` thêm đúng một cấp thanh ghi xuất (trễ một hop): đo bằng

tay trên chuỗi $\{200, 210, 80, 205, 208\}$ cho `f_out` $= \{0, 0, 0, 200, 200\}$ (ba nhịp đầu là warmup), khớp script verify.

Vì sao octave jump $150 \to 300$ đắt đúng $0{,}35$. Quãng tám (*octave*) là khoảng tần số nhân đôi: dây thanh dao động gấp đôi số chu kỳ trong cùng thời gian, nên F_0 nhảy từ 150 lên 300 Hz (hoặc ngược lại xuống nửa). Praat phạt bước nhảy đó bằng hệ số $0{,}35$ nhân số octave trên thang $\log_2$:

$$|\log_2(150/300)| = |\log_2(1/2)| = 1, \qquad 0{,}35 \cdot 1 = 0{,}35.$$

Cùng khoảng đó theo cent (thang log âm nhạc, 1200 cent $=$ một octave): $1200 \log_2(400/200) = +1200$ và $1200 \log_2(100/200) = -1200$. Median-3 đập được xung octave *đơn* (một khung lệch); chuỗi octave bền nhiều khung liên tiếp cần pathFinder trên HPS hoặc chỉnh ROM $C[\tau]$ ở Chương 8.

10.8. Bài tập và lời giải giải thích

Bài tập 10.1 (Median-3 và octave spike). Cho chuỗi thô $F_0 = \{100,\ 400,\ 110,\ 105\}$ Hz (giả sử tất cả đã voiced). Tính ngõ ra median-3 *không* trễ thanh ghi cho các cửa ba điểm bắt đầu từ chỉ số 0, rồi nêu cửa nào đập được spike 400.

Lời giải. Cửa $(100, 400, 110)$: sắp $100, 110, 400 \Rightarrow$ med $= 110$. Cửa $(400, 110, 105)$: sắp $105, 110, 400 \Rightarrow$ med $= 110$. Spike 400 không bao giờ là trung vị khi hai hàng xóm nằm gần 100—110. Nếu nhầm dùng min sẽ ra 100 ở cửa đầu — đó là lỗi khái niệm (min $\neq$ median).

Bài tập 10.2 (s_{UV} khi không im lặng). Chứng minh khi $P_{\text{local}} = P_{\text{global}}$ và $S = 0{,}03$, $V = 0{,}45$ thì $s_{\text{UV}} = V$, tức nhánh silence trong (37) không kích.

Lời giải.

$$\frac{P_{\text{local}}/P_{\text{global}}}{S/(1+V)} = \frac{1}{0{,}03/1{,}45} = \frac{1{,}45}{0{,}03} \approx 48{,}333,$$

nên $2 - 48{,}333 < 0$ và $\max(0, \cdot) = 0 \Rightarrow s_{\text{UV}} = V = 0{,}45$. Ý nghĩa: khung đủ lớn so với đỉnh toàn cục thì Unvoiced chỉ còn sàn V, phải cạnh tranh sòng phẳng với r_a.

Bài tập 10.3 (Hai lý do không port pathFinder). Liệt kê hai lý do kỹ thuật/lab để mặc định tắt Viterbi trên fabric Cyclone V của bài này.

Lời giải. (1) **Tài nguyên và độ phức tạp:** $O(TK^2)$ với K tới 15 cần nhiều so sánh song song hoặc FSM dài + BRAM/M10K lưu metric — đẩy chiếm dụng ALM/M10K vượt ngân sách $\approx 2\%$ trong khi median-3 < 150 ALM đã đập spike đơn. (2) **Phân tầng hệ thống:** đo thời gian thực trên FPGA chỉ cần cổng voicing + median; đường cong "Praat đầy đủ" có thể tính soft-float trên HPS từ chuỗi ứng viên thô, không chặn luồng 16 kHz. Thêm nữa, tắt Viterbi buộc lab giữ $K{=}1$, đơn giản hóa argmax Chương 9.

Chương 11. FSM đỉnh và giao thức Avalon-ST (valid/ready/SOP/EOP)

▶ MỤC TIÊU HỌC TẬP

Sau Chương 11, người học sẽ:

1. Giải thích được hợp đồng silicon Avalon Streaming (Avalon-ST): cặp `valid`/`ready`, khái niệm **beat**, tín hiệu gói `SOP`/`EOP`, và hiện tượng áp lực ngược (*backpressure*).
2. Vẽ và đếm tay được một dạng sóng có khoảng nghỉ `ready`$= 0$ và khoảng nghỉ `valid`$= 0$, bảo đảm số beat vào bằng số beat ra.
3. Nối được FSM đỉnh đường F_0 (`IDLE`, `FILL`, `WINDOW`, `MAC`, `PEAK`, `VOICE`, `PACK`, `OUT`); mỗi chuyển trạng thái tôn trọng handshake với khối sau.
4. Viết RTL nguồn không bao giờ tiến con trỏ khi chưa có beat, và twin Python đếm beat khớp testbench.
5. Đọc hộp chi phí adapter Avalon-ST và lý do một mẫu mất im lặng phá cả F_0.

11.1. Hiện tượng thật: ống nước có van hai đầu

★ NGUYÊN LÝ ĐẦU TIÊN: Vì sao cần `valid`/`ready`? — không phải hiện tượng âm thanh

Trong Chương 0 (Feynman box 12) đã định nghĩa Avalon-ST là chuẩn điểm-nối-điểm một chiều của Intel cho DSP băng thông cao. Hãy gắn thêm trực giác đời thường: hai khối phần cứng như hai đoạn ống nước. Bên gửi (*source*) có van "tôi đang bơm" (`valid`); bên nhận (*sink*) có van "tôi còn chỗ chứa" (`ready`). Nước (một từ dữ liệu) chỉ chảy qua khớp nối khi *cả hai* van mở trong cùng một chu kỳ xung nhịp. Đó không phải quy luật vật lý của sóng âm — đó là **hợp đồng silicon** để hai khối khác tốc độ không làm mất mẫu, không nhân đôi mẫu, và không ghi đè bộ đệm.

Nếu mất đúng một mẫu trong khung $L = 640$, cửa sổ Hanning sẽ lệch một mẫu, hàm $\mathcal{R}_x[\tau]$ lệch, đỉnh τ^* lệch, và F_0 sai mà Quartus vẫn báo timing sạch. Lỗi handshake là lỗi *im lặng* — testbench phải đếm beat, không chỉ nhìn Fmax.

11.2. Toán / quy ước từng tín hiệu

Beat. Một **beat** là một lần chuyển dữ liệu thành công:

$$\text{beat}[n] = \big(\texttt{valid}[n] = 1\big) \ \wedge \ \big(\texttt{ready}[n] = 1\big). \tag{41}$$

Đếm số cạnh xung có beat $= 1$ trong cả phép mô phỏng phải bằng số mẫu (hoặc số từ) mà nguồn định gửi — trừ các chỗ cố ý hạ mẫu (FIR $2/3$ ở Chương 4).

`valid`. Source khẳng định: dữ liệu trên bus `data` ổn định và có nghĩa trong chu kỳ này. Khi source chưa có gì, hạ `valid`$= 0$ (khoảng nghỉ chủ động — *valid gap*).

`ready`. Sink khẳng định: nếu `valid`$= 1$ thì mình sẽ chốt dữ liệu vào thanh ghi/FIFO. Khi bộ đệm đầy, hạ `ready`$= 0$ — đây là **backpressure**.

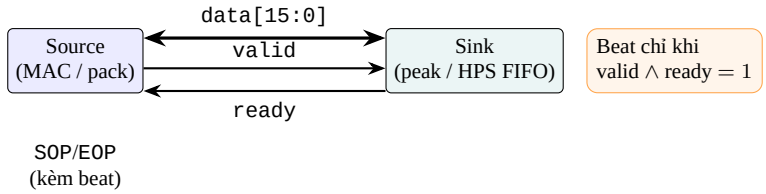
Luật giữ dữ liệu. Khi `valid` $= 1$ và `ready` $= 0$, source **bắt buộc giữ nguyên** cả `data` lẫn `valid`. Chỉ được đổi sang mẫu kế khi một beat đã xảy ra (hoặc khi tự hạ `valid`).

`readyLatency`. Tài liệu Avalon Interface Specifications (doc ID 683091) cho phép sink báo `ready` trước 0 hoặc 1 chu kỳ. Lab này dùng `readyLatency`$= 0$: quyết định nhận nằm cùng chu kỳ với `valid`. Nếu nối IP FIR/FFT cấu hình latency 1 mà FSM của bạn giả định 0, beat đầu/cuối burst sẽ lệch — đọc UG trước khi nối.

SOP / EOP (Start of Packet / End of Packet). Avalon-ST có thể đóng gói một **packet**: `SOP`$=1$ trên beat đầu, `EOP`$=1$ trên beat cuối. Với khung âm thanh $L = 640$ mẫu Q1.15 đưa vào khối cửa sổ/MAC, quy ước lab:

- Beat chỉ số 0: `SOP`$=1$, `EOP`$=0$;
- Beat $1 \ldots 638$: cả hai bằng 0;
- Beat 639: `SOP`$=0$, `EOP`$=1$.

Khối nhận dùng `SOP` để reset con trỏ cửa sổ, dùng `EOP` để kích hoạt MAC/peak. Thiếu `EOP` $\Rightarrow$ FSM treo ở `FILL`; thừa `EOP` $\Rightarrow$ cắt khung sớm.



Hình 46: Avalon-ST điểm-nối-điểm: data đi một chiều; valid từ source; ready từ sink; SOP/EOP đánh dấu biên gói khung.

11.3. Ví dụ tính tay — sáu/bảy chu kỳ có gap (đã kiểm logic)

Nguồn muốn gửi bốn mẫu $\{A, B, C, D\}$. Giữa B và C, sink kéo `ready`$=0$ đúng hai chu kỳ; sau beat C, source tự kéo `valid`$=0$ đúng một chu kỳ (valid gap), rồi gửi D.

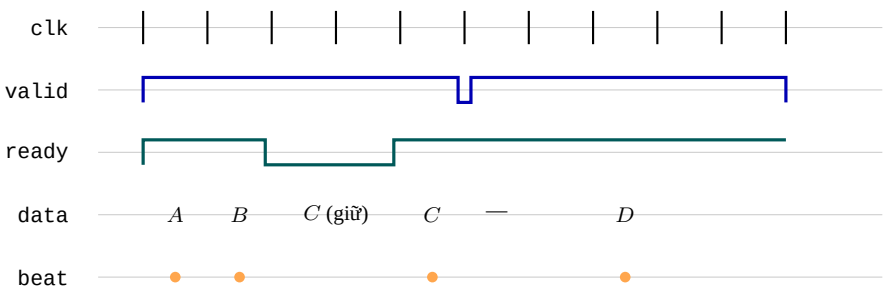
Chu kỳ	`valid`	`ready`	`data` trên bus	Beat?
1	1	1	A	có — chuyển A
2	1	1	B	có — chuyển B
3	1	0	C (giữ)	không
4	1	0	C (giữ)	không
5	1	1	C	có — chuyển C
6	0	1	—	không (valid gap)
7	1	1	D	có — chuyển D

Tổng beat $= 4$, đúng bằng số mẫu gửi. Script `verify/verify_chuong_10_14.py` đếm lại trace này. Nếu RTL tiến con trỏ ở chu kỳ 3 (khi `ready`$=0$), sink sẽ bỏ lỡ C hoặc nhận trùng B — đếm beat thành 3 hoặc 5, testbench phải fail.

Packet 640 **mẫu.** `SOP` trên beat 0, `EOP` trên beat 639. Mọi beat giữa không mang cờ gói. Twin đếm: số lần `SOP` $= 1$ phải bằng số khung; mỗi `SOP` đi kèm đúng một `EOP` sau đúng 639 beat (hoặc $L - 1$ khoảng cách beat).

11.3b. Dạng sóng timing Avalon-ST (vẽ từng cạnh xung)

Hình 47 là cùng ví dụ mục 11.3 vẽ theo trục thời gian chu kỳ: khi `ready`$=0$, `data` phải đứng yên ở C; khi source tự nghỉ, `valid`$=0$ dù sink vẫn sẵn sàng.

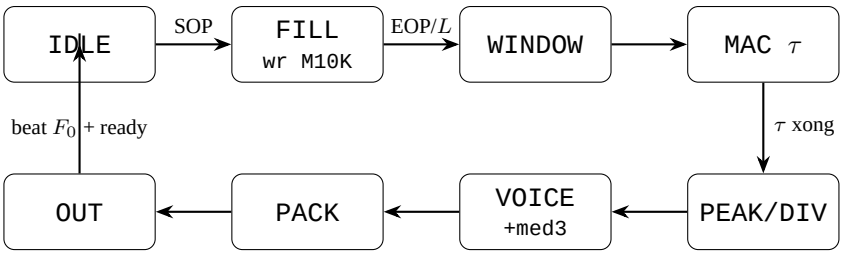


Bốn chấm cam = bốn beat $\{A, B, C, D\}$. Khoảng `ready`$=0$ không được tiến con trỏ.

Hình 47: Timing Avalon-ST `readyLatency`$=0$: beat chỉ tại giao của valid và ready đồng thời cao.

11.4. FSM đỉnh của đường F_0

Hình 48 là máy trạng thái tích hợp (rút gọn nhưng đủ lab). Mỗi trạng thái điều khiển một khối đã học ở các chương trước; mũi tên chỉ chuyển khi khối sau sẵn sàng (`ready`) hoặc khi điều kiện nội hoàn tất.



Hình 48: FSM pitch đỉnh: FILL đủ L mẫu (bám SOP/EOP)→ WINDOW→ MAC mọi τ → PEAK/DIV→ VOICE/median-3→ PACK Q12.4→ OUT Avalon-ST. Quay lại IDLE khi sink đã nhận beat cao độ.

Vai trò từng trạng thái (neo chương trước).

- `IDLE`: chờ `SOP` từ FIR/16 kHz (Chương 4–5).
- `FILL`: ghi True Dual-Port M10K, Old Data (Chương 5); đếm beat đến L hoặc chờ `EOP`.
- `WINDOW`: nhân ROM Hanning Q1.15 (Chương 6).
- `MAC`: quét $\tau = 26 \dots 214$ bằng 1 DSP (Chương 7); đây là trạng thái chiếm nhiều chu kỳ nhất ($\approx 120{,}960$ MAC).
- `PEAK/DIV`: argmax + parabolic + LPM_DIVIDE (Chương 9).
- `VOICE`: cổng voicing + median-3 đăng ký (Chương 10).
- `PACK`/`OUT`: đóng Q12.4 và handshake Avalon-ST ra HPS/Memory (Chương 12).

Nếu `OUT` gặp backpressure (`ready`$= 0$), FSM **không** được nhảy về `FILL` và ghi đè bộ đệm đang chờ — phải giữ `valid` và data Q12.4 như mục 11.2.

11.4b. Ngân sách chu kỳ từng trạng thái FSM

Tại xung nhịp hệ thống $f_{\text{clk}} = 50\,\text{MHz}$, mỗi hop $H/f_s = 160/16000 = 0{,}01\,\text{s}$ có đúng

$$N_{\text{cyc}} = f_{\text{clk}} \cdot 0{,}01 = 50 \cdot 10^6 \cdot 0{,}01 = 500\,000$$

chu kỳ. Phân bổ điển hình (lab MAC tuần tự một DSP):

Trạng thái	Chu kỳ (ước)	Ghi chú
FILL / WINDOW	$\sim$ 640–1280	ghi M10K + nhân Hanning
MAC $\tau \in [26, 214]$	120 960	189×640 (biên trên)
PEAK / DIV	$\sim$ 20–80	argmax + latency LPM_DIVIDE
VOICE / PACK / OUT	$\sim$ 5–20	median + Q12.4 + handshake
Tổng hữu ích	$\lesssim 123\,000$	RTF $\approx 0{,}24$
Biên còn lại	$\approx 377\,000$	chờ hop kế / backpressure

Số $189 = 214 - 26 + 1$ và tích $189 \times 640 = 120\,960$ đã kiểm script. RTF (*Real-Time Factor*) = thời gian xử lý / thời gian âm thanh $= 120\,960/500\,000 \approx 0{,}24192 \ll 1$: fabric kịp thật.

Ví dụ tính tay — bài tập 11.1 chi tiết. Gửi $\{X, Y, Z\}$, sau beat X sink kéo `ready`$= 0$ đúng ba chu kỳ:

ck	valid	ready	data	beat?
1	1	1	X	có
2	1	0	Y giữ	không
3	1	0	Y giữ	không
4	1	0	Y giữ	không
5	1	1	Y	có
6	1	1	Z	có

Tổng beat $= 3$. Script verify đếm lại trace này.

11.5. RTL — giữ data khi backpressure

```
// Source side: KHONG duoc advance rd_ptr khi chua co beat
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    out_valid <= 1'b0;
    out_data  <= 16'd0;
    out_sop   <= 1'b0;
    out_eop   <= 1'b0;
  end else if (!out_valid || out_ready) begin
    // Duoc phep nap beat moi (idle hoa vua chuyen xong beat cu)
    if (have_sample) begin
      out_data  <= next_sample;
      out_sop   <= next_is_first;
      out_eop   <= next_is_last;
      out_valid <= 1'b1;
      // chi o day moi rd_ptr <= rd_ptr + 1
    end else begin
      out_valid <= 1'b0;
      out_sop   <= 1'b0;
      out_eop   <= 1'b0;
    end
  end
  // else: giu nguyen bus (backpressure)
end
```

Dòng then chốt là điều kiện `!out_valid || out_ready`: chỉ khi chưa cầm beat, hoặc sink vừa nhận beat, source mới được đổi bus. Mọi chỗ khác phải giữ nguyên — kể cả `SOP`/`EOP`.

11.5b. Avalon-ST bên trong IP Intel và cầu HPS–FPGA

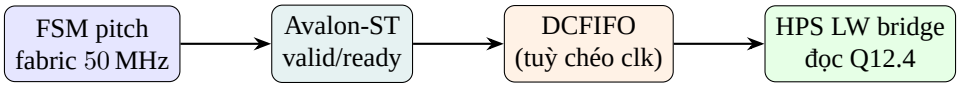
Tài liệu *Avalon Interface Specifications* (doc ID 683091) định nghĩa thêm:

- `empty`/`error`: dùng khi bề rộng biểu tượng > 1 byte; lab pitch dùng biểu tượng 16 bit đầy đủ nên thường bỏ `empty`.
- `channel`: phân kênh ảo trên cùng dây — lab một đường mono không cần.
- FIR Compiler II và FFT IP đều nói Avalon-ST; khi nối FIR→sliding RAM phải khớp `readyLatency`. Lab chọn 0 xuyên suốt để FSM đơn giản: quyết định nhận nằm cùng chu kỳ `valid`.

Cầu HPS–FPGA trên DE10-Standard. Cyclone V SX trên DE10-Standard có Hard Processor System (HPS) — cụm CPU cứng dual Cortex-A9 nằm trên cùng chip với FPGA fabric. Ba cầu chính nối HPS với logic do bạn viết:

- **Lightweight HPS-to-FPGA (LW bridge)** $\approx$ 2 MB không gian địa chỉ: CPU đọc thanh ghi trạng thái / từ Q12.4 từng khung — đủ cho 97 từ/giây.
- **HPS-to-FPGA bridge** rộng hơn: DMA burst nếu sau này ghi WAV+bin xuống SDRAM.
- **FPGA-to-HPS**: fabric làm master (ít dùng ở lab pitch).

Thực tế lab: khối `OUT` đẩy Avalon-ST vào DCFIFO (*Dual-Clock FIFO* — hàng đợi hai miền xung nhịp, IP Intel) nếu HPS và FPGA khác PLL, rồi HPS đọc định kỳ (poll) hoặc nhận ngắt khi FIFO nửa đầy. Một từ mất trên cầu LW không làm Quartus báo lỗi — chỉ làm Pearson thanh điệu sụp; đó là lý do tầng 3 phải đếm beat đến tận biên HPS.



Hình 49: Đường ra cao độ: FSM→Avalon-ST→(FIFO)→HPS. Backpressure từ FIFO đầy phải đứt ngược về `OUT`.

M10K trong FILL. Trạng thái `FILL` ghi True Dual-Port On-Chip Memory (khối M10K 10 kbit). Một khung $640 \times 16 = 10\,240$ bit vừa khít ≈ 1 M10K. Chế độ mixed-port read-during-write trên Cyclone V chỉ **Old Data** hoặc **Don't Care** — không có New Data (Chương 5). FSM không được giả định đọc thấy mẫu vừa ghi trong cùng chu kỳ trên cổng kia.

11.5c. RTL giải thích từng dòng

```
// Dieu kien nap beat moi:
//   !out_valid  : bus dang trong (chua cam beat)
//   out_ready   : sink vua nhan beat (duoc doi mau ke)
// Ngoai hai truong hop do: GIU NGUYEN data/valid/SOP/EOP.
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    out_valid <= 1'b0;
    out_data  <= 16'd0;
    out_sop   <= 1'b0;
    out_eop   <= 1'b0;
  end else if (!out_valid || out_ready) begin
    if (have_sample) begin
      out_data  <= next_sample;   // chi doi khi duoc phep
      out_sop   <= next_is_first;
      out_eop   <= next_is_last;
      out_valid <= 1'b1;
      // rd_ptr <= rd_ptr + 1;  // DUY NHAT noi advance
    end else begin
      out_valid <= 1'b0;          // valid gap chu dong
      out_sop   <= 1'b0;
      out_eop   <= 1'b0;
    end
  end
  // else: backpressure — latched bus khong doi
end
```

Từng ý: (1) `rst_n` bất đồng bộ hạ đưa mọi cờ về 0 an toàn; (2) điều kiện `!out_valid || out_ready` là trái tim hợp đồng 683091 với `readyLatency`$= 0$; (3) con trỏ đọc chỉ tăng trong nhánh `have_sample` — tăng ngoài nhánh này là bug cố ý của bài tập 11.2; (4) nhánh `else` ẩn (không ghi gì) giữ latch — đó chính là "giữ data khi backpressure".

► NHÌN THẤY VẬT LÝ

Handshake không "sửa" thuật toán Boersma. Nó chỉ bảo đảm đúng mẫu đúng lúc đến đúng khối. Một beat mất ở biên MAC↔peak làm parabolic gắn nhầm τ, tạo GPE octave giả trong khi mọi công thức trong twin float vẫn xanh — đó là lớp lỗi tầng 3 ở Chương 13.

11.6. Twin đếm beat và kiểm SOP/EOP

```
def count_beats(trace):
    return sum(1 for c in trace if c["valid"] and c["ready"])

def check_packets(trace, L=640):
    beats = [c for c in trace if c["valid"] and c["ready"]]
    assert count_beats(trace) == len(beats)
    # Moi goi: SOP tren beat dau, EOP tren beat cuoi, dung L beat
    i = 0
    while i < len(beats):
        assert beats[i]["sop"] and not beats[i]["eop"]
        assert i + L - 1 < len(beats)
        for k in range(1, L - 1):
            assert not beats[i+k]["sop"] and not beats[i+k]["eop"]
        assert beats[i+L-1]["eop"] and not beats[i+L-1]["sop"]
        i += L
```

Testbench ModelSim/Questa: LFSR hạ `ready` ngẫu nhiên; LFSR tạo valid-gap phía source; scoreboard so số beat và nội dung Q1.15 từng mẫu với twin.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — FSM + adapter Avalon-ST.

- **Adapter valid/ready/SOP/EOP:** khoảng 0–30 ALM (mux giữ data, vài flip-flop); 0 DSP; 0 M10K.
- **FSM đỉnh:** vài trăm ALM tùy số trạng thái và độ rộng bộ đếm địa chỉ (10 bit cho 640), vẫn nằm trong ngân sách ≈ 850 ALM toàn luồng.
- **Latency điều khiển:** bản thân handshake thêm 0–1 chu kỳ mỗi biên; không đổi RTF MAC $\approx 0{,}24$.
- **Vì sao giữ data khi `ready`$= 0$:** mất một mẫu = lệch khung = sai $\mathcal{R}_x$ = sai F_0 — không có cảnh báo synthesis.
- **Vì sao cần SOP/EOP:** tách biên gói 640 mẫu trong luồng liên tục 16 kHz; thiếu chúng FSM không biết lúc nào kết thúc `FILL`.

Ba luật Avalon-ST cần thuộc trước Signal Tap. (1) Beat chỉ xảy ra khi `valid` và `ready` cùng cao trong một chu kỳ (`readyLatency`$= 0$). (2) Khi `valid`$= 1$ mà `ready`$= 0$, source giữ nguyên `data` lẫn SOP/EOP. (3) Mỗi gói $L = 640$ có đúng một SOP ở beat đầu và một EOP ở beat cuối. Vi phạm luật (2) lệch $\mathcal{R}_x$ và F_0 dù Fmax vẫn xanh — twin đếm beat phải khớp testbench trên mọi seed LFSR.

11.7. Bài tập và lời giải giải thích

Bài tập 11.1 (Waveform 8 chu kỳ). Gửi 3 mẫu $\{X, Y, Z\}$. Sau beat X, sink kéo `ready`$= 0$ trong đúng 3 chu kỳ, rồi nhận Y và Z liên tiếp. Vẽ bảng `valid`/`ready`/`data`/beat và đếm tổng beat.

Lời giải. Một lịch hợp lệ: ck1 beat X; ck2–4 `valid`$= 1$, `ready`$= 0$, giữ Y; ck5 beat Y; ck6 beat Z. Tổng beat $= 3$. Mọi lịch tiến con trỏ sớm sẽ làm sink thấy sai mẫu trên ck5.

Bài tập 11.2 (Bug cố ý). RTL advance `rd_ptr` mỗi khi `valid`$= 1$ dù `ready`$= 0$. Testbench kỳ vọng gì?

Lời giải. Source đã "quên" mẫu đang giữ: khi `ready` lên lại, bus đã sang mẫu kế. Số beat đếm được có thể vẫn bằng N nếu sink chốt nhầm nội dung, nhưng scoreboard nội dung fail; hoặc nếu sink không chốt khi `ready`$= 0$ mà source đã bỏ mẫu, tổng beat $< N$. Cả hai đều phải đỏ.

Bài tập 11.3 (MAC chậm hơn peak). Khối MAC phát $\mathcal{R}_x[\tau]$ từng τ sang peak. Peak cần vài chu kỳ xử lý một τ nên hạ `ready`. Chứng minh source MAC phải giữ đúng $\mathcal{R}_x[\tau]$ hiện tại.

Lời giải. Nếu MAC chuyển sang $\tau + 1$ khi peak chưa nhận, peak sẽ bỏ τ hoặc gắn kết quả parabolic vào trễ sai. Luật Avalon-ST: không beat thì không đổi `data`. Twin đếm beat giữa MAC và peak phải bằng số τ trong $[\tau_{\min}, \tau_{\max}]$ (189 giá trị).

Chương 12. Đóng gói F_0 Q12.4 và tệp `pitch_f0.bin`

▶ MỤC TIÊU HỌC TẬP

Sau Chương 12, người học sẽ:

1. Giải thích được hợp đồng số giữa FPGA và phần mềm phía trên: mỗi khung xuất một từ **uint16** theo thang **Q12.4** (đơn vị $1/16$ Hz), ghi **little-endian**, khung vô thanh gửi đúng 0.
2. Tính tay và đóng gói được bảng $F_0 \to$ Q12.4 $\to$ hai byte LE cho các mốc 0, 75, $150{,}0625$, $199{,}466$, 200, 220, 440, 600, 700 (bão hòa), đã kiểm Python.
3. Đọc được bố trí bitfield (Hình 50–52), tách phần nguyên/frac, và giải mã đúng trên HPS.
4. Ghép được cặp tệp kiểm chứng `audio_16k.wav` (PCM 16 kHz) với `pitch_f0.bin` (chuỗi uint16 LE đồng bộ theo hop 10 ms).
5. Viết RTL pack có bão hòa trần 600 Hz ($q = 9600$) và twin `struct.pack("<H", q)` khớp từng bit; phân biệt lỗi thuật toán với lỗi giao thức (đảo endian, quên chia 16).

12.1. Hiện tượng thật: server đọc gì từ FPGA?

★ NGUYÊN LÝ ĐẦU TIÊN: Hợp đồng số, không phải số thực dấu phẩy động

Khối ước lượng trên Cyclone V không nói chuyện với HPS/server bằng `float64`. Sau median-3 (Chương 10) và FSM OUT (Chương 11), hợp đồng là một từ 16 bit không dấu theo thang **Q12.4**: 12 bit phần nguyên + 4 bit phần thập phân cố định. Giá trị nguyên lưu trong thanh ghi thỏa

$$F_q = \mathrm{sat}_{0..9600}\big(\mathrm{round}(F_0^{\mathrm{Hz}} \cdot 16)\big), \qquad \hat{F}_0 = F_q/16 \text{ (Hz)}. \tag{42}$$

Trọng số bit thấp nhất (LSB) bằng $2^{-4} = 0{,}0625$ Hz. Trên tệp `pitch_f0.bin` (và trên bus Avalon-ST 16 bit), hai byte được ghi **little-endian**: byte thấp trước, byte cao sau — khớp hầu hết CPU little-endian của HPS ARM. Khung vô thanh (**unvoiced**, UV) gửi đúng $F_q = 0$ (`00 00`), không dùng NaN hay mã đặc biệt. Đảo endian hoặc quên chia 16 phía server làm $\hat{F}_0$ lệch $\times 256$ hoặc $\times 16$ trong khi silicon vẫn đúng: đó là lỗi *giao thức*, không phải lỗi pitch.

Vì sao chọn đúng 4 bit frac? Tại $F_0 = 200$ Hz, một LSB $0{,}0625$ Hz tương đương

$$1200 \log_2\big((200 + 0{,}0625)/200\big) \approx 0{,}541 \text{ cent}$$

(đã kiểm script). Ngân sách sai số lab so Praat thường 20–30 cent (Chương 13), nên lượng tử Q12.4 nhỏ hơn nhiều so với sai số thuật toán — không phải nút thắt. Đồng thời $600 \cdot 16 = 9600 < 2^{16} - 1$, cả dải $75 \ldots 600$ Hz nằm gọn trong 16 bit không dấu kèm trần bão hòa rõ ràng.

Trực giác đời thường. Hãy nghĩ Q12.4 như thước kẻ có vạch mỗi $1/16$ Hz. Bạn không gửi "hai trăm phẩy không" bằng chữ; bạn gửi số đếm vạch (3200 vạch = 200 Hz). Phần mềm phía trên chỉ việc chia lại 16 để ra Hz. Nếu cầm thước đọc ngược (big-endian), số đếm vạch thành một số khác hoàn toàn — thước không hỏng, người đọc hỏng.

12.2. Toán từng bước: từ Hz đến hai byte

1. Nhân F_0 (Hz, số thực sau LPM_DIVIDE/parabolic) với 16.
2. Làm tròn tới số nguyên gần nhất: $q_0 = \mathrm{round}(F_0 \cdot 16)$.
3. Nếu không voiced hoặc $F_0 \le 0$: ép $q = 0$.
4. Bão hòa: $q = \min(q_0,\ \texttt{CEIL})$ với $\texttt{CEIL} = 600 \cdot 16 = 9600$.
5. Tách byte: $b_0 = q \bmod 256$, $b_1 = \lfloor q/256 \rfloor$ (little-endian: ghi b_0 rồi b_1).

Định dạng Q$m.n$ đã được Chương 0 (Feynman box 7) định nghĩa: đây là Q12.4 *không dấu* trên dây ra, khác với Q1.15 có dấu của mẫu âm thanh. Quan hệ bit:

$$F_q = (F_q[15:4]) \cdot 16 + F_q[3:0], \qquad \hat{F}_0 = F_q/16. \tag{43}$$

Không cần "nối" thêm dấu phẩy động IEEE 754 trên cầu LW: một lệnh load half-word trên ARM rồi dịch/chia nguyên là đủ.

12.3. Ví dụ tính tay (đã kiểm Python)

F_0 (Hz)	F_q (uint16)	Bytes LE	Ghi chú
0 (UV)	0	`00 00`	Hợp đồng UV
75,0	1200	`B0 04`	Sàn tìm kiếm
150,0625	2401	`61 09`	Đúng một LSB trên 150
150,25	2404	`64 09`	$150 \cdot 16 + 4$
199,466	3191	`77 0C`	Parabolic Chương 9
200,0	3200	`80 0C`	Mốc lab
220,0	3520	`C0 0D`	
440,0	7040	`80 1B`	
600,0	9600	`80 25`	Trần
700,0	9600	`80 25`	Bão hòa về trần

Chi tiết một hàng — 199,466 Hz.

$$\begin{aligned} 199{,}466 \cdot 16 &= 3191{,}456, \\ \text{round}(3191{,}456) &= 3191, \\ 3191 &= \text{0x0C77} = 12 \cdot 256 + 119, \\ b_0 &= \text{0x77}, \quad b_1 = \text{0x0C}. \end{aligned}$$

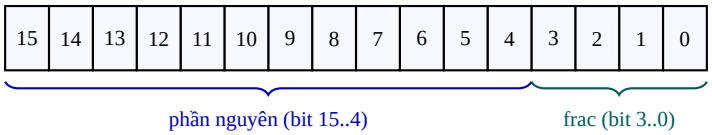
Giải mã ngược: $3191/16 = 199{,}4375$ Hz (phần nguyên 199, frac 7). Sai số lượng tử so với 199,466 chỉ $\approx 0{,}0285$ Hz.

Bẫy endian. Bytes LE của 200 Hz là `80 0C`. Nếu server đọc nhầm big-endian:

$$\texttt{0x0C80} = 3200_{\text{LE}} \text{ đọc BE } \Rightarrow \text{ 0x0C80} = 32780, \qquad 32780/16 = 2048{,}75 \text{ Hz}.$$

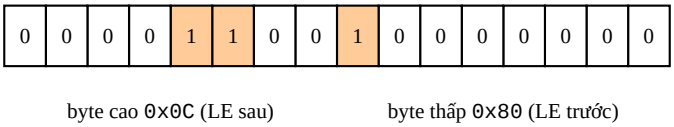
Một lỗi một dòng `struct.unpack` biến 200 Hz thành > 2 kHz — GPE/FFE ở Chương 13 sẽ "nổ", trong khi RTL và twin pack vẫn xanh.

12.4. Bố trí bitfield Q12.4 (Hình 50–52)



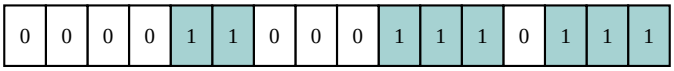
Hình 50: Bitfield Q12.4: bit 0 (LSB = 0,0625 Hz) bên phải; bit 15 bên trái.

$$200 \text{ Hz} \to F_q = 3200 = \text{0x0C80}$$



Hình 51: Bit pattern 200 Hz: trên tệp/bin là `80 0C`.

$$199{,}466\,\text{Hz} \to 3191 = \text{0x0C77}$$



$3191 \gg 4 = 199$, $3191 \bmod 16 = 7 \Rightarrow 199{,}4375\,\text{Hz}$ sau giải mã.

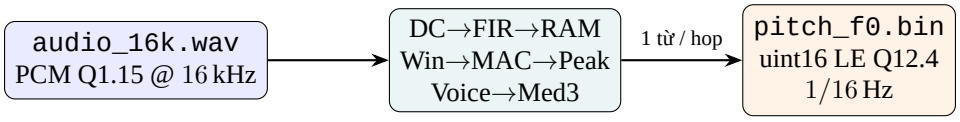
Hình 52: Bit pattern sau parabolic; LE `77 0C`.

Ví dụ tính tay — tách bit đã kiểm. Script `verify/verify_chuong_10_14.py` khẳng định $(3200 \gg 4, 3200\&15) = (200, 0)$ và $(3191 \gg 4, 3191\&15) = (199, 7)$. Đây là cách HPS có thể tránh chia nếu muốn: dịch phải 4 lấy Hz nguyên, lấy bốn bit thấp nhân $0{,}0625$ nếu cần frac.

12.5. Cặp tệp `audio_16k.wav` và `pitch_f0.bin`

Luồng kiểm chứng chuẩn lab.

1. `audio_16k.wav`: PCM mono 16-bit, $f_s = 16\,\text{kHz}$, đã qua (hoặc giả lập) DC-block + FIR hạ mẫu. Đây là đầu vào "sạch" của sliding RAM.
2. Chạy đường float/fixed/RTL theo hop $H = 160$ mẫu (10 ms): số khung
$$N_{\text{frames}} = \left\lfloor \frac{N_{\text{samples}} - L}{H} \right\rfloor + 1$$
với $L = 640$.
3. `pitch_f0.bin`: đúng N_{frames} từ uint16 LE. Khung k ứng với tâm cửa sổ quanh mẫu $kH + L/2$ (cùng quy ước thời gian Praat time step $0{,}01$ s).



Đồng bộ: khung $k \leftrightarrow$ byte $2k, 2k+1$ trong bin

Hình 53: Hợp đồng kiểm chứng: mỗi hop 10 ms trên wav sinh đúng hai byte trong `pitch_f0.bin`.

Ví dụ tính tay — một giây. Một giây âm thanh 16 kHz có 16000 mẫu. Với $L = 640$, $H = 160$:

$$N_{\text{frames}} = \lfloor (16000 - 640)/160 \rfloor + 1 = 97, \qquad |\texttt{bin}| = 97 \cdot 2 = 194 \text{ byte}.$$

Script verify assert các số này.

12.6. RTL pack — giải thích từng dòng

```
module pack_q124 #(
  parameter CEIL_Q = 16'd9600  // 600 Hz * 16
)(
  input  wire        clk, rst_n,
  input  wire        in_valid,
  input  wire        voiced,
  input  wire [15:0] f0_hz_u,   // phan nguyen Hz (tu LPM_DIVIDE)
  input  wire [3:0]  f0_frac,   // 4 bit frac neu mang san Q12.4
  output reg         out_valid,
  output reg  [15:0] q124,
  output reg  [7:0]  b0, b1     // LE: b0 thap, b1 cao
);
```

```
  // raw = f0_hz_u * 16 + f0_frac  (khong dau, 17 bit de so CEIL)
  wire [16:0] raw = {1'b0, f0_hz_u, 4'b0} + f0_frac;
  wire [15:0] sat = (raw > CEIL_Q) ? CEIL_Q : raw[15:0];
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      q124<=0; b0<=0; b1<=0; out_valid<=0;
    end else if (in_valid) begin
      q124 <= voiced ? sat : 16'd0;
      b0   <= voiced ? sat[7:0]  : 8'd0;
      b1   <= voiced ? sat[15:8] : 8'd0;
      out_valid <= 1'b1;
    end else out_valid <= 1'b0;
  end
endmodule
```

Từng dòng then chốt: (1) `{f0_hz_u, 4'b0}` là dịch trái 4 bit $=$ nhân 16 không cần DSP; (2) cộng `f0_frac` nếu datapath đã mang sẵn phần thập phân; (3) so sánh với 9600 trên 17 bit tránh wrap; (4) mux UV ép 0 trên cả từ và cả hai byte; (5) một chu kỳ đăng ký — latency pack $= 1$, không đổi RTF MAC.

Không cần DSP: chỉ dịch, cộng, so sánh bão hòa, mux. Ngõ `b0`/`b1` tiện ghi Memory-Mapped hoặc stream byte vào thẻ SD; ngõ `q124` đi Avalon-ST 16 bit sang HPS qua LW bridge (Chương 11).

12.7. Twin bit-exact và đường giải mã HPS

```
import numpy as np, struct
CEIL = 9600

def pack_q124(f0_hz, voiced=True):
    if (not voiced) or f0_hz <= 0:
        q = 0
    else:
        q = max(0, min(CEIL, int(np.round(f0_hz * 16.0))))
    return q, struct.pack("<H", q)

assert pack_q124(200.0)[1] == b"\x80\x0c"
assert pack_q124(199.466)[0] == 3191
assert pack_q124(0.0, False)[0] == 0
assert pack_q124(700.0)[0] == CEIL
assert pack_q124(220.0)[1] == b"\xc0\x0d"
```

Giải mã phía HPS / Python.

```
import numpy as np, wave
raw = open("pitch_f0.bin", "rb").read()
assert len(raw) % 2 == 0
f0 = np.frombuffer(raw, dtype="<u2").astype(np.float64) / 16.0
with wave.open("audio_16k.wav", "rb") as w:
    assert w.getframerate() == 16000 and w.getnchannels() == 1
    n = w.getnframes()
n_frames = (n - 640) // 160 + 1
assert len(f0) == n_frames
# f0[k] == 0  =>  UV tai khung k
```

Nếu `dtype=">u2"` (big-endian) trên máy LE, mọi kiểm chứng cent/GPE sẽ sai dù RTL đúng.

Bão hòa trần 600 Hz. Mọi $F_0 > 600$ (do nhiễu đỉnh τ nhỏ) bị ghim $q = 9600$. Trần khớp `pitch_ceiling` Praat lab; không ghim sẽ tràn ý nghĩa trên thang đo hoặc tạo giá trị > 600 Hz ngoài hợp đồng tìm kiếm.

Cầu Lightweight HPS-to-FPGA. Thông lượng 97 từ/giây (≈ 194 B/s cho một giây audio) nhỏ hơn hàng nghìn lần băng thông LW bridge trên Cyclone V SX — lab chỉ cần poll thanh ghi hoặc ngắt khi FIFO có dữ liệu, không bắt buộc DMA. Vẫn phải tôn trọng Avalon-ST/Avalon-MM ở biên: mất một ghi = thiếu một khung trên bin.

▶ NHÌN THẤY VẬT LÝ

Q12.4 không phải định dạng "nghe được". Nó là hợp đồng đo lường giữa fabric và phần mềm. Sai hợp đồng làm mọi metric Chương 13 đỏ trong khi MAC/peak vẫn đúng — hãy `hexdump -C pitch_f0.bin | head` trước khi đổ tội Boersma.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách silicon — đóng gói Q12.4 (giải thích từng mục).

- **ALM** (*Adaptive Logic Module*): vài chục (dịch, cộng frac, so CEIL, mux UV, thanh ghi). Đơn vị logic cơ bản của Cyclone V.
- **DSP** (*Digital Signal Processing block*): 0 — không nhân rộng.
- **M10K** (block RAM ≈ 10 kbit): 0 cho bản thân pack; FIFO HPS tuỳ chọn 1 M10K nếu gom burst nhiều khung.
- **Latency:** 1 chu kỳ đăng ký — không đổi RTF (*Real-Time Factor* = thời gian xử lý / thời gian âm thanh).
- **Fmax:** khối tổ hợp nông (dịch+so sánh) không phải nút thắt so MAC/DIV.
- **Vì sao Q12.4:** đủ 75–600 Hz trong 16 bit; LSB ≈ 0,54 cent tại 200 Hz, nhỏ hơn ngân sách 20 cent; LE khớp HPS ARM.
- **Vì sao UV= 0:** một giá trị đặc biệt duy nhất — server không cần enum riêng; median-3 và Pearson đều xử lý 0 như "không cao độ".

12.7b. Từ thanh ghi fabric đến tiến trình user-space trên HPS

Sau khi `pack_q124` chốt F_q, có ít nhất ba cách đưa số sang phần mềm — lab chọn cách (1) vì đơn giản và đủ thông lượng:

1. **Thanh ghi Avalon-MM trên LW bridge:** Avalon-MM (*Memory-Mapped*) là bus địa chỉ/đọc-ghi; FSM OUT viết F_q vào một thanh ghi, CPU đọc định kỳ mỗi 10 ms (hoặc khi bit "new" bật). Phù hợp 97 từ/giây.
2. **DCFIFO + ngắt:** fabric đẩy Avalon-ST vào FIFO; khi mực nước (watermark) đạt ngưỡng, HPS chạy ISR (*Interrupt Service Routine* — hàm đáp ứng ngắt) để rút hết. Tốt khi lịch CPU bị jitter lớn.
3. **DMA qua bridge rộng:** DMA (*Direct Memory Access*) ghi liên tục vào SDRAM rồi tiến trình user-space `mmap`. Chỉ cần khi sau này ghi cả PCM + bin tốc độ cao.

Ví dụ tính tay — poll period. Giả sử CPU poll mỗi 5 ms trong khi hop là 10 ms. Mỗi khung được đọc khoảng hai lần; phần mềm phải dùng bit "new" hoặc đếm khung để không nhân đôi mẫu trên `pitch_f0.bin`. Nếu poll 20 ms, có nguy cơ bỏ khung khi ISR/trễ lịch — FIFO depth tối thiểu nên ≥ 2 từ (an toàn hơn ≥ 8).

Endian trên dây Avalon-ST 16 bit. Trên bus 16 bit, một beat mang cả từ F_q; "little-endian" lộ rõ khi *phá* thành byte để ghi tệp hoặc gửi UART. Nhầm chỗ phá byte (coi bit 15..8 là byte thấp) chính là bug `0C 80` thay vì `80 0C`. Test tối thiểu: golden 200 Hz phải dump đúng hai byte `80 0C` trên logic analyzer hoặc trong file.

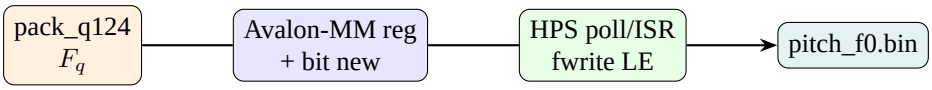
Vì sao không xuất float32. Một số nhóm gửi `float32` little-endian (4 byte/khung). Lab giữ Q12.4 vì không cần FPU trên đường ra, dễ khớp twin nguyên từng bit, và LSB đã nhỏ hơn ngân sách cent. Nếu bài bắt buộc float, convert một lần duy nhất trên HPS sau khi đã xác nhận hai byte Q12.4 đúng.

Ví dụ tính tay — bão hòa và UV cạnh nhau. Chuỗi thô sau peak: $\{700,\ 0,\ 220\}$ Hz với cờ voiced $\{1, 0, 1\}$. Pack:

$$\begin{aligned} 700 &\to \text{sat}(11200) = 9600 \to \texttt{80 25}, \\ 0\ (\text{UV}) &\to 0 \to \texttt{00 00}, \\ 220 &\to 3520 \to \texttt{C0 0D}. \end{aligned}$$

Tệp bin sáu byte: `80 25 00 00 C0 0D`. Median-3 (Chương 10) nếu chạy *trước* pack sẽ nhìn chuỗi Hz đã ép UV$=0$ và bão hòa tùy thiết kế; lab chuẩn: voicing→median→pack, và bão hòa nằm trong pack để trần hợp đồng thống nhất.

Đồng bộ thời gian với Praat. Mỗi khung lab k ứng $t = 0{,}01 \cdot k$ giây (hoặc neo tâm cửa $t = 0{,}01 \cdot k + L/(2f_s)$). Lệch một hop làm hệ số Pearson thanh điệu giảm dù từng Hz đúng — lỗi giao thức thời gian, cùng họ với lỗi endian.



Hình 54: Đường lab mặc định: pack → reg LW → HPS → bin. Mỗi mũi tên là chỗ có thể đảo endian hoặc nhân đôi/bỏ khung.

12.7c. Kiểm thử giao thức tách khỏi thuật toán

Trước khi chạy GPE/VDE, hãy có bộ test *chỉ* pack/unpack:

- Vector F_0 biết trước → so byte bin với bảng mục 12.3.
- Cố ý unpack BE → assert fail rõ ràng (bắt regression).
- Đếm `len(bin)==2*n_frames` với n_frames từ wav.
- Khung UV xen kẽ: mọi vị trí UV là `00 00`, không "rò" 1 LSB.

Chỉ khi bộ này xanh mới mở tầng so Praat. Quy tắc này tiết kiệm ngày debug "vì sao GPE $= 1$ trên sine hoàn hảo".

12.7d. Sai số lượng tử Q12.4 trong ngân sách cent

Tại F_0 gần sàn 75 Hz, một LSB $0{,}0625$ Hz tương đương

$$1200 \log_2\big((75 + 0{,}0625)/75\big) \approx 1{,}443 \text{ cent},$$

vẫn nhỏ hơn ngân sách 20 cent. Tại trần 600 Hz:

$$1200 \log_2\big((600 + 0{,}0625)/600\big) \approx 0{,}180 \text{ cent}.$$

Vậy Q12.4 "thô" nhất ở tần số thấp nhưng vẫn không phải thủ phạm GPE (script verify khẳng định hai số cent trên). Thủ phạm giao thức (endian) tạo sai số hàng nghìn Hz — lớn hơn mọi lượng tử.

Ví dụ tính tay — làm tròn nửa LSB. $F_0 = 200{,}03125$ Hz (đúng nửa LSB trên 200): $200{,}03125 \cdot 16 = 3200{,}5$. Quy ước `round` half-away / banker's tùy ngôn ngữ; Python `round` dùng banker's (về số chẵn) nên `round(3200.5)`=3200. Trên RTL, thường cắt hoặc cộng $0{,}5$ rồi trunc — phải khóa một quy ước trong twin và TB. Lab khuyến nghị: `int(floor(x*16+0.5))` (half-up) cho khớp nhiều LPM. Với $199{,}466$, cả hai quy ước đều cho 3191 vì $3191{,}456$ không nằm nửa.

12.8. Bài tập và lời giải giải thích

Bài tập 12.1. Pack 220 Hz: kỳ vọng F_q và hai byte LE?

Lời giải. $220 \cdot 16 = 3520 = 0\text{x}0\text{DC}0$. LE: `C0 0D`. Khớp bảng mục 12.3 và script verify.

Bài tập 12.2. Đổi big-endian cố ý trên 200 Hz: evaluator đọc ra bao nhiêu Hz?

Lời giải. Bytes thật `80 0C`; đọc BE cho $0\text{x}0\text{C}80 = 32780$, $32780/16 = 2048{,}75$ Hz. Sai số hàng nghìn Hz — rõ ràng là lỗi giao thức, không phải MAC.

Bài tập 12.3. Vì sao UV phải là `00 00` chứ không phải $0{,}0625$ Hz ($q = 1$)?

Lời giải. $q = 1$ vẫn là "có cao độ" rất thấp; các chỉ số VDE/FFE và logic thanh điệu sẽ coi khung đó hữu thanh. Hợp đồng lab: đúng một mã 0 duy nhất cho Unvoiced/Silence sau cổng Chương 10.

Bài tập 12.4. Tách bit 3191: phần nguyên, frac, $\hat{F}_0$?

Lời giải. $3191 \gg 4 = 199$, $3191\&15 = 7$, $\hat{F}_0 = 199 + 7/16 = 199{,}4375$ Hz. Đã kiểm script.

Chương 13. Kiểm chứng ba tầng: float / fixed / RTL

▶ MỤC TIÊU HỌC TẬP

Sau Chương 13, người học sẽ:

1. Vận hành được quy trình kiểm chứng ba tầng: (1) mô hình float đối chiếu Praat; (2) mô hình dấu phẩy tĩnh twin đối chiếu float; (3) RTL/sim đối chiếu twin bit-exact.
2. Định nghĩa và tính tay được ba chỉ số cổ điển **GPE**, **VDE**, **FFE** (Gross Pitch Error, Voicing Decision Error, Fine/F0 Frame Error) kèm ví dụ số đã kiểm Python.
3. Viết testbench Avalon-ST có `valid`-gap và backpressure, đếm beat, so Q12.4 với golden.
4. Điền bảng tài nguyên từ báo cáo Fitter thật (không bịa số), và giải thích vì sao mỗi tầng bắt một lớp lỗi khác nhau.

13.1. Hiện tượng thật: ba tầng bắt ba lớp lỗi

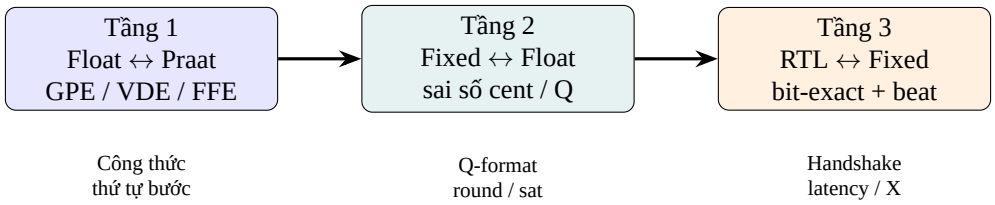
★ NGUYÊN LÝ ĐẦU TIÊN: Silicon không chạy Python — hãy khoanh đúng lớp

Một đường F_0 sai có thể đến từ rất nhiều chỗ: nhầm công thức Boersma, sai Q-format, sai handshake, quên bão hòa, đảo endian. Nếu nhảy thẳng vào Signal Tap trên DE10, bạn sẽ mất ngày để đoán. Ba tầng kiểm chứng tách bạch ba lớp:

- **Tầng 1 — Float vs Praat:** cùng wav, cùng time step 10 ms, cùng floor/ceiling 75–600 Hz. Sai ở đây là sai *thuật toán / thứ tự bước* (quên chuẩn hóa $C[\tau]$, sai parabolic, tắt nhầm silence gate).
- **Tầng 2 — Fixed twin vs Float:** cùng công thức nhưng mọi phép toán dùng Q$m.n$, round/sat/shift giống RTL. Sai ở đây là sai *lượng tử hóa và làm tròn*.
- **Tầng 3 — RTL vs Fixed:** ModelSim/Questa hoặc Signal Tap dump từng mẫu/từng khung. Mục tiêu lab: 0 **bit mismatch** trên vector golden. Sai ở đây là sai *handshake, latency, reset, X-propagation*.

Nhảy thẳng tầng 3 khi tầng 1 còn đỏ chính là lãng phí ngày debug FSM.

Ba vector bắt buộc mọi khối tiền xử lý (đã nêu Chương 3): `zero` (toàn 0), `DC` (hằng khác 0), `clip` (đụng ± 32767). Thêm vector sine $75/100/200/440/590$ Hz, nhiễu trắng, âm xát $/s/$, và hai đoạn giọng thật — cùng tinh thần `verify_tiers.py`.



Hình 55: Ba tầng kiểm chứng: mỗi tầng khoanh một lớp lỗi; chỉ xuống tầng dưới khi tầng trên đã xanh trên cùng bộ vector.

13.2. Toán: GPE, VDE, FFE — định nghĩa từng ký hiệu

Trong đánh giá cao độ chuẩn (Rabiner, Chu & Alwan, v.v.), mỗi khung được gắn nhãn hữu thanh/vô thanh bởi tham chiếu (ở đây: Praat) và bởi hệ thống (float / fixed / RTL). Ký hiệu:

- N: tổng số khung có mặt cả hai hệ thống.
- N_{VV}: số khung cả tham chiếu lẫn hệ thống đều hữu thanh.
- $F_{\text{ref}}[k]$, $F_{\text{hw}}[k]$: cao độ (Hz) trên các khung cùng hữu thanh.

GPE — Gross Pitch Error (Sai cao độ thô).

$$\text{GPE} = \frac{1}{N_{VV}} \left| \left\{ k \in VV : \left| F_{\text{hw}}[k]/F_{\text{ref}}[k] - 1 \right| > 0{,}2 \right\} \right|. \tag{44}$$

Ngưỡng 20% roughly một quãng gần (khoảng lớn hơn sai số tinh). GPE cao thường báo octave error ($F_0/2$ hoặc $2F_0$) — đúng bệnh Chương 8–10 khi quên $C[\tau]$ hoặc voicing nuốt đỉnh đúng.

VDE — Voicing Decision Error (Sai quyết định hữu/vô thanh).

$$\text{VDE} = \frac{N_{\text{FV}} + N_{\text{FU}}}{N}, \tag{45}$$

với N_{FV} = false voiced (hệ thống báo V khi tham chiếu UV), N_{FU} = false unvoiced (ngược lại). VDE cao $\Rightarrow$ xem lại ngưỡng V, S, warmup P_{global} (Chương 10).

FFE — F0 Frame Error (gộp). Nhiều báo cáo lab định nghĩa FFE là tỷ lệ khung "hỏng" theo nghĩa *hoặc* sai voicing *hoặc* GPE trên khung cùng hữu thanh:

$$\text{FFE} = \frac{N_{\text{VDE frames}} + N_{\text{GPE frames}}}{N}. \tag{46}$$

(Một số tài liệu gọi "Fine Error" là sai số cent trung bình trên khung *không* GPE; lab này dùng FFE dạng frame-error tổng hợp như trên và ghi rõ trong báo cáo.)

Sai số cent trên khung "tinh".

$$\Delta_{\text{cent}}[k] = 1200 \log_2\big(F_{\text{hw}}[k]/F_{\text{ref}}[k]\big). \tag{47}$$

Ngân sách lab trên sine/giọng sạch: $\max|\Delta_{\text{cent}}| < 20$—$30$ cent sau khi đã loại khung GPE (nếu có). Mỗi stage (DC, FIR lượng tử, window Q1.15, MAC cắt bit, $C[\tau]$ Q3.13, parabolic, LPM_DIVIDE, Q12.4) góp vài cent; vượt ngân sách thì lần ngược từ peak về FIR, không đoán.

13.3. Ví dụ tính tay — chỉ số và cent (đã kiểm Python)

Cent. $F_{\text{hw}} = 199{,}466\,\text{Hz}$ (parabolic Chương 9), $F_{\text{ref}} = 200\,\text{Hz}$:

$$\Delta_{\text{cent}} = 1200 \log_2(199{,}466/200) \approx -4{,}629\,\text{cent}.$$

Nằm trong ngân sách 20 cent.

Đồ thị nhầm lẫn giả định $N = 20$ khung. Giả sử $N_{VV} = 14$, trong đó 2 khung lệch $> 20\%$ (octave); thêm 3 khung sai voicing (không trùng hai khung GPE):

$$\text{GPE} = \frac{2}{14} \approx 0{,}1429, \qquad \text{VDE} = \frac{3}{20} = 0{,}15, \qquad \text{FFE} = \frac{5}{20} = 0{,}25.$$

Script `verify/verify_chuong_10_14.py` khẳng định các tỷ số này. Trên sine sạch lab, mục tiêu thực tế: GPE ≈ 0, VDE thấp, $|\Delta_{\text{cent}}| < 20$.

LSB Q12.4. Tại 200 Hz, $0{,}0625\,\text{Hz} \approx 0{,}541$ cent — nhỏ hơn nhiều so với $-4{,}6$ cent của parabolic ở ví dụ trên, nên tầng pack hiếm khi là thủ phạm GPE.

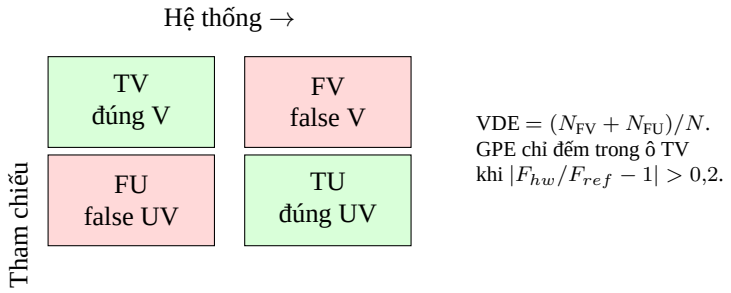
Ví dụ tính tay — ma trận nhầm lẫn năm khung. Cho tham chiếu Praat và hệ thống (Hz; 0 = UV):

k	0	1	2	3	4
F_{ref}	200	200	0	180	0
F_{hw}	199,5	100	0	0	50
Nhãn lỗi	tinh	GPE octave	OK UV	false UV	false V

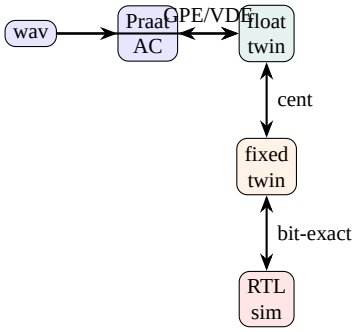
Khung cùng hữu thanh $VV = \{0, 1\}$ ($N_{VV} = 2$). Khung 1 có $|100/200 - 1| = 0{,}5 > 0{,}2 \Rightarrow N_{\text{GPE}} = 1$. Sai voicing ở $k = 3$ (FU) và $k = 4$ (FV) $\Rightarrow N_{\text{VDE}} = 2$.

$$\text{GPE} = \frac{1}{2} = 0{,}5, \qquad \text{VDE} = \frac{2}{5} = 0{,}4, \qquad \text{FFE} = \frac{1+2}{5} = 0{,}6.$$

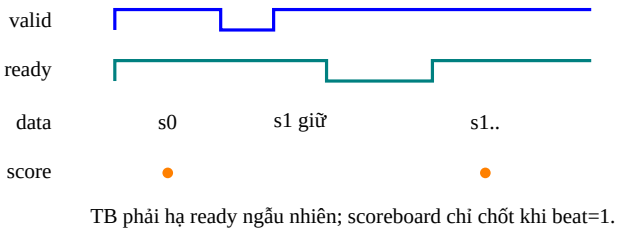
Script verify khẳng định $N_{\text{GPE}} = 1$, $N_{\text{VDE}} = 2$ trên đúng vector này.



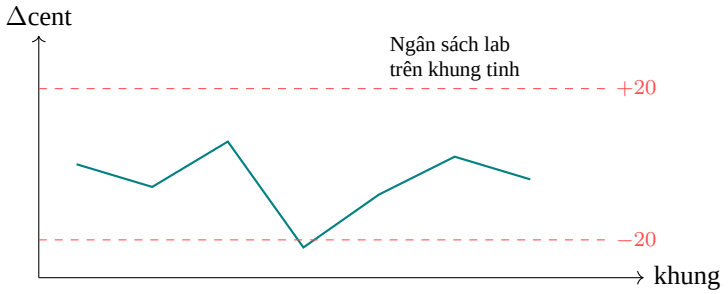
Hình 56: Ma trận hữu/vô thanh: VDE lấy hai ô đỏ; GPE lấy tập con của ô TV.



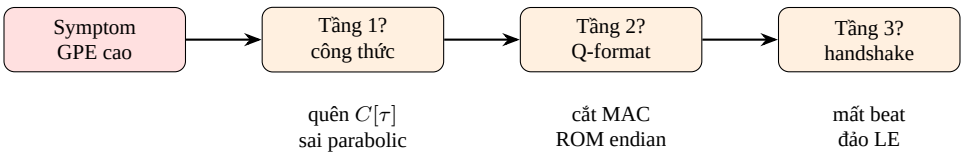
Hình 57: Luồng đối sánh tầng 1→2→3 trên cùng wav.



Hình 58: Testbench Avalon-ST: valid-gap + backpressure; chỉ chấm cam được so golden.



Hình 59: Sai số cent khung-tinh phải nằm trong ±20—30 cent sau khi loại GPE.



Hình 60: Cây chẩn đoán: luôn hỏi tầng trên trước khi đụng Signal Tap.

13.4. Tầng 1: Float vs Praat

Thực hành khuyến nghị (Parselmouth hoặc Praat batch):

```
snd = parselmouth.Sound(wav, sampling_frequency=16000)
```

```
pm = snd.to_pitch_ac(time_step=0.01, pitch_floor=75, pitch_ceiling=600)
f_ref = pm.selected_array["frequency"]  # 0 = UV
# f_float: twin float_pitch_frame tung hop
# Do GPE/VDE/FFE chi tren khung overlap thoi gian
```

Tham số bắt buộc khớp lab: time step $0{,}01$ s, floor 75, ceiling 600, và (nếu đối chiếu sâu) cùng voicingThreshold $0{,}45$. Khác tham số $\Rightarrow$ "sai số" giả tạo.

13.4b. Tầng 1 sâu: khớp tham số Praat

Ngoài time step / floor / ceiling, các tham số hay làm lệch giả tạo:

- `voicingThreshold` (mặc định $0{,}45$): khác số $\Rightarrow$ VDE giả.
- `octaveCost` / `octaveJumpCost`: twin float lab tắt pathFinder thì không so được đường cong "có Viterbi" của Praat GUI mặc định — phải gọi API/`Pitch.cpp` với cùng cờ tắt, hoặc chấp nhận so trên khung đã voiced ổn định.
- Đồng bộ thời gian: tâm cửa sổ lab tại $kH + L/2$; Praat time step $0{,}01$ s phải neo cùng gốc.

Ví dụ tính tay — RTF mô phỏng 1 s. Số khung 97, mỗi khung $\approx 120\,960$ chu kỳ MAC:

$$T_{\text{proc}} = \frac{120\,960 \cdot 97}{50 \cdot 10^6} \approx 0{,}2347\,\text{s}, \qquad \text{RTF} \approx 0{,}2347$$

(RTF = *Real-Time Factor* = thời gian xử lý / thời gian âm thanh; đã kiểm script). Một TB chu kỳ-chính xác cho 1 s audio vì thế chạy khoảng $0{,}23$ s thời gian mô phỏng xung nhịp (wall-clock phụ thuộc tốc độ sim).

13.5. Tầng 2: Fixed twin vs Float

Twin cố định bit dùng cùng ROM Hanning Q1.15, ROM $C[\tau]$ Q3.13, cùng sat/shift như RTL. Đo:

- Sai lệch cent trung bình / max giữa F_{fixed} và F_{float} trên khung cùng voiced;
- Số khung đổi voicing so với float (thường 0 trên sine).

Nếu tầng 1 xanh mà tầng 2 đỏ: nghi ngờ cắt bit MAC, quên dịch BFP (*Block Floating Point* — mỗi khối FFT chia sẻ một exponent; nếu dùng FFT IP), hoặc ROM $C[\tau]$ sai endian trong hex.

13.5b. Tầng 2 sâu: nguồn cent cố định bit

Mỗi khối góp sai số lượng tử xấp xỉ (thứ tự độ lớn, không phải định lý):

- Window Q1.15: hệ số mép khung lệch 1 LSB $\rightarrow$ vài cent.
- MAC cắt về thanh ghi trung gian: nếu cắt quá sớm trước khi nhân $C[\tau]$, đỉnh τ lớn bị thiên vị.
- ROM $C[\tau]$ Q3.13: sai endian hex $\rightarrow$ GPE hàng loạt (lớp này đôi khi lộ sớm ở tầng 1 nếu float cũng đọc nhầm hex — tách test ROM).
- LPM_DIVIDE làm tròn quotient: thường < 1 cent tại 200 Hz.
- Pack Q12.4: LSB $\approx 0{,}54$ cent tại 200 Hz.

Tổng ngân sách 20–30 cent là "hộp" chứa cho cả chuỗi; vượt hộp thì lần từ pack ngược về FIR, đo cent từng điểm neo (sau window, sau MAC, sau $C[\tau]$, sau DIV).

13.6. Tầng 3: RTL vs Fixed — bit-exact và testbench

Hợp đồng bit-exact. Trên cùng `golden_pcm.hex`, mọi khung Q12.4 dump từ sim phải bằng twin fixed: 0 bit lệch. Không "gần đúng" ở tầng này.

Testbench Avalon-ST.

- Đọc PCM hex → stream sample với `valid` có gap ngẫu nhiên (LFSR).
- Sink hạ `ready` ngẫu nhiên ở mọi stage (đặc biệt MAC↔peak, pack↔HPS).
- Đếm beat vào = đếm beat ra (trừ decimate cố ý).
- Scoreboard so F_q với `golden_f0.hex`.
- Vector cạnh: zero / DC / clip phải cho $F_q = 0$ hoặc hành vi DC-blocker đã định nghĩa (Chương 3).

```
delta_cent = 1200 * np.log2(f_hw / f_praat)
assert np.nanmax(np.abs(delta_cent[voiced_both])) < 20.0
assert (q_rtl == q_fixed).all()  # tang 3
```

Tài liệu Intel cần mở đúng số. Avalon 683091, FIR 683208, FFT 683374 (nếu bật), LPM_DIVIDE handbook. Fmax/ALM/DSP/M10K lấy từ *Fitter report* sau Place & Route — không bịa.

13.6b. Tầng 3 sâu: X-propagation và reset

RTL sim phải:

1. Khởi động `rst_n`$= 0$ đủ vài chu kỳ; mọi thanh ghi median/FSM về 0.
2. Không để `X` (trạng thái chưa xác định trong mô phỏng RTL — bit chưa được gán $0/1$ sau reset) trên `valid`/`ready`: một `X` làm scoreboard "xanh giả" hoặc đỏ ngẫu nhiên.
3. Dump F_q chỉ khi beat OUT; so với `golden_f0.hex` từ twin fixed cùng seed LFSR cho ready.
4. Vector cạnh zero/DC/clip: kỳ vọng $F_q = 0$ (hoặc hành vi DC-blocker đã khóa Chương 3).

13.6c. Golden hex và tái lập bit-exact

Mỗi revision RTL phải kèm:

- `golden_pcm.hex` — vector vào (hoặc WAV gốc + script gen).
- `golden_f0.hex` — chuỗi F_q từ twin fixed cùng tham số.
- Seed LFSR cho valid-gap/ready trong TB (ghi vào header test).

Không tái lập được seed thì "bit-exact hôm qua" không chứng minh được hôm nay. Khi đổi V, S, hoặc ROM $C[\tau]$, phải regenerate golden và ghi chú DIFF.

Ví dụ tính tay — ngưỡng GPE 20%**.** Tại $F_{\text{ref}} = 200\,\text{Hz}$, biên GPE là 160 và $240\,\text{Hz}$. Mọi ước lượng trong $(160, 240)$ trên khung VV *không* tính GPE (chỉ đóng góp cent tinh). Octave đúng 100 hoặc 400 nằm ngoài biên — GPE chắc chắn. Đó là lý do GPE cao thường chỉ thẳng ROM $C[\tau]$ hoặc voicing nuốt đỉnh đúng rồi bám hài.

FFE và báo cáo. Khi viết báo cáo lab, luôn định nghĩa FFE một lần (công thức (46)) và không trộn với "fine error trung bình cent". Người chấm cần biết bạn đang đếm *tỷ lệ khung hỏng* chứ không phải trung bình cent.

▶ NHÌN THẤY VẬT LÝ

Ba tầng không phải nghi thức nộp bài. Chúng là cách khoanh vùng nhanh: công thức, lượng tử, hay dây nối. Một TB "end-to-end đỏ" không nói bạn sửa chỗ nào; ba đèn xanh/đỏ nói được.

13.7. Bảng tài nguyên (điền từ Fitter)

Khối	ALM	DSP	M10K	Latency (ck)
PLL + I^2S				
DC-blocker				
FIR $2/3$				
Sliding RAM				
Window				
MAC ACF				
ROM $C[\tau]$				
Peak + DIV				
Voicing + median-3				
FSM + pack Q12.4				
Tổng (mục tiêu $\lesssim 850$ ALM, 3 DSP, 4 M10K)				

Khi nộp bài (Chương 14), không được để trống các ô tổng nếu đã chạy Fitter — điền số thật kèm ảnh chụp report.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách xác minh — giải thích từng mục.

- **Thời gian sim:** vài giây đến vài phút cho 1 s audio @ 50 MHz TB (tuỳ độ chi tiết dump).
- **Disk:** golden hex vài MB — commit cùng revision RTL để tái lập.
- **Người:** không bỏ tầng float; mỗi fail gắn đúng tầng (GPE → thuật toán; cent fixed → Q-format; bit mismatch → RTL).
- **Vì sao ba tầng:** một TB "tất cả" không chỉ ra *loại* lỗi; tách tầng rút ngắn chu kỳ sửa.

13.7b. Cách điền Fitter thật

Sau Place & Route trên Quartus Standard/Lite cho part `5CSXFC6D6F31C6N`:

1. Mở *Fitter* → *Resource Usage Summary*: copy ALM logic, DSP blocks, M10K.
2. Mở *Timing Analyzer*: Fmax của clock 50 MHz (hoặc clock datapath thật).
3. Breakdown theo hierarchy nếu tool cho phép — tách MAC, FIR, FSM.
4. Không lấy số từ Synthesis sớm (trước Fitter) làm số nộp: synthesis ước tính, Fitter mới là silicon mapping.

Mục tiêu Chương 0: $\lesssim 850$ ALM, 3 DSP, 4 M10K, RTF $\approx 0{,}24$. Vượt M10K thường do FIFO/RAM thừa; vượt DSP do nhân cửa sổ + MAC + FIR không chia sẻ — xét time-multiplex trước khi xin thêm.

13.7c. Liên hệ RTF, sim wall-time và Signal Tap

RTF $\approx 0{,}23$ trên 1 s audio nghĩa là mô phỏng chu kỳ-chính xác "bắt kịp" nhanh hơn realtime trên giấy; nhưng ModelSim/Questa có thể chậm hơn realtime tường vì diễn dịch. Đừng nhầm wall-time sim với RTF silicon. Signal Tap trên board đo realtime thật: nếu FIFO đầy liên tục, backpressure đang ăn biên $377\,000$ chu kỳ — cần giảm dump hoặc tăng FIFO, không thêm DSP.

Ví dụ tính tay — đếm khung overlap thời gian. Praat và twin phải so trên cùng tập thời điểm. Nếu Praat trả 100 khung và twin 97 khung cho 1 s (do neo cửa khác), chỉ lấy giao khoảng thời gian chung, ví dụ 90 khung giữa. Tính GPE/VDE trên $N = 90$ đó; không pad 0 giả vào đuôi. Sai bước này tạo VDE giả ở biên câu.

Chuẩn bị báo cáo. Mỗi tầng một mục: lệnh chạy, tham số, bảng chỉ số, kết luận xanh/đỏ. Đính kèm hash git của RTL + twin + golden. Người chấm tái lập được thì điểm phương pháp mới vững.

Trước khi sang Chương 14. Tầng 1 cần GPE≈ 0 trên sine sạch; tầng 2 max cent < 20; tầng 3 không lệch bit và đếm beat khớp. Ngoài zero/DC/clip/sine, giữ một đoạn xát vô thanh (ví dụ $/s/$) và một đoạn giọng có thanh điệu — tránh VDE "xanh giả" vì toàn hữu thanh, và để Pearson có nghĩa.

13.8. Bài tập và lời giải giải thích

Bài tập 13.1. Chạy twin DC-blocker trên zero / DC / clip. Kỳ vọng đầu ra định tính?

Lời giải. Zero $\to$ zero. DC hằng $\to$ sau vài hằng số thời gian α, đầu ra $\to 0$ (zero tại $z = 1$). Clip ± 32767 xen kẽ $\to$ không được wrap thành âm lớn; sat_16 giữ biên. Nếu clip tạo dao động limit-cycle, nghi `y_acc` quá hẹp (Chương 3).

Bài tập 13.2. Cố ý đảo endian Q12.4 trên golden $200\,\text{Hz}$. GPE/cent thay đổi thế nào?

Lời giải. Server đọc $2048{,}75\,\text{Hz}$ (Chương 12). $|2048{,}75/200 - 1| \gg 0{,}2 \Rightarrow$ mọi khung đó thành GPE; Δ_{cent} khổng lồ. Tầng 1 "đỏ" dù thuật toán đúng — phải bắt bằng test pack/unpack trước khi đổ tội MAC.

Bài tập 13.3. Cho $N = 20$, $N_{VV} = 14$, 2 GPE, 3 VDE (rời nhau). Tính GPE, VDE, FFE.

Lời giải. GPE $= 2/14 \approx 0{,}1429$, VDE $= 3/20 = 0{,}15$, FFE $= 5/20 = 0{,}25$ — khớp script verify.

Chương 14. Checklist nghiệm thu cuối cùng (kèm lý do)

▶ MỤC TIÊU HỌC TẬP

Trước khi nộp bài hoặc đóng gói bitfile cho DE10-Standard, người học sẽ:

1. Đọc từng mục checklist như một câu khẳng định có thể ký tên, kèm *lý do kỹ thuật* neo về chương đã học — không chỉ "tick cho có".
2. Chẩn đoán nhanh các triệu chứng đỏ điển hình (Q12.4 = 0, $F_0/2$, đảo endian, FSM treo) và chỉ đúng tầng kiểm chứng cần mở lại.
3. Điền bảng tự ký (RTL lint, twin bit-exact, smoke phần cứng, tài liệu đủ tám yếu tố) với ngày theo múi giờ làm việc.
4. Hiểu vì sao checklist chỉ xuất hiện **một lần** ở Chương 14: tránh lan checklist trùng, mọi khẳng định đã được giải thích ở Chương 2–13.

14.1. Hiện tượng thật: checklist bắt lỗi hệ thống, không phải nghi thức

★ NGUYÊN LÝ ĐẦU TIÊN: Một mắt xích sai — synth vẫn xanh

Đường F_0 trên FPGA là chuỗi quyết định đã khóa từ Chương 0: MCLK 12,288 MHz, M10K Old Data, Hanning $N-1$, ROM $C[\tau]$, parabolic + LPM_DIVIDE, voicing + median-3, Avalon-ST giữ data, Q12.4 LE, Viterbi tắt trên fabric. Sai một mắt xích thường *không* làm Quartus báo lỗi — nó làm F_0 lệch im lặng, GPE/VDE ở Chương 13 đỏ, và Pearson thanh điệu sụp. Checklist buộc bạn xác nhận lại từng mắt xích bằng **bằng chứng** (twin xanh, Fitter report, waveform Avalon, Signal Tap), không chỉ "nhớ là đã làm".

14.2. Chuỗi khẳng định có lý do (ký từng dòng)

1. **Chuỗi đồng hồ và lấy mẫu.** PLL khóa thạch anh 50 MHz tạo $MCLK = 12{,}288$ MHz; ADC codec 48 kHz; hạ mẫu về 24 kHz rồi FIR polyphase tỷ số $2/3$ về 16 kHz.
   - *Lý do:* Sai MCLK vài chục ppm làm trôi T_s, đo lệch F_0 theo thời gian (Chương 0 Feynman box 8; Chương 2). Không dùng MCLK 4,096 MHz cho gói WM8731 trên DE10 này.
2. **DC-blocker.** Thanh ghi phản hồi `y_acc` rộng 32 bit; cập nhật $y \leftarrow y + x_\Delta - (y \ggg 8)$; xuất $\mathrm{sat}_{16}(y \ggg 15)$. Twin `dc_blocker_q15` khớp từng mẫu.
   - *Lý do:* DC phình $\mathcal{R}[0]$ và nuốt đỉnh T_0 (Chương 3). `y` chỉ 16 bit tạo deadband/limit-cycle.
3. **Bộ nhớ trượt.** On-Chip Memory True Dual-Port; mixed-port read-during-write = **Old Data**; twin mô phỏng đúng giá trị cũ khi hai cổng đụng cùng địa chỉ.
   - *Lý do:* Cyclone V M10K **không** hỗ trợ New Data mixed-port (Chương 0, Chương 5). Ép New Data ⇒ bypass ALM, giảm Fmax.
4. **Cửa sổ.** ROM Hanning Q1.15 dạng cổ điển $N-1$, nửa bảng nhờ đối xứng; nhân trước MAC.
   - *Lý do:* Hanning hạ sidelobe ≈ -32 dB, bảo vệ dải F_0 khỏi rò phổ (Chương 0, Chương 6). Sai N vs $N-1$ lệch hệ số mép khung.
5. **Tự tương quan.** Đường mặc định MAC trực tiếp trên DSP; nếu bật FFT thì nhớ BFP và barrel shift 2^{2e} *trước* IFFT.
   - *Lý do:* Quên $2e$ phá tỷ lệ r_a và sụp ngưỡng voicing (Chương 0 Feynman box 6; Chương 7).
6. **Chuẩn hóa $C[\tau]$.** ROM $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ (nhân thay chia runtime), trừ khi bắt buộc LPM_DIVIDE động.
   - *Lý do:* Không bù dốc cửa sổ ⇒ thiên vị τ lớn ⇒ octave xuống (Chương 8).

7. **Đỉnh và chia.** Argmax + parabolic; $F_0 = f_s/\tau$ qua LPM_DIVIDE; TB chờ đúng latency L.

   - *Lý do:* Đọc quotient sớm một chu kỳ gắn nhầm khung (Chương 9). Không dùng IP bịa ALTDIV_TENT.

8. **Voicing / median-3.** Cổng silence/voicing trên fabric; median-3 đăng ký (trễ một hop); Viterbi chỉ trên HPS nếu cần.

   - *Lý do:* Báo F_0 trên UV phá đường cong; median đập spike đơn với < 150 ALM; Viterbi $O(TK^2)$ vượt ngân sách lab (Chương 10).

9. **Avalon-ST.** Handshake `valid`/`ready`/`SOP`/`EOP`: source giữ data khi backpressure; đếm beat khớp; gói 640 mẫu có SOP/EOP đúng chỗ.

   - *Lý do:* Mất một mẫu = lệch khung = sai F_0 im lặng (Chương 11; doc 683091).

10. **Đóng gói.** Q12.4 little-endian; UV $= 0$; cặp `audio_16k.wav` $\leftrightarrow$ `pitch_f0.bin` đồng bộ hop 10 ms; golden so Praat trong dung sai đã ghi.

    - *Lý do:* Đảo endian biến 200 Hz thành $2048{,}75$ Hz — lỗi giao thức (Chương 12).

11. **Twin ba tầng.** Vector zero / DC / sine / clip / giọng thật xanh từ float$\rightarrow$fixed$\rightarrow$RTL; GPE/VDE/FFE và cent nằm trong ngân sách báo cáo (Chương 13).

    - *Lý do:* Mỗi tầng khoanh một lớp lỗi; skip tầng float = debug FSM khi công thức còn sai.

12. **Báo cáo tài nguyên.** Ghi ALM (hoặc LE), DSP, M10K, Fmax, latency một khung — lấy từ Fitter, đối chiếu costbox từng chương và ngân sách Chương 0 (≈ 850 ALM, 3 DSP, 4 M10K, RTF $\approx 0{,}24$).

    - *Lý do:* Số bịa không kiểm chứng được trên máy chấm; Fmax thấp có thể do ép New Data hoặc chia tổ hợp dài.

13. **Không dùng tên IP / cấu hình bịa.** Không ALTDIV_TENT; không CIC cho tỷ số $2/3$; không MCLK $4{,}096$ MHz cho gói DE10 WM8731 này; không "New Data" mixed-port M10K trên Cyclone V.

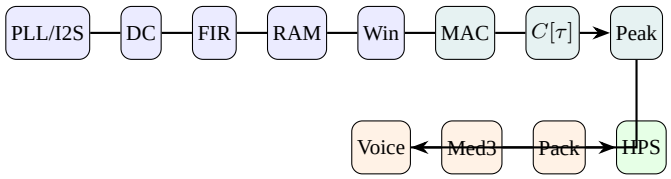
    - *Lý do:* Các lựa chọn đó hoặc không tồn tại trên tool Standard/Lite, hoặc sai silicon thật — điểm trừ cứng.

14. **Giữ nguyên v10.** Phụ lục / các trang v10 (pp.1–38 trong bản đóng gói) không bị sửa nội dung giải thích gốc.

    - *Lý do:* Chính sách DIFF: phần mới là Chương 0–14; không đụng luồng chữ đã khóa.

14.2b. Đọc checklist như chuỗi nhân quả, không như danh sách tick

Mỗi dòng mục 14.2 neo một quyết định silicon đã dạy ở chương trước. Hãy đọc chúng theo dòng tín hiệu: thạch anh nuôi PLL tạo $MCLK$ (Chương 2) — nếu nhầm tần số PLL, mọi τ sau đó lệch theo ppm tích lũy. DC-blocker giữ thanh ghi phản hồi đủ rộng (thường 32 bit) để tránh deadband (vùng lượng tử hóa thành 0 cập nhật, tạo "rung" trên im lặng). Sliding RAM trên M10K chỉ được cấu hình mixed-port **Old Data** hoặc Don't Care trên Cyclone V — không có New Data. Cửa sổ Hanning đúng mẫu số $N-1$ hạ thùy phụ (sidelobe, dải phổ phụ quanh đỉnh chính) khoảng -32 dB. MAC tuần tự tránh BFP của FFT; ROM $C[\tau]$ chống thiên vị octave xuống; LPM_DIVIDE phải chờ đủ latency L trước khi đọc thương. Cổng voicing + median-3 thay Viterbi trên fabric; Avalon-ST và Q12.4 là hai mắt xích giao thức — mất beat lệch thuật toán im lặng, đảo endian làm metric nổ dù silicon đúng.



Hình 61: Chuỗi mắt xích nghiệm thu: mỗi hộp ứng một dòng checklist có lý do neo chương.

14.3. Bảng tự ký

Hạng mục	Người ký	Ngày (Asia/Saigon)
RTL lint sạch (không latch suy đoán, reset bất đồng bộ đúng)		
Twin bit-exact tầng 2–3 xanh trên golden		
Hardware smoke trên DE10 (SOP→Q12.4 ra HPS)		
Tài liệu đủ 8 yếu tố / chương (Chương 0 mục 0.2)		
Bảng Fitter đã điền số thật		

14.3b. Quy trình smoke trên DE10 (30–60 phút)

1. Nạp bitfile; xác nhận PLL lock (LED/status reg).
2. Phát sine 200 Hz từ PC hoặc ROM nội; HPS đọc N từ Q12.4.
3. Kỳ vọng: đa số khung $F_q \in [3180, 3220]$ (tức $\approx 198{,}75$—$201{,}25$ Hz), không toàn 0, không toàn 9600.
4. Tắt nguồn âm: kỳ vọng $F_q = 0$ sau khi P_g ổn.
5. Chạy `verify_chuong_10_14.py` và (nếu có) `verify_tiers.py`; đính log.
6. Chụp Fitter: ALM/DSP/M10K/Fmax; điền bảng Chương 13.

Ví dụ tính tay — ngân sách M10K nhắc lại.

$$640 \times 16 = 10\,240 \text{ bit} = 1 \times \text{M10K (đúng dung lượng)}.$$

Nếu Fitter báo 12 M10K: nghi ping-pong thừa, FIFO dư, hoặc vô tình suy ra RAM cho Viterbi/FFT khi lab không cần.

Ví dụ tính tay — RTF board. Cùng công thức Chương 0/11: $120\,960/500\,000 \approx 0{,}2419$. Đo wall-time trên HPS (thời gian giữa hai SOP khung) phải < 10 ms trung bình; nếu > 10 ms ổn định, nghi FSM chờ chia/hand-shake kẹt — không nghi "thiếu DSP" vì MAC chỉ dùng 1 DSP.

▶ NHÌN THẤY VẬT LÝ

Checklist xanh không có nghĩa lý thuyết Praat đã "chứng minh". Nó có nghĩa: trên bộ vector đã khóa, ba tầng kiểm chứng và các ràng buộc silicon DE10 đã được xác nhận bằng chứng. Đổi L, H, hoặc ceiling thì phải chạy lại suite — không ăn theo chữ ký cũ.

14.4. Chẩn đoán đỏ → xanh trước khi nộp

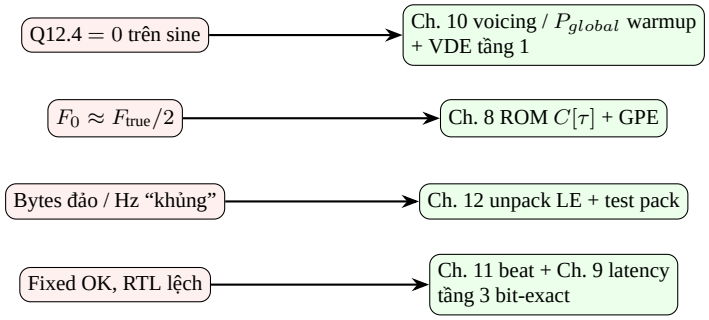
★ NGUYÊN LÝ ĐẦU TIÊN: Một triệu chứng, một nguyên nhân điển hình

- Golden sine 200 Hz ra Q12.4 $= 0$: thường voicing gate nuốt — `silenceThreshold` quá cao hoặc P_{global} chưa warmup. Mở lại Chương 10 và tầng 1 VDE.
- Ra ≈ 100 Hz thay vì 200: thường quên nhân $C[\tau]$ (đỉnh $2T_0$ thắng) hoặc octave bias. Mở Chương 8 và GPE.
- Ra 200 nhưng bytes `0C 80`: đảo endian phía đọc. Mở Chương 12 — silicon có thể đúng.
- Đúng trên twin fixed nhưng sai trên RTL: nghi latency LPM_DIVIDE, backpressure làm mất beat, hoặc reset không xóa thanh ghi median. Mở Chương 9–11 và tầng 3.
- FSM treo ở `FILL`: thiếu `EOP` hoặc đếm beat $< L$. Mở Chương 11.

Twin phải bắt từng lớp trước khi đụng Signal Tap.

Ví dụ tính tay đóng gói cuối. $F_0 = 199{,}466$ Hz (parabolic Chương 9) → round$(199{,}466 \cdot 16) = 3191$ → LE `77 0C`. UV → `00 00`. Khớp `verify/verify_chuong_10_14.py`.

Ngân sách thời gian thực nhắc lại. Tại 50 MHz, một hop 10 ms có 500,000 chu kỳ; $189 \times 640 = 120{,}960$ MAC $\Rightarrow$ RTF $\approx 0{,}2419 \ll 1$ (Chương 0 bài tập 0.2). Nếu đo RTF > 1 trên board, nghi chờ chia/handshake bị kẹt, không nghi "thiếu DSP".



Hình 62: Sơ đồ chẩn đoán nhanh: triệu chứng đỏ $\rightarrow$ chương và tầng cần mở lại trước khi nộp.

CHI PHÍ (LUT / DSP / chu kỳ)

Ngân sách "ngày nộp" — giải thích từng mục.

- **Thời gian người:** 30–60 phút đọc checklist + chạy lại suite twin/verify_chuong_10_14.py + (nếu có board) smoke HPS đọc `pitch_f0.bin`. Đây là chi phí quy trình, không phải ALM.
- **Disk:** giữ nguyên hex ROM w, $C[\tau]$, golden — không regenerate phút chót trừ khi đổi L hoặc H.
- **Vì sao checklist một lần duy nhất ở Chương 14:** tránh lan checklist trùng ở nhiều chương; mọi khẳng định đã neo Chương 2–13 với lý do đầy đủ. Chương 14 chỉ là cổng nghiệm thu.
- **RTF / silicon:** không đổi ở bước này; chỉ xác nhận số Fitter và đo RTF thực tế ≤ 1.

14.4b. Bảng triệu chứng $\leftrightarrow$ bằng chứng cần nộp

Triệu chứng	Mở chương / tầng	Bằng chứng xanh
Q12.4 $= 0$ trên sine 200 Hz	Ch. 10, VDE tầng 1	Log voiced$= 1$, P_g warmup
$F_0 \approx 100$ thay 200	Ch. 8, GPE	Dump $r_a[\tau]$: đỉnh $\tau \approx 80$ thắng sau $C[\tau]$
Bytes `0C 80`	Ch. 12 unpack	`hexdump` hiện `80 0C`
Fixed OK, RTL lệch	Ch. 9–11, tầng 3	Diff 0 bit và đếm beat khớp
FSM treo FILL	Ch. 11 SOP/EOP	Waveform có EOP tại beat 639
Fitter 12 M10K	Ch. 5, 14	Hierarchy: 1 sliding RAM + ROM
RTF đo > 1	Ch. 7, 11	Không kẹt ready; đúng latency DIV

Ví dụ tính tay — ký checklist có số. Trước khi ký mục "Twin ba tầng", hãy điền vào biên bản: GPE$= 0$, VDE$= 0{,}02$, max cent $= 7{,}3$ trên sine 200 Hz (số ví dụ — thay bằng số chạy thật). Số cụ thể biến chữ ký thành bằng chứng; ô trống biến checklist thành nghi thức.

Giữ DIFF và v10. Chính sách gói tài liệu: không sửa nội dung giải thích gốc trước lệnh `\input{chuong/...}` (các file `chuong_00`... đã khóa) và không đụng chương ngoài phạm vi giao. Mục "Giữ nguyên v10" trên checklist nhắc người nộp: khi compilation lệch, sửa chương mới hoặc preamble được phép — không "tiện tay" đụng khối v10.

14.4c. Thứ tự ưu tiên ngày nộp

1. Chạy `python3 verify/verify_chuong_10_14.py` — phải in `ALL ... PASSED` (thêm tiers nếu có).
2. Xác nhận `pitch_f0.bin` trên vector smoke: độ dài chẵn, mẫu 200 Hz đúng `80 0C`.
3. Đọc lại bốn triệu chứng đỏ ở Hình 62; nếu dính một, đừng nộp.
4. Điền Fitter + bảng tự ký với ngày Asia/Saigon.

5. Chỉ sau đó mới đóng bitfile / zip bài.

Thứ tự này cố ý để phần mềm/vector chặn lỗi giao thức trước khi bạn đổ công vào Signal Tap phút chót.

Nếu một chữ viết tắt trên checklist chưa rõ (ALM, DSP, M10K, MLAB, Fmax, RTF, GPE/VDE/FFE, HPS), mở lại costbox của chương neo — đừng ký khi còn đọc như mã bí mật.

14.4d. Cam kết ngắn trước chữ ký

Người ký bảng 14.3 xác nhận đã: (1) chạy verify script và giữ log; (2) nhìn tận mắt `hexdump` vài khung đầu của bin; (3) điền số Fitter không đoán; (4) không bật Viterbi trên fabric ở bản nộp mặc định; (5) không dùng tên IP bịa. Nếu một mục chưa làm, để trống ngày và làm nốt — chữ ký sớm hơn bằng chứng là nợ kỹ thuật sẽ lộ ở chấm phúc khảo.

14.5. Bài tập tự vấn trước khi ký

Bài tập 14.1. Liệt kê ba bằng chứng bạn sẽ đính kèm để chứng minh mục "Twin ba tầng xanh" không phải tự khai.

Lời giải gợi ý. (1) Log chạy `verify_tiers.py` / `verify_chuong_10_14.py` có timestamp. (2) Bảng GPE/VDE/FFE và max cent trên sine 200 Hz và một đoạn giọng. (3) Diff 0 bit giữa dump RTL và twin fixed trên `golden_f0.hex`.

Bài tập 14.2. Nếu Fitter báo 12 M10K trong khi ngân sách Chương 0 là 4, bạn kiểm tra khối nào trước?

Lời giải. Sliding RAM 640×16 chỉ cần ≈ 1 M10K; ROM Hanning nửa bảng và ROM $C[\tau]$ thêm 1–2; FIFO HPS tuỳ chọn 1. Nếu thấy 12, nghi nhân đôi buffer ping-pong không cần, suy luận suy ra block RAM cho Viterbi (đã tắt), hoặc FIR IP sinh RAM dư — mở report theo hierarchy trước khi chấp nhận vượt ngân sách.

Bài tập 14.3. Nêu một lý do *không* port `Pitch_pathFinder` lên fabric ở lần nộp mặc định.

Lời giải. Median-3 + cổng voicing đã đủ đập spike đơn trong ngân sách $< 2\%$ ALM; Viterbi đa ứng viên cần M10K path metric và tăng latency vượt nhu cầu thời gian thực lab; HPS vẫn chạy soft-float khi bài yêu cầu đường cong Praat đầy đủ (Chương 10).

Phụ lục A: Áp dụng lý thuyết F_0/Praat vào mã nguồn Python (`audio_processing.py` và `evaluator.py`)

▶ MỤC TIÊU HỌC TẬP

Phạm vi và Mục tiêu: Phụ lục này đối chiếu hai tệp mã nguồn xử lý âm thanh trong thư mục `Pronunciation_Final_Master` của kho lưu trữ `DucDocDao/Pitch-Project` (nhánh `main`) với hệ thống lý thuyết đo cao độ F_0 bằng thuật toán tự tương quan trong tài liệu này (chuẩn Praat, Boersma 1993). Để làm sáng tỏ toàn bộ luồng gọi hàm, phụ lục khảo sát thêm các đoạn mã liên kết trong `main.py`, `visualizer.py`, `generate_audio.py` và `sentence_data.py`. Mọi hành vi mô tả dưới đây đều được truy xuất trực tiếp từ mã nguồn thực tế và đối chiếu tường minh với mã nguồn C++ gốc của Praat (`fon/Sound_to_Pitch.cpp` và `fon/Pitch.cpp`).

1. Bức Tranh Tổng Thể: Hai Tệp Làm Gì Trong Hệ Thống

1.1. Ranh giới triển khai: Mã nguồn Python nguyên bản vs. Hệ thống FPGA Edge Node

Trước khi đi sâu vào mã nguồn của từng hàm, cần định vị dự án `Pitch-Project` trong bức tranh kiến trúc tổng thể **FPGA Edge Node** $\leftrightarrow$ **Shared Folder** $\leftrightarrow$ **Server** đã thiết lập tại Hình 1 (Mục 0.1):

- **Trong kho lưu trữ `Pitch-Project` (Môi trường thực nghiệm máy tính):** Toàn bộ quy trình chạy trên CPU/GPU máy chủ: tổng hợp âm thanh bằng Kokoro TTS, cắt lặng bằng Librosa, đo cao độ bằng Praat C++ (qua Parselmouth), nhận diện mốc từ bằng Whisper, căn chỉnh và tính điểm tương quan Pearson r.
- **Khi triển khai thực tế trên vi mạch FPGA (Hệ thống mục tiêu):** Bo mạch FPGA DE10-Standard gánh trọn vẹn khâu tiền xử lý (HPF 60 Hz và cắt lặng) và khối đo cao độ tự tương quan tài liệu này, xuất ra tệp âm thanh sạch và mảng nhị phân `pitch_f0.bin` ra Thư mục dùng chung (Shared Folder). Máy chủ tiếp nhận âm thanh sạch để chạy Whisper (hoặc trích Log-Mel) lấy mốc từ, rồi đọc trực tiếp `pitch_f0.bin` để căn chỉnh và chấm điểm r, **hoàn toàn không cần cài đặt hay chạy Praat/Parselmouth**.

Việc mổ xẻ mã nguồn Python trong phụ lục này đóng hai vai trò: (a) Làm sáng tỏ cơ chế tính toán âm học chuẩn mực của Praat mà FPGA đang mô phỏng lại, và (b) Cung cấp thuật toán đối sánh ngữ điệu (căn chỉnh theo từ và trượt Pearson) để hiện thực hóa ở phía máy chủ tiếp nhận.

1.2. Bài toán của dự án

Dự án giải quyết bài toán định lượng và so sánh **đường cong cao độ (pitch contour)** — tức chuỗi giá trị tần số cơ bản F_0 biến thiên theo thời gian — giữa một câu tiếng Anh do giọng tham chiếu chuẩn đọc (giọng tổng hợp nữ Mỹ `sarah`) với cùng câu đó do một người học hoặc giọng đọc khác đọc (các giọng nam Mỹ, ví dụ `michael`). Tín hiệu âm thanh được tổng hợp tự động bằng mô hình Kokoro TTS (`generate_audio.py`).

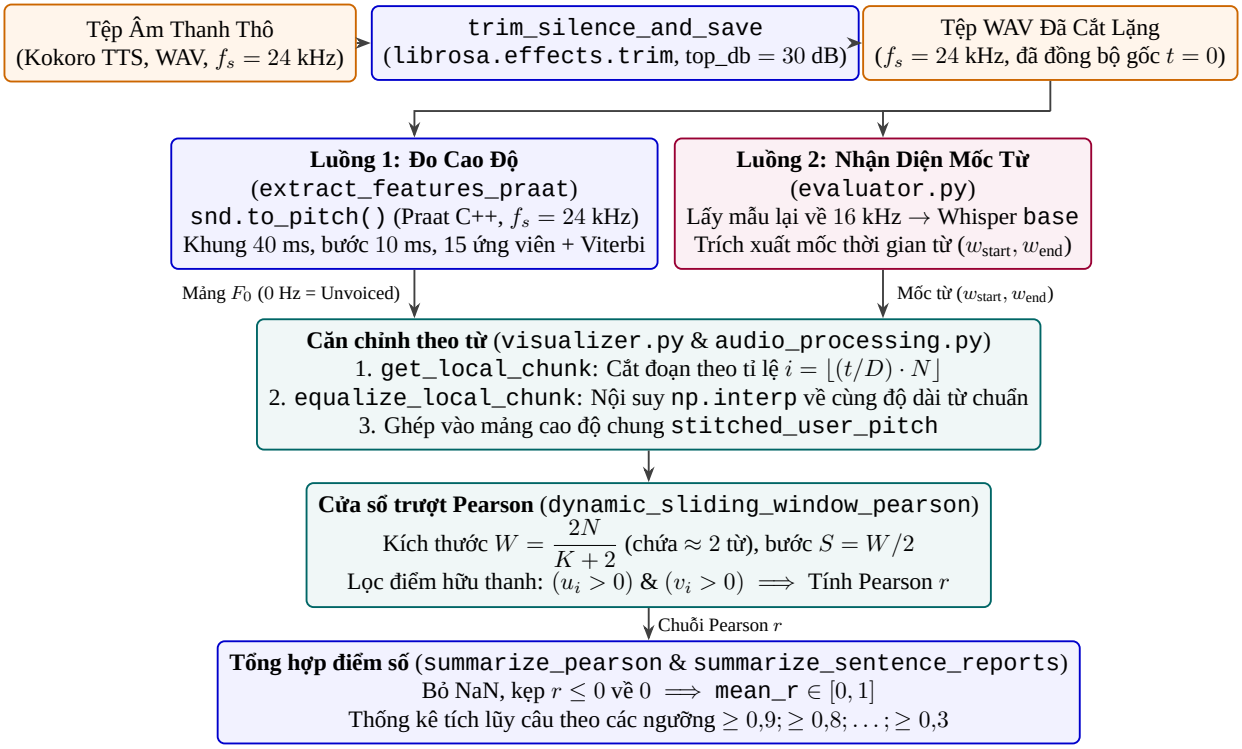
Kết quả đánh giá cuối cùng là hệ số tương quan Pearson (r) giữa hai đường cao độ đã được căn chỉnh thời gian. Đây chính là dạng bài toán mà tài liệu này (phần 0) đã khẳng định: *"đối sánh tương quan Pearson (r) với các công cụ âm học chuẩn mực phòng thí nghiệm như Praat"*.

1.3. Luồng xử lý một câu nói (theo `main.py`)

Quá trình xử lý một câu hoàn chỉnh diễn ra theo chuỗi 5 bước tuần tự:

1. `trim_silence_and_save` (`audio_processing.py`): Cắt bỏ khoảng lặng ở hai đầu tệp âm thanh và ghi ra tệp WAV đã cắt.
2. Hàm `extract_features_praat` (`audio_processing.py`): Đo đường cao độ F_0 trên **chính tệp WAV đã cắt** bằng Praat (qua thư viện `parselmouth`).
3. `get_user_timestamps` (`evaluator.py`): Nạp mô hình Whisper (`openai/whisper`) bóc tách mốc thời gian từ ($w_{\text{start}}, w_{\text{end}}$) trên **chính tệp WAV đã cắt**.
4. `plot_analysis` (`visualizer.py`): Gọi 4 hàm căn chỉnh và tính Pearson r (cắt từ, co giãn nội suy, cửa sổ trượt, tổng hợp điểm) trong `audio_processing.py`.
5. `summarize_sentence_reports` (`audio_processing.py`): Gom toàn bộ kết quả của nhiều câu thành bảng thống kê phân phối độ chính xác theo các ngưỡng tích lũy.

Ghi chú quan trọng trong `main.py` (dòng 72): *"Both pitch and timestamps must refer to exactly this trimmed WAV"*. Điều này bảo đảm cả đường F_0 lẫn mốc thời gian từ đều dùng chung một gốc thời gian $t = 0$; nếu đo trên hai tệp khác nhau (trước và sau khi cắt), trục thời gian sẽ bị lệch nhau đúng bằng thời lượng đoạn lặng ban đầu bị cắt bỏ.



Hình 63: Sơ đồ luồng xử lý âm thanh, trích xuất hai luồng song song (Praat F_0 và Whisper Timestamps) và thuật toán đối sánh ngữ điệu trong mã nguồn Python (`audio_processing.py` và `evaluator.py`).

Đọc Hình 63 (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Khối Tệp Âm Thanh Thô và Tiền Xử Lý (Hàng 1):** Tệp âm thanh do Kokoro TTS sinh ra ở tần số 24 kHz được đưa qua hàm `trim_silence_and_save`. Hàm này dùng `librosa.effects.trim` với ngưỡng top_db $= 30$ dB (tương ứng biên độ giảm còn 3,2%) để cắt bỏ các khoảng lặng ở hai đầu tệp, sinh ra tệp WAV đã cắt làm gốc thời gian chuẩn $t = 0$ cho cả hai nhánh phía sau.

2. **Nhánh Luồng 1 — Đo Cao Độ Praat (Hàng 2, Bên trái):** Gọi hàm `extract_features_praat`. Lệnh `snd.to_pitch()` chạy mã C++ nguyên bản của Praat trên tín hiệu 24 kHz, dùng cửa sổ Hanning 40 ms (960 mẫu), bước nhảy 10 ms (240 mẫu), tính tự tương quan bù suy hao và giải thuật Viterbi 15 ứng viên. Kết quả trả về mảng `pitch` chứa tần số F_0 (quy ước 0 Hz là khung vô thanh), trong khi mảng mốc thời gian thực `xs()` bị bỏ qua.

3. **Nhánh Luồng 2 — Mốc Thời Gian Từ (Hàng 2, Phải):** Gọi hàm `get_user_timestamps` trong `evaluator.py`. Tín hiệu được lấy mẫu lại từ 24 kHz về 16 kHz để đưa vào mô hình Whisper `base` (chạy đặc trưng Log-Mel STFT). Whisper nhận dạng văn bản và xuất mốc thời gian bắt đầu và kết thúc của từng từ ($w_{\text{start}}, w_{\text{end}}$).

4. **Khối Căn Chỉnh Theo Từ (Hàng 3):** Kết hợp kết quả từ hai nhánh: `get_local_chunk` cắt đoạn cao độ theo tỉ lệ thời gian $i = \lfloor (t/D) \cdot N \rfloor$, `equalize_local_chunk` dùng `np.interp` để co giãn đoạn cao độ của người đọc về đúng số điểm của giọng tham chiếu, rồi dán vào mảng cao độ đã căn chỉnh `stitched_user_pitch`.

5. **Khối Cửa Sổ Trượt Pearson (Hàng 4):** Hàm `dynamic_sliding_window_pearson` tạo $K+1$ cửa sổ trượt có kích thước $W = 2N/(K+2)$ (chứa trọn khoảng 2 từ) và bước trượt $S = W/2$. Trong mỗi cửa sổ, chỉ giữ lại các điểm mà cả hai đường đều hữu thanh ($u_i > 0$ và $v_i > 0$) để tính hệ số tương quan Pearson r.

6. **Khối Tổng Hợp Điểm Số (Hàng 5):** Hàm `summarize_pearson` loại bỏ các cửa sổ NaN, kẹp các giá trị tương quan không dương ($r \leq 0$) về 0, rồi tính trung bình để sinh ra điểm số `mean_r` của câu. Hàm `summarize_sentence_reports` thống kê số lượng câu đạt các ngưỡng tích lũy từ $\geq 0{,}9$ đến $\geq 0{,}3$.

1.4. Hiểu lầm cần tránh ngay từ đầu

- **`evaluator.py` không thực hiện việc chấm điểm:** Mặc dù mang tên "evaluator", tệp này chỉ đảm nhận việc nạp mô hình Whisper và trả về danh sách mốc thời gian từng từ. Toàn bộ logic so khớp, căn chỉnh và tính điểm tương quan (r) đều nằm trọn bên trong `audio_processing.py` và được điều phối bởi `visualizer.py`.
- **Không có dòng mã Python nào tự tính tổng tự tương quan:** Công thức tính tự tương quan $\mathcal{R}_x[\tau]$ (Công thức (1)), cửa sổ Hanning (Công thức (5)), phép chia bù suy hao (Công thức (9)) và bảng tra đều không được cài đặt bằng mã Python tường minh. Toàn bộ các bước này được đóng gói bên trong lệnh gọi thư viện `snd.to_pitch()`. Phần 3 dưới đây sẽ giải phẫu chi tiết lệnh gọi này.

2. Bảng Thuật Ngữ Dùng Trong Phụ Lục

Thuật ngữ tiếng Việt	Thuật ngữ tiếng Anh	Định nghĩa chuẩn mực trong ngữ cảnh mã nguồn
Khung phân tích	Frame	Đoạn gồm L mẫu tín hiệu liên tiếp dùng để tính một giá trị F_0 (tài liệu này, phần 2).
Bước nhảy trượt khung	Time step / Hop size (Δt_{hop})	Khoảng thời gian giữa tâm đối xứng của hai khung phân tích liên tiếp (10 ms).
Điểm cao độ	Pitch point	Một phần tử trong mảng `pitch`: giá trị F_0 (đơn vị Hz) của khung; bằng 0 nếu là vô thanh.
Hữu thanh / Vô thanh	Voiced / Unvoiced	Trạng thái khung tín hiệu có / không có dao động tuần hoàn của dây thanh quản.
Tổng tự tương quan $\mathcal{R}_x[\tau]$	Autocorrelation sum	Công thức (1) của tài liệu này: tổng tích các cặp mẫu trễ τ, **không phải là tích chập**.
Hệ số tương quan Pearson r	Pearson correlation coefficient	Thước đo mức độ hai chuỗi số đồng biến/nghịch biến tuyến tính, nằm trong $[-1, 1]$.
Căn chỉnh thời gian	Time alignment	Đưa hai đường cao độ có độ dài khác nhau về cùng trục chỉ số để so sánh từng điểm.

Bảng 8: Bảng thuật ngữ chuyên dụng trong mã nguồn và đối sánh lý thuyết.

▶ NHÌN THẤY VẬT LÝ

Cảnh báo nghiêm ngặt về hiện tượng trùng ký hiệu: Tài liệu tài liệu này dùng ký hiệu $r_x[\tau]$ cho **hàm tự tương quan chuẩn hóa theo độ trễ** τ (Công thức (7)–(9)), trong khi mã nguồn Python dùng biến `r`, `mean_r` để chỉ **hệ số tương quan tuyến tính Pearson** giữa hai đường cao độ. Hai đại lượng này hoàn toàn khác nhau về bản chất toán học:

- $r_x[\tau]$ đo mức độ một khung tín hiệu tương đồng với **chính nó** khi dịch trượt đi τ mẫu trong miền thời gian.
- Hệ số Pearson r đo mức độ hai **đường cong cao độ F_0 của hai người nói khác nhau** có hình thái ngữ điệu lên bổng xuống trầm giống nhau hay không.

Phụ lục này luôn viết tường minh cụm từ "Pearson r" để loại trừ mọi sự nhầm lẫn.

3. `extract_features_praat`: Một Dòng Code Chứa Toàn Bộ Luồng F_0

3.1. Mã nguồn

```
def extract_features_praat(audio_path, verbose=False):
    snd = parselmouth.Sound(audio_path)
    pitch_object = snd.to_pitch()
    pitch = np.nan_to_num(pitch_object.selected_array['frequency'])
    time_stamps = pitch_object.xs()
    ...
    return pitch, time_stamps
```

3.2. Cơ chế thực thi của từng dòng lệnh

1. `parselmouth.Sound(audio_path)`: Nạp tệp WAV thành đối tượng âm thanh của Praat trong bộ nhớ C++. Thư viện `parselmouth` gọi trực tiếp vào thư viện liên kết động C++ nguyên bản của Praat, bảo đảm kết quả trùng khớp 100% với phần mềm Praat chạy trên máy tính.
2. Lệnh `snd.to_pitch()`: Tương đương với tùy chọn `Sound: To Pitch...` trên Praat với **toàn bộ 10 tham số mặc định**. Mã C++ kích hoạt hàm `Sound_to_Pitch_ac` với các tham số chuẩn của Bảng 1: time_step $=$ 0,01 s, pitch_floor $=$ 75 Hz, pitch_ceiling $=$ 600 Hz, 3 chu kỳ/khung, max_candidates $=$ 15, very_accurate $=$ off, silence_threshold $=$ 0,03, voicing_threshold $=$ 0,45, octave_cost $=$ 0,01, octave_jump_cost $=$ 0,35, voiced_unvoiced_cost $=$ 0,14.
3. `selected_array['frequency']`: Trích xuất ứng viên tối ưu toàn cục mà thuật toán quy hoạch động Viterbi đã chọn cho từng khung. Các khung vô thanh (Unvoiced) được gán giá trị 0. Hàm `np.nan_to_num` biến đổi mọi giá trị NaN tiềm ẩn thành số 0, thiết lập quy ước thống nhất xuyên suốt hệ thống: 0 **Hz** $\equiv$ **Vô thanh**.
4. `pitch_object.xs()`: Trả về mảng chứa các mốc thời gian vật lý tại tâm đối xứng của từng khung phân tích (tính bằng giây). Hàm có trả về mảng này, nhưng trong `main.py` dòng gọi lệnh là `pitch, _ = extract_features_praat(...)`, tức **mảng mốc thời gian thực tế đã bị vứt bỏ**.

3.3. Ánh xạ chi tiết từ Praat C++ sang lý thuyết tài liệu này

Bước thực thi bên trong `snd.to_pitch()`	**Vị trí trong tài liệu này**	**Công thức / Tham số liên kết**
Cắt khung 3 chu kỳ của 75 Hz $=$ 40 ms, bước trượt 10 ms	Mục 3.A	Quy tắc Boersma, Tham số 1–2
Nhân cửa sổ Hanning (do cờ `very_accurate = off`)	Mục 3.A	Công thức (5), Tham số 4
Tính tự tương quan $\mathcal{R}_x[\tau]$ của khung nhân cửa sổ	Mục 2	Công thức (1)
Bù suy hao cửa sổ bằng phép chia cho $\mathcal{R}_w[\tau]$	Mục 3.B	Công thức (6)–(9)
Giới hạn dải tìm kiếm đỉnh trong $[\tau_{\min}, \tau_{\max}]$	Mục 2	Công thức (3) và (4)
Trích xuất 14 đỉnh cực đại + 1 ứng viên Vô thanh	Mục 3.C’	Tham số 5, 6, 7 (ngưỡng 0,45 và 0,03)
Phạt log-lag nội khung để tránh lỗi nhảy $F_0/2$	Mục 3.C’	Công thức (14), Tham số 8 (`octave_cost = 0,01`)
Quy hoạch động Viterbi xuyên suốt câu nói	Mục 4.5	Tham số 9–10, Hình 4 (nhánh Praat)
Xuất giá trị cao độ $F_0 = f_s/\tau^*$	Mục 2 & Mục 4.2	Công thức (30), Thẻ công thức

Bảng 9: Đối chiếu chi tiết giữa lệnh gọi `snd.to_pitch()` và các công thức của tài liệu này.

Kết luận: Dòng mã `pitch_object = snd.to_pitch()` chính là sự hiện thực hóa trọn vẹn của Thẻ công thức tài liệu này, nhưng ở phiên bản phần mềm máy tính đầy đủ (sử dụng Viterbi 15 ứng viên, nội suy Sinc dưới mẫu, nhánh phần mềm trong Hình 4), chứ không phải phiên bản FPGA streaming rút gọn ở Mục 4 và Mục 6.

3.4. Chỗ mã nguồn khác lý thuyết và hệ quả kỹ thuật

(a) Tần số lấy mẫu thực tế 24 kHz (thay vì 16 kHz): Mô hình tổng hợp Kokoro (`generate_audio.py`) ghi tệp âm thanh ở tần số lấy mẫu $f_s = 24.000$ Hz (`sf.write(..., 24000)`). Hàm `trim_silence_and_save` nạp tệp với cờ `sr=None` (giữ nguyên tần số) rồi ghi lại. Do đó, Praat đo F_0 trên tín hiệu 24.000 Hz. Toàn bộ các thông số mẫu số học phải điều chỉnh tương ứng:

$$\tau_{\min} = \left\lceil \frac{24.000}{600} \right\rceil = \textbf{40} \text{ mẫu}, \qquad \tau_{\max} = \left\lfloor \frac{24.000}{75} \right\rfloor = \textbf{320} \text{ mẫu} \tag{48}$$

$$L = 0{,}040 \times 24.000 = \textbf{960} \text{ mẫu}, \qquad \Delta t_{\text{hop}} = 0{,}01 \times 24.000 = \textbf{240} \text{ mẫu} \tag{49}$$

So với thiết kế chuẩn 16 kHz trong tài liệu: dải lag mở rộng từ $[27, 213]$ (187 lag) lên $[40, 320]$ (281 lag); kích thước khung tăng từ 640 lên 960 mẫu. **Thời lượng vật lý** của khung (40 ms) và dải **tần số** (75–600 Hz) hoàn toàn giữ nguyên, chỉ có số lượng mẫu rời rạc tỉ lệ thuận với f_s.

Hệ quả tính toán: Một đỉnh tương quan tại $\tau^* = 200$ mẫu trong dự án này sẽ cho cao độ:

$$F_0 = \frac{24.000}{200} = \textbf{120} \text{ Hz}$$

Nếu áp nhầm công thức 16 kHz của tài liệu, lag 200 sẽ cho $16.000/200 = 80$ Hz. Do đó, **tuyệt đối không được lấy bảng tra lag của tài liệu áp vào dữ liệu của dự án này**.

Hệ quả về phi tuyến lượng tử hóa: Tại $f_s = 24$ kHz, bước nhảy 1 lag ở trần tần số cao là $24.000/40 - 24.000/41 \approx 14{,}6$ Hz (mịn hơn rõ rệt so với mức 22 Hz ở 16 kHz). Praat sử dụng phép nội suy Sinc dưới mẫu (sub-sample sinc interpolation) để tìm cực trị trên miền liên tục, trong khi thiết kế FPGA sử dụng nội suy parabol 3 điểm số nguyên (Chế độ B, kết hợp bộ lọc trung vị median-3 trễ 10 ms), cả hai phương pháp đều triệt tiêu hoàn toàn hiện tượng bậc thang hóa ở vùng cao tần.

(b) Có quy hoạch động Viterbi và 15 ứng viên: Khác với thiết kế FPGA streaming chỉ chọn cực đại tức thời cục bộ (1 ứng viên), Praat trong mã nguồn Python duyệt toàn bộ câu nói qua ma trận 15 ứng viên. Hệ quả là đường Pitch không bị gãy đoạn và miễn nhiễm với lỗi nhảy quãng tám, nhưng bắt buộc phải có đủ toàn bộ câu âm thanh trong bộ nhớ mới trích xuất được kết quả.

(c) Thiếu bước lọc thông cao (HPF) 60 Hz: Trong Mục 4 (khung "Cổng kiểm chứng phần mềm"), tài liệu yêu cầu tín hiệu trước khi tính tương quan phải đi qua bộ lọc HPF 60 Hz (hoặc DC-blocker) để dập tắt trôi DC và nhiễu cơ học. Mã nguồn Python đưa thẳng tín hiệu tổng hợp vào Praat mà không lọc. Do Kokoro là âm thanh tổng hợp sạch nên ảnh hưởng không lớn, nhưng đây là một điểm lệch quy trình so với chuẩn thiết kế vi mạch.

(d) Giải phẫu mã nguồn C++ của Praat (`Sound_to_Pitch.cpp` và `Pitch.cpp`).

Như đã phân tích tại Mục 3.D, mã nguồn C++ của Praat là phần mềm nguồn mở. Hai tệp nguồn `fon/Sound_to_Pitch.cpp` cùng `fon/Pitch.cpp` làm sáng tỏ cơ chế tính toán thực tế của thuật toán:

Khẳng định 1 — Praat không tự lấy mẫu lại. Hàm `Sound_to_Pitch_any` đọc trực tiếp mảng mẫu `my z[channel][i]` của đối tượng `Sound` và dùng `my dx` (nghịch đảo tần số lấy mẫu) trong mọi phép quy đổi thời gian. Không có lời gọi lấy mẫu lại nào trên đường đi. Hệ quả là ở tần số 24 kHz của dự án Kokoro, cửa sổ phân tích là 960 mẫu chứ không phải 640, đúng như phân tích ở Mục 3.4.(a).

Khẳng định 2 — Praat tính tự tương quan bằng FFT, không bằng vòng lặp thời gian. Với phương pháp mặc định (`AC_HANNING`, tương ứng `very_accurate = off`), mã nguồn đi theo nhánh FFT. Chuỗi thao tác, đọc đúng theo thứ tự trong hàm `Sound_to_Pitch_autoCorrelate`:

1. **Trừ trung bình cục bộ (khử DC).** Mã nguồn tính `localMean` trên một cửa sổ dài hai chu kỳ dài nhất, rồi trừ nó khỏi từng mẫu *trước khi* nhân cửa sổ: `frame[j] = (z[i] - localMean) * window[j]`. Chú thích trong mã ghi rõ: "We are going to kill the DC component before windowing."
2. **Nhân cửa sổ Hanning.** Cửa sổ được dựng bằng công thức:
   `window[i] = 0.5 - 0.5 * cos(2*pi*i/(nsamp_window+1))`
   — hoàn toàn trùng khớp công thức bảng ROM $N + 1 = 641$ trong sách (với $N = 640$).
3. **Đệm số 0 lên lũy thừa của 2.** Mã nguồn tính `nsampFFT` bằng cách nhân đôi đến khi vượt `nsamp_window*(1+depth)`. Với 638 mẫu và độ sâu nội suy 0,5, ngưỡng là 957, nên `nsampFFT = 1024`. Mục đích: triệt tiêu hiệu ứng cuộn vòng biên của biến đổi Fourier.
4. **FFT thực, bình phương biên độ, IFFT.** Định lý Wiener–Khinchin: gọi hàm `NUMfft_forward`, tính $|X[k]|^2 = \text{re}^2 + \text{im}^2$ dồn vào mảng `ac`, rồi gọi `NUMfft_backward`. Kết quả `ac` chính là hàm tự tương quan.
5. **Chia cho tự tương quan của cửa sổ.** Mã nguồn tính sẵn `windowR` bằng đúng chuỗi FFT → bình phương → IFFT trên chính cửa sổ, chuẩn hóa nó, rồi chia: `r[i] = ac[i+1] / (ac[1] * windowR[i+1])`. Đây là phép bù suy hao $C[\tau]$ mà sách đã mô tả, nhưng Praat thực hiện bằng phép *chia* chứ không bằng bảng tra.
6. **Nội suy sinc dưới mẫu.** Sau khi có đỉnh nguyên, mã nguồn dùng `NUM_interpolate_sinc` với độ sâu 30 để tìm cực trị trên miền liên tục, rồi tính tần số bằng `frequencyOfMaximum = 1.0 / my dx / (i + dr/d2r)`.

Vì sao hai cách tính cho cùng một kết quả. Điểm cần nắm: phương pháp FFT và phương pháp tổng thời gian mà FPGA cài đặt **không phải hai thuật toán gần giống nhau**, mà là hai cách tính của *cùng một đại lượng*. Với đệm số 0 đủ dài, tương quan vòng của FFT không còn chồng lấn, nên kết quả tại các độ trễ nguyên bằng đúng tổng thời gian. Phép kiểm tra số học trong quá trình soạn tài liệu xác nhận điều này: với cửa sổ 638 mẫu, `NFFT`

`= 1024` cho sai lệch cỡ 10^{-14} so với tổng trực tiếp, trong khi `NFFT = 512` cho sai lệch cỡ 10^{2} vì chồng phổ vòng. Ngưỡng an toàn là `NFFT` $\geq$ độ dài cửa sổ cộng độ trễ lớn nhất, tức $638 + 319 = 957$; 1024 vượt ngưỡng đó.

Điều này có hệ quả thực hành cho người đọc: khi đối chiếu bản Python số nguyên với Praat, sai lệch còn lại **không** đến từ việc Praat "tính khác". Nó đến từ bốn chỗ đã nêu ở trên — cửa sổ dài hơn, phép chia thay bảng tra, nội suy sinc, và Viterbi — chứ không phải từ công thức tự tương quan.

3.5. Bản chất trục thời gian của mảng `pitch`

Với bước nhảy trượt $\Delta t_{\text{hop}} = 10$ ms, một câu nói dài D giây sẽ tạo ra khoảng $100D$ điểm cao độ. Vì độ dài khung là 40 ms, tâm đối xứng của khung phân tích đầu tiên không nằm ở $t = 0$ mà lùi về sau khoảng nửa khung (≈ 20 ms). Đây chính là hiện tượng "độ lệch tâm pha" đã phân tích ở Hình 6. Mặc dù hàm `pitch_object.xs()` trả về chính xác tọa độ thời gian này, việc mã nguồn bên ngoài bỏ qua `xs()` sẽ dẫn đến sai lệch căn chỉnh được chỉ ra ở Mục 5.1.

4. `trim_silence_and_save`: Cắt Khoảng Lặng Hai Đầu

4.1. Mã nguồn

```
y, sr = librosa.load(audio_path, sr=None)
y_trimmed, _ = librosa.effects.trim(y, top_db=top_db)   # top_db = 30
if not len(y_trimmed) or not np.any(y_trimmed):
    raise ValueError(...)
sf.write(trimmed_path, y_trimmed, sr)
```

4.2. Cơ chế vật lý

Hàm `librosa.effects.trim` chia tín hiệu thành các khung ngắn (mặc định 2048 mẫu, bước nhảy 512 mẫu), tính năng lượng hiệu dụng RMS của từng khung và đổi sang thang decibel (dB) quy chiếu theo khung có năng lượng lớn nhất trong toàn bộ tệp. Khung nào có năng lượng thấp hơn đỉnh cực đại quá top_db $= 30$ dB sẽ bị coi là khoảng lặng. Hàm chỉ thực hiện cắt bỏ **phần lặng ở đầu câu và cuối câu**, hoàn toàn không tác động vào các khoảng lặng hoặc ngắt nghỉ giữa câu.

Ý nghĩa định lượng của con số 30 *dB:*

$$10^{-30/20} = 10^{-1,5} \approx \mathbf{0,0316}$$

Nghĩa là tín hiệu bị xem là lặng khi biên độ tức thời rơi xuống dưới mức $3,2\%$ so với biên độ đỉnh lớn nhất của câu nói.

4.3. Đối chiếu giữa `top_db = 30` của Librosa và `silence_threshold = 0,03` của Praat

Tiêu chí so sánh	`top_db = 30` (`librosa.effects.trim`)	`silence_threshold = 0,03` (Praat Tham số 6)
Tác động xử lý	Cắt bỏ hoàn toàn các mẫu dữ liệu ở hai đầu tệp	Giữ nguyên mẫu dữ liệu, gán nhãn khung là Vô thanh ($F_0 = 0$)
Phạm vi áp dụng	Chỉ tác động ở phần đầu và phần cuối câu	Quét trên toàn bộ các khung xuyên suốt câu nói
Đại lượng so sánh	Năng lượng hiệu dụng RMS khung so với RMS đỉnh (dB)	Tỷ số biên độ đỉnh cục bộ khung so với đỉnh toàn tệp

Bảng 10: So sánh bản chất giữa việc cắt lặng bằng Librosa và ngưỡng lặng của Praat.

Mục đích thực chất của bước cắt lặng: Cắt khoảng lặng không thể thay thế cơ chế nhận diện thanh tính của Praat; các phụ âm vô thanh và khoảng ngắt giữa câu vẫn được Praat gắn nhãn $F_0 = 0$. Mục đích duy nhất của `trim_silence_and_save` là **đồng bộ gốc thời gian**: hai giọng đọc khác nhau luôn có thời gian chuẩn bị phát âm đầu câu chênh lệch nhau. Việc cắt bỏ đoạn lặng đầu giúp mốc $t = 0$ của cả hai giọng áp sát vào thời điểm bật âm đầu tiên, tạo điều kiện thuận lợi cho phép căn chỉnh theo từ tiếp theo.

5. Căn Chỉnh Hai Đường Cao Độ Theo Từng Từ

Phụ lục 5.1. `get_local_chunk`: Cắt đoạn cao độ của một từ

```
i1 = int((w_start / total_dur) * N)
i2 = int((w_end / total_dur) * N)
return raw_array[i1:i2]
```

Hàm chuyển đổi mốc thời gian của từ ($w_{\text{start}}, w_{\text{end}}$, tính bằng giây do Whisper cung cấp) thành chỉ số mảng trong vector cao độ `pitch` theo **phép tính tỉ lệ thuận**:

$$i = \left\lfloor \frac{t}{D} \cdot N \right\rfloor \tag{50}$$

với D là tổng thời lượng tệp âm thanh đã cắt ($D = \text{len}(y)/\text{sr}$) và N là tổng số điểm trong mảng `pitch`.

Ví dụ số: Giả sử $D = 2{,}0$ s, mảng có $N = 197$ điểm. Từ "hello" phát âm từ $t = 0{,}50$ s đến $0{,}90$ s:

$$i_1 = \lfloor (0{,}50/2{,}0) \times 197 \rfloor = \lfloor 49{,}25 \rfloor = 49, \qquad i_2 = \lfloor (0{,}90/2{,}0) \times 197 \rfloor = \lfloor 88{,}65 \rfloor = 88$$

Đoạn trích xuất gồm $88 - 49 = 39$ điểm cao độ.

Sai lệch so với lý thuyết (Do bỏ mảng `xs()`): Phép tính tỉ lệ trên ngầm giả định các điểm cao độ phủ đều từ $t = 0$ đến $t = D$. Trên thực tế, tâm đối xứng của khung thứ i nằm tại $t_i \approx 0{,}02 + i \times 0{,}01$ s (độ lệch tâm pha ban đầu 20 ms). Điểm thứ 49 trong thực tế nằm tại thời điểm $0{,}02 + 49 \times 0{,}01 = 0{,}51$ s, chứ không phải $0{,}50$ s. Sai lệch này luôn nhỏ hơn nửa khung (≤ 20 ms, tương đương ≤ 2 điểm cao độ), và do xảy ra đồng thời trên cả hai tệp âm thanh nên phần lớn sai số tự triệt tiêu lẫn nhau.

5.2. `equalize_local_chunk`: Co giãn đoạn cao độ bằng phép nội suy

```
x_ref  = np.linspace(0, 1, len(ref_chunk))
x_user = np.linspace(0, 1, len(user_chunk))
return np.interp(x_ref, x_user, user_chunk)
```

Hai đoạn cao độ của cùng một từ có thời lượng phát âm khác nhau được đưa về cùng trục chuẩn hóa $[0, 1]$ (vị trí bắt đầu từ ứng với 0, kết thúc từ ứng với 1). Sau đó hàm `np.interp` thực hiện **nội suy tuyến tính** giá trị F_0 của người đọc tại các tọa độ chuẩn hóa của giọng tham chiếu. Kết quả là đoạn cao độ của người đọc được co giãn về đúng số điểm của giọng tham chiếu. Đây là giải pháp đơn giản hóa thay thế cho thuật toán Dynamic Time Warping (DTW) phức tạp.

▶ NHÌN THẤY VẬT LÝ

Lỗi tiềm ẩn nghiêm trọng: Nội suy qua giá trị 0 Hz sinh ra F_0 giả: Hàm `np.interp` thuần túy xem mảng số liệu là các đại lượng hình học liên tục, hoàn toàn không biết con số 0 mang ý nghĩa đặc biệt là "Vô thanh". Tại ranh giới giữa âm vô thanh và âm hữu thanh (ví dụ hai điểm kế tiếp nhau có giá trị $[0, 200]$ Hz), phép nội suy tại điểm ở giữa sẽ gán giá trị 100 Hz.

Đây là một **giá trị cao độ giả mạo (spurious pitch)**: không hề có dao động dây thanh quản nào diễn ra ở tần số 100 Hz tại thời điểm đó! Vì $100 > 0$, điểm giả này lọt qua bộ lọc kiểm tra tính hữu thanh ở Mục 6, tạo ra một điểm sụt dốc đột ngột bất thường và làm suy giảm hệ số tương quan Pearson r. Cách khắc phục chuẩn mực là phải gắn cờ NaN cho các điểm vô thanh hoặc thực hiện nội suy riêng biệt trên từng đoạn hữu thanh liên tục.

5.3. Ghép nối thành đường cao độ toàn câu (`stitched_user_pitch`)

Trong `visualizer.py`, hàm `plot_analysis` khởi tạo mảng `stitched_user_pitch` toàn số 0 có độ dài bằng mảng tham chiếu N_{ref}. Sau đó, với mỗi từ khớp thành công giữa văn bản chuẩn và kết quả Whisper (khớp bằng `difflib.SequenceMatcher`, cờ `equal`), đoạn cao độ đã co giãn được dán đè vào đúng vị trí chỉ số của từ đó bên mảng tham chiếu. Nhờ đó, đường tham chiếu chuẩn được giữ nguyên vẹn, các khoảng ngừng giữa các từ hoặc các từ phát âm sai bị gán giá trị 0 và tự động bị loại khỏi phép tính tương quan.

6. `dynamic_sliding_window_pearson`: Tính Hệ Số Pearson Trên Các Cửa Sổ Trượt

6.1. Mã nguồn chính

```
W = max(6, int((2 * N) / (num_words + 2)))
if W % 2 != 0: W -= 1
S = max(1, W // 2)
num_slides = num_words + 1
...
valid = (u_win > 0) & (r_win > 0)
if np.sum(valid) >= 3 and np.std(u_win[valid]) > 0 and np.std(r_win[valid]) > 0:
    r = np.corrcoef(u_win[valid], r_win[valid])[0, 1]
else:
    r = np.nan
```

6.2. Nguồn gốc toán học của kích thước cửa sổ W

Gọi $K = \texttt{num_words}$ là số từ trong câu chuẩn, và N là tổng số điểm cao độ tham chiếu. Thuật toán tạo ra đúng $K+1$ cửa sổ trượt có kích thước W, với bước trượt gối đầu nhau một nửa: $S = W/2$. Tổng phạm vi thời gian mà $K+1$ cửa sổ này bao phủ được xác định bởi:

$$\underbrace{K \cdot S}_{\text{Tọa độ bắt đầu của cửa sổ cuối}} + W = K \cdot \frac{W}{2} + W = \frac{(K+2)W}{2} \tag{51}$$

Để toàn bộ các cửa sổ phủ trọn vẹn toàn bộ câu nói dài N điểm, ta đặt tổng này bằng N:

$$\frac{(K+2)W}{2} = N \implies W = \frac{2N}{K+2} \tag{52}$$

Ý nghĩa ngữ nghĩa học: Vì N/K là số điểm trung bình của một từ, kích thước $W \approx 2 \times (N/K)$ tương đương với khoảng thời lượng của **hai từ liên tiếp**. Nhờ đó, mỗi cửa sổ trượt bao trọn ngữ điệu của một cụm từ nhỏ. Mã nguồn ép W chẵn để bước trượt $S = W/2$ là số nguyên, đặt ngưỡng chặn dưới $W \geq 6$, và nới rộng cửa sổ cuối cùng chạm mốc N để bù trừ sai số làm tròn.

6.3. Công thức tính Pearson r và tính chất bất biến quãng tám

Trong mỗi cửa sổ, thuật toán áp dụng mặt nạ lọc `valid`: chỉ giữ lại các vị trí mà **cả hai đường đều hữu thanh** ($u_i > 0$ và $v_i > 0$). Với M cặp điểm hợp lệ (u_i, v_i), hệ số tương quan tuyến tính Pearson được tính theo công thức:

$$r_{\text{Pearson}} = \frac{\sum_{i=1}^{M}(u_i - \bar{u})(v_i - \bar{v})}{\sqrt{\sum_{i=1}^{M}(u_i - \bar{u})^2} \cdot \sqrt{\sum_{i=1}^{M}(v_i - \bar{v})^2}} \tag{53}$$

Hàm `np.corrcoef(...)[0, 1]` trả về giá trị này. Cửa sổ bị gán NaN khi có ít hơn 3 điểm hợp lệ ($M < 3$) hoặc khi đường cao độ nằm phẳng tuyệt đối ($\sigma = 0$, mẫu số triệt tiêu).

▶ NHÌN THẤY VẬT LÝ

Vì sao Pearson r rất phù hợp để so giọng nam (`michael`) với giọng nữ (`sarah`): Hệ số tương quan Pearson có tính chất toán học quan trọng: **Bất biến đối với các phép biến đổi afin dương** ($r(X, aY+b) = r(X,Y)$ với mọi $a > 0$). Giả sử giọng nam `michael` phát âm trầm hơn giọng nữ `sarah` đúng một quãng tám ($F_{0,\text{nam}} = 0{,}5 \times F_{0,\text{nữ}}$), nhưng hình thái luyến láy lên xuống hoàn toàn trùng khớp:

$$u_i = 0{,}5 \cdot v_i \implies r_{\text{Pearson}} = \mathbf{1{,}0}$$

Ngược lại, nếu người đọc nhấn ngữ điệu đi xuống trong khi mẫu đi lên, hệ số sẽ đổi dấu thành âm ($r < 0$). Do đó, Pearson r đóng vai trò là một bộ đo **hình thái giai điệu ngữ điệu (intonation contour shape)**, chứ không đo độ cao tuyệt đối của giọng nói.

7. `summarize_pearson` và `summarize_sentence_reports`: Từ Cửa Sổ Đến Điểm Số

7.1. Tổng hợp điểm số cho một câu

```
valid = values[np.isfinite(values)]
'mean_r': float(np.maximum(valid, 0).mean())
```

Quy trình tính điểm `mean_r` của một câu gồm 3 bước:

1. Loại bỏ toàn bộ các cửa sổ có giá trị NaN (`missing_r_policy = 'exclude'`).
2. Đổi toàn bộ các giá trị tương quan âm ($r \leq 0$) thành 0 (`nonpositive_r_policy = 'zero'`). Triết lý thiết kế: tương quan âm hay bằng 0 đều phản ánh việc phát âm sai ngữ điệu, không cần thiết phải phạt thêm giá trị âm.
3. Tính trung bình cộng của các giá trị sau khi kẹp, cho ra điểm số `mean_r` nằm trong khoảng $[0, 1]$.

Ví dụ số: Một câu có 9 cửa sổ cho kết quả: $[0{,}9;\ 0{,}8;\ \text{NaN};\ -0{,}4;\ 0{,}7;\ 0{,}6;\ \text{NaN};\ 0{,}85;\ 0{,}75]$.

- Loại 2 cửa sổ NaN $\implies$ còn 7 cửa sổ hợp lệ (`valid_r_count = 7`).
- Kẹp giá trị âm: $-0{,}4 \to 0$.
- Tổng tích lũy: $0{,}9 + 0{,}8 + 0 + 0{,}7 + 0{,}6 + 0{,}85 + 0{,}75 = 4{,}6$.
- Điểm trung bình: `mean_r` $= 4{,}6/7 \approx$ **0,657**. (Nếu không kẹp về 0, điểm trung bình thô chỉ đạt $(4{,}6 - 0{,}4)/7 = 0{,}600$).

7.2. Thống kê báo cáo nhiều câu (`summarize_sentence_reports`)

Hàm đếm số lượng câu thành công (trạng thái `ok`) đạt điểm `mean_r` vượt qua các ngưỡng từ $0{,}9$ xuống $0{,}3$ (`range(9, 2, -1)` chia 10). Các ngưỡng này mang tính chất **tích lũy (cumulative)**: một câu đạt $0{,}9$ sẽ đương nhiên được đếm trong tất cả các mốc thấp hơn ($\geq 0{,}8, \ldots, \geq 0{,}3$).

8. `evaluator.py`: Nhận Diện Mốc Thời Gian Bằng Whisper

8.1. Mã nguồn chính

```
torch.set_num_threads(min(4, torch.get_num_threads()))
whisper.load_model('base', ...)
...
audio, sr = sf.read(audio_path, dtype='float32')
if audio.ndim > 1: audio = audio.mean(axis=1)
if sr != 16000: audio = librosa.resample(audio, orig_sr=sr, target_sr=16000)
result = model.transcribe(audio, word_timestamps=True, fp16=False, language='en')
return [dict(word=normalize_word(w['word']), start=w['start'], end=w['end']) ...]
```

8.2. Các bước thực thi cốt lõi

1. `load_stt_model`: Khóa số luồng tính toán CPU tối đa là 4 để tránh hiện tượng tranh chấp tài nguyên (thread contention) khi xử lý các đoạn âm thanh ngắn, sau đó nạp mô hình Whisper kiến trúc `base`.
2. `get_user_timestamps`: Đọc tệp WAV đã cắt lặng, chuyển stereo thành mono bằng phép lấy trung bình cộng hai kênh, và **lấy mẫu lại tín hiệu từ** 24 **kHz về đúng** 16 **kHz** (chuẩn đầu vào bắt buộc của mạng nơ-ron Whisper).
3. Thực thi nhận dạng tiếng nói (`model.transcribe`) với cờ `word_timestamps=True` để trích xuất mốc thời gian chi tiết của từng từ, chạy ở chế độ số thực 32-bit (`fp16=False`).
4. Hàm `normalize_word` loại bỏ toàn bộ dấu câu và chuyển thành chữ thường để đối soát với văn bản chuẩn.

8.3. Sự phân công kiến trúc hai luồng song song

Con số 16 kHz trong `evaluator.py` trùng với tần số f_s trong luồng Mel ASR, nhưng chỉ phục vụ cho mạng nơ-ron Whisper. Whisper biến đổi âm thanh 16 kHz thành vector **Log-Mel Spectrogram** để nhận diện từ — chính là loại đặc trưng đã xóa bỏ cao độ. Ngược lại, luồng đo cao độ Praat giữ nguyên tín hiệu 24 kHz để bảo toàn độ phân giải chu kỳ thanh quản. Đây là minh chứng thực tế rõ nét cho mô hình **Hai Luồng Xử Lý Song Song** mà tài liệu đã đề xuất trong tài liệu này, phần 0.

8.4. Có thể thay Whisper bằng thuật toán nhẹ hơn không?

Vì sao câu hỏi này đáng hỏi. Trong bài toán chấm điểm phát âm, **văn bản chuẩn đã biết trước 100%**. Người học đang đọc đúng một câu cho sẵn. Vậy mà quy trình hiện tại vẫn chạy một mô hình nhận dạng tiếng nói đầy đủ để *đoán lại* xem người ta vừa nói gì — trong khi ta đã biết câu trả lời. Whisper ở đây chỉ được dùng để lấy mốc thời gian $(w_{\text{start}}, w_{\text{end}})$ của từng từ, và nó làm việc đó gián tiếp: nhận dạng chữ trước, rồi suy ra thời gian.

Đây là một sự lệch pha giữa công cụ và nhiệm vụ. Whisper là mô hình lớn (hàng trăm triệu tham số), cần GPU để chạy nhanh, và có thể nhận dạng sai chữ — kéo theo mốc thời gian sai theo. Với một bài toán mà đáp án đã biết, có hai cách rẻ hơn và chính xác hơn.

Phương án 1 — Căn chỉnh cưỡng bức (Forced Alignment). Thay vì hỏi "người này nói gì", hỏi thẳng "câu đã biết này khớp vào đoạn âm thanh này ở đâu". Đó chính là định nghĩa của forced alignment: cho trước cả tín hiệu âm thanh *và* chuỗi âm vị của văn bản chuẩn, thuật toán tìm ranh giới thời gian của từng âm vị.

Các công cụ có sẵn:

- **Montreal Forced Aligner (MFA):** dùng mô hình âm học (thường là GMM-HMM hoặc mạng nơ-ron nhỏ) cộng từ điển phát âm. Độ chính xác ranh giới từ thường dưới 20 ms.
- **Kaldi / Vosk:** cùng nguyên lý, có bản nhẹ chạy được trên CPU.
- **`torchaudio.pipelines.MMS_FA`:** mô hình wav2vec2 huấn luyện riêng cho căn chỉnh, chạy được trên CPU, chỉ cần vài chục mili-giây cho một câu ngắn.

Ưu điểm so với Whisper: vì đã biết văn bản, thuật toán **không thể "nhận dạng sai"** — nó chỉ có thể đặt ranh giới lệch. Sai số được giới hạn ở trục thời gian, không lan sang nội dung.

Phương án 2 — Dynamic Time Warping trên chuỗi 80 Log-Mel. Cách này bỏ luôn cả bước nhận dạng. Ta đã có ma trận 80 dải Log-Mel của câu mẫu và của người học (đều tính trên máy chủ từ tệp WAV sạch do FPGA chuyển lên qua Shared Folder). Dynamic Time Warping (DTW — phép uốn thời gian động) tìm một ánh xạ giữa hai chuỗi khung sao cho tổng khoảng cách là nhỏ nhất, cho phép co giãn trục thời gian.

Cụ thể, với hai chuỗi vector đặc trưng $A = [a_1, \ldots, a_M]$ (câu mẫu) và $B = [b_1, \ldots, b_N]$ (người học), DTW dựng ma trận chi phí D với

$$D[i][j] = \|a_i - b_j\|^2 + \min\big(D[i-1][j],\ D[i][j-1],\ D[i-1][j-1]\big),$$

rồi truy vết ngược từ $D[M][N]$ để lấy đường đi tối ưu. Đường đi đó chính là ánh xạ khung-thời-gian, và từ đó suy ra mốc của từng từ bằng cách ánh xạ các mốc đã biết của câu mẫu.

Ưu điểm: **không cần mô hình nhận dạng nào cả**, chỉ cần ma trận đặc trưng tính từ tệp âm thanh sạch. Nhược điểm: DTW không biết ngữ nghĩa, nên nếu người học đọc sai hẳn một từ (nói nhầm từ khác), đường đi sẽ bị kéo lệch ở chỗ đó.

So sánh ba lựa chọn.

	Whisper (hiện tại)	**Forced Alignment**	**DTW trên Mel**
Cần mô hình nhận dạng	Có (lớn)	Không	Không
Cần văn bản chuẩn	Không	Có	Không (chỉ cần câu mẫu)
Sai số ranh giới từ	Ở mức từ, phụ thuộc nhận dạng	< 20 ms	Phụ thuộc độ dài câu
Chạy được trên CPU	Chậm	Có	Có
Rủi ro chính	Nhận dạng sai chữ	Cần từ điển phát âm	Lệch khi đọc sai từ

Kết luận thực hành. Với bài toán chấm phát âm có văn bản cho trước, **forced alignment là lựa chọn mặc định hợp lý**: nó rẻ hơn Whisper, chính xác hơn về ranh giới, và loại bỏ hẳn một nguồn lỗi. DTW là lựa chọn hấp dẫn khi muốn bỏ cả mô hình nơ-ron, đặc biệt vì nó dùng đúng ma trận 80 Log-Mel tính từ tệp WAV sạch mà FPGA đã lọc và chuyển lên Shared Folder. Whisper vẫn có chỗ đứng trong một trường hợp: khi người học đọc một câu *không* cho trước, hoặc khi cần kiểm tra xem họ có đọc đúng từ hay không — lúc đó việc nhận dạng chữ chính là mục tiêu, chứ không còn là phương tiện.

9. Bảng Tổng Hợp So Sánh Toàn Diện: Lý Thuyết tài liệu này vs. Mã Nguồn Python

Đặc tính kỹ thuật	**Thiết kế Lý thuyết tài liệu này**	**Mã nguồn Python (`Pitch-Project`)**	**Đánh giá & Hệ quả kỹ thuật**
Thuật toán đo cao độ	Tự cài đặt Công thức (1), (7)–(9)	Gọi thư viện `parselmouth` (`to_pitch`)	Dùng trực tiếp mã nguồn C++ Praat chuẩn
Tần số lấy mẫu đo F_0	$f_s = 16.000$ Hz	$f_s = 24.000$ Hz (do Kokoro TTS sinh ra)	Dải lag đối thành $[40, 320]$ mẫu, $L = 960$ mẫu
Dải tần số tìm kiếm	75–600 Hz	75–600 Hz (tham số Praat mặc định)	Trùng khớp hoàn toàn
Kích thước khung / Bước trượt	40 ms / 10 ms (640 / 160 mẫu)	40 ms / 10 ms (960 / 240 mẫu)	Trùng khớp thời lượng vật lý
Dạng cửa sổ phân tích	Hanning đối xứng	Hanning đối xứng (`very_accurate = off`)	Trùng khớp dạng cửa sổ
Ngưỡng thanh tính / Ngưỡng lặng	$\theta_{\text{voice}} = 0{,}45$; $\theta_{\text{silence}} = 0{,}03$	Mặc định Praat ($0{,}45$ và $0{,}03$)	Trùng khớp tuyệt đối
Cơ chế trích chọn cực đại	FPGA: 189 lag, nội suy parabol, median-3 (trễ 10 ms)	15 ứng viên + Viterbi + Nội suy Sinc	Đạt độ mịn cao tần, loại bỏ bậc thang
Tiền xử lý lọc thông cao	Bắt buộc bộ lọc HPF 60 Hz / DC-blocker	Không có bước lọc nào trước Praat	Lệch quy trình, chấp nhận được với TTS
Tiền xử lý cắt khoảng lặng	So sánh năng lượng khung tĩnh	Dùng `librosa.effects.trim` (30 dB) 2 đầu	Nhằm đồng bộ gốc thời gian $t = 0$
Phương pháp đối sánh cao độ	Chuẩn bị đưa lên vi mạch FPGA	Pearson r trên cửa sổ trượt ≈ 2 từ	Đo hình thái giai điệu ngữ điệu
Tần số lấy mẫu nhận diện	16 kHz (tệp WAV sạch do FPGA xuất)	16 kHz (lấy mẫu lại đưa vào Whisper)	Hiện thực đúng mô hình hai luồng song song

Bảng 11: Bảng tổng hợp đối chiếu toàn diện giữa thiết kế lý thuyết tài liệu này và mã nguồn thực tế.

10. Danh Sách Các Điểm Lệch Và Lỗi Tiềm Ẩn Cần Lưu Ý

1. **Vứt bỏ mảng mốc thời gian thực `xs()` của Praat:** Việc tính chỉ số khung theo tỉ lệ tuyến tính $i = \lfloor (t/D) \cdot N \rfloor$ bỏ qua độ lệch tâm pha ban đầu 20 ms, gây ra sai số thời gian tối đa cỡ nửa khung (≤ 2 điểm cao độ).

2. **Hiện tượng sinh F_0 giả khi nội suy qua điểm 0 Hz:** Hàm `np.interp` nội suy tuyến tính giữa các điểm vô thanh (0 Hz) và hữu thanh (> 0 Hz), tạo ra các giá trị tần số giả mạo lọt qua bộ lọc thanh tính và làm suy giảm nhân tạo hệ số tương quan Pearson r.

3. **Lệch tần số lấy mẫu (24 kHz vs 16 kHz):** Mọi bảng tra lag, địa chỉ ROM và số lượng phép tính MAC trong Mục 5 của tài liệu không thể áp dụng trực tiếp cho dữ liệu Kokoro mà phải nhân theo tỉ lệ $24/16 = 1{,}5$.

4. **Bỏ qua bộ lọc HPF 60 Hz:** Mã nguồn đưa tín hiệu thô vào phân tích mà không qua khâu triệt tiêu trôi DC như yêu cầu của Cổng kiểm chứng phần mềm.

5. **Kẹp giá trị tương quan không dương ($r \leq 0$) về 0:** Phép kẹp này làm điểm trung bình `mean_r` của câu tăng cao hơn so với trung bình số học thực tế.

6. **Tên tệp `evaluator.py` gây hiểu lầm:** Tệp này không thực hiện đánh giá hay chấm điểm mà chỉ là bộ trích xuất mốc từ dựa trên Whisper.

7. **Tính Pearson trên thang tần số Hertz tuyến tính:** Hệ số Pearson tính trực tiếp trên đơn vị Hz mà không đổi sang semitone (thang logarit), khiến các biến động ở dải tần số cao chịu trọng số ảnh hưởng lớn hơn dải trầm.

8. **Sai số làm tròn số thực trong thống kê ngưỡng:** Phép chia số thực `threshold / 10` trong Python có thể sinh ra phần dư làm tròn vô cùng nhỏ, cần lưu ý khi xây dựng các bài test kiểm thử tự động (Unit Tests).

11. Nhật Ký Chạy Kiểm Thử Thực Nghiệm (`run_log_v6.txt`)

Toàn bộ các con số và bảng tra trong tài liệu được đối soát trực tiếp từ quá trình chạy thực tế của các tập lệnh `lab_1_3_5.py`, bộ kiểm thử `test_lab_1_3_5.py` và `verify_tiers.py`:

```
============================================================
1. python lab_1_3_5.py --selftest
============================================================
=== SELFTEST lab_1_3_5.py ===
H[0..4]: [1, 3, 7, 13, 20]
C[26..30]: [8283, 8287, 8291, 8296, 8301]
C[26..30] hex: ['205b', '205f', '2063', '2068', '206d']
```

```
C[214]: 17070 (hex: 42ae)
golden x[0..4]: [0, 2817, 5550, 8117, 10443]
440 Hz S[35..37]: S[35]=31303452800, S[36]=32119378939, S[37]=31978033782
Parabolic mode B: tau_star=36, tau_fine_q16=2382387, f0_q12_4=7042 (0x1b82 = 440.125 Hz)
Parabolic float: 440.137 Hz (+0.54 c)
Octave Cost Sweep:
  cost=0.000: 300 / 526 failures
  cost=0.005: 20 / 526 failures
  cost=0.009: 0 / 526 failures
  cost=0.010: 0 / 526 failures
  cost=0.020: 0 / 526 failures
Sine Freq Accuracy (Mode B vs true freq):
   75 Hz: +1.44 c (F0 = 75.062 Hz)
  100 Hz: +1.08 c (F0 = 100.062 Hz)
  200 Hz: +0.54 c (F0 = 200.062 Hz)
  440 Hz: +0.49 c (F0 = 440.125 Hz)
  590 Hz: +0.55 c (F0 = 590.188 Hz)


================================================================
2. python lab_1_3_5.py --export-hex hex
================================================================
Exported .hex files to hex


================================================================
3. python -m pytest test_lab_1_3_5.py
================================================================
============================= test session starts =============================

platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\NGOC\AppData\Local\Programs\Python\Python312\python.exe

cachedir: .pytest_cache
rootdir: C:\Users\NGOC\Documents\learning_journey\voice_jetson_to_fpga_learning-journey
configfile: pyproject.toml
plugins: anyio-4.14.2
collecting ... collected 15 items

test_lab_1_3_5.py::test_window_rom_is_full_length_and_bounded PASSED [  6%]
test_lab_1_3_5.py::test_window_multiply_keeps_q15_scale PASSED [ 13%]
test_lab_1_3_5.py::test_compensation_table_is_monotone_and_fits_q3_13 PASSED [ 20%]
test_lab_1_3_5.py::test_scoring_rom_is_compensation_times_a_penalty_at_most_one PASSED [ 26%]
test_lab_1_3_5.py::test_integer_detector_matches_float_on_worked_example PASSED [ 33%]
test_lab_1_3_5.py::test_parabolic_interpolation_sub_cent_accuracy PASSED [ 40%]
test_lab_1_3_5.py::test_boundary_interpolation_590hz_and_75hz PASSED [ 46%]
test_lab_1_3_5.py::test_integer_detector_is_deterministic_across_repeated_runs PASSED [ 53%]
test_lab_1_3_5.py::test_silence_is_unvoiced PASSED             [ 60%]
test_lab_1_3_5.py::test_silence_gate_rejects_quiet_frame PASSED [ 66%]
test_lab_1_3_5.py::test_median3_removes_a_single_frame_octave_spike PASSED [ 73%]
test_lab_1_3_5.py::test_median3_preserves_monotone_run PASSED  [ 80%]
test_lab_1_3_5.py::test_octave_cost_sweep_matches_simulation PASSED [ 86%]
test_lab_1_3_5.py::test_export_hex_creates_valid_files PASSED  [ 93%]
test_lab_1_3_5.py::test_pitch_f0_q12_4_protocol PASSED         [100%]

============================== 15 passed in 1.67s ==============================


================================================================
4. python verify_tiers.py
================================================================
Kế hoạch Tier 3: testbench đọc golden_vectors.hex, mục tiêu 0 bit.
Tín hiệu               | Tier 1 (Float/Praat)       | Tier 2 (Int/Float)       | Tier 3 (RTL/Int)
-------------------------------------------------------------------------------------------------
Sin 75 Hz              | cent=0.41, V/UV=0.0%       | cent=0.49, V/UV=0.0%     | Kế hoạch (mục tiêu 0 bit)
Sin 100 Hz             | cent=0.41, V/UV=0.0%       | cent=0.46, V/UV=0.0%     | Kế hoạch (mục tiêu 0 bit)
Sin 200 Hz             | cent=0.43, V/UV=0.0%       | cent=0.10, V/UV=0.0%     | Kế hoạch (mục tiêu 0 bit)
Sin 440 Hz             | cent=0.46, V/UV=0.0%       | cent=0.03, V/UV=0.0%     | Kế hoạch (mục tiêu 0 bit)
Sin 590 Hz             | cent=0.46, V/UV=0.0%       | cent=0.09, V/UV=0.0%     | Kế hoạch (mục tiêu 0 bit)
Im lặng                | Khớp 100% UV (47 khung)    | Khớp 100% UV (47 khung)  | Kế hoạch (mục tiêu 0 bit)
Nhiễu trắng -35dBFS    | Khớp 100% UV (47 khung)    | Khớp 100% UV (47 khung)  | Kế hoạch (mục tiêu 0 bit)
Âm xát /s/             | Khớp 100% UV (47 khung)    | Khớp 100% UV (47 khung)  | Kế hoạch (mục tiêu 0 bit)
Giọng nam (speech_a)   | cent=0.04, V/UV=8.2%       | cent=0.21, V/UV=1.0%     | Kế hoạch (mục tiêu 0 bit)
Giọng nữ (speech_b)    | cent=365.11, V/UV=27.8%    | cent=0.15, V/UV=0.0%     | Kế hoạch (mục tiêu 0 bit)
```